

# P2 Documentation and Code Project

You are viewing draft content  
created with the use of Claude.AI



# P2 I/O & Smart Pins User Guide

*Complete P2 Pin I/O and Smart Pin Reference*

July 2026

[Version 1.0.1](#)

## User Guide Organization

### Complete P2 Smart Pin Documentation

#### Part I: Fundamentals

- Direct I/O Basics
- Enhanced Direct I/O
- Smart Pin Architecture
- Configuration Instructions
- Working with Smart Pins

#### Part II: Output Modes

- Digital Output
- Pulse & Transition
- NCO Frequency/Duty
- PWM Triangle/Sawtooth/SMPS
- DAC Output
- Serial Transmit

#### Part III: Input Modes

- Digital Input
- Timing Measurement
- Counting Modes
- Period/Frequency
- ADC Input
- Serial Receive

#### Part IV: Special Modes

- Repository Mode
- USB Support

#### Part V: Appendices

- Intent Index
- P\_ Constants Reference
- Formulas & Reference Tables
- Mode Comparison Charts
- Troubleshooting Guide
- Complete Mode Reference
- FPGA Board Differences

Iron Sheep Productions, LLC

P2 Knowledge Base Project

# Contents

|                                                                |               |
|----------------------------------------------------------------|---------------|
| <b>Copyright and License</b>                                   | <b>14</b>     |
| Trademarks . . . . .                                           | 14            |
| <b>Acknowledgments</b>                                         | <b>15</b>     |
| <b>How to Use This Guide</b>                                   | <b>16</b>     |
| Path 1: Learning Path (New to P2 I/O) . . . . .                | 16            |
| Path 2: Task-Oriented Path (Know What to Accomplish) . . . . . | 16            |
| Path 3: Reference Path (Know the Mode) . . . . .               | 17            |
| <b>Document Conventions</b>                                    | <b>18</b>     |
| Typography . . . . .                                           | 18            |
| Register Notation . . . . .                                    | 18            |
| Code Examples . . . . .                                        | 18            |
| Terminology . . . . .                                          | 19            |
| Cross-References . . . . .                                     | 19            |
| <b>Quick Mode Selection Matrix</b>                             | <b>20</b>     |
| Output Modes . . . . .                                         | 20            |
| Input Modes . . . . .                                          | 21            |
| Special Modes . . . . .                                        | 21            |
| Mode Categories Quick Reference . . . . .                      | 22            |
| <br><b>Part I: P2 Pin System Fundamentals</b>                  | <br><b>23</b> |
| <b>Chapter 1: Direct I/O</b>                                   | <b>23</b>     |
| 1.1 The Hardware Model . . . . .                               | 23            |
| 1.2 Timing . . . . .                                           | 25            |
| 1.3 Drive Instructions (DRVx) . . . . .                        | 26            |
| 1.4 Output Instructions (OUTx) . . . . .                       | 32            |
| 1.5 Direction Instructions (DIRx) . . . . .                    | 36            |
| 1.6 Float Instructions (FLTx) . . . . .                        | 40            |
| 1.7 Test Pin Instructions . . . . .                            | 44            |
| 1.8 Spin2 Pin Methods . . . . .                                | 46            |
| 1.9 Pin Span Operations . . . . .                              | 48            |
| 1.10 Instruction Quick Reference . . . . .                     | 50            |

|                                                            |           |
|------------------------------------------------------------|-----------|
| 1.11 Common Patterns . . . . .                             | 51        |
| <b>Chapter 2: Enhanced Direct I/O</b>                      | <b>53</b> |
| 2.1 Overview . . . . .                                     | 53        |
| 2.2 Drive Strength . . . . .                               | 55        |
| 2.3 Input Conditioning . . . . .                           | 57        |
| 2.4 Input Source Selection . . . . .                       | 59        |
| 2.5 ADC Input Modes (Basic) . . . . .                      | 61        |
| 2.6 DAC Output Modes (Basic) . . . . .                     | 61        |
| 2.7 Level Comparison Modes . . . . .                       | 62        |
| 2.8 Synchronous I/O Mode . . . . .                         | 63        |
| 2.9 Polarity Control . . . . .                             | 63        |
| 2.10 DIR/OUT Control . . . . .                             | 64        |
| 2.11 Combining Constants . . . . .                         | 64        |
| 2.12 Complete Configuration Examples . . . . .             | 65        |
| 2.13 Resetting to Default . . . . .                        | 67        |
| 2.14 Quick Reference . . . . .                             | 67        |
| <b>Chapter 3: Smart Pin Architecture</b>                   | <b>68</b> |
| 3.1 The Autonomous Operation Concept . . . . .             | 68        |
| 3.2 The X / Y / Z Registers (plus the mode word) . . . . . | 69        |
| 3.3 The IN Bit - Event Signaling . . . . .                 | 70        |
| 3.4 The Smart Pin State Machine . . . . .                  | 72        |
| 3.5 Mode Bits and the 32 Modes . . . . .                   | 74        |
| 3.6 The Layered Configuration Model . . . . .              | 75        |
| 3.7 When to Use Smart Pins . . . . .                       | 76        |
| 3.8 Architectural Constraints . . . . .                    | 77        |
| 3.9 Chapter Summary . . . . .                              | 78        |
| <b>Chapter 4: Smart Pin Configuration</b>                  | <b>79</b> |
| 4.1 Configuration Instructions Overview . . . . .          | 79        |
| 4.2 WRPIN - Write Pin Configuration . . . . .              | 79        |
| 4.3 WXPIN - Write X Register . . . . .                     | 81        |
| 4.4 WYPIN - Write Y Register . . . . .                     | 82        |
| 4.5 RDPIN - Read Z Register with Acknowledge . . . . .     | 84        |
| 4.6 RQPIN - Read Z Register Quietly . . . . .              | 85        |
| 4.7 AKPIN - Acknowledge Only . . . . .                     | 86        |
| 4.8 The Standard Configuration Sequence . . . . .          | 87        |
| 4.9 P_OE - Output Enable . . . . .                         | 89        |
| 4.10 Input Routing . . . . .                               | 90        |
| 4.11 Span Operations . . . . .                             | 91        |
| 4.12 Reading the C Flag . . . . .                          | 91        |
| 4.13 The 2-Clock Acknowledge Delay . . . . .               | 91        |

|                                                  |                |
|--------------------------------------------------|----------------|
| 4.14 Configuration Quick Reference . . . . .     | 92             |
| <b>Chapter 5: Working with Smart Pins</b>        | <b>94</b>      |
| 5.1 The Read/Acknowledge Cycle . . . . .         | 94             |
| 5.2 Continuous vs One-Shot Modes . . . . .       | 97             |
| 5.3 Multi-Pin Patterns . . . . .                 | 99             |
| 5.4 PINSTART and PINCLEAR . . . . .              | 101            |
| 5.5 Debugging Smart Pins . . . . .               | 102            |
| 5.6 Performance Considerations . . . . .         | 103            |
| 5.7 Troubleshooting Quick Reference . . . . .    | 104            |
| 5.8 Best Practices Summary . . . . .             | 106            |
| <br><b>Part II: Output Modes</b>                 | <br><b>107</b> |
| <b>Chapter 6: Digital Output</b>                 | <b>107</b>     |
| 6.1 Overview . . . . .                           | 107            |
| 6.2 Output Configurations . . . . .              | 108            |
| 6.3 Software-Timed Output Patterns . . . . .     | 112            |
| 6.4 Timing Analysis . . . . .                    | 113            |
| 6.5 Complete Examples . . . . .                  | 114            |
| 6.6 Configuration Quick Reference . . . . .      | 116            |
| <br><b>Chapter 7: Pulse &amp; Transition</b>     | <br><b>117</b> |
| 7.1 Overview . . . . .                           | 117            |
| 7.2 P_PULSE Mode (%00100) . . . . .              | 118            |
| 7.3 P_TRANSITION Mode (%00101) . . . . .         | 120            |
| 7.4 Applicable P_ Constants . . . . .            | 122            |
| 7.5 Timing Calculations . . . . .                | 122            |
| 7.6 Comparison: When to Use Each Mode . . . . .  | 124            |
| 7.7 Complete Examples . . . . .                  | 124            |
| 7.8 Quick Reference . . . . .                    | 126            |
| <br><b>Chapter 8: Frequency Generation (NCO)</b> | <br><b>128</b> |
| 8.1 NCO Concept . . . . .                        | 128            |
| 8.2 P_NCO_FREQ Mode (%00110) . . . . .           | 128            |
| 8.3 P_NCO_DUTY Mode (%00111) . . . . .           | 131            |
| 8.4 Phase Synchronization . . . . .              | 132            |
| 8.5 Analog Output with DAC . . . . .             | 134            |
| 8.6 Frequency Accuracy Analysis . . . . .        | 134            |
| 8.7 Complete Examples . . . . .                  | 135            |
| 8.8 Quick Reference . . . . .                    | 137            |
| <br><b>Chapter 9: PWM Output</b>                 | <br><b>139</b> |
| 9.1 PWM Fundamentals . . . . .                   | 139            |

|                                            |            |
|--------------------------------------------|------------|
| 9.2 P_PWM_TRIANGLE Mode (%01000)           | 140        |
| 9.3 P_PWM_SAWTOOTH Mode (%01001)           | 142        |
| 9.4 P_PWM_SMPS Mode (%01010)               | 144        |
| 9.5 Dynamic Duty Cycle Updates             | 147        |
| 9.6 PWM Resolution and Frequency Tradeoffs | 148        |
| 9.7 Complete Examples                      | 149        |
| 9.8 Quick Reference                        | 152        |
| <b>Chapter 10: DAC Output</b>              | <b>154</b> |
| 10.1 DAC Architecture Overview             | 154        |
| 10.2 Resistor DAC Options                  | 155        |
| 10.3 Direct 8-bit DAC Control              | 157        |
| 10.4 16-bit Dithered DAC Modes             | 158        |
| 10.5 DAC with Other Modes                  | 161        |
| 10.6 ADC Feedback                          | 162        |
| 10.7 Voltage Calculation                   | 162        |
| 10.8 Complete Examples                     | 163        |
| 10.9 Design Considerations                 | 166        |
| 10.10 Quick Reference                      | 167        |
| <b>Chapter 11: Serial Transmission</b>     | <b>169</b> |
| 11.1 Serial Transmission Overview          | 169        |
| 11.2 P_ASYNC_TX Mode (%11110)              | 169        |
| 11.3 P_SYNC_TX Mode (%11100)               | 173        |
| 11.4 Clock Generation                      | 176        |
| 11.5 Complete Examples                     | 177        |
| 11.6 Baud Rate Error Analysis              | 180        |
| 11.7 Quick Reference                       | 181        |
| <b>Part III: Input Modes</b>               | <b>183</b> |
| <b>Chapter 12: Digital Input</b>           | <b>183</b> |
| 12.1 Input Architecture                    | 183        |
| 12.2 Reading Input State                   | 184        |
| 12.3 Input Conditioning Options            | 185        |
| 12.4 Pull-Up and Pull-Down Resistors       | 187        |
| 12.5 Floating Input Behavior               | 188        |
| 12.6 Multi-Pin Input Patterns              | 189        |
| 12.7 Software Debouncing                   | 190        |
| 12.8 Active-Low Signals                    | 191        |
| 12.9 Complete Examples                     | 192        |
| 12.10 Input Timing Analysis                | 194        |
| 12.11 Quick Reference                      | 195        |

|                                                             |            |
|-------------------------------------------------------------|------------|
| <b>Chapter 13: Timing Measurement</b>                       | <b>197</b> |
| 13.1 Timing Measurement Overview . . . . .                  | 197        |
| 13.2 P_STATE_TICKS Mode (%10000) . . . . .                  | 198        |
| 13.3 P_HIGH_TICKS Mode (%10001) . . . . .                   | 200        |
| 13.4 P_EVENTS_TICKS Mode (%10010) . . . . .                 | 202        |
| 13.5 Input Signal Routing . . . . .                         | 205        |
| 13.6 Accuracy Analysis . . . . .                            | 206        |
| 13.7 Complete Examples . . . . .                            | 207        |
| 13.8 Quick Reference . . . . .                              | 210        |
| <b>Chapter 14: Counting Modes</b>                           | <b>212</b> |
| 14.1 Counting Mode Overview . . . . .                       | 212        |
| 14.2 P_QUADRATURE Mode (%01011) . . . . .                   | 213        |
| 14.3 P_REG_UP Mode (%01100) . . . . .                       | 215        |
| 14.4 P_REG_UP_DOWN Mode (%01101) . . . . .                  | 217        |
| 14.5 P_COUNT_RISES Mode (%01110) . . . . .                  | 218        |
| 14.6 P_COUNT_HIGHS Mode (%01111) . . . . .                  | 219        |
| 14.7 Input Signal Routing . . . . .                         | 221        |
| 14.8 Counter Overflow and Range . . . . .                   | 222        |
| 14.9 Mode Selection Guide . . . . .                         | 223        |
| 14.10 Complete Examples . . . . .                           | 223        |
| 14.11 Quick Reference . . . . .                             | 226        |
| <b>Chapter 15: Frequency Measurement</b>                    | <b>228</b> |
| 15.1 Measurement Philosophy . . . . .                       | 228        |
| 15.2 Period-Based Modes (Measure X Periods) . . . . .       | 230        |
| 15.3 Time-Based Modes (Measure in X Clocks) . . . . .       | 231        |
| 15.4 Combined Measurements . . . . .                        | 234        |
| 15.5 PASM2 Implementation . . . . .                         | 236        |
| 15.6 Application Examples . . . . .                         | 237        |
| 15.7 Precision Considerations . . . . .                     | 240        |
| 15.8 Mode Selection Guide . . . . .                         | 240        |
| 15.9 Quick Reference . . . . .                              | 241        |
| <b>Chapter 16: ADC (Analog Input)</b>                       | <b>244</b> |
| 16.1 ADC Architecture . . . . .                             | 244        |
| 16.2 ADC Input Modes . . . . .                              | 245        |
| 16.3 Mode %11000: P_ADC (Internal Clock) . . . . .          | 246        |
| 16.4 Mode %11001: P_ADC_EXT (External Clock) . . . . .      | 251        |
| 16.5 Mode %11010: P_ADC_SCOPE (Triggered Capture) . . . . . | 252        |
| 16.6 Multi-Channel ADC . . . . .                            | 253        |
| 16.7 Practical Examples . . . . .                           | 254        |
| 16.8 Accuracy Considerations . . . . .                      | 257        |

|                                                               |                |
|---------------------------------------------------------------|----------------|
| 16.9 Quick Reference . . . . .                                | 259            |
| <b>Chapter 17: Serial Receive</b>                             | <b>262</b>     |
| 17.1 Serial Receive Modes Overview . . . . .                  | 262            |
| 17.2 Mode %11111: P_ASYNC_RX (Asynchronous Receive) . . . . . | 263            |
| 17.3 Mode %11101: P_SYNC_RX (Synchronous Receive) . . . . .   | 266            |
| 17.4 Full-Duplex UART . . . . .                               | 268            |
| 17.5 Buffered Reception . . . . .                             | 270            |
| 17.6 Error Detection . . . . .                                | 271            |
| 17.7 RS-232 Signal Inversion . . . . .                        | 272            |
| 17.8 Multi-Drop Networks . . . . .                            | 272            |
| 17.9 Application Examples . . . . .                           | 273            |
| 17.10 Quick Reference . . . . .                               | 275            |
| <br><b>Part IV: Special Modes</b>                             | <br><b>278</b> |
| <b>Chapter 18: Repository</b>                                 | <b>278</b>     |
| 18.1 Repository Concept . . . . .                             | 278            |
| 18.2 Long Repository (Non-DAC Mode) . . . . .                 | 279            |
| 18.3 Mode %00001: DAC Noise . . . . .                         | 281            |
| 18.4 Mode %00010: DAC PRNG Dither . . . . .                   | 282            |
| 18.5 Mode %00011: DAC PWM Dither . . . . .                    | 284            |
| 18.6 Comparison with Other Inter-Cog Mechanisms . . . . .     | 286            |
| 18.7 Application Examples . . . . .                           | 287            |
| 18.8 Quick Reference . . . . .                                | 290            |
| <br><b>Chapter 19: USB Host/Device</b>                        | <br><b>292</b> |
| 19.1 USB Overview . . . . .                                   | 292            |
| 19.2 Pin Configuration . . . . .                              | 293            |
| 19.3 USB Signaling . . . . .                                  | 294            |
| 19.4 Register Usage . . . . .                                 | 294            |
| 19.5 Host vs Device Mode . . . . .                            | 297            |
| 19.6 Using USB Libraries . . . . .                            | 298            |
| 19.7 Basic USB Configuration Example . . . . .                | 299            |
| 19.8 Hardware Considerations . . . . .                        | 300            |
| 19.9 Limitations . . . . .                                    | 300            |
| 19.10 Quick Reference . . . . .                               | 301            |
| <br><b>Part V: Appendices</b>                                 | <br><b>303</b> |
| <b>Appendix A: Intent Index</b>                               | <b>303</b>     |
| Generate Signals . . . . .                                    | 303            |
| Measure Signals . . . . .                                     | 304            |



|                                                   |            |
|---------------------------------------------------|------------|
| Control Outputs . . . . .                         | 304        |
| Read Inputs . . . . .                             | 305        |
| Communicate . . . . .                             | 305        |
| Coordinate and Synchronize . . . . .              | 306        |
| Quick Mode Lookup . . . . .                       | 306        |
| <b>Appendix B: P_ Constants Quick Reference</b>   | <b>308</b> |
| How to Use P_ Constants . . . . .                 | 308        |
| Smart Pin Modes (pick one) . . . . .              | 308        |
| A Input Selection (pick one) . . . . .            | 310        |
| B Input Selection (pick one) . . . . .            | 310        |
| A/B Input Logic (pick one) . . . . .              | 311        |
| Input Conditioning Modes (pick one) . . . . .     | 311        |
| ADC Input Modes (pick one) . . . . .              | 312        |
| DAC Output Modes (pick one) . . . . .             | 312        |
| Sync/Async I/O (pick one) . . . . .               | 312        |
| IN/OUT Polarity (pick one each) . . . . .         | 313        |
| Drive Strength - High (pick one) . . . . .        | 313        |
| Drive Strength - Low (pick one) . . . . .         | 314        |
| TT Bits / DIR-OUT Control (pick one) . . . . .    | 314        |
| Common Combinations . . . . .                     | 314        |
| <b>Appendix C: Formulas Reference</b>             | <b>317</b> |
| NCO Frequency Generation . . . . .                | 317        |
| PWM Output . . . . .                              | 318        |
| Serial Communication (UART) . . . . .             | 319        |
| ADC (Analog Input) . . . . .                      | 320        |
| DAC (Analog Output) . . . . .                     | 321        |
| Timing Measurement . . . . .                      | 322        |
| Period/Frequency Measurement . . . . .            | 323        |
| Quadrature Encoder . . . . .                      | 324        |
| Common sysclk Values . . . . .                    | 325        |
| Accuracy Notes . . . . .                          | 328        |
| <b>Appendix D: Mode Comparison Charts</b>         | <b>330</b> |
| Output Mode Comparison . . . . .                  | 330        |
| Input Mode Comparison . . . . .                   | 331        |
| Frequency Generation Comparison . . . . .         | 332        |
| Counting Mode Comparison . . . . .                | 333        |
| Period/Frequency Measurement Comparison . . . . . | 334        |
| Serial Mode Comparison . . . . .                  | 335        |
| ADC Mode Comparison . . . . .                     | 336        |
| DAC Mode Comparison . . . . .                     | 337        |

|                                                   |            |
|---------------------------------------------------|------------|
| Quick Selection Trees . . . . .                   | 338        |
| <b>Appendix E: Troubleshooting</b>                | <b>340</b> |
| Pin Not Responding . . . . .                      | 340        |
| No Output Visible . . . . .                       | 341        |
| Wrong Frequency or Timing . . . . .               | 342        |
| Noisy or Unstable Signal . . . . .                | 344        |
| Serial Not Working . . . . .                      | 345        |
| ADC Readings Wrong . . . . .                      | 346        |
| Encoder Counts Incorrect . . . . .                | 348        |
| Mode Stops Working . . . . .                      | 349        |
| Interference Between Pins . . . . .               | 350        |
| Debugging Techniques . . . . .                    | 351        |
| Quick Diagnostic Checklist . . . . .              | 353        |
| <b>Appendix F: Complete Mode Reference</b>        | <b>355</b> |
| Mode Number Cross-Reference . . . . .             | 355        |
| Mode %00000: P_NORMAL . . . . .                   | 356        |
| Mode %00001: P_REPOSITORY / P_DAC_NOISE . . . . . | 357        |
| Mode %00010: P_DAC_DITHER_RND . . . . .           | 358        |
| Mode %00011: P_DAC_DITHER_PWM . . . . .           | 359        |
| Mode %00100: P_PULSE . . . . .                    | 360        |
| Mode %00101: P_TRANSITION . . . . .               | 361        |
| Mode %00110: P_NCO_FREQ . . . . .                 | 362        |
| Mode %00111: P_NCO_DUTY . . . . .                 | 363        |
| Mode %01000: P_PWM_TRIANGLE . . . . .             | 364        |
| Mode %01001: P_PWM_SAWTOOTH . . . . .             | 365        |
| Mode %01010: P_PWM_SMPS . . . . .                 | 366        |
| Mode %01011: P_QUADRATURE . . . . .               | 367        |
| Mode %01100: P_REG_UP . . . . .                   | 368        |
| Mode %01101: P_REG_UP_DOWN . . . . .              | 369        |
| Mode %01110: P_COUNT_RISES . . . . .              | 370        |
| Mode %01111: P_COUNT_HIGHS . . . . .              | 371        |
| Mode %10000: P_STATE_TICKS . . . . .              | 372        |
| Mode %10001: P_HIGH_TICKS . . . . .               | 373        |
| Mode %10010: P_EVENTS_TICKS . . . . .             | 374        |
| Mode %10011: P_PERIODS_TICKS . . . . .            | 375        |
| Mode %10100: P_PERIODS_HIGHS . . . . .            | 376        |
| Mode %10101: P_COUNTER_TICKS . . . . .            | 377        |
| Mode %10110: P_COUNTER_HIGHS . . . . .            | 378        |
| Mode %10111: P_COUNTER_PERIODS . . . . .          | 379        |
| Mode %11000: P_ADC . . . . .                      | 380        |
| Mode %11001: P_ADC_EXT . . . . .                  | 381        |

|                                           |            |
|-------------------------------------------|------------|
| Mode %11010: P_ADC_SCOPE . . . . .        | 382        |
| Mode %11011: P_USB_PAIR . . . . .         | 383        |
| Mode %11100: P_SYNC_TX . . . . .          | 384        |
| Mode %11101: P_SYNC_RX . . . . .          | 385        |
| Mode %11110: P_ASYNC_TX . . . . .         | 386        |
| Mode %11111: P_ASYNC_RX . . . . .         | 387        |
| <b>Appendix G: FPGA Board Differences</b> | <b>388</b> |
| USB — No Built-In Resistors . . . . .     | 388        |
| Clock — Fixed 20 MHz or 80 MHz . . . . .  | 388        |
| Other Board-Level Differences . . . . .   | 388        |
| <b>Index</b>                              | <b>389</b> |

# List of Figures

|      |                                                                                   |     |
|------|-----------------------------------------------------------------------------------|-----|
| 1.1  | Output timing: 3-clock pipeline delay . . . . .                                   | 25  |
| 1.2  | INA/INB read latency (3 clocks) . . . . .                                         | 25  |
| 1.3  | TESTP/TESTPN read latency (2 clocks) . . . . .                                    | 26  |
| 2.1  | Enhanced Direct I/O — the active WRPIN fields (full 32-bit map in §4.2) . . . . . | 54  |
| 3.1  | Smart pin lifecycle . . . . .                                                     | 72  |
| 4.1  | WRPIN configuration value — 32-bit field map . . . . .                            | 80  |
| 7.1  | Pulse output (counted cycles) . . . . .                                           | 118 |
| 7.2  | Transition output (counted edges) . . . . .                                       | 121 |
| 8.1  | NCO architecture . . . . .                                                        | 128 |
| 9.1  | PWM architecture . . . . .                                                        | 139 |
| 9.2  | PWM triangle duty . . . . .                                                       | 142 |
| 9.3  | PWM sawtooth duty . . . . .                                                       | 144 |
| 9.4  | SMPS mode block diagram . . . . .                                                 | 145 |
| 9.5  | Typical SMPS (buck) circuit . . . . .                                             | 146 |
| 10.1 | DAC structure . . . . .                                                           | 154 |
| 10.2 | Op-amp output buffer (unity-gain follower) . . . . .                              | 166 |
| 10.3 | RC dither filter . . . . .                                                        | 167 |
| 11.1 | UART transmit frame . . . . .                                                     | 170 |
| 12.1 | Input signal path . . . . .                                                       | 183 |
| 12.2 | TESTP vs INA read latency . . . . .                                               | 185 |
| 13.1 | Pulse-width / state-time measurement . . . . .                                    | 198 |
| 13.2 | P_HIGH_TICKS high-time measurement . . . . .                                      | 200 |
| 14.1 | Quadrature A/B signals . . . . .                                                  | 213 |
| 14.2 | Gated edge counter . . . . .                                                      | 216 |
| 14.3 | Up/down counter . . . . .                                                         | 217 |
| 16.1 | Sigma-delta ADC chain . . . . .                                                   | 244 |
| 16.2 | ADC pin-configuration fields . . . . .                                            | 244 |

---

|                                                |     |
|------------------------------------------------|-----|
| 17.1 UART receive frame . . . . .              | 263 |
| 18.1 Pin repository handoff . . . . .          | 279 |
| B.1 Structure of a P_ constant value . . . . . | 308 |
| D.1 Output mode selection . . . . .            | 338 |
| D.2 Input mode selection . . . . .             | 339 |

# Copyright and License

Copyright © 2026 Iron Sheep Productions, LLC and Parallax Inc.

This work is licensed under the Creative Commons Attribution–NonCommercial–NoDerivatives 4.0 International License (CC BY-NC-ND 4.0).

You are free to:

- **Share** — copy and redistribute the material in any medium or format

Under the following terms:

- **Attribution** — You must give appropriate credit, provide a link to the license, and indicate if changes were made (for example, formatting or excerpting).
- **NonCommercial** — You may not use the material for commercial purposes.
- **NoDerivatives** — If you remix, transform, translate, or build upon the material, you may not distribute the modified material.

**Commercial use:** For uses that may be commercial (including paid courses, kits, or redistribution with products), please contact Iron Sheep Productions, LLC and Parallax Inc. (info@ironsheep.biz) for separate permission.

To view the full license, visit: <https://creativecommons.org/licenses/by-nc-nd/4.0/>

## Trademarks

Parallax, Propeller, Spin, and the Parallax logo are trademarks of Parallax Inc.

# Acknowledgments

This guide would not exist without the contributions of many individuals and organizations:

**Parallax Inc.** for creating the Propeller 2 microcontroller and providing comprehensive reference documentation that forms the foundation of this work.

**Chip Gracey** for the brilliant design of the P2 smart pin system and for maintaining detailed technical specifications.

**The P2 Community** for extensive testing, feedback, and real-world usage that has refined our understanding of the smart pin modes and identified critical details worth documenting.

**Jon Titus** for the *Propeller 2 Smart Pin Supplementary Documentation* — a commenting-enabled Google Doc that supplements the *Parallax Propeller 2 Documentation v35 - Rev B/C* with examples and further explanation — whose detailed smart pin mode descriptions informed and enriched much of this guide. Titus is also the historical designer of the 1974 Mark-8, one of the world's earliest personal hobbyist microcomputers.

This guide is a community-developed resource, created to make the P2's smart pin system more accessible to developers at all skill levels.

# How to Use This Guide

The P2 I/O & Smart Pins User Guide supports three distinct reading paths, each designed for different needs:

## Path 1: Learning Path (New to P2 I/O)

Readers unfamiliar with the P2 pin system should progress through Part I sequentially:

1. **Chapter 1: Direct I/O** - Fundamental pin control via DIR, OUT, and IN registers
2. **Chapter 2: Enhanced Direct I/O** - P\_ constants for drive strength, input conditioning, and basic analog
3. **Chapter 3: Smart Pin Architecture** - The autonomous I/O concept and state machine
4. **Chapter 4: Smart Pin Configuration** - Configuration instructions and patterns
5. **Chapter 5: Working with Smart Pins** - Common patterns and debugging

After completing Part I, proceed to specific mode chapters in Parts II-IV as needed, using the appendices for reference.

## Path 2: Task-Oriented Path (Know What to Accomplish)

Readers who know what they want to accomplish but not which mode to use should start with **Appendix A: Intent Index**. The Intent Index provides entries in the form:

**I want to... [task]**

- Chapter N: [chapter name]
- Specifically: [mode or technique]
- Also consider: [alternatives]

The Intent Index covers common tasks including:

- Generating signals (clocks, PWM, analog, serial)
- Measuring signals (timing, counting, analog, serial)
- Controlling outputs (digital, DAC)
- Reading inputs (digital, ADC)
- Communication protocols (SPI, I<sup>2</sup>C, UART, USB)



## Path 3: Reference Path (Know the Mode)

Readers who know which mode or feature they need can navigate directly:

- **Quick Mode Selection Matrix** (below) - Visual overview of all 32 smart pin modes
- **Appendix F: Complete Mode Reference** - Condensed reference for all modes
- **Chapter index** - Direct chapter access by topic

Each mode chapter stands alone with complete configuration details, all applicable P\_ constants, working examples in both Spin2 and PASM2, and decision guidance.

# Document Conventions

## Typography

| Element             | Convention                  | Example                        |
|---------------------|-----------------------------|--------------------------------|
| PASM2 instructions  | Bold uppercase              | <b>DRVH, WRPIN, RDPIN</b>      |
| Spin2 methods       | Bold mixed case             | <b>PINHIGH, PINREAD, WRPIN</b> |
| P_ constants        | Monospace                   | P_NCO_FREQ, P_HIGH_15K, P_OE   |
| Register references | Name with bit range         | X[15:0], Z[31]                 |
| Mode values         | Binary with percent prefix  | %00110, %11110                 |
| Numeric values      | Underscores for readability | 200_000_000, 4_294_967_296     |

## Register Notation

The P2 smart pin system uses three internal registers:

| Register   | Notation            | Description                           |
|------------|---------------------|---------------------------------------|
| X register | X[31:0] or X[range] | Configuration and parameters          |
| Y register | Y[31:0] or Y[range] | Input data or secondary configuration |
| Z register | Z[31:0] or Z[range] | Accumulator / working register        |

Bit ranges use the notation X[high:low], where X[31:0] indicates all 32 bits and X[15:0] indicates the lower 16 bits.

## Code Examples

All code examples appear in both Spin2 and PASM2:

### Spin2 Example:

```
' Example description
WRPIN(PIN, mode_value)    ' Configuration step
WXPIN(PIN, x_value)       ' Parameter setting
PINL(PIN)                 ' Enable Smart Pin
```

**PASM2 Example:**

```
' Example description
    WRPIN    ##mode_value, pin    ' Configuration step
    WXPIN    ##x_value, pin      ' Parameter setting
    DRVL     pin                  ' Enable Smart Pin
```

## Terminology

| Term       | Definition                                                 |
|------------|------------------------------------------------------------|
| Direct I/O | Fundamental pin control via DIR, OUT, and IN registers     |
| Smart Pin  | Autonomous pin mode providing hardware-based I/O functions |
| DIR bit    | Direction control (0 = input/disabled, 1 = output/enabled) |
| OUT bit    | Output state when DIR = 1                                  |
| IN bit     | Input state or Smart Pin status flag                       |
| sysclk     | System clock frequency (typically 200 MHz)                 |
| mode bits  | Bits [4:0] in WRPIN value selecting Smart Pin mode         |

## Cross-References

Cross-references use the format:

- “See Chapter N: Title” for chapter references
- “See Appendix X” for appendix references
- “See MODE\_NAME (%XXXXX)” for mode references

# Quick Mode Selection Matrix

The following matrix provides a one-page overview of all 32 smart pin modes organized by function. Use this for quick navigation to the appropriate chapter.

## Output Modes

| Mode                 | P_Constant                    | Mode Bits | Chapter      | Description                                    |
|----------------------|-------------------------------|-----------|--------------|------------------------------------------------|
| Normal               | P_NORMAL                      | %00000    | Ch 2         | Direct I/O, no Smart Pin (Enhanced Direct I/O) |
| Repository/DAC Noise | P_REPOSITORY /<br>P_DAC_NOISE | %00001    | Ch 18, Ch 10 | Long repository or DAC noise output            |
| DAC Dither RND       | P_DAC_DITHER_RND              | %00010    | Ch 10        | DAC 16-bit random dither                       |
| DAC Dither PWM       | P_DAC_DITHER_PWM              | %00011    | Ch 10        | DAC 16-bit PWM dither                          |
| Pulse/Cycle          | P_PULSE                       | %00100    | Ch 7         | Pulse or cycle output                          |
| Transition           | P_TRANSITION                  | %00101    | Ch 7         | Timed transition output                        |
| NCO Frequency        | P_NCO_FREQ                    | %00110    | Ch 8         | NCO frequency output (square wave)             |
| NCO Duty             | P_NCO_DUTY                    | %00111    | Ch 8         | NCO duty cycle output                          |
| PWM Triangle         | P_PWM_TRIANGLE                | %01000    | Ch 9         | PWM triangle wave output                       |
| PWM Sawtooth         | P_PWM_SAWTOOTH                | %01001    | Ch 9         | PWM sawtooth wave output                       |
| PWM SMPS             | P_PWM_SMPS                    | %01010    | Ch 9         | Switch-mode power supply PWM                   |
| Sync Serial TX       | P_SYNC_TX                     | %11100    | Ch 11        | Synchronous serial transmit                    |
| Async Serial TX      | P_ASYNC_TX                    | %11110    | Ch 11        | Asynchronous serial transmit                   |

## Input Modes

| Mode            | P_Constant        | Mode Bits | Chapter | Description                                            |
|-----------------|-------------------|-----------|---------|--------------------------------------------------------|
| Quadrature      | P_QUADRATURE      | %01011    | Ch 14   | A-B quadrature encoder input (within Counting chapter) |
| Reg Up          | P_REG_UP          | %01100    | Ch 14   | Increment on A-rise when B-high                        |
| Reg Up/Down     | P_REG_UP_DOWN     | %01101    | Ch 14   | Increment/decrement accumulator                        |
| Count Rises     | P_COUNT_RISES     | %01110    | Ch 14   | Count A-rises, optionally subtract B-rises             |
| Count Highs     | P_COUNT_HIGHS     | %01111    | Ch 14   | Count A-high ticks, optionally subtract B-high         |
| State Ticks     | P_STATE_TICKS     | %10000    | Ch 13   | Measure A-low and A-high durations                     |
| High Ticks      | P_HIGH_TICKS      | %10001    | Ch 13   | Measure A-high duration                                |
| Events/Timeout  | P_EVENTS_TICKS    | %10010    | Ch 13   | Count events or timeout detection                      |
| Periods Ticks   | P_PERIODS_TICKS   | %10011    | Ch 15   | For X periods, count ticks                             |
| Periods Highs   | P_PERIODS_HIGHS   | %10100    | Ch 15   | For X periods, count highs                             |
| Counter Ticks   | P_COUNTER_TICKS   | %10101    | Ch 15   | For periods in X+ ticks, count ticks                   |
| Counter Highs   | P_COUNTER_HIGHS   | %10110    | Ch 15   | For periods in X+ ticks, count highs                   |
| Counter Periods | P_COUNTER_PERIODS | %10111    | Ch 15   | For periods in X+ ticks, count periods                 |
| ADC Internal    | P_ADC             | %11000    | Ch 16   | ADC sample/filter, internal clock                      |
| ADC External    | P_ADC_EXT         | %11001    | Ch 16   | ADC sample/filter, external clock                      |
| ADC Scope       | P_ADC_SCOPE       | %11010    | Ch 16   | ADC oscilloscope with trigger                          |
| Sync Serial RX  | P_SYNC_RX         | %11101    | Ch 17   | Synchronous serial receive                             |
| Async Serial RX | P_ASYNC_RX        | %11111    | Ch 17   | Asynchronous serial receive                            |

## Special Modes

| Mode     | P_Constant | Mode Bits | Chapter | Description              |
|----------|------------|-----------|---------|--------------------------|
| USB Pair | P_USB_PAIR | %11011    | Ch 19   | USB host/device pin pair |

## Mode Categories Quick Reference

| Category                            | Modes                                            | Chapters |
|-------------------------------------|--------------------------------------------------|----------|
| <b>Digital Output</b>               | Pulse, Transition                                | Ch 7     |
| <b>Frequency Generation</b>         | NCO Freq, NCO Duty                               | Ch 8     |
| <b>PWM Output</b>                   | Triangle, Sawtooth, SMPS                         | Ch 9     |
| <b>DAC Output</b>                   | Repository/Noise, Dither RND, Dither PWM         | Ch 10    |
| <b>Serial Transmit</b>              | Sync TX, Async TX                                | Ch 11    |
| <b>Timing Measurement</b>           | State Ticks, High Ticks, Events/Timeout          | Ch 13    |
| <b>Counting</b>                     | Reg Up, Reg Up/Down, Count Rises, Count Highs    | Ch 14    |
| <b>Quadrature Encoder</b>           | Quadrature                                       | Ch 14    |
| <b>Period/Frequency Measurement</b> | Periods Ticks/Highs, Counter Ticks/Highs/Periods | Ch 15    |
| <b>ADC Input</b>                    | ADC, ADC Ext, ADC Scope                          | Ch 16    |
| <b>Serial Receive</b>               | Sync RX, Async RX                                | Ch 17    |
| <b>Inter-Cog Sharing</b>            | Repository                                       | Ch 18    |
| <b>USB</b>                          | USB Pair                                         | Ch 19    |

*This front matter provides navigation tools for all readers. Proceed to Part I for foundational knowledge, or use the Intent Index (Appendix A) for task-oriented guidance.*

# Part I: P2 Pin System Fundamentals

## Chapter 1: Direct I/O

### The Foundation

Direct I/O is the fundamental layer of P2 pin control. Every pin operation—from simple LED blinking to complex smart pin configurations—ultimately depends on three core concepts: **direction**, **output state**, and **input sensing**. This chapter documents the hardware model and all Direct I/O instructions.

### 1.1 The Hardware Model

#### Pin Control Registers

Each cog maintains its own set of pin control registers:

| Register | Cog Address | Function                                      |
|----------|-------------|-----------------------------------------------|
| DIRA     | \$1FA       | Output enable bits for P0..P31 (active high)  |
| DIRB     | \$1FB       | Output enable bits for P32..P63 (active high) |
| OUTA     | \$1FC       | Output state bits for P0..P31                 |
| OUTB     | \$1FD       | Output state bits for P32..P63                |
| INA      | \$1FE       | Input state bits for P0..P31                  |
| INB      | \$1FF       | Input state bits for P32..P63                 |

## The Three-State Model

Every pin operates according to three independent states:

1. **Direction (DIR)**: Controls whether the pin is an output (DIR=1) or input/floating (DIR=0)
2. **Output State (OUT)**: The logic level driven when the pin is an output
3. **Input State (IN)**: The current logic level present on the pin

**Critical relationship:** The OUT register value only affects the physical pin when DIR=1. When DIR=0, the pin floats (high impedance) and the OUT register has no effect on the pin, though the OUT value is preserved for when the pin later becomes an output.

## Multiple Cog Arbitration

Multiple cogs can control the same pin. The P2 uses OR logic to combine control signals:

- **DIR**: If any cog sets DIR=1 for a pin, the pin becomes an output
- **OUT**: The output state is the OR of all cogs' OUT bits

This means:

- Any cog can “claim” a pin by setting its DIR bit
- When multiple cogs drive the same pin, the output is high if any cog drives high

## Pin Output Driver

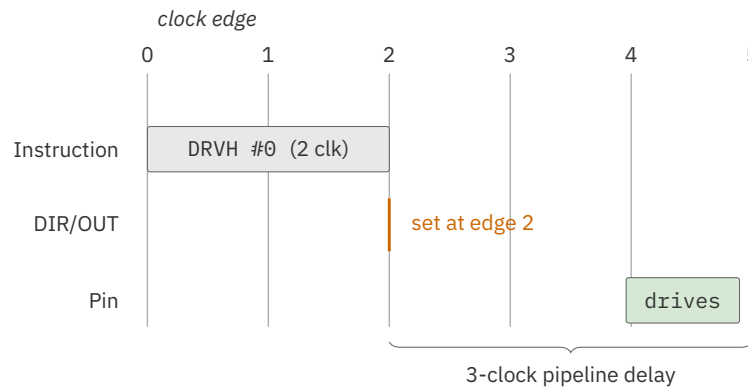
When DIR=1, the pin's output driver connects to the pad. The driver strength is configurable via WRPIN (see Chapter 2). The default is “fast” drive providing approximately 30mA source/sink capability.



## 1.2 Timing

### Output Timing: 3-Clock Delay

When a DIR or OUT bit is changed by any instruction, **three additional clock cycles pass** after the instruction completes before the pin begins transitioning to the new state.

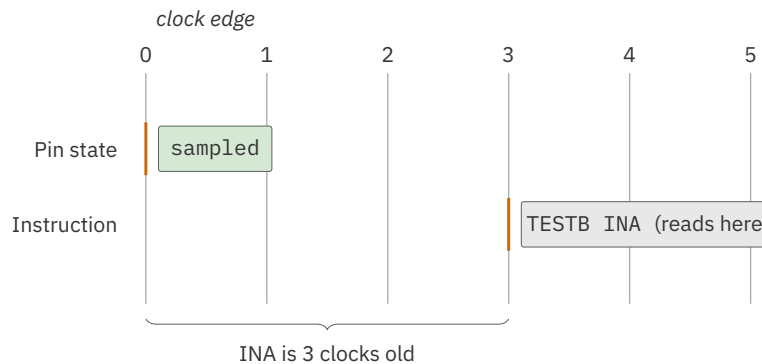


**Figure 1.1.** Output timing: 3-clock pipeline delay

**Total latency from instruction start to pin transition:** 5 clock cycles (2 for instruction execution + 3 pipeline delay).

### Input Timing via INx Registers: 3 Clocks Old

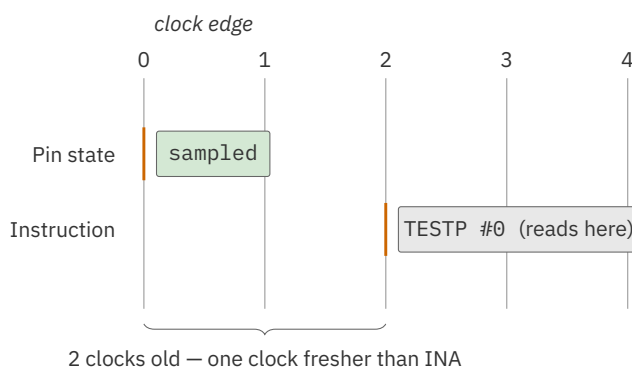
When an INx register is read by an instruction, it reflects the state of the pins registered **three clocks before** the start of the instruction.



**Figure 1.2.** INA/INB read latency (3 clocks)

## Input Timing via TESTP/TESTPN: 2 Clocks Old

The TESTP and TESTPN instructions provide “fresher” input data—the value read reflects the state of the pin registered **two clocks before** the start of the instruction.



**Figure 1.3.** TESTP/TESTPN read latency (2 clocks)

**Recommendation:** Use TESTP/TESTPN for time-critical input sensing. The one-clock fresher data can matter in tight timing loops.

## Timing Summary

| Operation                                   | Latency                     | Notes                  |
|---------------------------------------------|-----------------------------|------------------------|
| Output change (any DRV/OUT/DIR instruction) | 3 clocks after instruction  | Before pin transitions |
| Input via INx register (MOV, TESTB, etc.)   | 3 clocks before instruction | Older data             |
| Input via TESTP/TESTPN                      | 2 clocks before instruction | Fresher data           |

## 1.3 Drive Instructions (DRVx)

Drive instructions set both the DIR bit (set to 1) and the OUT bit in a single atomic operation. These are the most common pin control instructions.

### Common Properties

- **Execution time:** 2 clock cycles
- **Output latency:** 3 additional clock cycles after instruction
- **Flags:** With the optional WCZ effect, C and Z are **both** set to the pin’s prior OUT-bit state (its output level before the instruction executes); without WCZ, neither flag changes
- **Pin range:** D[5:0] specifies base pin (0-63); D[10:6] specifies span (0-31 additional pins)

## DRVH - Drive High

Drives pin high by setting DIR=1 and OUT=1.

### PASM2 Syntax

```
DRVH    {#}D    {WCZ}
```

### Operation:

1. Set DIR bit for pin D to 1 (output mode)
2. Set OUT bit for pin D to 1 (high state)
3. If WC/WZ is specified, set C and Z to the pin's prior OUT bit (otherwise leave flags unchanged)

**Timing:** 2 clock cycles; pin begins driving 3 clocks after instruction completes

**Spin2 Equivalent:** PINHIGH(pin)

### Example - Spin2:

```
CON
  LED_PIN = 56                ' Onboard LED on P2 Eval board

PUB main()
  PINHIGH(LED_PIN)           ' Drive LED pin high (LED on)
```

### Example - PASM2:

```
DRVH    #56    ' Drive pin 56 high
```

**Related:** DRVL, DRVNOT, OUTH, DIRH

## DRVL - Drive Low

Drives pin low by setting DIR=1 and OUT=0.

### PASM2 Syntax

```
DRVL    {#}D    {WCZ}
```

### Operation:

1. Set DIR bit for pin D to 1 (output mode)
2. Set OUT bit for pin D to 0 (low state)
3. If WC/WZ is specified, set C and Z to the pin's prior OUT bit (otherwise leave flags unchanged)

**Timing:** 2 clock cycles; pin begins driving 3 clocks after instruction completes

**Spin2 Equivalent:** PINLOW(pin)

**Example - Spin2:**

```
CON
    LED_PIN = 56

PUB main()
    PINLOW(LED_PIN)           ' Drive LED pin low (LED off)
```

**Example - PASM2:**

```
        DRVL        #56        ' Drive pin 56 low
```

**Related:** DRVH, DRVNOT, OUTL, DIRL

## DRVNOT - Drive Toggle

Toggles the output state while keeping the pin as an output.

### PASM2 Syntax

```
        DRVNOT    {#}Dest    {WCZ}
```

**Operation:**

1. Set DIR bit for pin D to 1 (output mode)
2. Toggle OUT bit for pin D (0→1 or 1→0)
3. If WC/WZ is specified, set C and Z to the pin's prior OUT bit (otherwise leave flags unchanged)

**Timing:** 2 clock cycles

**Spin2 Equivalent:** PINTOGGLE(pin)

**Example - Spin2:**

```

CON
  LED_PIN = 56

PUB main()
  PINHIGH(LED_PIN)      ' Start with LED on
  repeat
    WAITMS(500)          ' Wait 500ms
    PINTOGGLE(LED_PIN)    ' Toggle LED state

```

**Example - PASM2:**

```

      DRVH      #56      ' Start high
.loop
      WAITX     delay    ' Wait
      DRVNOT    #56      ' Toggle pin 56
      JMP       #.loop
delay  LONG     100_000_000 ' 0.5 sec at 200 MHz

```

**Related:** DRVH, DRVL, OUTNOT**DRVC - Drive to C**

Drives pin to the current state of the C flag.

**PASM2 Syntax**

```
DRVC    {#}D      {WCZ}
```

**Operation:**

1. Set DIR bit for pin D to 1 (output mode)
2. Set OUT bit for pin D to C flag value
3. With WCZ, set C and Z flags to the prior OUT bit state

**Timing:** 2 clock cycles**Spin2 Equivalent:** None (use conditional logic with PINHIGH/PINLOW)**Example - PASM2:**

```

      TESTP     #10 wc    ' Read pin 10 into C
      DRVC      #11      ' Drive pin 11 to same state as pin 10

```

**Related:** DRVNC, OUTC

## DRVNC - Drive to Not C

Drives pin to the inverse of the C flag.

### PASM2 Syntax

```
DRVNC    {#}D    {WCZ}
```

### Operation:

1. Set DIR bit for pin D to 1 (output mode)
2. Set OUT bit for pin D to !C (inverted C flag)
3. With WCZ, set C and Z flags to the prior OUT bit state

**Timing:** 2 clock cycles

**Spin2 Equivalent:** None (use conditional logic)

### Example - PASM2:

```
TESTP    #10 wc    ' Read pin 10 into C
DRVNC    #11        ' Drive pin 11 to opposite state
```

**Related:** DRVNC, OUTNC

## DRVZ - Drive to Z

Drives pin to the current state of the Z flag.

### PASM2 Syntax

```
DRVZ    {#}D    {WCZ}
```

### Operation:

1. Set DIR bit for pin D to 1 (output mode)
2. Set OUT bit for pin D to Z flag value
3. With WCZ, set C and Z flags to the prior OUT bit state

**Timing:** 2 clock cycles

**Spin2 Equivalent:** None (use conditional logic)

**Example - PASM2:**

```

CMP      value, #0 wz      ' Z=1 if value is zero
DRVZ     #1led             ' Drive LED based on Z

```

**Related:** DRVNZ, OUTZ**DRVNZ - Drive to Not Z**

Drives pin to the inverse of the Z flag.

**PASM2 Syntax**

```
DRVNZ    {#}D              {WCZ}
```

**Operation:**

1. Set DIR bit for pin D to 1 (output mode)
2. Set OUT bit for pin D to !Z (inverted Z flag)
3. With WCZ, set C and Z flags to the prior OUT bit state

**Timing:** 2 clock cycles**Spin2 Equivalent:** None (use conditional logic)**Example - PASM2:**

```

CMP      value, #0 wz      ' Z=1 if value is zero
DRVNZ    #1led             ' Drive high if value != 0

```

**Related:** DRVZ, OUTNZ**DRVRND - Drive Random**

Drives pin to a random state.

**PASM2 Syntax**

```
DRVRND   {#}Dest          {WCZ}
```

**Operation:**

1. Set DIR bit for pin D to 1 (output mode)
2. Set OUT bit for pin D to a random value (0 or 1)
3. If WC/WZ is specified, set C and Z to the pin's prior OUT bit (otherwise leave flags unchanged)

**Timing:** 2 clock cycles

**Spin2 Equivalent:** None (use GETRND() with conditional logic)

**Example - PASM2:**

```
DRVNRND    #1ed    ' Drive LED to random state
```

**Related:** OUTRND, DIRRND

## 1.4 Output Instructions (OUTx)

Output instructions modify only the output state register bit. The direction register is unchanged. The output state only affects the physical pin when DIR=1.

### Common Properties

- **Execution time:** 2 clock cycles
- **Flags:** With the optional WCZ effect, C and Z are **both** set to the pin's prior OUT-bit state (its output level before the instruction executes); without WCZ, neither flag changes
- **Note:** If DIR=0, the instruction changes the OUT register but has no immediate effect on the pin

### OUTH - Output High

Sets the output state to high without changing direction.

#### PASM2 Syntax

```
OUTH    {#}D    {WCZ}
```

#### Operation:

1. Set OUT bit for pin D to 1
2. If WC/WZ is specified, set C and Z to the pin's prior OUT bit (otherwise leave flags unchanged)
3. DIR is unchanged

**Timing:** 2 clock cycles

**Spin2 Equivalent:** None (PINHIGH also sets direction; for OUT-only, use register access)



**Example - PASM2:**

```

DIRH      #1ed      ' Make pin output (once)
' ...later...
OUTH      #1ed      ' Set high without touching DIR
OUTL      #1ed      ' Set low without touching DIR

```

**Related:** OUTL, OUTNOT, DRVH

**OUTL - Output Low**

Sets the output state to low without changing direction.

**PASM2 Syntax**

```
OUTL      {#}D      {WCZ}
```

**Operation:**

1. Set OUT bit for pin D to 0
2. If WC/WZ is specified, set C and Z to the pin's prior OUT bit (otherwise leave flags unchanged)
3. DIR is unchanged

**Timing:** 2 clock cycles

**Spin2 Equivalent:** None (PINLOW also sets direction)

**Example - PASM2:**

```
OUTL      #1ed      ' Set output register low
```

**Related:** OUTH, OUTNOT, DRVL

**OUTNOT - Output Toggle**

Toggles the output state without changing direction.

**PASM2 Syntax**

```
OUTNOT    {#}Dest    {WCZ}
```

**Operation:**

1. Toggle OUT bit for pin D

2. If WC/WZ is specified, set C and Z to the pin's prior OUT bit (otherwise leave flags unchanged)
3. DIR is unchanged

**Timing:** 2 clock cycles

**Spin2 Equivalent:** None (PINTOGGLE also sets direction)

**Example - PASM2:**

```
OUTNOT    #1ed    ' Toggle output state only
```

**Related:** OUTH, OUTL, DRVNOT

## OUTC - Output to C

Sets output state to the C flag value without changing direction.

### PASM2 Syntax

```
OUTC    {#}D    {WCZ}
```

**Operation:**

1. Set OUT bit for pin D to C flag value
2. With WCZ, set C and Z flags to the prior OUT bit state
3. DIR is unchanged

**Timing:** 2 clock cycles

**Related:** OUTNC, DRVC

## OUTNC - Output to Not C

Sets output state to the inverse of C flag without changing direction.

### PASM2 Syntax

```
OUTNC    {#}D    {WCZ}
```

**Operation:**

1. Set OUT bit for pin D to !C
2. With WCZ, set C and Z flags to the prior OUT bit state
3. DIR is unchanged

**Timing:** 2 clock cycles

**Related:** OUTC, DRVNC

## OUTZ - Output to Z

Sets output state to the Z flag value without changing direction.

### PASM2 Syntax

```
OUTZ    {#}D    {WCZ}
```

### Operation:

1. Set OUT bit for pin D to Z flag value
2. With WCZ, set C and Z flags to the prior OUT bit state
3. DIR is unchanged

**Timing:** 2 clock cycles

**Related:** OUTNZ, DRVZ

## OUTNZ - Output to Not Z

Sets output state to the inverse of Z flag without changing direction.

### PASM2 Syntax

```
OUTNZ    {#}D    {WCZ}
```

### Operation:

1. Set OUT bit for pin D to !Z
2. With WCZ, set C and Z flags to the prior OUT bit state
3. DIR is unchanged

**Timing:** 2 clock cycles

**Related:** OUTZ, DRVNZ

## OUTRND - Output Random

Sets output state to a random value without changing direction.

### PASM2 Syntax

```
OUTRND    {#}Dest    {WCZ}
```

### Operation:

1. Set OUT bit for pin D to a random value
2. If WC/WZ is specified, set C and Z to the pin's prior OUT bit (otherwise leave flags unchanged)
3. DIR is unchanged

**Timing:** 2 clock cycles

**Related:** DRVRND

## 1.5 Direction Instructions (DIRx)

Direction instructions modify only the direction register bit. The output state register is unchanged.

### Common Properties

- **Execution time:** 2 clock cycles
- **Flags:** With the optional WCZ effect, C and Z are **both** set to the pin's prior DIR-bit state (its direction before the instruction executes); without WCZ, neither flag changes

### DIRH - Direction High (Output)

Sets the pin to output mode.

#### PASM2 Syntax

```
DIRH    {#}D    {WCZ}
```

#### Operation:

1. Set DIR bit for pin D to 1 (output mode)
2. If WC/WZ is specified, set C and Z to the pin's prior DIR bit (otherwise leave flags unchanged)
3. OUT is unchanged; pin drives current OUT value

**Timing:** 2 clock cycles

**Spin2 Equivalent:** None directly. PINHIGH / PINLOW set DIR=1 and OUT in a single CALL (direction control is part of the operation, not a side effect).

#### Example - PASM2:

```
OUTH    #1ed    ' Pre-set output high
DIRH    #1ed    ' Now enable output (no glitch)
```

**Related:** DIRL, DIRNOT, DRVH

## DIRL - Direction Low (Input/Float)

Sets the pin to input mode (floating).

### PASM2 Syntax

```
DIRL    {#}D    {WCZ}
```

### Operation:

1. Set DIR bit for pin D to 0 (input mode, pin floats)
2. If WC/WZ is specified, set C and Z to the pin's prior DIR bit (otherwise leave flags unchanged)
3. OUT is unchanged

**Timing:** 2 clock cycles

**Spin2 Equivalent:** PINFLOAT(pin)

### Example - Spin2:

```
PUB main()
  PINFLOAT(10)           ' Float pin 10 (high impedance)
```

### Example - PASM2:

```
DIRL    #10    ' Float pin 10
```

**Related:** DIRH, DIRNOT, FLTL

## DIRNOT - Direction Toggle

Toggles the direction between input and output.

### PASM2 Syntax

```
DIRNOT  {#}Dest    {WCZ}
```

### Operation:

1. Toggle DIR bit for pin D
2. If WC/WZ is specified, set C and Z to the pin's prior DIR bit (otherwise leave flags unchanged)

**Timing:** 2 clock cycles

**Example - PASM2:**

```
DIRNOT    #10    ' Toggle pin 10 direction
```

**Related:** DIRH, DIRL**DIRC - Direction to C**

Sets direction based on C flag.

**PASM2 Syntax**

```
DIRC    {#}D    {WCZ}
```

**Operation:**

1. Set DIR bit for pin D to C flag value
2. With WCZ, set C and Z flags to the prior DIR bit state

**Timing:** 2 clock cycles**Related:** DIRNC, DRVC**DIRNC - Direction to Not C**

Sets direction based on inverse of C flag.

**PASM2 Syntax**

```
DIRNC    {#}D    {WCZ}
```

**Operation:**

1. Set DIR bit for pin D to !C
2. With WCZ, set C and Z flags to the prior DIR bit state

**Timing:** 2 clock cycles**Related:** DIRC, DRVNC**DIRZ - Direction to Z**

Sets direction based on Z flag.

**PASM2 Syntax**

```
DIRZ    {#}D    {WCZ}
```

**Operation:**

1. Set DIR bit for pin D to Z flag value
2. With WCZ, set C and Z flags to the prior DIR bit state

**Timing:** 2 clock cycles**Related:** DIRNZ, DRVZ**DIRNZ - Direction to Not Z**

Sets direction based on inverse of Z flag.

**PASM2 Syntax**

DIRNZ {#}D {WCZ}

**Operation:**

1. Set DIR bit for pin D to !Z
2. With WCZ, set C and Z flags to the prior DIR bit state

**Timing:** 2 clock cycles**Related:** DIRZ, DRVNZ**DIRRND - Direction Random**

Sets direction to a random value.

**PASM2 Syntax**

DIRRND {#}Dest {WCZ}

**Operation:**

1. Set DIR bit for pin D to a random value
2. If WC/WZ is specified, set C and Z to the pin's prior DIR bit (otherwise leave flags unchanged)

**Timing:** 2 clock cycles**Related:** DRVRND, OUTRND

## 1.6 Float Instructions (FLT<sub>x</sub>)

Float instructions set the pin to input mode (DIR=0) AND pre-set the output state. This is useful for preparing the output level before switching to output mode, avoiding glitches.

### Common Properties

- **Execution time:** 2 clock cycles
- **Flags:** With the optional WCZ effect, C and Z are **both** set to the pin's prior OUT-bit state (its output level before the instruction executes); without WCZ, neither flag changes
- **Effect:** DIR=0 (floating) AND OUT=specified value

### FLTH - Float with Output High

Floats pin and pre-sets output register high.

#### PASM2 Syntax

```
FLTH    {#}D    {WCZ}
```

#### Operation:

1. Set DIR bit for pin D to 0 (float)
2. Set OUT bit for pin D to 1 (high)
3. If WC/WZ is specified, set C and Z to the pin's prior OUT bit (otherwise leave flags unchanged)

**Timing:** 2 clock cycles

**Use Case:** Pre-set output high so that when DIRH is later executed, the pin immediately drives high without a glitch.

#### Example - PASM2:

```
FLTH    #led    ' Float pin, prepare to drive high
' ...later...
DIRH    #led    ' Enable output - immediately high
```

**Related:** FLTL, DRVH



## FLTL - Float with Output Low

Floats pin and pre-sets output register low.

### PASM2 Syntax

```
FLTL    {#}D    {WCZ}
```

### Operation:

1. Set DIR bit for pin D to 0 (float)
2. Set OUT bit for pin D to 0 (low)
3. If WC/WZ is specified, set C and Z to the pin's prior OUT bit (otherwise leave flags unchanged)

**Timing:** 2 clock cycles

**Spin2 Equivalent:** `PINFLOAT(pin)` is approximately equivalent (floats pin)

### Example - PASM2:

```
FLTL    #led    ' Float pin, prepare to drive low
```

**Related:** FLTH, DRVL, DIRL

## FLTNOT - Float with Output Toggle

Floats pin and toggles the output register.

### PASM2 Syntax

```
FLTNOT  {#}Dest    {WCZ}
```

### Operation:

1. Set DIR bit for pin D to 0 (float)
2. Toggle OUT bit for pin D
3. If WC/WZ is specified, set C and Z to the pin's prior OUT bit (otherwise leave flags unchanged)

**Timing:** 2 clock cycles

**Related:** FLTH, FLTL

## FLTC - Float with Output to C

Floats pin and sets output register to C flag.

### PASM2 Syntax

```
FLTC    {#}D    {WCZ}
```

#### Operation:

1. Set DIR bit for pin D to 0 (float)
2. Set OUT bit for pin D to C flag value
3. With WCZ, set C and Z flags to the prior OUT bit state

**Timing:** 2 clock cycles

**Related:** FLTNC

## FLTNC - Float with Output to Not C

Floats pin and sets output register to inverse of C flag.

### PASM2 Syntax

```
FLTNC   {#}D    {WCZ}
```

#### Operation:

1. Set DIR bit for pin D to 0 (float)
2. Set OUT bit for pin D to !C
3. With WCZ, set C and Z flags to the prior OUT bit state

**Timing:** 2 clock cycles

**Related:** FLTC

## FLTZ - Float with Output to Z

Floats pin and sets output register to Z flag.

### PASM2 Syntax

```
FLTZ    {#}D    {WCZ}
```

#### Operation:

1. Set DIR bit for pin D to 0 (float)
2. Set OUT bit for pin D to Z flag value

3. With WCZ, set C and Z flags to the prior OUT bit state

**Timing:** 2 clock cycles

**Related:** FLTNZ

## FLTNZ - Float with Output to Not Z

Floats pin and sets output register to inverse of Z flag.

### PASM2 Syntax

```
FLTNZ    {#}D        {WCZ}
```

### Operation:

1. Set DIR bit for pin D to 0 (float)
2. Set OUT bit for pin D to !Z
3. With WCZ, set C and Z flags to the prior OUT bit state

**Timing:** 2 clock cycles

**Related:** FLTZ

## FLTRND - Float with Output Random

Floats pin and sets output register to random value.

### PASM2 Syntax

```
FLTRND    {#}Dest    {WCZ}
```

### Operation:

1. Set DIR bit for pin D to 0 (float)
2. Set OUT bit for pin D to random value
3. If WC/WZ is specified, set C and Z to the pin's prior OUT bit (otherwise leave flags unchanged)

**Timing:** 2 clock cycles

**Related:** DRVRND, OUTRND

## 1.7 Test Pin Instructions

TEST instructions read the physical pin state and affect the C and/or Z flags. These instructions do NOT change pin direction or output state.

### TESTP - Test Pin

Reads the physical pin state and affects C or Z flags.

#### PASM2 Syntax

```
TESTP    {#}D        WC/WZ
```

#### Operation:

1. Read the physical state of pin D
2. Apply the specified operation to C or Z flag

#### Flag Operations:

| Modifier | Effect              |
|----------|---------------------|
| WC       | C = pin state       |
| WZ       | Z = pin state       |
| ANDC     | C = C AND pin state |
| ANDZ     | Z = Z AND pin state |
| ORC      | C = C OR pin state  |
| ORZ      | Z = Z OR pin state  |
| XORC     | C = C XOR pin state |
| XORZ     | Z = Z XOR pin state |

**Timing:** 2 clock cycles

**Input Latency:** Reads pin state from 2 clock cycles before instruction start (fresher than INx registers)

**Spin2 Equivalent:** PINREAD(pin)

**Example - Spin2:**

```

CON
    BUTTON_PIN = 10

PUB main() | state
    repeat
        state := PINREAD(BUTTON_PIN) ' Read button state
        if state == 0                  ' Button pressed (active low)
            PINHIGH(56)                ' Turn on LED
        else
            PINLOW(56)                 ' Turn off LED

```

**Example - PASM2:**

```

        TESTP    #10 wc    ' Read pin 10 into C flag
if_c    DRVH     #56      ' If pin high, drive LED high
if_nc   DRVL     #56      ' If pin low, drive LED low

```

**Related:** TESTPN**TESTPN - Test Pin Negated**

Reads the physical pin state inverted and affects C or Z flags.

**PASM2 Syntax**

```
TESTPN  {#}D      WC/WZ
```

**Operation:**

1. Read the physical state of pin D
2. Invert the value
3. Apply the specified operation to C or Z flag

**Timing:** 2 clock cycles

**Use Case:** Useful for active-low inputs (buttons, sensors) where high means “not pressed” and low means “pressed”.

**Example - PASM2:**

```

        TESTPN   #button wc    ' C=1 if button pressed (active-low)
if_c    CALL     #handle_button

```

**Related:** TESTP

## 1.8 Spin2 Pin Methods

Spin2 provides high-level methods for common pin operations. These methods execute from hub RAM and have additional overhead compared to inline PASM2.

Spin2 also accepts short-form aliases for the three most common of these: PINH for PINHIGH, PINL for PINLOW, and PINF for PINFLOAT. The two forms are interchangeable; this guide uses both.

### PINHIGH(PinField)

Drives pin(s) high.

**Function:** Sets DIR=1 and OUT=1 for specified pins

**Equivalent PASM2:** DRVH instruction

**Parameter:** PinField - Single pin number (0-63), range (Bottom..Top), or ADDPINS expression

**Example:**

```
PINHIGH(56)           ' Drive pin 56 high
PINHIGH(0..7)         ' Drive pins 0-7 all high
PINHIGH(16 ADDPINS 3) ' Drive pins 16-19 high
```

### PINLOW(PinField)

Drives pin(s) low.

**Function:** Sets DIR=1 and OUT=0 for specified pins

**Equivalent PASM2:** DRVL instruction

**Example:**

```
PINLOW(56)           ' Drive pin 56 low
```

### PINTOGGLE(PinField)

Toggles pin output state.

**Function:** Toggles OUT bit and sets DIR=1

**Equivalent PASM2:** DRVNOT instruction

**Example:**

```
PINTOGGLE(56)         ' Toggle pin 56
```

## PINFLOAT(PinField)

Floats pin(s) (sets to input mode).

**Function:** Sets DIR=0 for specified pins

**Equivalent PASM2:** DIRL instruction

**Example:**

```
PINFLOAT(10)           ' Float pin 10 (high impedance)
```

## PINWRITE(PinField, Value)

Writes value to pin(s).

**Function:** Sets OUT to Value and DIR=1

**Parameters:**

- PinField: Pin specification
- Value: 0 or 1 (or multi-bit value for pin ranges)

**Equivalent PASM2:** DRVL (value=0) or DRVH (value=1)

**Example:**

```
PINWRITE(56, 1)         ' Same as PINHIGH(56)
PINWRITE(56, 0)         ' Same as PINLOW(56)
PINWRITE(0..7, %10101010) ' Write pattern to pins 0-7
```

## PINREAD(PinField)

Reads pin input state.

**Function:** Returns current state of pin(s)

**Returns:** 0 or 1 for single pin; multi-bit value for pin ranges

**Equivalent PASM2:** TESTP (approximately)

**Example:**

```

VAR
    LONG button_state

PUB main()
    button_state := PINREAD(10)      ' Read single pin

    ' For pin range, returns value with LSB = lowest pin
    byte_val := PINREAD(0..7)       ' Read 8 pins as byte

```

**PINCLEAR(PinField)**

Clears smart pin configuration.

**Function:** Resets pin to normal mode (P\_NORMAL)

**Equivalent PASM2:** WRPIN #0, pin

**Example:**

```

PINCLEAR(10)                        ' Reset pin 10 to normal mode

```

**Note:** Use this to disable smart pin modes and return to basic Direct I/O.

## 1.9 Pin Span Operations

All DRV/OUT/DIR/FLT instructions support operating on multiple pins simultaneously using the span encoding in the D operand or via SETQ.

### Span Encoding

- D[5:0]: Base pin number (0-63)
- D[10:6]: Number of additional pins (0-31)

Bit 5 of the base-pin field is what selects the target port: a base pin in 0–31 lands the operation on Port A (the DIRA/OUTA registers), and 32–63 lands it on Port B (DIRB/OUTB). That is also why a span never crosses the 32-pin boundary — see *Wrap Behavior* below.

### Using SETQ for Span

```

SETQ      #7      ' Set span to 8 pins (0 + 7 additional)
DRVH      #0      ' Drive pins 0-7 high

```



## Using ADDPINS for Span

ADDPINS sets the additional-pins field (D[10:6]) inline, without a preceding SETQ — convenient when the span is known at assembly time:

```
DRVH      #10 ADDPINS 7      ' P10..P17 high (base 10 + 7)
```

As with every span operation, an ADDPINS range cannot cross a 32-pin port boundary.

## Wrap Behavior

Span operations wrap within the same 32-pin port. Pins 0-31 (Port A) and 32-63 (Port B) are independent. A span starting at pin 28 with span 7 affects pins 28-31, then wraps to 0-3.

## 1.10 Instruction Quick Reference

Table 1.1.

| Instruction   | Effect            | DIR    | OUT    | Flags   |
|---------------|-------------------|--------|--------|---------|
| <b>DRVH</b>   | Drive high        | 1      | 1      | C/Z=OUT |
| <b>DRVL</b>   | Drive low         | 1      | 0      | C/Z=OUT |
| <b>DRVNOT</b> | Drive toggle      | 1      | toggle | C/Z=OUT |
| <b>DRVC</b>   | Drive to C        | 1      | C      | C/Z=OUT |
| <b>DRVNC</b>  | Drive to !C       | 1      | !C     | C/Z=OUT |
| <b>DRVZ</b>   | Drive to Z        | 1      | Z      | C/Z=OUT |
| <b>DRVNZ</b>  | Drive to !Z       | 1      | !Z     | C/Z=OUT |
| <b>DRVRND</b> | Drive random      | 1      | rnd    | C/Z=OUT |
| <b>OUTH</b>   | Output high       | -      | 1      | C/Z=OUT |
| <b>OUTL</b>   | Output low        | -      | 0      | C/Z=OUT |
| <b>OUTNOT</b> | Output toggle     | -      | toggle | C/Z=OUT |
| <b>OUTC</b>   | Output to C       | -      | C      | C/Z=OUT |
| <b>OUTNC</b>  | Output to !C      | -      | !C     | C/Z=OUT |
| <b>OUTZ</b>   | Output to Z       | -      | Z      | C/Z=OUT |
| <b>OUTNZ</b>  | Output to !Z      | -      | !Z     | C/Z=OUT |
| <b>OUTRND</b> | Output random     | -      | rnd    | C/Z=OUT |
| <b>DIRH</b>   | Direction output  | 1      | -      | C/Z=DIR |
| <b>DIRL</b>   | Direction input   | 0      | -      | C/Z=DIR |
| <b>DIRNOT</b> | Direction toggle  | toggle | -      | C/Z=DIR |
| <b>DIRC</b>   | Direction to C    | C      | -      | C/Z=DIR |
| <b>DIRNC</b>  | Direction to !C   | !C     | -      | C/Z=DIR |
| <b>DIRZ</b>   | Direction to Z    | Z      | -      | C/Z=DIR |
| <b>DIRNZ</b>  | Direction to !Z   | !Z     | -      | C/Z=DIR |
| <b>DIRRND</b> | Direction random  | rnd    | -      | C/Z=DIR |
| <b>FLTH</b>   | Float, out high   | 0      | 1      | C/Z=OUT |
| <b>FLTL</b>   | Float, out low    | 0      | 0      | C/Z=OUT |
| <b>FLTNOT</b> | Float, toggle out | 0      | toggle | C/Z=OUT |
| <b>FLTC</b>   | Float, out to C   | 0      | C      | C/Z=OUT |

Continued on next page

Table 1.1. (Continued)

| Instruction   | Effect            | DIR | OUT | Flags    |
|---------------|-------------------|-----|-----|----------|
| <b>FLTNC</b>  | Float, out to !C  | 0   | !C  | C/Z=OUT  |
| <b>FLTZ</b>   | Float, out to Z   | 0   | Z   | C/Z=OUT  |
| <b>FLTNZ</b>  | Float, out to !Z  | 0   | !Z  | C/Z=OUT  |
| <b>FLTRND</b> | Float, out random | 0   | rnd | C/Z=OUT  |
| <b>TESTP</b>  | Test pin          | -   | -   | C/Z=pin  |
| <b>TESTPN</b> | Test pin negated  | -   | -   | C/Z=!pin |

**Legend:** “-” = unchanged, “toggle” = inverts current value, “rnd” = random. **Flag effects (with the optional WCZ effect):** DRV/OUT/FLT set **both C and Z** to the pin’s prior OUT-bit state, and DIR sets **both C and Z** to the pin’s prior DIR-bit state — i.e. the output/direction level *before* the instruction executes. TESTP/TESTPN set both C and Z to the pin’s input state. Without WC/WZ, no flag is written. The single value shown in the Flags column above is the value delivered to both flags. (Source: *P2 Assembly Language Reference*.)

## 1.11 Common Patterns

### LED Blink (Spin2)

```
CON
    _clkfreq = 200_000_000
    LED_PIN = 56

PUB main()
    repeat
        PINTOGGLE(LED_PIN)
        WAITMS(500)
```

### LED Blink (PASM2)

```
CON
    _clkfreq = 200_000_000

DAT                ORG

                    DRVH    #56            ' Start with LED on

.loop              WAITX    delay          ' Wait
                    DRVNOT  #56            ' Toggle LED
```

continues on next page →

*↔ continued from previous page*

```

                JMP      #.loop      ' Repeat
delay          LONG     100_000_000  ' 0.5 seconds at 200 MHz

```

## Button-Controlled LED (Spin2)

```

CON
  _clkfreq = 200_000_000
  BUTTON_PIN = 10
  LED_PIN = 56

PUB main()
  repeat
    if PINREAD(BUTTON_PIN) == 0      ' Active-low button
      PINHIGH(LED_PIN)
    else
      PINLOW(LED_PIN)

```

ch01-button-led.spin2

## Button-Controlled LED (PASM2)

```

CON
  _clkfreq = 200_000_000

DAT          ORG

.loop        TESTP    #10 wc      ' Read button into C
             if_nc    DRVH    #56  ' Button pressed: LED on
             if_c     DRVL    #56  ' Button released: LED off
             JMP      #.loop

```

## Glitch-Free Output Start

```

FLTH    #motor      ' Prepare output high, but float
' ... other setup ...
DIRH    #motor      ' Enable output - immediately high

```

*This chapter establishes the foundational concepts of P2 pin control. All smart pin modes (Chapters 6-19) build upon these Direct I/O principles. See Chapter 2 for enhanced pin configuration via P\_ constants.*

# Chapter 2: Enhanced Direct I/O

## Low-Level Pin Modes

Enhanced Direct I/O extends basic pin control with configurable drive strength, input conditioning, and basic analog capabilities—all without entering smart pin modes. These features are configured via WRPIN using P\_ constants with mode bits [5:1] = %00000 (P\_NORMAL).

## 2.1 Overview

### What Enhanced Direct I/O Provides

While Chapter 1 covered the fundamental DIR/OUT/IN operations, Enhanced Direct I/O adds:

- **Drive Strength Selection:** 8 options for high-side drive, 8 options for low-side drive
- **Input Conditioning:** Logic level, Schmitt trigger, and comparator modes
- **Input Routing:** Select from local pin or adjacent pins (-3 to +3)
- **Basic Analog:** DAC output and ADC input without smart pin modes
- **Polarity Control:** Invert input or output signals

### Configuration Method

All Enhanced Direct I/O features are configured via WRPIN:

#### Spin2:

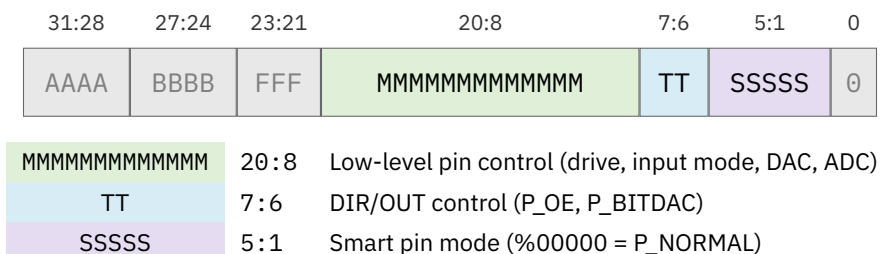
```
WRPIN(pin, P_constant1 | P_constant2 | ...)
```

#### PASM2:

```
WRPIN    ##(P_constant1 | P_constant2), pin
```

## The P\_ Constant Architecture

P\_ constants are 32-bit values where specific bit fields control different aspects of pin behavior. The three fields most relevant to Enhanced Direct I/O are lit below; the muted fields (A/B input routing in bits [31:21] and the always-0 bit 0) belong to smart pin modes and are covered by the full field map in §4.2.



**Figure 2.1.** Enhanced Direct I/O — the active WRPIN fields (full 32-bit map in §4.2)

When mode bits [5:1] = %00000, the pin operates in P\_NORMAL mode with enhanced characteristics from other bit fields.

## 2.2 Drive Strength

The P2 provides configurable drive strength for both high-side (driving to VIO) and low-side (driving to ground) independently. This enables open-drain configurations, current limiting, and power optimization.

### Drive-High Options

Select one drive-high constant. These control the high-side output driver.

| Constant                 | Bits[13:11] | Drive          | Current/Impedance | Use Case                     |
|--------------------------|-------------|----------------|-------------------|------------------------------|
| P_HIGH_FAST<br>(default) | %000        | Fast CMOS      | ~30mA / ~100Ω     | Standard digital, LEDs       |
| P_HIGH_1K5               | %001        | Resistive      | ~2mA / 1.5kΩ      | Current limiting, protection |
| P_HIGH_15K               | %010        | Resistive      | ~200μA / 15kΩ     | Pull-up resistor             |
| P_HIGH_150K              | %011        | Resistive      | ~20μA / 150kΩ     | Weak pull-up                 |
| P_HIGH_1MA               | %100        | Current source | 1mA               | Constant current             |
| P_HIGH_100UA             | %101        | Current source | 100μA             | Low-power pull-up            |
| P_HIGH_10UA              | %110        | Current source | 10μA              | Very low power               |
| P_HIGH_FLOAT             | %111        | Float          | High-Z            | Open-drain output            |

## Drive-Low Options

Select one drive-low constant. These control the low-side output driver.

| Constant             | Bits[10:8] | Drive        | Current/Impedance | Use Case               |
|----------------------|------------|--------------|-------------------|------------------------|
| P_LOW_FAST (default) | %000       | Fast CMOS    | ~30mA / ~100Ω     | Standard digital, LEDs |
| P_LOW_1K5            | %001       | Resistive    | ~2mA / 1.5kΩ      | Current limiting       |
| P_LOW_15K            | %010       | Resistive    | ~200μA / 15kΩ     | Pull-down resistor     |
| P_LOW_150K           | %011       | Resistive    | ~20μA / 150kΩ     | Weak pull-down         |
| P_LOW_1MA            | %100       | Current sink | 1mA               | Constant current       |
| P_LOW_100UA          | %101       | Current sink | 100μA             | Low-power pull-down    |
| P_LOW_10UA           | %110       | Current sink | 10μA              | Very low power         |
| P_LOW_FLOAT          | %111       | Float        | High-Z            | Totem-pole disable     |

## Common Drive Configurations

### Standard Digital (Default):

```
WRPIN(pin, P_HIGH_FAST | P_LOW_FAST)    ' Maximum drive both directions
```

### Open-Drain (I<sup>2</sup>C style):

```
WRPIN(pin, P_HIGH_FLOAT | P_LOW_FAST)    ' OUT=1 floats, OUT=0 drives low
```

### Open-Source:

```
WRPIN(pin, P_HIGH_FAST | P_LOW_FLOAT)    ' OUT=1 drives high, OUT=0 floats
```

### Pull-Up Resistor:

```
WRPIN(pin, P_HIGH_15K | P_LOW_FLOAT)    ' 15kΩ pull-up, no low drive
```

### Current-Limited Output:

```
WRPIN(pin, P_HIGH_1K5 | P_LOW_1K5)      ' ~2mA max in either direction
```



## Resistive vs Current Source

**Resistive modes** (1K5, 15K, 150K):

- Voltage-dependent current
- Current decreases as pin approaches target voltage
- Suitable for bus pull-ups/pull-downs
- Rise/fall time depends on load capacitance

**Current source modes** (1MA, 100UA, 10UA):

- Constant current regardless of voltage
- Useful for driving LEDs without external resistor
- Linear charging of capacitive loads
- More predictable timing

**Example - LED without external resistor:**

```
' 1mA current source - suitable for indicator LED
WRPIN(led_pin, P_HIGH_1MA | P_LOW_FAST)
PINHIGH(led_pin)           ' LED on at 1mA
```

## 2.3 Input Conditioning

Input conditioning selects how the pin's analog signal is converted to the digital IN bit.

### Logic-Level Modes

Standard digital input with selectable input source.

| Constant               | Description                              |
|------------------------|------------------------------------------|
| P_LOGIC_A<br>(default) | Logic level A → IN, output from OUT bit  |
| P_LOGIC_A_FB           | Logic level A → IN, output from feedback |
| P_LOGIC_B_FB           | Logic level B → IN, output from feedback |

**Note:** “Feedback” routes the actual pin state (after driver) back to the output stage, useful for tri-state bus sensing.

### Schmitt Trigger Modes

Schmitt trigger input provides hysteresis, making the input more resistant to noise on slowly-changing signals.

| Constant       | Description                                  |
|----------------|----------------------------------------------|
| P_SCHMITT_A    | Schmitt trigger A → IN, output from OUT      |
| P_SCHMITT_A_FB | Schmitt trigger A → IN, output from feedback |
| P_SCHMITT_B_FB | Schmitt trigger B → IN, output from feedback |

#### When to use Schmitt trigger:

- Slow edge rates on input signals
- Noisy environments
- Mechanical switch debouncing (combined with software)
- Signals with long wiring

#### Example - Schmitt trigger for button:

```
WRPIN(button_pin, P_SCHMITT_A)      ' Schmitt trigger input
PINFLOAT(button_pin)                ' Make it an input
```

## Comparator Modes

Pin-to-pin comparison for analog signal detection.

| Constant        | Description                      |
|-----------------|----------------------------------|
| P_COMPARE_AB    | A > B → IN, output from OUT      |
| P_COMPARE_AB_FB | A > B → IN, output from feedback |

**Use case:** Compare two analog voltages without ADC.

#### Example - Voltage comparator:

```
' Compare pin 10 (A input) to pin 11 (B input)
' IN=1 when pin 10 > pin 11
WRPIN(10, P_COMPARE_AB | P_PLUS1_B)      ' A=local (10), B=pin+1 (11)
```

## 2.4 Input Source Selection

The A and B inputs can be sourced from the local pin or adjacent pins.

### A Input Selection

| Constant            | Source             |
|---------------------|--------------------|
| P_LOCAL_A (default) | Local pin          |
| P_PLUS1_A           | Pin + 1            |
| P_PLUS2_A           | Pin + 2            |
| P_PLUS3_A           | Pin + 3            |
| P_OUTBIT_A          | OUT bit (internal) |
| P_MINUS3_A          | Pin - 3            |
| P_MINUS2_A          | Pin - 2            |
| P_MINUS1_A          | Pin - 1            |

### B Input Selection

| Constant            | Source             |
|---------------------|--------------------|
| P_LOCAL_B (default) | Local pin          |
| P_PLUS1_B           | Pin + 1            |
| P_PLUS2_B           | Pin + 2            |
| P_PLUS3_B           | Pin + 3            |
| P_OUTBIT_B          | OUT bit (internal) |
| P_MINUS3_B          | Pin - 3            |
| P_MINUS2_B          | Pin - 2            |
| P_MINUS1_B          | Pin - 1            |

## Input Polarity

| Constant           | Effect               |
|--------------------|----------------------|
| P_TRUE_A (default) | Non-inverted A input |
| P_INVERT_A         | Inverted A input     |
| P_TRUE_B (default) | Non-inverted B input |
| P_INVERT_B         | Inverted B input     |

## A,B Input Logic

Combine A and B inputs logically before use.

| Constant            | Result                |
|---------------------|-----------------------|
| P_PASS_AB (default) | Pass A, B unchanged   |
| P_AND_AB            | A AND B, B            |
| P_OR_AB             | A OR B, B             |
| P_XOR_AB            | A XOR B, B            |
| P_FILT0_AB          | FILT0 filter settings |
| P_FILT1_AB          | FILT1 filter settings |
| P_FILT2_AB          | FILT2 filter settings |
| P_FILT3_AB          | FILT3 filter settings |

## 2.5 ADC Input Modes (Basic)

Basic ADC modes provide analog-to-digital conversion without smart pin modes. The result appears in the IN bit based on comparison.

| Constant    | Gain  | Description            |
|-------------|-------|------------------------|
| P_ADC_GIO   | -     | ADC GIO → IN           |
| P_ADC_VIO   | -     | ADC VIO → IN           |
| P_ADC_FLOAT | -     | ADC float → IN         |
| P_ADC_1X    | 1×    | Standard gain          |
| P_ADC_3X    | 3.16× | Moderate amplification |
| P_ADC_10X   | 10×   | High gain              |
| P_ADC_30X   | 31.6× | Higher gain            |
| P_ADC_100X  | 100×  | Maximum gain           |

**Note:** These modes provide single-bit output (comparator-style). For multi-bit ADC conversion, use smart pin ADC modes (Chapter 16).

### Example - Simple threshold detection:

```
' Detect when analog input exceeds ~1.65V (mid-scale)
WRPIN(adc_pin, P_ADC_1X)
PINFLOAT(adc_pin)
```

## 2.6 DAC Output Modes (Basic)

Basic DAC modes provide digital-to-analog conversion without smart pin modes. The DAC value is encoded in the WRPIN configuration word.

| Constant      | Impedance | Peak Voltage | Description                     |
|---------------|-----------|--------------|---------------------------------|
| P_DAC_990R_3V | 990Ω      | 3.3V         | High impedance, full swing      |
| P_DAC_600R_2V | 600Ω      | 2.0V         | Medium impedance, reduced swing |
| P_DAC_124R_3V | 124Ω      | 3.3V         | Low impedance, full swing       |
| P_DAC_75R_2V  | 75Ω       | 2.0V         | Lowest impedance, reduced swing |

### DAC Value Encoding:

The 8-bit DAC value is encoded in bits [15:8] of the WRPIN configuration:

```
' Set pin to output DAC level
' dac_value: 0-255 (0V to peak voltage)
dac_config := P_DAC_990R_3V | (dac_value << 8)
WRPIN(pin, dac_config)
PINH(pin)                                ' Enable output
```

### Selecting DAC Mode:

| Need                        | Use                                         |
|-----------------------------|---------------------------------------------|
| Maximum voltage swing       | P_DAC_990R_3V or P_DAC_124R_3V              |
| Driving low impedance loads | P_DAC_124R_3V or P_DAC_75R_2V               |
| Lower power                 | P_DAC_990R_3V or P_DAC_600R_2V              |
| Audio output                | P_DAC_75R_2V (low impedance for headphones) |

### Example - Static analog voltage:

```
CON
  DAC_PIN = 40
  DAC_MIDPOINT = 128                                ' Half of 256

PUB main()
  ' Output ~1.65V (half of 3.3V)
  WRPIN(DAC_PIN, P_DAC_990R_3V | (DAC_MIDPOINT << 8))
  PINH(DAC_PIN)                                ' Enable DAC output
```

**Note:** For dynamic DAC output with waveform generation, use smart pin DAC modes (Chapter 10).

## 2.7 Level Comparison Modes

Level comparison modes compare the input voltage to a programmable 8-bit threshold level.

| Constant      | Description                              |
|---------------|------------------------------------------|
| P_LEVEL_A     | A > Level → IN, output from OUT          |
| P_LEVEL_A_FBN | A > Level → IN, output negative feedback |
| P_LEVEL_B_FBP | B > Level → IN, output positive feedback |
| P_LEVEL_B_FBN | B > Level → IN, output negative feedback |

### Level Encoding:

The 8-bit comparison level is encoded in bits [15:8]:

```
' Compare pin A input to threshold level
' level: 0-255 (0V to VIO)
level_config := P_LEVEL_A | (level << 8)
WRPIN(pin, level_config)
```

**Note:** When DIR=1, output drive is 1.5kΩ.

#### Feedback Modes:

- **FBN (Negative Feedback):** Output opposes input (stabilizing)
- **FBP (Positive Feedback):** Output reinforces input (hysteresis/latching)

## 2.8 Synchronous I/O Mode

| Constant             | Effect                                         |
|----------------------|------------------------------------------------|
| P_ASYNC_IO (default) | Asynchronous I/O (inputs sampled continuously) |
| P_SYNC_IO            | Synchronous I/O (inputs sampled on clock edge) |

Synchronous mode is used for clocked interfaces where input sampling must be synchronized to a clock signal.

## 2.9 Polarity Control

### IN Bit Polarity

| Constant            | Effect                        |
|---------------------|-------------------------------|
| P_TRUE_IN (default) | IN bit reflects actual input  |
| P_INVERT_IN         | IN bit is inverted from input |

### Output Polarity

| Constant                             | Effect                          |
|--------------------------------------|---------------------------------|
| P_TRUE_OUTPUT / P_TRUE_OUT (default) | Output matches OUT bit          |
| P_INVERT_OUTPUT / P_INVERT_OUT       | Output is inverted from OUT bit |

**Example - Active-low LED:**

```
' LED connected to VCC, turns on when pin is low
WRPIN(led_pin, P_INVERT_OUT)
PINHIGH(led_pin)           ' Actually drives low, LED on
```

## 2.10 DIR/OUT Control

| Constant           | TT Bits | Effect                               |
|--------------------|---------|--------------------------------------|
| P_TT_00 (default)  | %00     | Normal operation                     |
| P_TT_01 / P_OE     | %01     | Output enable (for Smart Pin output) |
| P_TT_10 / P_BITDAC | %10     | BITDAC enable                        |
| P_TT_11            | %11     | Combined                             |
| P_CHANNEL          | %01     | DAC channel enable (alias for P_OE)  |

**P\_OE** is required when using smart pin modes that produce output. For P\_NORMAL mode, it is not needed as DIR controls output directly.

## 2.11 Combining Constants

P\_ constants are combined using the OR operator. Constants from different categories can be freely combined; constants from the same category (marked “pick one”) are mutually exclusive.

### Combination Rules

1. **Pick one from each category** - Only one drive-high, one drive-low, one input mode, etc.
2. **OR them together** - Use | operator in Spin2 or PASM2
3. **Order doesn't matter** - Constants can be combined in any order
4. **Defaults apply if omitted** - P\_HIGH\_FAST, P\_LOW\_FAST, P\_LOGIC\_A, etc. are defaults

### Examples

**Open-drain with Schmitt input:**

```
WRPIN(pin, P_HIGH_FLOAT | P_LOW_FAST | P_SCHMITT_A)
```



**Weak pull-up with inverted input:**

```
WRPIN(pin, P_HIGH_15K | P_LOW_FLOAT | P_INVERT_IN)
```

**Current-limited output with inverted polarity:**

```
WRPIN(pin, P_HIGH_1K5 | P_LOW_1K5 | P_INVERT_OUT)
```

**Comparator using adjacent pin:**

```
WRPIN(pin, P_COMPARE_AB | P_PLUS1_B)
```

## 2.12 Complete Configuration Examples

### I<sup>2</sup>C Open-Drain Configuration

```
CON
    SDA_PIN = 0
    SCL_PIN = 1

PUB setup_i2c()
    ' Open-drain (most I2C setups use external pull-ups instead)
    WRPIN(SDA_PIN, P_HIGH_FLOAT | P_LOW_FAST | P_SCHMITT_A)
    WRPIN(SCL_PIN, P_HIGH_FLOAT | P_LOW_FAST | P_SCHMITT_A)

    ' Start with lines released (high via external pull-ups)
    PINHIGH(SDA_PIN)           ' Float (open-drain high)
    PINHIGH(SCL_PIN)           ' Float (open-drain high)
```

### Button Input with Internal Pull-Up

```
CON
    BUTTON_PIN = 10

PUB setup_button()
    ' Internal 15kΩ pull-up, Schmitt trigger for noise immunity
    WRPIN(BUTTON_PIN, P_HIGH_15K | P_LOW_FLOAT | P_SCHMITT_A)
    PINHIGH(BUTTON_PIN)           ' Enable pull-up

    ' Now PINREAD returns 1 when released, 0 when pressed
```

## LED Current Source

```

CON
    LED_PIN = 56

PUB setup_led()
    ' 1mA current source - no external resistor needed
    WRPIN(LED_PIN, P_HIGH_1MA | P_LOW_FAST)

    ' PINHIGH turns LED on at 1mA
    ' PINLOW turns LED off

```

## Static DAC Output

```

CON
    DAC_PIN = 40

PUB set_voltage(level) | config
    ' Output analog voltage proportional to level (0-255)
    config := P_DAC_990R_3V | (level << 8)
    WRPIN(DAC_PIN, config)
    PINH(DAC_PIN)                    ' Enable output

```

## PASM2 Configuration Examples

```

' Open-drain configuration
    WRPIN ##(P_HIGH_FLOAT | P_LOW_FAST | P_SCHMITT_A), sda_pin
    DRVH      sda_pin                ' Release line (floats high)

' Internal pull-up button
    WRPIN ##(P_HIGH_15K | P_LOW_FLOAT | P_SCHMITT_A), btn_pin
    DRVH      btn_pin                ' Enable pull-up

' Current-source LED
    WRPIN      ##(P_HIGH_1MA | P_LOW_FAST), led_pin
    DRVH      led_pin                ' LED on at 1mA

' DAC output (1.65V = 128 at 3.3V range)
    WRPIN      ##(P_DAC_990R_3V | (128 << 8)), dac_pin
    DIRH      dac_pin                ' Enable DAC

```

## 2.13 Resetting to Default

`PINCLEAR(pin)` — or equivalently `WRPIN(pin, 0)` — clears all enhanced configuration and smart pin modes, returning the pin to basic Direct I/O operation. See §4.14 for the full reset-to-normal reference, including the fact that `WRPIN #0` takes effect even while a smart pin is running.

## 2.14 Quick Reference

### Drive Strength Summary

| High-Side | Low-Side | Configuration          |
|-----------|----------|------------------------|
| Fast      | Fast     | Default digital (30mA) |
| Float     | Fast     | Open-drain             |
| Fast      | Float    | Open-source            |
| 15K       | Float    | Pull-up only           |
| Float     | 15K      | Pull-down only         |
| 1K5       | 1K5      | Current-limited        |
| 1MA       | Fast     | LED current source     |

### Input Mode Summary

| Mode         | Hysteresis | Use Case                    |
|--------------|------------|-----------------------------|
| P_LOGIC_A    | None       | Fast digital signals        |
| P_SCHMITT_A  | Yes        | Slow/noisy signals, buttons |
| P_COMPARE_AB | None       | Analog comparison           |
| P_ADC_*      | None       | Analog threshold detection  |
| P_LEVEL_*    | Optional   | Programmable threshold      |

*This chapter covers pin configuration without smart pin modes. For autonomous pin operations (PWM, serial, ADC, etc.), see Chapters 6-19. For the smart pin configuration process, see Chapter 4.*

# Chapter 3: Smart Pin Architecture

## Autonomous I/O

Smart pins transform P2 I/O pins from simple input/output points into autonomous peripheral engines. Once configured, a smart pin operates independently of the cog—generating waveforms, measuring signals, counting events, or performing analog conversions without consuming cog cycles. This chapter establishes the mental model for understanding all smart pin modes documented in Parts II through IV.

### 3.1 The Autonomous Operation Concept

#### What Makes Smart Pins “Smart”

A traditional microcontroller pin requires continuous software attention. To generate a PWM signal, for example, software must toggle the pin at precise intervals. To measure an input pulse width, software must sample the pin and track timing. These operations consume CPU cycles and require precise interrupt handling.

Smart pins invert this model. Once configured:

- **The hardware generates signals autonomously** - PWM, serial data, NCO waveforms run without cog intervention
- **The hardware measures signals autonomously** - Pulse widths, frequencies, quadrature positions accumulate in registers
- **The cog interacts only when needed** - To read results, update parameters, or reconfigure

This autonomous operation enables:

- **Precise timing** - Hardware-level timing accuracy, independent of software execution
- **Multi-channel operation** - Every pin can run its own smart pin mode simultaneously
- **Reduced cog load** - cogs spend time on computation rather than I/O bit-banging
- **Deterministic behavior** - Hardware timing is unaffected by software complexity

#### The Relationship Between Cog and Smart Pin

Each smart pin operates as an independent state machine. The cog’s role is:

1. **Configure** the smart pin (mode, parameters)
2. **Enable** the smart pin (set DIR=1)
3. **Monitor** the IN flag for events
4. **Read** results when ready
5. **Update** parameters as needed

## 6. **Disable** when finished

Between these interactions, the smart pin runs autonomously.

## 3.2 The X / Y / Z Registers (plus the mode word)

Every smart pin has four 32-bit registers: a mode-configuration register written by WRPIN (the mode-control word, see §3.5), plus the three parameter/result registers—X, Y, and Z—described here. The mode register establishes *what* the pin does; X, Y, and Z carry the data it works with:

### X Register - Configuration and Parameters

The X register holds configuration parameters that define **how** the smart pin operates. Its meaning varies by mode:

| Mode Category | Typical X Usage                   |
|---------------|-----------------------------------|
| Timing modes  | Base period in clock cycles       |
| Counter modes | Measurement window duration       |
| Serial modes  | Bit timing / baud rate parameters |
| ADC modes     | Sample period and filter settings |

X is written via the **WXPIN** instruction. Some modes use only X[15:0]; others use the full 32 bits. Many modes also use X[31:16] for secondary parameters (frame period, initial phase, etc.).

### Y Register - Input Data or Secondary Configuration

The Y register holds:

- **Output data** for transmit modes (serial data to send)
- **Target values** for output modes (PWM duty cycle, DAC level)
- **Mode modifiers** for some input modes (sensitivity selection)

Y is written via the **WYPIN** instruction. For many modes, Y is updated repeatedly during operation to provide new output data or adjust behavior.

### Z Register - Accumulator and Results

The Z register is the smart pin's working register:

- **Accumulators** - Counting events, timing measurements
- **Phase accumulators** - NCO modes track phase in Z
- **Output buffers** - Results for cog to read

Z is read via **RDPIN** or **RQPIN**. Software cannot write Z directly—it is managed by the smart pin hardware.

## Register Initialization

When a smart pin is reset (DIR transitions from 1 to 0), the registers are initialized according to the mode. Specific initialization behavior is documented in each mode's chapter.

## 3.3 The IN Bit - Event Signaling

### Purpose of the IN Bit

Each smart pin has an **IN bit** that signals events to cogs. This is the same IN bit readable via TESTP/TESTPN, but its meaning changes when a smart pin mode is active.

In P\_NORMAL mode (no smart pin):

- IN reflects the physical pin state

In smart pin modes:

- IN signals mode-specific events (data ready, measurement complete, overflow, etc.)

### When IN is Raised

Each mode defines when it raises IN:

| Mode Category     | IN is raised when...                       |
|-------------------|--------------------------------------------|
| Output modes      | Cycle completes, buffer ready for new data |
| Measurement modes | Measurement period completes               |
| Serial TX         | Ready for next data word                   |
| Serial RX         | Data word received                         |
| Counter modes     | Measurement window expires                 |

### Acknowledging IN

When a cog interacts with a smart pin, the IN bit is **acknowledged** (lowered). This prepares the smart pin to signal the next event.

#### Instructions that acknowledge:

- WRPIN - Configure (also acknowledges)

- WXPIN - Write X (also acknowledges)
- WYPIN - Write Y (also acknowledges)
- RDPIN - Read Z and acknowledge
- AKPIN - Acknowledge without reading

#### Instructions that do NOT acknowledge:

- RQPIN - Read Z quietly (no acknowledge)

## Polling and the 2-Clock Delay

After an acknowledge, it takes **two clock cycles** before the IN bit can be polled again:

1. Cog executes RDPIN/AKPIN/etc. (acknowledges smart pin)
2. One clock elapses
3. Second clock elapses
4. IN can now be polled via TESTP

This timing matters in tight polling loops.

## RQPIN for Multi-Cog Access

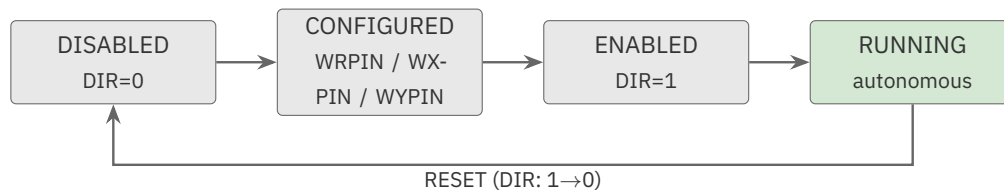
The **RQPIN** (read quiet) instruction reads the Z register without acknowledging. This allows multiple cogs to read the same smart pin's result without interfering with each other. Only one cog should acknowledge; others can use RQPIN to observe.

### CAUTION

**Multi-cog bus collisions.** Every cog reaches each smart pin over a shared 34-bit bus for write data and acknowledgment, and the smart pin **ORs** the incoming buses from all cogs together — the same way DIR and OUT bits are OR'd before reaching a pin. So if two or more cogs issue **WRPIN, WXPIN, WYPIN, RDPIN, or AKPIN** to the *same* smart pin simultaneously, their bus data collides and corrupts. **RQPIN is the lone exception** — it does not use the acknowledge bus, so any number of cogs may RQPIN the same smart pin at once. Design multi-cog access so that only one cog configures and acknowledges a given smart pin; the others observe with RQPIN.

## 3.4 The Smart Pin State Machine

Every smart pin follows a consistent state progression:



**Figure 3.1.** Smart pin lifecycle

### State 1: Disabled (DIR=0)

When DIR=0, the smart pin is held in reset:

- The smart pin state machine is stopped
- IN is forced low
- Registers are initialized to mode-specific values
- Configuration instructions (WRPIN/WXPIN/WYPIN) are accepted

**Important:** Always issue WRPIN (mode/routing configuration) while DIR=0 — changing the mode word while the pin is running (DIR=1) can cause unpredictable behavior. WXPIN and WYPIN are different: they are the normal runtime update path and work freely while the smart pin is running (DIR=1).

### State 2: Configured

Configuration consists of up to three instructions:

1. **WRPIN** - Sets the mode and low-level pin configuration
2. **WXPIN** - Sets X register parameters
3. **WYPIN** - Sets Y register parameters (if mode uses Y)

Not all modes require all three. Some modes only need WRPIN; others need WRPIN + WXPIN; complex modes use all three.

### State 3: Enabled (DIR=1)

When DIR transitions from 0 to 1:

- The smart pin begins autonomous operation
- The state machine starts running
- Mode-specific behavior commences

The direction can be set via:



- DIRH - Set direction high (output mode)
- DRVH/DRVL - Set direction high with specific output state
- Note: For smart pins with P\_OE set, output is enabled regardless of DIR

## State 4: Running

The smart pin operates autonomously:

- Generates outputs according to mode
- Measures inputs according to mode
- Updates Z register with results
- Raises IN when events occur

Cog interaction during running:

- Read results via RDPIN/RQPIN
- Update parameters via WXPIN/WYPIN
- Monitor events via TESTP

## Reset

To reset a smart pin without reconfiguring:

1. Clear DIR (DIRL or FLTL)
2. Set DIR (DIRH or DRVH/DRVL)

This restarts the smart pin with current configuration. No need to repeat WRPIN/WXPIN/WYPIN.

To completely disable and return to normal mode:

- Execute WRPIN with value 0 (or use PINCLEAR in Spin2)

## 3.5 Mode Bits and the 32 Modes

### Mode Selection via WRPIN

The smart pin mode is selected by bits [5:1] of the WRPIN D operand (bit 0 is always 0). With 5 bits, there are 32 possible modes (0-31, or %00000 through %11111).

| Mode Bits     | Category                     |
|---------------|------------------------------|
| %00000        | P_NORMAL - No Smart Pin mode |
| %00001-%00011 | Repository / DAC modes       |
| %00100-%00101 | Pulse and Transition output  |
| %00110-%00111 | NCO frequency and duty       |
| %01000-%01010 | PWM modes                    |
| %01011        | Quadrature encoder           |
| %01100-%01111 | Counter modes                |
| %10000-%10010 | Timing measurement modes     |
| %10011-%10111 | Period/frequency measurement |
| %11000-%11010 | ADC modes                    |
| %11011        | USB                          |
| %11100-%11111 | Serial TX/RX modes           |

### Mode Constants

Spin2 and PASM2 provide named constants for each mode (P\_NCO\_FREQ, P\_PWM\_TRIANGLE, etc.). These constants have the mode bits properly positioned within the 32-bit WRPIN value.

### Mode Categories

**Output Modes (Chapters 6-11):** Generate signals on the pin—pulses, waveforms, serial data.

**Input Modes (Chapters 12-17):** Measure signals on the pin—timing, counting, frequency, analog levels.

**Special Modes (Chapters 18-19):** Inter-cog data sharing (Repository) and USB.

## 3.6 The Layered Configuration Model

Smart pin configuration is layered. Multiple aspects combine to define complete behavior:

### Layer 1: Smart Pin Mode (bits [5:1])

Selects which of the 32 modes is active. Each mode defines fundamental behavior (PWM, ADC, serial, etc.).

### Layer 2: Low-Level Pin Configuration (bits [20:8])

Controls the analog characteristics of the pin:

- Input mode (logic, Schmitt, comparator, ADC)
- Drive strength (resistive or current source options)
- DAC configuration (when applicable)

These are the same settings documented in Chapter 2 (Enhanced Direct I/O).

### Layer 3: Input Routing (bits [31:24])

Selects input sources:

- A input source: local pin, adjacent pins (-3 to +3), or OUT bit
- B input source: local pin, adjacent pins (-3 to +3), or OUT bit
- Input polarity: true or inverted
- Input logic: pass, AND, OR, XOR, or filter

### Layer 4: DIR/OUT Control (bits [7:6])

The TT bits control output behavior:

- P\_OE (%01): Output enabled regardless of DIR
- Without P\_OE: DIR controls output enable

For smart pin output modes, P\_OE is required.

## Combining Layers

Configuration constants from different layers are combined with OR:

```
mode := P_NCO_FREQ | P_OE | P_HIGH_FAST | P_LOW_FAST
```

This combines:

- Smart pin mode (P\_NCO\_FREQ)
- Output enable (P\_OE)
- Drive strength (P\_HIGH\_FAST, P\_LOW\_FAST)

## 3.7 When to Use Smart Pins

### Smart Pins Excel At

#### **Precise timing requirements:**

- Clock generation with exact frequencies
- PWM with consistent timing unaffected by software
- Pulse measurement with clock-cycle accuracy

#### **Continuous autonomous operation:**

- Free-running oscillators
- Ongoing signal measurement
- Serial communication without polling

#### **High-frequency signals:**

- MHz-range waveforms (limited only by sysclk)
- High baud-rate serial
- Fast ADC sampling

#### **Multi-channel parallel operation:**

- Every pin can run independently
- 64 simultaneous smart pin operations possible

### Direct I/O May Be Better For

#### **Simple on/off control:**

- LEDs, relays, simple outputs
- Occasional pin reads

#### **One-time or irregular operations:**

- Configuration signals
- Status reads

#### **Complex conditional logic:**

- Where software decision-making determines output

- Irregular patterns that don't FIT smart pin modes

**Low pin count applications:**

- When cog cycles are abundant
- When flexibility outweighs hardware efficiency

**Hybrid Approaches**

Many applications combine smart pins with Direct I/O:

- Smart pin for timing-critical signals (PWM, serial)
- Direct I/O for control signals (enable, reset, status)

## 3.8 Architectural Constraints

**One Mode Per Pin**

Each pin can run exactly one smart pin mode at a time. To change modes:

1. Disable the smart pin (DIR=0)
2. Reconfigure with WRPIN
3. Re-enable (DIR=1)

**Z Register is Read-Only to Software**

Software cannot directly write the Z register. Z is managed entirely by smart pin hardware. To “preset” a counter or phase, use the mode-specific mechanisms (often via X or Y registers, or by reset timing).

**Acknowledge Timing**

The 2-clock delay after acknowledge means polling loops must account for this latency. Tight loops polling IN immediately after RDPIN will miss the first event.

**Mode-Specific Behaviors**

Each mode has unique characteristics:

- Which registers are used
- When IN is raised
- What Z contains
- How reset behaves

These details are documented in each mode's chapter.

## 3.9 Chapter Summary

Smart pins provide autonomous I/O operations through:

1. **Three registers** (X, Y, Z) for configuration, input, and results
2. **The IN bit** for event signaling
3. **A state machine** progressing from disabled through configured to running
4. **32 modes** selected by bits [5:1] of WRPIN
5. **Layered configuration** combining mode, pin settings, input routing, and output control

The key insight: once configured and enabled, smart pins operate independently. The cog is free to perform other work, interacting with the smart pin only to read results or update parameters.

*This conceptual foundation applies to all smart pin modes. Proceed to Chapter 4 for the practical configuration process, or to Part II (Chapters 6-11) for specific output modes.*

# Chapter 4: Smart Pin Configuration

This chapter documents the instructions and methods for configuring and interacting with smart pins. The configuration instructions—WRPIN, WXPIN, WYPIN—establish smart pin behavior. The read instructions—RDPIN, RQPIN—retrieve results. The acknowledge instruction—AKPIN—signals the smart pin without reading.

## 4.1 Configuration Instructions Overview

| Instruction  | Purpose                        | Effect on IN             |
|--------------|--------------------------------|--------------------------|
| <b>WRPIN</b> | Set mode and pin configuration | Acknowledges (lowers IN) |
| <b>WXPIN</b> | Set X register parameters      | Acknowledges (lowers IN) |
| <b>WYPIN</b> | Set Y register parameters      | Acknowledges (lowers IN) |
| <b>RDPIN</b> | Read Z register                | Acknowledges (lowers IN) |
| <b>RQPIN</b> | Read Z register quietly        | Does NOT acknowledge     |
| <b>AKPIN</b> | Acknowledge only               | Acknowledges (lowers IN) |

All configuration and acknowledge instructions execute in 2 clock cycles.

## 4.2 WRPIN - Write Pin Configuration

### Function

WRPIN establishes the complete pin configuration including:

- Smart pin mode selection
- Low-level pin configuration (drive strength, input mode)
- Input routing and polarity
- DIR/OUT control options

#### PASM2 Syntax

```
WRPIN    {#}D, {#}S
```

- **D**: 32-bit configuration value
- **S**: Pin number (0-63) or pin field with span

#### Spin2 Syntax

```
WRPIN(PinField, Mode)
```

- **PinField:** Single pin, range, or ADDPINS expression
- **Mode:** 32-bit configuration value (P\_ constants OR'd together)

## Configuration Value Format

The D operand is a 32-bit value divided into the fields below. Each field selects one aspect of pin behavior; a configuration is built by OR-ing the P\_ constants for the needed fields.

| 31:28 | 27:24 | 23:21 | 20:8           | 7:6 | 5:1   | 0 |
|-------|-------|-------|----------------|-----|-------|---|
| AAAA  | BBBB  | FFF   | MMMMMMMMMMMMMM | TT  | SSSSS | 0 |

|                |       |                                             |
|----------------|-------|---------------------------------------------|
| AAAA           | 31:28 | A input selection and polarity              |
| BBBB           | 27:24 | B input selection and polarity              |
| FFF            | 23:21 | A, B input logic and filtering              |
| MMMMMMMMMMMMMM | 20:8  | Low-level pin mode (drive, input, DAC, ADC) |
| TT             | 7:6   | DIR/OUT control                             |
| SSSSS          | 5:1   | Smart pin mode (0–31)                       |
| 0              | 0     | Reserved (always 0)                         |

**Figure 4.1.** WRPIN configuration value — 32-bit field map

## Timing

- Execution: 2 clock cycles
- After WRPIN, 2 additional clocks must elapse before IN can be polled

## Effect

1. Pin is configured according to the D value
2. IN bit is acknowledged (lowered)
3. If DIR=0, smart pin remains in reset state
4. If DIR=1 and mode changes, behavior is unpredictable (always configure while DIR=0)

## Critical Requirements

**Configure while DIR=0:** Smart pins must be configured while held in reset (DIR=0). The proper sequence is:

1. DIRM to reset smart pin
2. WRPIN to configure
3. WXPIN/WYPIN as needed
4. DRVH/DRVL to enable



**Reset to normal mode:** To return a pin to Direct I/O mode:

```
WRPIN    ##0, pin    ' Reset to P_NORMAL
```

## Examples

**Spin2 - Configure NCO frequency mode:**

```
WRPIN(pin, P_NCO_FREQ | P_OE)    ' NCO mode with output enable
```

**PASM2 - Configure NCO frequency mode:**

```
WRPIN    ##(P_NCO_FREQ | P_OE), pin
```

**Spin2 - Configure with drive strength:**

```
WRPIN(pin, P_PWM_TRIANGLE | P_OE | P_HIGH_FAST | P_LOW_FAST)
```

## 4.3 WXPIN - Write X Register

### Function

WXPIN writes the X register, which holds configuration parameters. The meaning of X varies by mode.

#### PASM2 Syntax

```
WXPIN    {##}D, {##}S
```

- **D:** Value to write to X register
- **S:** Pin number (0-63) or pin field with span

#### Spin2 Syntax

```
WXPIN(PinField, Xvalue)
```

## Common X Register Uses

| Mode Category       | X[15:0]              | X[31:16]              |
|---------------------|----------------------|-----------------------|
| Timing output modes | Base period (clocks) | Frame period or phase |
| Counter modes       | Measurement window   | -                     |
| Serial TX           | Bit period           | Data format           |
| Serial RX           | Bit period           | Data format           |
| ADC modes           | Sample period/mode   | -                     |

## Timing

- Execution: 2 clock cycles
- Acknowledges IN (2-clock delay before polling)

## Special Behavior

Some modes capture X[31:16] to Z[31:16] upon WXPIN, allowing phase initialization (NCO modes, for example).

## Examples

### Spin2 - Set base period:

```
WXPIN(pin, base_period)      ' X = base_period
```

### PASM2 - Set base period with frame count:

```
WXPIN      x_value, pin      ' X[15:0] = base, X[31:16] = frame
```

## 4.4 WYPIN - Write Y Register

### Function

WYPIN writes the Y register, which holds input data or secondary parameters.

#### PASM2 Syntax

```
WYPIN      {#}D, {#}S
```

- **D**: Value to write to Y register
- **S**: Pin number (0-63) or pin field with span

### Spin2 Syntax

WYPIN(PinField, Yvalue)

## Common Y Register Uses

| Mode Category     | Y Usage                   |
|-------------------|---------------------------|
| PWM modes         | Duty cycle value          |
| NCO modes         | Frequency/phase increment |
| DAC modes         | Output level              |
| Serial TX         | Data to transmit          |
| Transition output | Number of transitions     |
| Counter modes     | Mode modifier (Y[0])      |

## Timing

- Execution: 2 clock cycles
- Acknowledges IN (2-clock delay before polling)

## Capture Behavior

Many modes capture Y on specific events (frame start, period end). Writing Y immediately before the capture point ensures the new value is used.

## Examples

### Spin2 - Set PWM duty:

```
WYPIN(pin, duty_value)      ' Y = duty cycle
```

### PASM2 - Send serial data:

```
WYPIN    data, tx_pin      ' Y = data to transmit
```

## 4.5 RDPIN - Read Z Register with Acknowledge

### Function

RDPIN reads the Z register and acknowledges the smart pin (lowers IN).

#### PASM2 Syntax

```
RDPIN    D, {#}S    {WC}
```

- **D**: Destination register for Z value
- **S**: Pin number (0-63)

#### Spin2 Syntax

```
result := RDPIN(Pin)
```

### Effect

1. Z register value is read to D
2. C flag receives mode-specific flag (often Z[31] or event indicator)
3. IN bit is acknowledged (lowered)

### Timing

- Execution: 2 clock cycles
- After RDPIN, 2 additional clocks before IN can be polled again

### Z Register Content by Mode

| Mode Category     | Z Contains                |
|-------------------|---------------------------|
| Measurement modes | Accumulated count or time |
| Counter modes     | Event count               |
| ADC modes         | Conversion result         |
| Serial RX         | Received data             |
| NCO modes         | Phase accumulator         |

## When to Use RDPIN

Use RDPIN when:

- The cog needs the result AND
- The smart pin should be signaled that the result was consumed

This is the normal read operation for single-cog access.

## Examples

### Spin2 - Read measurement:

```
measurement := RDPIN(pin)           ' Read Z, acknowledge
```

### PASM2 - Read and check flag:

```
                RDPIN      result, #pin wc  ' Read Z, C = flag
if_c JMP        #handle_event  ' Act on flag
```

## 4.6 RQPIN - Read Z Register Quietly

### Function

RQPIN reads the Z register WITHOUT acknowledging the smart pin. IN remains in its current state.

#### PASM2 Syntax

```
RQPIN    D, {#}S      {WC}
```

#### Spin2 Syntax

```
result := RQPIN(Pin)
```

### Effect

1. Z register value is read to D
2. C flag receives mode-specific flag
3. IN bit is NOT affected (no acknowledge)

## When to Use RQPIN

**Multi-cog observation:** When multiple cogs need to read the same smart pin's result, only one should use RDPIN; others use RQPIN to avoid acknowledging multiple times. This matters because WRPIN/WXPIN/WYPIN/RDPIN/AKPIN all share the OR'd 34-bit smart pin bus and collide if two cogs issue them to the same pin at once — RQPIN is the one access that does not use that bus (see the multi-cog caution in §3.3).

**Non-destructive peek:** When checking results without signaling consumption.

**Continuous modes:** Some modes (like totalizer counters) benefit from RQPIN for intermediate reads while RDPIN resets for the next period.

## Example - Multi-Cog Access

```
' Cog 0 (primary) uses RDPIN
    RDPIN    result, #sensor ' Read and acknowledge

' Cog 1 (observer) uses RQPIN
    RQPIN    result, #sensor ' Read without acknowledge
```

## 4.7 AKPIN - Acknowledge Only

### Function

AKPIN acknowledges the smart pin without reading the Z register.

#### PASM2 Syntax

```
AKPIN    {#}Src
```

- **S:** Pin number (0-63) or pin field

### Spin2 Equivalent

There is no direct Spin2 equivalent. Use RDPIN with a discard variable:

```
ack := RDPIN(pin)           ' Read (discard result) to acknowledge
```

Or configure in PASM2 if needed.

## When to Use AKPIN

- Resetting the IN flag without needing the data
- Synchronizing smart pin timing without data consumption
- Discarding an unwanted result

## Example

```
AKPIN    #pin    ' Acknowledge without reading
```

## 4.8 The Standard Configuration Sequence

All smart pin modes follow a common configuration pattern:

### Step 1: Reset the Smart Pin

```
PINFLOAT(pin)    ' DIR=0, hold in reset
' or
PINF(pin)        ' Same effect (short form of PINFLOAT)
```

```
DIRL    #pin    ' Reset Smart Pin
```

### Step 2: Configure Mode (WRPIN)

```
WRPIN(pin, mode | P_OE | ...)    ' Set mode and options
```

```
WRPIN    ##(mode | P_OE), #pin
```

### Step 3: Set Parameters (WXPIN)

```
WXPIN(pin, x_value)    ' Set X register
```

```
WXPIN    x_value, #pin
```

## Step 4: Set Data/Secondary Parameters (WYPIN) - If Needed

```
WYPIN(pin, y_value)           ' Set Y register
```

```
WYPIN      y_value, #pin
```

## Step 5: Enable Smart Pin

```
PINLOW(pin)                   ' DIR=1, start Smart Pin
' or
PINHIGH(pin)                  ' DIR=1, start Smart Pin
```

```
          DRVL      #pin      ' Enable Smart Pin
' or
          DRVH      #pin      ' Enable Smart Pin
```

**Note:** For output modes, DRVL vs DRVH doesn't affect the smart pin output (which is controlled by the mode). Use whichever is appropriate for the pre-enabled output state.

## Complete Example - NCO Frequency

**Spin2:**

```
CON
  _clkfreq = 200_000_000
  OUT_PIN = 10
  TARGET_FREQ = 1000           ' 1 kHz output

PUB setup_nco() | y_value
  ' Calculate Y for target frequency
  ' frequency = (Y × sysclk) / 2^32
  ' Y = (frequency × 2^32) / sysclk
  y_value := TARGET_FREQ FRAC _clkfreq

  ' Configuration sequence
  PINFLOAT(OUT_PIN)           ' Step 1: Reset
  WRPIN(OUT_PIN, P_NCO_FREQ | P_OE) ' Step 2: Mode
  WXPIN(OUT_PIN, 1)           ' Step 3: Base period = 1 clock
  WYPIN(OUT_PIN, y_value)     ' Step 4: Frequency value
  PINLOW(OUT_PIN)             ' Step 5: Enable
```

The FRAC operator computes  $(\text{operand1} \times 2^{32}) / \text{operand2}$  with a 64-bit intermediate (no overflow) — here it scales TARGET\_FREQ into the NCO frequency word.



**PASM2:**

```

DIRL      #OUT_PIN      ' Step 1: Reset
WRPIN     ##(P_NCO_FREQ | P_OE), #OUT_PIN  ' Step 2: Mode
WXPIN     #1, #OUT_PIN   ' Step 3: Base period = 1
WYPIN     y_val, #OUT_PIN ' Step 4: Frequency
DRV_L     #OUT_PIN      ' Step 5: Enable

```

## 4.9 P\_OE - Output Enable

### Purpose

The P\_OE constant (TT bits = %01) enables smart pin output regardless of the DIR bit state.

### When P\_OE is Required

**Output modes:** All smart pin modes that produce output require P\_OE:

- NCO frequency/duty (%00110, %00111)
- PWM modes (%01000, %01001, %01010)
- Pulse/Transition (%00100, %00101)
- Serial TX (%11100, %11110)
- DAC modes (%00001, %00010, %00011 in DAC mode)
- USB (%11011)

**Without P\_OE:** The smart pin calculates output but doesn't drive the pin. This can be useful for:

- Preparing output before enabling
- Running the mode for internal timing without external output

### When P\_OE is Not Needed

**Input-only modes:** Modes that only measure input don't need P\_OE:

- All timing measurement modes (%10000-%10010)
- Quadrature and counter modes (%01011-%01111) unless driving output
- Period/frequency modes (%10011-%10111)
- ADC modes (%11000-%11010)
- Serial RX (%11101, %11111)

## Including P\_OE

```
WRPIN(pin, P_NCO_FREQ | P_OE)      ' Output enabled
WRPIN(pin, P_NCO_FREQ)              ' Output NOT enabled (internal only)
```

## 4.10 Input Routing

Smart pins draw their A and B inputs using the same input-routing constants introduced for Enhanced Direct I/O in §2.4: `P_LOCAL_A/P_PLUS1_A...P_MINUS1_A` (and the `_B` equivalents) select the source pin, `P_TRUE_A/P_INVERT_A` set the polarity, and `P_PASS_AB/P_AND_AB/P_OR_AB/P_XOR_AB/P_FILTER0_AB...P_FILTER3_AB` combine the A and B inputs before use. The A input is the primary input for most modes; the B input carries secondary signals (clock, quadrature channel B, etc.). See §2.4 for the full constant tables.

When a pin is **not** in a smart pin mode, the A result produced here (after this logic and any filtering) is what drives the pin's IN signal. So these combinations — and the `P_FILTERx_AB` options — also shape the value an ordinary TESTP/IN read sees on a plain direct-I/O pin, not just the input to a smart pin.

### Example - Quadrature Encoder

Quadrature encoder uses two input channels (A and B):

```
' Pin 10 = A input (local)
' Pin 11 = B input (pin + 1)
WRPIN(10, P_QUADRATURE | P_PLUS1_B)  ' A = pin 10, B = pin 11
WXPIN(10, 0)                        ' Continuous measurement
PINLOW(10)                          ' Enable
```

### Example - External Clock

Synchronous serial RX with external clock on adjacent pin:

```
' Pin 20 = data (A input, local)
' Pin 21 = clock (B input, pin + 1)
WRPIN(20, P_SYNC_RX | P_PLUS1_B)    ' A = data, B = clock
WXPIN(20, bit_config)               ' Configure bit format
PINLOW(20)                          ' Enable
```

## 4.11 Span Operations

Smart pin instructions operate on a span of pins exactly as the Direct I/O instructions do (§1.9), with one difference: the span travels in the **S** operand (the pin-number operand) rather than the D operand. S[5:0] is the base pin and S[10:6] the additional-pin count, set inline or via a preceding SETQ; as always, a span wraps within its 32-pin port. See §1.9 for the full span model.

### Spin2 Pin Ranges

```
WRPIN(0..7, P_NCO_FREQ | P_OE)      ' Configure pins 0-7
WXPIN(0..7, period)                  ' Set X for pins 0-7
```

## 4.12 Reading the C Flag

RDPIN and RQPIN set the C flag based on mode-specific information:

| Mode Category     | C Flag Meaning     |
|-------------------|--------------------|
| NCO modes         | Z[31] (phase MSB)  |
| Measurement modes | State indicator    |
| Counter modes     | Overflow indicator |
| Serial RX         | Parity or error    |

### Checking C After Read

```
        RDPIN      result, #pin wc      ' Read Z, C = flag
if_c    JMP        #handle_flag
```

```
result := RDPIN(pin)
if result & $8000_0000      ' Check bit 31 (mode-dependent)
    ' Handle condition
```

## 4.13 The 2-Clock Acknowledge Delay

After any instruction that acknowledges the smart pin (WRPIN, WXPIN, WYPIN, RDPIN, AKPIN), two clock cycles must elapse before IN can be polled:

|       |              |                                  |
|-------|--------------|----------------------------------|
| RDPIN | result, #pin | ' Acknowledge Smart Pin          |
| NOP   |              | ' Wait 2 clocks (NOP = 2 clocks) |
| TESTP | #pin wc      | ' Now safe to poll IN            |

In practice, other instructions between the acknowledge and the poll often provide sufficient delay.

## 4.14 Configuration Quick Reference

### Minimum Configuration (Mode Only)

```
PINFLOAT(pin)
WRPIN(pin, mode | P_OE)
PINLOW(pin)
```

### Standard Configuration (Mode + X)

```
PINFLOAT(pin)
WRPIN(pin, mode | P_OE)
WXPIN(pin, x_value)
PINLOW(pin)
```

### Full Configuration (Mode + X + Y)

```
PINFLOAT(pin)
WRPIN(pin, mode | P_OE)
WXPIN(pin, x_value)
WYPIN(pin, y_value)
PINLOW(pin)
```

### Reconfiguration (Change Mode)

|                             |                  |
|-----------------------------|------------------|
| PINFLOAT(pin)               | ' Reset first    |
| WRPIN(pin, NEW_MODE   P_OE) | ' New mode       |
| WXPIN(pin, new_x)           | ' New parameters |
| WYPIN(pin, new_y)           |                  |
| PINLOW(pin)                 | ' Re-enable      |

## Reset Without Reconfiguration

|               |                           |
|---------------|---------------------------|
| PINFLOAT(pin) | ' Reset                   |
| PINLOW(pin)   | ' Re-enable (same config) |

## Return to Direct I/O

|               |                     |
|---------------|---------------------|
| PINCLEAR(pin) | ' Reset to P_NORMAL |
| ' or          |                     |
| PINFLOAT(pin) |                     |
| WRPIN(pin, 0) |                     |

WRPIN(pin, 0) clears a smart pin to P\_NORMAL **at any time, including while it is running** — no DIRL /DIRH cycle is required. The reset-before-configure rule (§4.2) applies when *changing* to another active mode; returning to direct I/O with #0 takes effect immediately.

*This chapter covers the mechanics of smart pin configuration. For specific mode behaviors, see the mode chapters in Parts II-IV. For common usage patterns and debugging, see Chapter 5.*

# Chapter 5: Working with Smart Pins

This chapter covers practical patterns for smart pin operation, debugging techniques, and common troubleshooting scenarios. The concepts here apply across all smart pin modes documented in Parts II through IV.

## 5.1 The Read/Acknowledge Cycle

### How IN and Acknowledge Work

When a smart pin event occurs (measurement complete, data ready, etc.), the smart pin raises its IN flag. This signals to cogs that attention is needed.

**Acknowledging instructions** (WRPIN, WXPIN, WYPIN, RDPIN, AKPIN) lower the IN flag, signaling to the smart pin that the event was handled. This allows IN to be raised again for the next event.

**Non-acknowledging read** (RQPIN) reads the result without lowering IN.

### Polling Patterns

**Check-then-read (recommended):**

```
repeat
  if PINREAD(pin) == 1          ' IN high?
    result := RDPIN(pin)        ' Read and acknowledge
  ' Process result
```

```
.loop      TESTP      #pin wc      ' Check IN
          if_nc JMP      #.loop      ' Wait if low
          RDPIN      result, #pin    ' Read and acknowledge
          ' Process result
          JMP        #.loop
```

**Blocking wait (PASM2):**

```
          TESTP      #pin wc      ' Check IN
          if_nc JMP      #$.1      ' Tight loop until high
          RDPIN      result, #pin  ' Read result
```

**Time-limited wait:**

```

start := GETCT()
repeat until PINREAD(pin) == 1 OR (GETCT() - start) > timeout
if PINREAD(pin) == 1
    result := RDPIN(pin)
else
    ' Handle timeout

```

**Waiting Strategies**

Every pattern above keeps the cog **executing** — the poll-spin loops on TESTP/PINREAD, and the time-limited wait re-reads GETCT() on each pass. That burns instruction cycles (and power) for the whole wait. When a cog has nothing to do until the smart pin is ready, the P2's event system offers a true **stall**: the cog halts and resumes the instant the pin acts. These are PASM2 patterns; Spin2 code reaches them through inline PASM.

**Blocking wait via the event system (the true stall).** A selectable event (SE1–SE4, four per cog) can watch a pin's IN flag. SETSE1 arms it for the rising edge of IN; WAITSE1 then halts the cog — no instructions execute — until that edge occurs:

```

.wait          SETSE1    #%001<<6 + pin    ' Arm SE1 on IN rising edge

              WAITSE1                    ' Cog halts until IN rises
              RDPIN      result, #pin      ' Read + ack (lowers IN)
              JMP        #.wait

```

WAITSE1 auto-clears the SE1 flag as it releases, so the next WAITSE1 waits for the next edge. An acknowledging read (RDPIN) is still required to retrieve the result and lower IN. The four slots are independent, letting one cog track up to four sources — but each WAITSE waits on exactly one.

**Wait with a timeout (never hang).** WAITSE1 stalls *indefinitely* — if the smart pin never completes, the cog never wakes. To bound the wait, arm a system-counter deadline for the wait *itself*: a SETQ holding a future CT target, placed immediately before WAITSE1, makes that one stalling instruction release on **whichever comes first** — the pin event or the deadline — and report which in a flag. The cog still truly stalls; no cycles or power are spent waiting:

```

              GETCT      deadline          ' Now
              ADD        deadline, ##timeout ' Deadline = now + wait
              SETSE1     #%001<<6 + pin    ' Arm SE1 on IN rising edge
              SETQ       deadline          ' Arm CT timeout (next instr)
              WAITSE1    wc                ' Stall for event OR timeout
if_c          JMP        #.timedout        ' C=1 timeout, C=0 event
              RDPIN      result, #pin      ' Event won: read + ack

.timedout
              ' Handle the timeout

```

The SETQ arms the timeout for the single instruction that follows it; WAITSE1 WC then sets c if the deadline arrived first, or clears it if the pin event did. (c and z carry the same timeout result, so one flag is all you need.) This keeps the zero-cost stall of a plain WAITSE1 while guaranteeing the cog can never hang. The same SETQ-then-wait timeout works for every event wait — WAITSE1–WAITSE4, WAITCT1–WAITCT3, WAITPAT, WAITATN, and the rest.

**If you need to do other work while waiting**, poll the event against the counter in a loop instead of stalling, branching on whichever fires first:

```

        GETCT    deadline      ' Read current time
        ADDCT1   deadline, ##timeout  ' Deadline = now + wait
        SETSE1   #%001<<6 + pin    ' Arm SE1 on IN rising edge

    .race
        POLLSE1  wc             ' Pin ready?
        if_c     JMP          #.ready    ' Yes - go read it
        POLLCT1  wc             ' Timeout reached?
        if_nc    JMP          #.race     ' Neither yet - keep polling
        JMP      #.timedout          ' Timed out

    .ready
        RDPIN    result, #pin        ' Pin won the race

    .timedout
        ' Handle the timeout

```

ADDCT1 sets counter-comparator 1 to a deadline; POLLCT1 WC reports (and clears) whether that deadline has passed, exactly as POLLSE1 WC does for the pin event. This costs a few instructions per pass and keeps the cog running — use it when you have real work to do between checks; otherwise prefer the SETQ-armed stall above. For background servicing, that same SE1 event can instead drive an interrupt (via SETINT1), freeing the cog to run other code between events.

**Let the smart pin time itself out.** Several input modes carry the timeout in hardware, removing the software race entirely. P\_EVENTS\_TICKS (mode %10010) with Y[2] = 1 raises IN either when the event arrives *or* after X clocks with no event (Chapter 13), so a single WAITSE1 covers both outcomes — read the result, then decide whether it was a real event or a timeout. The windowed measurement modes (%10101–%10111, Chapter 15) instead raise IN after a fixed number of clocks, giving a “wait exactly this long, then read” cadence. When one of these fits, prefer it: the blend is done in silicon at zero cog cost.



## Checking Without Clearing

To inspect the IN state without affecting it:

```
state := PINREAD(pin)           ' Just checks, doesn't acknowledge
```

```
TESTP      #pin wc             ' Checks IN, no acknowledge
```

To read the Z value without acknowledging:

```
value := RQPIN(pin)           ' Read quietly
```

## The 2-Clock Delay

After acknowledging, wait 2 clocks before polling IN again:

```
RDPIN      result, #pin       ' Acknowledge
NOP        ' Wait 2 clocks (NOP = 2 clocks)
TESTP      #pin wc           ' Safe to poll
```

Processing between reads provides sufficient delay.

## 5.2 Continuous vs One-Shot Modes

### Continuous Modes

These modes run indefinitely once enabled, producing periodic output or ongoing measurements:

| Mode                  | Behavior                                     |
|-----------------------|----------------------------------------------|
| NCO Frequency/Duty    | Runs continuously, IN raised on overflow     |
| PWM Triangle/Sawtooth | Runs continuously, IN raised each frame      |
| Quadrature Encoder    | Runs continuously (or periodically with X>0) |
| Counter modes (X=0)   | Totalizer mode, counts indefinitely          |
| ADC modes             | Samples continuously at configured rate      |

#### Using continuous modes:

- Configure once
- Read results as needed (RDPIN/RQPIN)
- Update parameters anytime (WYPIN for new values)

- Mode runs until DIR cleared or reconfigured

## Periodic Modes

These modes repeat automatically but generate periodic events:

| Mode                | Period Control               |
|---------------------|------------------------------|
| Counter modes (X>0) | X defines measurement window |
| Period measurement  | X defines number of periods  |
| Serial TX/RX        | Operates per data word       |

### Using periodic modes:

- Each period, IN is raised
- RDPIN retrieves period result and starts next period
- Missing reads can cause data loss (results overwritten)

## One-Shot Modes

These modes complete a defined action then stop:

| Mode               | Behavior                          |
|--------------------|-----------------------------------|
| Pulse/Cycle Output | Outputs Y pulses, then stops      |
| Transition Output  | Outputs Y transitions, then stops |

### Using one-shot modes:

- Configure and enable
- Wait for IN (operation complete)
- Write new Y value to restart
- Or reconfigure for new parameters

### Restarting one-shot:

```
' Wait for completion
repeat until PINREAD(pin) == 1
result := RDPIN(pin)           ' Acknowledge

' Start new operation
WYPIN(pin, new_count)          ' New Y value triggers restart
```

## 5.3 Multi-Pin Patterns

### Configuring Pin Groups

Use pin ranges for identical configuration:

```
' Configure pins 0-7 for PWM
WRPIN(0..7, P_PWM_TRIANGLE | P_OE)
WXPIN(0..7, base_period | (frame << 16))
PINLOW(0..7)

' Set individual duty cycles
WYPIN(0, duty_0)
WYPIN(1, duty_1)
' ...
```

```
SETQ      #7          ' 8 pins
WRPIN     ##(P_PWM_TRIANGLE | P_OE), #0
SETQ      #7
WXPIN     x_value, #0
SETQ      #7
DRVL      #0          ' Enable all 8
```

### Relative Pin Addressing

Smart pins can use adjacent pins for input:

**Quadrature encoder (A on pin N, B on pin N+1):**

```
WRPIN(encoder_pin, P_QUADRATURE | P_PLUS1_B)
```

**Synchronous serial (data on pin N, clock on pin N+1):**

```
WRPIN(data_pin, P_SYNC_RX | P_PLUS1_B)
```

**Comparator (compare pin N to pin N+1):**

```
WRPIN(comp_pin, P_COMPARE_AB | P_PLUS1_B)
```

## Synchronized Multi-Pin Output

For phase-synchronized outputs (audio, motor control):

1. Configure all pins with same base period
2. Use NCO mode with appropriate phase offsets in X[31:16]
3. Enable all simultaneously

```
' Three-phase motor control
WRPIN(phase_a, P_NCO_FREQ | P_OE)
WRPIN(phase_b, P_NCO_FREQ | P_OE)
WRPIN(phase_c, P_NCO_FREQ | P_OE)

' Same frequency, different phases (0°, 120°, 240°)
WXPIN(phase_a, 1 | (0 << 16))      ' Phase = 0
WXPIN(phase_b, 1 | (21845 << 16))  ' Phase ≈ 120°
WXPIN(phase_c, 1 | (43690 << 16))  ' Phase ≈ 240°

' Same frequency value
WYPIN(phase_a, freq_value)
WYPIN(phase_b, freq_value)
WYPIN(phase_c, freq_value)

' Drive all pins low simultaneously for coordinated startup
PINLOW(phase_a..phase_c)
```

## Multi-Cog Access

When multiple cogs need the same smart pin data:

**Pattern: One owner, multiple observers**

```
' Cog 0 - Owner (uses RDPIN)
      TESTP      #sensor wc
      if_c RDPIN      result, #sensor  ' Read and acknowledge

' Cog 1..N - Observers (use RQPIN)
      RQPIN      result, #sensor  ' Read without acknowledge
```

The owner controls the timing; observers passively read.

## 5.4 PINSTART and PINCLEAR

### PINSTART - One-Call Configuration

PINSTART combines WRPIN, WXPIN, WYPIN, and enable into one CALL:

#### Spin2 Syntax

```
PINSTART(Pin, Mode, Xval, Yval)
```

#### Example:

```
' Instead of:
PINFLOAT(pin)
WRPIN(pin, P_NCO_FREQ | P_OE)
WXPIN(pin, 1)
WYPIN(pin, freq)
PINLOW(pin)

' Use:
PINSTART(pin, P_NCO_FREQ | P_OE, 1, freq)
```

### When PINSTART Helps

- Quick setup of known configurations
- Reducing code size
- Prototyping

### When to Use Raw Configuration

- Mode doesn't need all three registers
- Partial reconfiguration (WYPIN alone, for example)
- Precise control over enable timing
- PASM2 code (PINSTART is Spin2 only)

### PINCLEAR - Reset to Normal

PINCLEAR(pin) disables smart pin mode and returns the pin to Direct I/O — equivalent to PINFLOAT(pin) followed by WRPIN(pin, 0). See §4.14 for the complete reset-to-normal reference.

## 5.5 Debugging Smart Pins

### Verifying Configuration

**Check that mode is active:**

```
' After configuration, wait for first IN
repeat 1000                                ' Timeout after many loops
  if PINREAD(pin) == 1
    result := RDPIN(pin)
    DEBUG("Smart Pin active, first result: ", UDEC_(result))
  quit
DEBUG("Smart Pin NOT responding")
```

**Inspect Z register:**

```
value := RQPIN(pin)                        ' Read without disturbing
DEBUG("Z register: ", UHEX_(value))
```

### Common Configuration Errors

**No output (output modes):**

- Missing P\_OE in WRPIN
- DIR still low (not enabled)
- WRPIN value has wrong mode bits

**No events (IN never goes high):**

- Mode not enabled (DIR=0)
- X register has invalid period (0 when period required)
- Mode waiting for input that isn't present

**Wrong timing:**

- X register calculation error
- Using wrong clock frequency assumption
- Frame period vs base period confusion

**Erratic behavior:**

- Configured while DIR=1 (should configure while DIR=0)
- Multiple cogs acknowledging same pin
- X or Y values out of valid range

## Debugging Checklist

1. **Is DIR=1?** - Smart pin must be enabled
2. **Is P\_OE included?** - Required for output modes
3. **Is mode correct?** - Verify mode bits in WRPIN value
4. **Is X valid?** - Check period/parameter calculations
5. **Was configured while reset?** - DIR should be 0 during WRPIN
6. **Is input present?** - For input modes, verify signal at pin

## Using Scope/Logic Analyzer

For timing issues:

1. Capture the pin output
2. Verify frequency/period matches expectations
3. Check for glitches during configuration
4. Verify phase relationships in multi-pin setups

## 5.6 Performance Considerations

### Instruction Timing

| Instruction       | Cycles |
|-------------------|--------|
| WRPIN/WXPIN/WYPIN | 2      |
| RDPIN/RQPIN       | 2      |
| AKPIN             | 2      |
| TESTP/TESTPN      | 2      |

### Configuration Overhead

Full configuration (DIRL + WRPIN + WXPIN + WYPIN + DRVL) = 10 cycles.

For frequently-reconfigured modes, consider:

- Just updating Y (WYPIN) when only output value changes
- Using reset (DIRL + DRVL = 4 cycles) instead of full reconfig

## Read Overhead

RDPIN every event: 2 cycles + polling overhead.

For high-frequency events:

- Use larger measurement windows (X register)
- Read less frequently
- Some events are lost when results are overwritten before they are read

## When Overhead Matters

- Events faster than ~10 MHz at 200 MHz sysclk
- Tight timing loops
- Multiple smart pins requiring attention

## When Overhead is Negligible

- Events slower than 1 MHz
- Occasional configuration changes
- Asynchronous operation (smart pin runs independently)

## 5.7 Troubleshooting Quick Reference

### “Pin Not Responding”

| Check        | Action                                                   |
|--------------|----------------------------------------------------------|
| DIR state    | Ensure DRVL/DRVH/PINLOW was executed after configuration |
| WRPIN value  | Verify mode bits are correct (%SSSSS field)              |
| Pin number   | Confirm correct pin in all instructions                  |
| Cog conflict | Check if another Cog is controlling the pin              |

### “No Output”

| Check               | Action                                       |
|---------------------|----------------------------------------------|
| P_OE                | Add P_OE to WRPIN value for output modes     |
| Drive strength      | Ensure not set to P_HIGH_FLOAT / P_LOW_FLOAT |
| Mode requires Y     | Some modes need WYPIN before output starts   |
| Reset during config | Configure with DIR=0, then enable            |



## “Wrong Frequency/Timing”

| Check           | Action                                |
|-----------------|---------------------------------------|
| X register      | Verify base period calculation        |
| Y register      | For NCO, verify frequency calculation |
| Clock frequency | Confirm _clkfreq matches actual clock |
| Frame vs base   | X[31:16] is frame, X[15:0] is base    |

## “Events Too Fast/Slow”

| Check              | Action                                   |
|--------------------|------------------------------------------|
| Measurement window | X register sets window for counter modes |
| Sample period      | Check ADC sample rate setting            |
| Base period        | NCO/PWM timing derived from base period  |

## “IN Never Goes High”

| Check              | Action                                         |
|--------------------|------------------------------------------------|
| Mode enabled       | DIR must be 1                                  |
| Input present      | For input modes, verify signal at pin          |
| X = 0 issue        | Some modes need X > 0 to generate events       |
| Acknowledge timing | Wait 2 clocks after acknowledge before polling |

## “Data Corrupted/Wrong”

| Check       | Action                                      |
|-------------|---------------------------------------------|
| Read timing | Read before next event overwrites result    |
| Bit width   | Ensure Z interpretation matches mode        |
| C flag      | Some modes put extra data in C flag         |
| Multi-Cog   | Only one Cog should RDPIN; others use RQPIN |

## “Works Then Stops”

| Check           | Action                              |
|-----------------|-------------------------------------|
| One-shot mode   | May need WYPIN to restart           |
| Y exhausted     | Pulse/Transition modes count down Y |
| Buffer overflow | Reading too slowly loses data       |

## 5.8 Best Practices Summary

### Configuration

1. Always configure while DIR=0 (reset state)
2. Include P\_OE for output modes
3. Verify calculations for X and Y values
4. Enable last (DRVH/DRVL after WRPIN/WXPIN/WYPIN)

### Operation

1. Poll IN before reading (avoid unnecessary reads)
2. Use RQPIN for observers, RDPIN for owner
3. Update Y for new output values (don't reconfigure)
4. Reset (DIRL + DRVL) is faster than reconfigure

### Multi-Pin

1. Use pin ranges for identical configurations
2. Enable simultaneously for synchronization
3. Use relative addressing for related pins
4. Designate one cog as owner for shared pins

### Debugging

1. Start simple - verify basic operation first
2. Check DIR and P\_OE before investigating further
3. Use RQPIN to inspect without disturbing
4. Verify calculations independently

*This chapter completes Part I: Fundamentals. For specific smart pin mode documentation, proceed to Part II (Output Modes), Part III (Input Modes), or Part IV (Special Modes).*

## Part II: Output Modes

# Chapter 6: Digital Output

This chapter covers digital output configurations using P\_NORMAL mode (%00000) with enhanced pin settings. While not technically a “smart pin mode,” these configurations use WRPIN to set drive characteristics, polarity, and output topology—extending basic Direct I/O with hardware-configurable behavior.

## 6.1 Overview

### P\_NORMAL Mode

When WRPIN bits [5:1] = %00000, the pin operates in P\_NORMAL mode—basic Direct I/O with enhanced characteristics. The pin is controlled by DIR and OUT bits (via DRVH, DRVL, etc.) but with configurable:

- Drive strength (high and low independently)
- Output polarity (inverted or normal)
- Input conditioning (Schmitt trigger, comparator)

### When to Use P\_NORMAL Output

**Use P\_NORMAL output for:**

- Simple on/off control (LEDs, relays, enables)
- Software-timed signals (bit-banging)
- Irregular patterns not suited to smart pin automation
- Open-drain/open-collector interfaces
- When cog control is preferred over autonomy

**Consider smart pin modes (Chapters 7-11) for:**

- Precise timing requirements
- Free-running oscillators
- PWM at high frequencies
- Serial communication

- Autonomous operation

## 6.2 Output Configurations

### Push-Pull Output (Standard)

The default configuration: both high and low are actively driven.

#### Configuration:

```
WRPIN(pin, P_HIGH_FAST | P_LOW_FAST)    ' Default drive strength
```

Or use Direct I/O without WRPIN (defaults apply):

```
PINHIGH(pin)    ' Drives high
PINLOW(pin)     ' Drives low
```

#### Drive Strength Options:

| High Drive  | Low Drive  | Use Case                 |
|-------------|------------|--------------------------|
| P_HIGH_FAST | P_LOW_FAST | Standard digital (30mA)  |
| P_HIGH_1K5  | P_LOW_1K5  | Current-limited (~2mA)   |
| P_HIGH_1MA  | P_LOW_1MA  | Current-source LED drive |

#### Example - LED with current limiting:

```
CON
  LED_PIN = 56

PUB setup()
  ' 1.5kΩ series resistance limits current
  WRPIN(LED_PIN, P_HIGH_1K5 | P_LOW_FAST)

PUB led_on()
  PINHIGH(LED_PIN)

PUB led_off()
  PINLOW(LED_PIN)
```

```
WRPIN    ##(P_HIGH_1K5 | P_LOW_FAST), #LED_PIN
DRVH     #LED_PIN    ' LED on
DRVL     #LED_PIN    ' LED off
```

## Open-Drain Output

Drives low actively; floats when logically high. Requires external pull-up resistor. Used for I<sup>2</sup>C, 1-Wire, and multi-master buses.

### Configuration:

```
WRPIN(pin, P_HIGH_FLOAT | P_LOW_FAST)
```

### Behavior:

- PINHIGH/DRVH → Pin floats (external pull-up pulls high)
- PINLOW/DRVL → Pin drives low

### Example - I<sup>2</sup>C-style bus:

```
CON
  SDA_PIN = 0
  SCL_PIN = 1

PUB setup_i2c()
  ' Open-drain with fast low drive
  WRPIN(SDA_PIN, P_HIGH_FLOAT | P_LOW_FAST | P_SCHMITT_A)
  WRPIN(SCL_PIN, P_HIGH_FLOAT | P_LOW_FAST | P_SCHMITT_A)

  ' Release lines (high via external pull-ups)
  PINHIGH(SDA_PIN)
  PINHIGH(SCL_PIN)

PUB sda_low()
  PINLOW(SDA_PIN)          ' Drive low

PUB sda_release()
  PINHIGH(SDA_PIN)         ' Float (pull-up makes high)

PUB sda_read() : state
  state := PINREAD(SDA_PIN) ' Read current state
```

```
' Open-drain configuration
      WRPIN ##(P_HIGH_FLOAT | P_LOW_FAST | P_SCHMITT_A), #SDA_PIN

' Drive low
      DRVL      #SDA_PIN

' Release (float high)
      DRVH      #SDA_PIN

' Read
      TESTP     #SDA_PIN wc      ' C = SDA state
```

TESTP is used for the read-back (rather than reading the INA register) deliberately: it sees the pin two clocks old versus three for INA — one clock fresher, which matters on a fast bus where a line is driven and immediately sampled (see §1.2).

### With Internal Pull-Up:

If external pull-up isn't available, use internal resistive drive:

```
WRPIN(pin, P_HIGH_15K | P_LOW_FAST)    ' 15kΩ pull-up when high
```

Note: Internal pull-ups are weaker than typical external pull-ups and may not meet bus specifications for higher speeds.

## Open-Source Output

Drives high actively; floats when logically low. Less common than open-drain.

### Configuration:

```
WRPIN(pin, P_HIGH_FAST | P_LOW_FLOAT)
```

### Behavior:

- PINHIGH/DRVH → Pin drives high
- PINLOW/DRVL → Pin floats (external pull-down pulls low)

## Inverted Output

Output logic is inverted from the OUT bit.

### Configuration:

```
WRPIN(pin, P_INVERT_OUTPUT)
```

### Behavior:

- PINHIGH/DRVH (OUT=1) → Pin drives LOW
- PINLOW/DRVL (OUT=0) → Pin drives HIGH

**Use Case:** Active-low devices where software logic is more natural as active-high.

**Example - Active-low LED:**

```

CON
    LED_PIN = 56                                ' LED connected to VCC, active low

PUB setup()
    WRPIN(LED_PIN, P_INVERT_OUTPUT)

PUB led_on()
    PINHIGH(LED_PIN)                            ' Drives LOW, LED on

PUB led_off()
    PINLOW(LED_PIN)                             ' Drives HIGH, LED off

```

**Tri-State Output**

Explicit control of output enable separate from output value.

**Pattern 1: Using DIR for enable**

```

' Output disabled (floating)
PINFLOAT(pin)

' Output enabled, driving last OUT value
PINHIGH(pin)                                ' or PINLOW(pin)

```

**Pattern 2: Pre-setting value before enable**

To avoid glitches, pre-set the output value while floating:

```

' Prepare output value while disabled
WRPIN(pin, P_HIGH_FAST | P_LOW_FAST)        ' Configure
PINFLOAT(pin)                               ' Ensure disabled

' Set intended output state
' (Internal OUT register is set but pin is floating)
' Use FLT instructions to set OUT while keeping DIR=0

' Then enable
PINHIGH(pin)                                ' Enable and drive high

```

```

' Pre-set output high, pin floating
    FLTH    #pin                            ' DIR=0, OUT=1

' Later, enable output (immediately high, no glitch)
    DIRH    #pin                            ' DIR=1, drives high

```

## 6.3 Software-Timed Output Patterns

### Bit-Banging Serial

For non-standard protocols or when smart pin serial modes don't FIT:

```
CON
  TX_PIN = 20
  BIT_TIME = 1000                                ' Clocks per bit

PUB send_byte(data) | i
  ' Start bit (low)
  PINLOW(TX_PIN)
  WAITCT(GETCT() + BIT_TIME)

  ' Data bits (LSB first)
  repeat i from 0 to 7
    if data & 1
      PINHIGH(TX_PIN)
    else
      PINLOW(TX_PIN)
    data >>= 1
    WAITCT(GETCT() + BIT_TIME) ' wait one bit time

  ' Stop bit (high)
  PINHIGH(TX_PIN)
  WAITCT(GETCT() + BIT_TIME)
```

### Pulse Generation

```
PUB pulse(pin, width_us) | start
  PINHIGH(pin)
  WAITUS(width_us)
  PINLOW(pin)
```

```
pulse    DRVH    pin
          WAITX   width
          DRVL    pin
          RET
```

' Width in clocks

### Fast Toggle

Maximum software toggle rate:



```
.fast_toggle
    DRVH    #pin        ' 2 cycles
    DRVL    #pin        ' 2 cycles
    JMP     #.fast_toggle ' 4 cycles

' Total: 8 cycles per complete cycle = 25 MHz at 200 MHz sysclk
' (The 3-clock output latency is a one-time pipeline offset,
' not a per-edge cost)
```

The 3-clock output latency is a fixed pipeline delay — it sets *when* each edge reaches the pad (3 clocks after the instruction completes), not *how often* edges can be produced. It does not lower the toggle frequency; throughput is set by the instruction count in the loop.

## 6.4 Timing Analysis

### Instruction Timing

| Quantity                                                                        | Cycles | At 200 MHz |
|---------------------------------------------------------------------------------|--------|------------|
| DRVH/DRVL execution — cost <b>per transition</b> (throughput)                   | 2      | 10 ns      |
| Output pipeline delay — fixed <b>latency</b> (instruction completes → pin edge) | 3      | 15 ns      |
| Latency, instruction <i>start</i> → pin edge (2 + 3)                            | 5      | 25 ns      |

**How to read this table:** the per-transition cost is the **2-clock** instruction time — that is what limits how fast edges can be emitted, and because back-to-back instructions pipeline, edges follow at the instruction rate. The **3-clock pipeline delay is latency, not throughput**: it shifts *when* an edge reaches the pad (5 clocks total from instruction start) but is a one-time offset, *not* added to every transition. Do **not** sum 2 + 3 to compute a per-edge rate.

### Maximum Toggle Rate

#### Tight loop toggle:

```
    DRVNOT   #pin        ' 2 cycles
    JMP      #$.-1        ' 4 cycles (taken branch)
```

Period: 6 cycles = 30 ns → ~33 MHz toggle (edge) rate at 200 MHz sysclk — i.e. a ~16.5 MHz square wave.

The 3-clock output latency shifts *when* edges reach the pad but does not reduce the edge rate; the actual rate is set by the loop's instruction count (the per-transition cost), not by the latency.

## When Direct I/O is Faster

Direct I/O is faster than smart pins for:

- Infrequent, irregular pulses
- One-shot signals
- Quick on/off without setup overhead

Smart pins are faster when:

- Continuous waveforms are needed
- Cog should be free for other work
- Precise timing independent of software

## Smart Pin Overhead

Smart pin configuration takes ~10 cycles (DIRL + WRPIN + WXPIN + WYPIN + DRVL). For a single pulse, Direct I/O is more efficient. For continuous operation, smart pin overhead is negligible.

## 6.5 Complete Examples

### Example 1: Status LED with Blink

```
CON
  _clkfreq = 200_000_000
  LED_PIN = 56
  BLINK_MS = 500

PUB main()
  ' Current-source for consistent brightness
  WRPIN(LED_PIN, P_HIGH_1MA | P_LOW_FAST)

  repeat
    PINHIGH(LED_PIN)
    WAITMS(BLINK_MS)
    PINLOW(LED_PIN)
    WAITMS(BLINK_MS)
```

ch06-current-drive-blink.spin2

```
CON
  _clkfreq = 200_000_000

DAT          ORG

              WRPIN    ##(P_HIGH_1MA | P_LOW_FAST), #56
```

*continues on next page →*

↪ continued from previous page

```

.loop          DRVH      #56
               WAITX     half_sec
               DRVL      #56
               WAITX     half_sec
               JMP        #.loop

half_sec       LONG      100_000_000      ' 0.5 sec at 200 MHz

```

## Example 2: I<sup>2</sup>C Bit-Bang (Open-Drain)

```

CON
  SDA = 0
  SCL = 1

PUB i2c_init()
  WRPIN(SDA, P_HIGH_FLOAT | P_LOW_FAST | P_SCHMITT_A)
  WRPIN(SCL, P_HIGH_FLOAT | P_LOW_FAST | P_SCHMITT_A)
  PINHIGH(SDA)                ' Release both
  PINHIGH(SCL)

PUB i2c_start()
  PINHIGH(SDA)
  PINHIGH(SCL)
  WAITUS(5)
  PINLOW(SDA)                  ' SDA low while SCL high
  WAITUS(5)
  PINLOW(SCL)

PUB i2c_stop()
  PINLOW(SDA)
  PINHIGH(SCL)
  WAITUS(5)
  PINHIGH(SDA)                ' SDA high while SCL high

```

## Example 3: Stepper Motor Pulses

```

CON
  STEP_PIN = 10
  DIR_PIN = 11
  STEPS_PER_REV = 200

PUB step_forward(steps) | i
  PINHIGH(DIR_PIN)            ' Direction: forward
  repeat i from 1 to steps
    PINHIGH(STEP_PIN)
    WAITUS(10)                ' Pulse width
    PINLOW(STEP_PIN)
    WAITUS(1000)              ' Step delay

```

continues on next page →

```

PUB step_reverse(steps) | i
  PINLOW(DIR_PIN)           ' Direction: reverse
  repeat i from 1 to steps
    PINHIGH(STEP_PIN)
    WAITUS(10)
    PINLOW(STEP_PIN)
    WAITUS(1000)

```

## 6.6 Configuration Quick Reference

| Topology                      | WRPIN Value               |
|-------------------------------|---------------------------|
| Push-pull (standard)          | P_HIGH_FAST   P_LOW_FAST  |
| Push-pull (current limit)     | P_HIGH_1K5   P_LOW_1K5    |
| Open-drain                    | P_HIGH_FLOAT   P_LOW_FAST |
| Open-drain + internal pull-up | P_HIGH_15K   P_LOW_FAST   |
| Open-source                   | P_HIGH_FAST   P_LOW_FLOAT |
| Inverted                      | P_INVERT_OUTPUT           |
| LED current source            | P_HIGH_1MA   P_LOW_FAST   |

*This chapter covered software-controlled digital output. For hardware-automated pulse and transition output, see Chapter 7. For continuous waveform generation, see Chapters 8 (NCO) and 9 (PWM).*

# Chapter 7: Pulse & Transition

## Signal Generation

This chapter covers hardware-generated pulses and transitions using two smart pin modes: **P\_PULSE** (%00100) for generating counted pulse cycles, and **P\_TRANSITION** (%00101) for generating counted signal transitions.

## 7.1 Overview

### Pulse vs Transition

#### **P\_PULSE (Pulse/Cycle Output):**

- Generates a programmable number of pulse cycles
- Each cycle has configurable high-time and low-time
- Output returns to low when complete
- Y register controls the number of cycles

#### **P\_TRANSITION (Transition Output):**

- Generates a programmable number of signal transitions (edges)
- Each transition occurs at a fixed base period
- Output remains at final state when complete
- Y register controls the number of transitions

### When to Use These Modes

#### **Use P\_PULSE for:**

- Stepper motor step pulses
- Trigger pulses with specific counts
- Timed burst generation
- PWM with controlled duration

#### **Use P\_TRANSITION for:**

- Precise edge generation
- RS-485 direction control timing
- Delayed signal assertion
- Clock bursts with known edge count

## 7.2 P\_PULSE Mode (%00100)

### Function

P\_PULSE generates a specified number of pulse cycles. Each cycle consists of a programmable high-time followed by a programmable low-time. When the cycle count reaches zero, the output remains low and IN is raised.

### Configuration

| Register | Field         | Purpose                                                           |
|----------|---------------|-------------------------------------------------------------------|
| X[15:0]  | Base period   | Total cycle length in clock cycles                                |
| X[31:16] | Compare value | Output HIGH while counter > this; numerically the LOW-time clocks |
| Y[31:0]  | Cycle count   | Number of cycles to generate                                      |

### Output Behavior

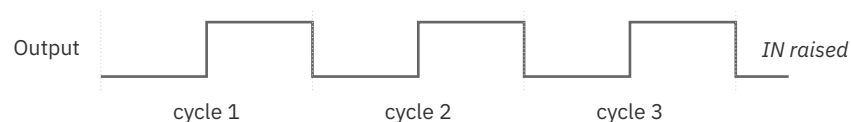
On each clock, the base period counter counts from X[15:0] down to 1, then restarts. The output is:

- **HIGH** when counter > X[31:16] AND Y > 0
- **LOW** otherwise

After each complete base period (counter reaches 1), Y is decremented. When Y reaches 0, the output stays low and IN is raised.

### Timing Diagram

For X[15:0] = 4, X[31:16] = 2, Y = 3:



**Figure 7.1.** Pulse output (counted cycles)

## Duty Cycle Calculation

The duty cycle is determined by the compare value relative to the base period:

```
High time = X[15:0] - X[31:16] clocks
Low time  = X[31:16] clocks
Duty cycle = (X[15:0] - X[31:16]) / X[15:0]
```

### Special cases:

- $X[31:16] = 0$ : Output stays high for entire period (100% duty)
- $X[31:16] = X[15:0]$ : Output stays low (0% duty)

## Configuration Sequence

### Spin2:

```
CON
  PULSE_PIN = 10
  BASE_PERIOD = 1000                ' 1000 clocks per cycle
  LOW_TIME = 500                    ' X[31:16]: 500 lo, 500 hi (50%)
  CYCLE_COUNT = 10                  ' Generate 10 pulses

PUB generate_pulses() | ack
  PINFLOAT(PULSE_PIN)              ' Reset
  WRPIN(PULSE_PIN, P_PULSE | P_OE)  ' Configure mode
  WXPIN(PULSE_PIN, BASE_PERIOD | (LOW_TIME << 16)) ' Set timing
  PINLOW(PULSE_PIN)                 ' Enable

  WYPIN(PULSE_PIN, CYCLE_COUNT)     ' Trigger: generate 10 pulses

  ' Wait for completion
  repeat until PINREAD(PULSE_PIN) == 1
  ack := RDPIN(PULSE_PIN)           ' Acknowledge completion (discard value)
```

**PASM2:**

```

                                DIRL      #PULSE_PIN
                                WRPIN      ##(P_PULSE | P_OE), #PULSE_PIN
                                WXPIN      ##(BASE_PERIOD | (LOW_TIME << 16)), #PULSE_PIN
                                DRVL       #PULSE_PIN

                                WYPIN      #CYCLE_COUNT, #PULSE_PIN      ' Trigger

.wait                          TESTP      #PULSE_PIN wc
    if_nc JMP                    #.wait
                                RDPIN      result, #PULSE_PIN          ' Acknowledge

```

**Retriggering**

Writing a new Y value while pulses are in progress:

- If Y > 0: New value is loaded at next base period boundary
- If Y = 0: New value triggers a new pulse sequence immediately

This allows continuous pulse generation or mid-stream adjustment.

**7.3 P\_TRANSITION Mode (%00101)****Function**

P\_TRANSITION generates a specified number of signal transitions (edges). Each transition occurs at the base period boundary. The output toggles on each boundary until the transition count reaches zero.

**Configuration**

| Register | Field            | Purpose                     |
|----------|------------------|-----------------------------|
| X[15:0]  | Base period      | Clocks between transitions  |
| Y[31:0]  | Transition count | Number of edges to generate |

**Output Behavior**

When Y is written with a non-zero value:

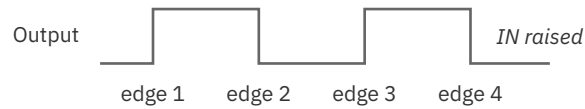
1. At each base period, the output toggles
2. Y is decremented after each toggle
3. When Y reaches 0, toggling stops



4. IN is raised
5. Output remains at its final state

## Timing Diagram

For  $X[15:0] = 100$ ,  $Y = 4$ , starting from low:



**Figure 7.2.** Transition output (counted edges)

## Configuration Sequence

### Spin2:

```
CON
  TRANS_PIN = 11
  EDGE_PERIOD = 200           ' 200 clocks between edges
  EDGE_COUNT = 8              ' Generate 8 edges (4 cycles)

PUB generate_transitions() | ack
  PINFLOAT(TRANS_PIN)
  WRPIN(TRANS_PIN, P_TRANSITION | P_OE)
  WXPIN(TRANS_PIN, EDGE_PERIOD)
  PINLOW(TRANS_PIN)

  WYPIN(TRANS_PIN, EDGE_COUNT)      ' Trigger

  repeat until PINREAD(TRANS_PIN) == 1
  ack := RDPIN(TRANS_PIN)           ' Acknowledge (discard value)
```

### PASM2:

```
        DIRL      #TRANS_PIN
        WRPIN     ##(P_TRANSITION | P_OE), #TRANS_PIN
        WXPIN     #EDGE_PERIOD, #TRANS_PIN
        DRVL      #TRANS_PIN

        WYPIN     #EDGE_COUNT, #TRANS_PIN

.wait   TESTP     #TRANS_PIN wc
        if_nc     JMP      #.wait
        RDPIN     result, #TRANS_PIN
```

## Transition Count and Final State

The final output state depends on:

- Initial state (low after reset)
- Number of transitions (odd = opposite state, even = same state)

| Initial | Transitions | Final |
|---------|-------------|-------|
| Low     | 1           | High  |
| Low     | 2           | Low   |
| Low     | 3           | High  |
| Low     | 4           | Low   |

## 7.4 Applicable P\_ Constants

Both modes support these configuration options:

| Constant               | Purpose                         |
|------------------------|---------------------------------|
| P_OE                   | <b>Required</b> - Enable output |
| P_INVERT_OUTPUT        | Invert output polarity          |
| P_HIGH_FAST            | Fast high drive (default)       |
| P_LOW_FAST             | Fast low drive (default)        |
| P_HIGH_1K5 / P_LOW_1K5 | Current-limited drive           |

**Example - Inverted output:**

```
WRPIN(pin, P_PULSE | P_OE | P_INVERT_OUTPUT)
```

## 7.5 Timing Calculations

### Pulse Width Calculation

For P\_PULSE:

```
Pulse period = X[15:0] × (1 / sysclk)
High time = (X[15:0] - X[31:16]) × (1 / sysclk)
```

**Example at 200 MHz:**

```

X[15:0] = 2000, X[31:16] = 500
Period   = 2000 / 200MHz = 10 µs
High time = (2000 - 500) / 200MHz = 7.5 µs
Low time  = 500 / 200MHz = 2.5 µs
Duty cycle = 1500 / 2000 = 75%

```

**Transition Period Calculation**

For P\_TRANSITION:

```
Time between edges = X[15:0] × (1 / sysclk)
```

**Example at 200 MHz:**

```

X[15:0] = 1000
Edge period = 1000 / 200MHz = 5 µs

```

**Timing at Different Clock Frequencies**

| sysclk  | X value for 1 µs | X value for 10 µs |
|---------|------------------|-------------------|
| 100 MHz | 100              | 1000              |
| 180 MHz | 180              | 1800              |
| 250 MHz | 250              | 2500              |
| 350 MHz | 350              | 3500              |

*P2 is rated to 180 MHz; 250 MHz is a common overclock and 350 MHz is the practical ceiling. Operation above the rated frequency depends on cooling and duty cycle — sustained high-throughput work generates heat that limits the usable maximum. (P2 Datasheet / Parallax Propeller 2 Documentation v35 - Rev B/C)*

## 7.6 Comparison: When to Use Each Mode

| Requirement                 | Use                           |
|-----------------------------|-------------------------------|
| Fixed number of pulses      | P_PULSE                       |
| Specific duty cycle         | P_PULSE                       |
| Single delayed edge         | P_TRANSITION with Y=1         |
| Clock burst with edge count | P_TRANSITION                  |
| Asymmetric high/low times   | P_PULSE                       |
| Equal high/low times        | Either (P_TRANSITION simpler) |
| Stay high after pulse train | P_TRANSITION (odd count)      |
| Return to low after         | P_PULSE                       |

### Pulse vs Transition vs Software

| Approach             | Best For                              |
|----------------------|---------------------------------------|
| P_PULSE              | Precise pulse trains, stepper motors  |
| P_TRANSITION         | Edge counting, clock bursts           |
| Software (DRVH/DRVL) | Irregular patterns, conditional logic |

## 7.7 Complete Examples

### Example 1: Stepper Motor Step Pulse

```

CON
  _clkfreq = 200_000_000
  STEP_PIN = 10
  STEP_PERIOD = 400           ' 2 µs period
  STEP_HIGH = 200             ' 1 µs high time

PUB step_motor(steps) | ack
  PINFLOAT(STEP_PIN)
  WRPIN(STEP_PIN, P_PULSE | P_OE)
  WXPIN(STEP_PIN, STEP_PERIOD | (STEP_HIGH << 16))
  PINLOW(STEP_PIN)

  WYPIN(STEP_PIN, steps)      ' Generate step pulses

  ' Wait for completion

```

*continues on next page →*

*↪ continued from previous page*

```
repeat until PINREAD(STEP_PIN) == 1
ack := RDPIN(STEP_PIN)           ' Acknowledge (discard value)
```

ch07-step-motor-pulses.spin2

## Example 2: RS-485 Transmit Disable Delay

After transmitting, delay before releasing the line:

```
CON
_clkfreq = 200_000_000
DE_PIN = 20           ' Driver Enable
DISABLE_DELAY = 2000  ' 10 µs delay

PUB setup_de()
  PINFLOAT(DE_PIN)
  WRPIN(DE_PIN, P_TRANSITION | P_OE | P_INVERT_OUTPUT)
  WXPIN(DE_PIN, DISABLE_DELAY)
  PINLOW(DE_PIN)       ' DE starts low (disabled)

PUB tx_complete()
  ' After transmission, trigger delayed disable
  ' DE is currently high (enabled) due to inversion
  WYPIN(DE_PIN, 1)      ' Single transition: high → low
  ' After DISABLE_DELAY clocks, DE goes low (driver disabled)
```

## Example 3: Trigger Pulse Burst

```
CON
TRIG_PIN = 15
PULSE_WIDTH = 100      ' 500 ns at 200 MHz
PULSE_COUNT = 5

PUB trigger_burst() | ack
  PINFLOAT(TRIG_PIN)
  WRPIN(TRIG_PIN, P_PULSE | P_OE)
  WXPIN(TRIG_PIN, (PULSE_WIDTH * 2) | (PULSE_WIDTH << 16)) ' 50% duty
  PINLOW(TRIG_PIN)

  WYPIN(TRIG_PIN, PULSE_COUNT)

  repeat until PINREAD(TRIG_PIN) == 1
  ack := RDPIN(TRIG_PIN)      ' Acknowledge (discard value)
```

## Example 4: PASM2 Continuous Step Generation

```

CON
    _clkfreq = 200_000_000

DAT          ORG

' Setup
            DIRL      #STEP_PIN
            WRPIN      ##(P_PULSE | P_OE), #STEP_PIN
            WXPIN      ##(STEP_PERIOD | (STEP_HIGH << 16)), #STEP_PIN
            DRVL       #STEP_PIN

' Generate steps as needed
step_loop
            WYPIN      steps_needed, #STEP_PIN

.wait
            TESTP      #STEP_PIN wc
            if_nc JMP   #.wait
            RDPIN      result, #STEP_PIN

            ' Check for more steps
            CMP        more_steps, #0 wz
            if_nz JMP   #step_loop

            JMP        #$          ' Done

STEP_PIN    LONG      10
STEP_PERIOD LONG      400
STEP_HIGH   LONG      200
steps_needed LONG     100
more_steps   LONG      0
result       LONG      0

```

## 7.8 Quick Reference

### P\_PULSE Configuration

| Parameter     | Register | Range             | Notes                                             |
|---------------|----------|-------------------|---------------------------------------------------|
| Base period   | X[15:0]  | 1-65535           | Clocks per cycle                                  |
| Compare value | X[31:16] | 0-65535           | Output HIGH when counter > this (low-time clocks) |
| Cycle count   | Y[31:0]  | 1-2 <sup>32</sup> | Pulses to generate                                |

## P\_TRANSITION Configuration

| Parameter   | Register | Range             | Notes                |
|-------------|----------|-------------------|----------------------|
| Edge period | X[15:0]  | 1-65535           | Clocks between edges |
| Edge count  | Y[31:0]  | 1-2 <sup>32</sup> | Transitions to make  |

## Reset State

Both modes when DIR=0:

- IN = low
- Output = low
- Y = 0

*This chapter covered hardware-timed pulse and transition generation. For continuous waveform generation, see Chapter 8 (NCO) and Chapter 9 (PWM).*

# Chapter 8: Frequency Generation (NCO)

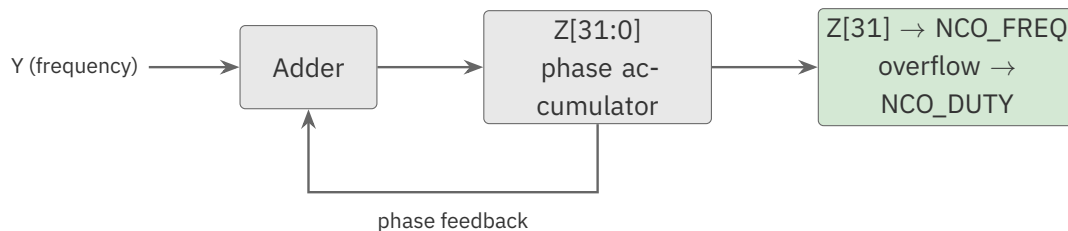
This chapter covers the two Numerically Controlled Oscillator (NCO) modes: **P\_NCO\_FREQ** (%00110) for precise frequency generation with 50% duty cycle, and **P\_NCO\_DUTY** (%00111) for frequency generation with variable duty cycle.

## 8.1 NCO Concept

### What is an NCO?

A Numerically Controlled Oscillator generates precise frequencies by accumulating a phase value. On each clock (or base period), a frequency control word is added to a phase accumulator. When the accumulator overflows (or crosses a threshold), the output toggles.

### P2 NCO Architecture



**Figure 8.1.** NCO architecture

### Key Properties

- **Frequency resolution:** the 32-bit phase accumulator gives a resolution of  $\text{sysclk} / 2^{32}$  — about 0.047 Hz at 200 MHz. Because it scales with the system clock, the resolution is finer at a lower sysclk and coarser at a higher one (e.g. ~0.023 Hz at 100 MHz, ~0.075 Hz at 320 MHz)
- **Phase coherence:** Multiple NCOs can be phase-locked via initial phase setting
- **Deterministic timing:** Hardware-based, independent of software execution

## 8.2 P\_NCO\_FREQ Mode (%00110)

### Function

P\_NCO\_FREQ generates a square wave at a precise frequency. The output reflects the MSB of the phase accumulator (Z[31]), creating a 50% duty cycle output.



## Configuration

| Register | Field             | Purpose                            |
|----------|-------------------|------------------------------------|
| X[15:0]  | Base period       | Clock cycles between phase updates |
| X[31:16] | Initial phase     | Written to Z[31:16] on WXPIN       |
| Y[31:0]  | Frequency control | Added to Z each base period        |
| Z[31:0]  | Phase accumulator | Z[31] drives output                |

## Output Behavior

On each base period (every X[15:0] clocks):

1. Y is added to Z
2. Output = Z[31]
3. If Z overflows, IN is raised

The output toggles when Z[31] changes, creating a square wave.

Writing **Y = 0** produces no output: with nothing added to Z each period, Z[31] never changes and the pin holds static. A free-running waveform requires a non-zero Y.

## Frequency Formula

$$\text{frequency} = (Y \times \text{sysclk}) / (X[15:0] \times 2^{32})$$

For X[15:0] = 1 (maximum update rate):

$$\text{frequency} = (Y \times \text{sysclk}) / 2^{32}$$

Solving for Y:

$$Y = (\text{frequency} \times 2^{32}) / \text{sysclk}$$

## Worked Examples

### Example 1: 1 kHz at 200 MHz sysclk

$$\begin{aligned} Y &= (1000 \times 4,294,967,296) / 200,000,000 \\ Y &= 4,294,967,296,000 / 200,000,000 \\ Y &= 21,475 \end{aligned}$$

**Example 2: 44.1 kHz (audio sample rate) at 200 MHz**

```

Y = (44100 × 4,294,967,296) / 200,000,000
Y = 189,408,057,753,600 / 200,000,000
Y = 947,040

```

**Example 3: 1 MHz at 200 MHz**

```

Y = (1,000,000 × 4,294,967,296) / 200,000,000
Y = 21,474,836

```

**Configuration Sequence**

FRAC computes  $(\text{operand1} \times 2^{32}) / \text{operand2}$  using a 64-bit intermediate (so it never overflows) — here it scales the desired frequency into the NCO's Y word.

**Spin2:**

```

CON
  _clkfreq = 200_000_000
  NCO_PIN = 10

PUB nco_frequency(freq_hz | y_value
  ' Calculate Y for desired frequency
  y_value := freq_hz FRAC _clkfreq

  PINFLOAT(NCO_PIN)
  WRPIN(NCO_PIN, P_NCO_FREQ | P_OE)
  WXPIN(NCO_PIN, 1)           ' Base period = 1 clock
  WYPIN(NCO_PIN, y_value)
  PINLOW(NCO_PIN)

```

**PASM2:**

```

DIRL      #NCO_PIN
WRPIN     ##(P_NCO_FREQ | P_OE), #NCO_PIN
WXPIN     #1, #NCO_PIN           ' Base period = 1
WYPIN     freq_y, #NCO_PIN       ' Frequency value
DRVL      #NCO_PIN

```

## Resolution vs Update Rate Tradeoff

Using  $X[15:0] > 1$  reduces update rate but can smooth jitter:

| <b>X[15:0]</b> | <b>Updates/sec at 200 MHz</b> | <b>Effect</b>         |
|----------------|-------------------------------|-----------------------|
| 1              | 200,000,000                   | Maximum resolution    |
| 10             | 20,000,000                    | Reduced jitter        |
| 100            | 2,000,000                     | Lower cog access rate |

For most applications,  $X[15:0] = 1$  provides best frequency resolution.

## 8.3 P\_NCO\_DUTY Mode (%00111)

### Function

P\_NCO\_DUTY generates a frequency with variable duty cycle. The output reflects the phase accumulator overflow state, allowing duty cycle control.

### Key Difference from P\_NCO\_FREQ

| <b>Mode</b> | <b>Output Based On</b> | <b>Duty Cycle</b> |
|-------------|------------------------|-------------------|
| P_NCO_FREQ  | Z[31]                  | Always 50%        |
| P_NCO_DUTY  | Z overflow             | Variable          |

### Duty Cycle Control

In P\_NCO\_DUTY, Y sets the duty cycle directly — the accumulator (Z) is incremented by Y each base period, and the output is high for one base period on every overflow, so the fraction of high time is  $Y / 2^{32}$ :

- Larger Y values → Higher duty cycle
- Smaller Y values → Lower duty cycle

The duty cycle is approximately:

$$\text{duty\_cycle} \approx Y / 2^{32}$$

**Example:**

```

Y = $8000_0000 → 50% duty cycle
Y = $4000_0000 → 25% duty cycle
Y = $C000_0000 → 75% duty cycle

```

**Configuration****Spin2:**

```

CON
    _clkfreq = 200_000_000
    NCO_PIN = 10

PUB nco_duty(duty_percent) | y_value
    ' In NCO_DUTY, Y sets the duty cycle directly: duty = Y / 2^32.
    ' (Base period is 1 clock, so Z accumulates Y every clock.)
    y_value := duty_percent FRAC 100          ' 50->$8000_0000, 25->$4000_0000

    PINFLOAT(NCO_PIN)
    WRPIN(NCO_PIN, P_NCO_DUTY | P_OE)
    WXPIN(NCO_PIN, 1)
    WYPIN(NCO_PIN, y_value)
    PINLOW(NCO_PIN)

```

**Worked Example: Fixed Pulse Width, Variable Period**

Because the output is high for exactly one base period on each overflow, P\_NCO\_DUTY is also a clean way to produce a **fixed-width pulse at an adjustable period** — X sets the pulse width, Y sets the period.

Suppose Fclk = 25 MHz and the goal is a 1 μs pulse repeating every 18 μs:

- Base period X = 25 clocks (= 1 μs at 25 MHz) — the high time is one base period, so 1 μs.
- The pin overflows (emits a pulse) once every 18 base periods (18 μs), so  $Y = 2^{32} / 18$ .

```

X = Fclk × t_pulse          = 25 MHz × 1 μs   = 25
Y = 232 × t_pulse / t_period = 232 × 1 / 18   = 238,609,294 ($0E38_E38E)

```

**8.4 Phase Synchronization****Setting Initial Phase**

X[31:16] sets the initial phase when WXPIN is executed:

```
' Phase offset in 16-bit units (0 = 0°, 32768 = 180°, 65535 = ~360°)
phase_offset := 32768           ' 180° offset
WXPIN(pin, 1 | (phase_offset << 16))
```

## Multi-Pin Phase Lock

For phase-locked outputs (e.g., three-phase motor control):

```
CON
  _clkfreq = 200_000_000
  PHASE_A = 10
  PHASE_B = 11
  PHASE_C = 12
  FREQ_HZ = 1000

PUB three_phase_nco() | y_val, phase_120, phase_240
  y_val := FREQ_HZ FRAC _clkfreq

  ' Phase offsets: 0°, 120°, 240°
  phase_120 := 65536 / 3           ' 21845
  phase_240 := 65536 * 2 / 3      ' 43691

  ' Configure all three
  PINFLOAT(PHASE_A)
  PINFLOAT(PHASE_B)
  PINFLOAT(PHASE_C)

  WRPIN(PHASE_A, P_NCO_FREQ | P_OE)
  WRPIN(PHASE_B, P_NCO_FREQ | P_OE)
  WRPIN(PHASE_C, P_NCO_FREQ | P_OE)

  WXPIN(PHASE_A, 1 | (0 << 16))   ' 0° phase
  WXPIN(PHASE_B, 1 | (phase_120 << 16)) ' 120° phase
  WXPIN(PHASE_C, 1 | (phase_240 << 16)) ' 240° phase

  ' Same frequency for all
  WYPIN(PHASE_A, y_val)
  WYPIN(PHASE_B, y_val)
  WYPIN(PHASE_C, y_val)

  ' Enable all simultaneously
  PINLOW(PHASE_A..PHASE_C)
```

ch08-three-phase-nco.spin2

## Phase Coherence

When multiple NCOs use the same Y value and are enabled simultaneously:

- They maintain constant phase relationship
- Phase offset is set by X[31:16] at configuration
- No drift between channels

## 8.5 Analog Output with DAC

### NCO + DAC for Sine Wave Approximation

An NCO can drive the resistor DAC directly (with an external RC filter) for filtered-sine output — see §10.5.

### Direct DAC Control

For true analog waveform generation, use the DAC modes with software updates (see Chapter 10) rather than NCO modes.

## 8.6 Frequency Accuracy Analysis

### Maximum Frequency

Maximum output frequency is  $\text{sysclk} / 2$  (Nyquist limit):

```
At 200 MHz: max frequency = 100 MHz
Achieved with Y = $8000_0000
```

### Minimum Frequency

Minimum frequency with X[15:0] = 1:

```
min_freq = sysclk / 232
At 200 MHz: min_freq ≈ 0.047 Hz
```

### Frequency Error

Frequency error depends on the fractional part of Y:

```
Actual frequency = round(Y) × sysclk / 232
Error = |target - actual| / target × 100%
```

**Example: 1 kHz target at 200 MHz**

```

Y_exact = 21474.83648
Y_rounded = 21475
Actual freq = 21475 × 200,000,000 / 4,294,967,296 = 1000.0076 Hz
Error = 0.00076%

```

**CAUTION**

**The average frequency is exact; individual periods are not.** Rounding Y bounds only the long-term *average* — the integer accumulator still quantizes each cycle to a whole number of base periods, and the leftover fraction accumulates and occasionally lengthens (or shortens) one period by a single base period. For the fixed-pulse example in §8.3 ( $Y = 2^{32}/18$ , true value 238,609,294.222), the average period is 18  $\mu\text{s}$ , but the dropped .222 surfaces as a rare  $\sim 19 \mu\text{s}$  period roughly every 1,073 s. Invisible for most timing; for jitter-sensitive work (precise pulse trains, sampling clocks) budget for it, or choose X/Y so the fraction is zero.

**Frequency Resolution Table**

| sysclk  | Resolution (X=1) |
|---------|------------------|
| 100 MHz | 0.0233 Hz        |
| 180 MHz | 0.0419 Hz        |
| 250 MHz | 0.0582 Hz        |
| 350 MHz | 0.0815 Hz        |

**8.7 Complete Examples****Example 1: Audio Tone Generator**

```

CON
    _clkfreq = 200_000_000
    SPEAKER_PIN = 56

PUB play_tone(frequency, duration_ms) | y_val
    y_val := frequency FRAC _clkfreq

    PINFLOAT(SPEAKER_PIN)
    WRPIN(SPEAKER_PIN, P_NCO_FREQ | P_OE)
    WXPIN(SPEAKER_PIN, 1)
    WYPIN(SPEAKER_PIN, y_val)

```

continues on next page →

↪ continued from previous page

```

    PINLOW(SPEAKER_PIN)

    WAITMS(duration_ms)

    PINFLOAT(SPEAKER_PIN)          ' Stop tone

PUB play_scale()
    play_tone(262, 500)             ' C4
    play_tone(294, 500)             ' D4
    play_tone(330, 500)             ' E4
    play_tone(349, 500)             ' F4
    play_tone(392, 500)             ' G4
    play_tone(440, 500)             ' A4
    play_tone(494, 500)             ' B4
    play_tone(523, 500)             ' C5

```

## Example 2: Variable Frequency Clock

```

CON
    _clkfreq = 200_000_000
    CLK_PIN = 20

VAR
    LONG current_y

PUB setup_clock(initial_freq)
    current_y := initial_freq FRAC _clkfreq

    PINFLOAT(CLK_PIN)
    WRPIN(CLK_PIN, P_NCO_FREQ | P_OE)
    WXPIN(CLK_PIN, 1)
    WYPIN(CLK_PIN, current_y)
    PINLOW(CLK_PIN)

PUB set_frequency(new_freq)
    current_y := new_freq FRAC _clkfreq
    WYPIN(CLK_PIN, current_y)          ' Update on the fly

```

## Example 3: PASM2 Frequency Sweep

```

CON
    _clkfreq = 200_000_000

DAT                ORG

' Setup NCO at 1 kHz
    DIRL            #NCO_PIN
    WRPIN            ##(P_NCO_FREQ | P_OE), #NCO_PIN

```

continues on next page →



→ continued from previous page

```

                WXPIN    #1, #NCO_PIN
                WYPIN    y_start, #NCO_PIN
                DRVL     #NCO_PIN

        ' Sweep frequency upward
sweep_loop
                ADD      y_current, y_step
                WYPIN    y_current, #NCO_PIN
                WAITX    sweep_delay
                CMP      y_current, y_end wc
        if_c    JMP      #sweep_loop

                JMP      #$$

NCO_PIN      LONG      10
y_start      LONG      21475          ' 1 kHz
y_end        LONG      214748        ' 10 kHz
y_step       LONG      215           ' ~10 Hz step
y_current     LONG      0
sweep_delay  LONG      2_000_000     ' 10 ms between steps

```

## 8.8 Quick Reference

### P\_NCO\_FREQ Configuration

| Parameter     | Register | Formula                                       |
|---------------|----------|-----------------------------------------------|
| Base period   | X[15:0]  | 1 for maximum resolution                      |
| Initial phase | X[31:16] | 0-65535 (0°-360°)                             |
| Frequency     | Y        | $(\text{freq} \times 2^{32}) / \text{sysclk}$ |

### P\_NCO\_DUTY Configuration

| Parameter     | Register | Formula                  |
|---------------|----------|--------------------------|
| Base period   | X[15:0]  | 1 for maximum resolution |
| Initial phase | X[31:16] | 0-65535 (0°-360°)        |
| Freq × duty   | Y        | Varies by desired duty   |

## Common Y Values at 200 MHz

| Frequency | Y Value     |
|-----------|-------------|
| 100 Hz    | 2,147       |
| 1 kHz     | 21,475      |
| 10 kHz    | 214,748     |
| 100 kHz   | 2,147,484   |
| 1 MHz     | 21,474,836  |
| 10 MHz    | 214,748,365 |

## Reset State

Both modes when DIR=0:

- IN = low
- Output = low
- Z = 0

*This chapter covered NCO-based frequency generation. For PWM output with variable duty cycle, see Chapter 9. For DAC analog output, see Chapter 10.*

# Chapter 9: PWM Output

This chapter covers the three Pulse Width Modulation (PWM) modes: **P\_PWM\_TRIANGLE** (%01000) for symmetric triangle-wave PWM, **P\_PWM\_SAWTOOTH** (%01001) for asymmetric sawtooth-wave PWM, and **P\_PWM\_SMPS** (%01010) for switch-mode power supply control with feedback.

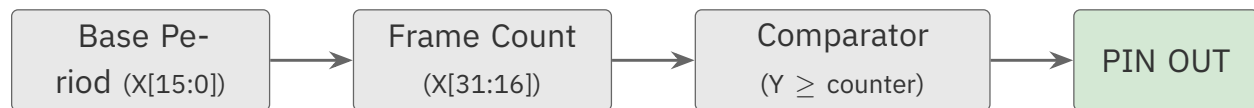
## 9.1 PWM Fundamentals

### What is PWM?

Pulse Width Modulation controls the average power delivered to a load by varying the duty cycle of a digital signal. The duty cycle is the percentage of time the signal is high during each period.

### P2 PWM Architecture

All three PWM modes share a common architecture:



**Figure 9.1.** PWM architecture

### Key Terminology

| Term                | Definition                                      |
|---------------------|-------------------------------------------------|
| <b>Base period</b>  | X[15:0] clock cycles between counter updates    |
| <b>Frame period</b> | X[31:16] base periods forming one counter cycle |
| <b>PWM period</b>   | Time for complete PWM cycle (depends on mode)   |
| <b>Duty value</b>   | Y[15:0] comparison threshold                    |

### Mode Comparison

| Mode           | Counter Pattern  | PWM Period       | Best For           |
|----------------|------------------|------------------|--------------------|
| P_PWM_TRIANGLE | Up-down          | 2 × frame period | Smooth transitions |
| P_PWM_SAWTOOTH | Up only          | 1 × frame period | Fast switching     |
| P_PWM_SMPS     | Up with feedback | Variable         | Power supply       |

## 9.2 P\_PWM\_TRIANGLE Mode (%01000)

### Function

P\_PWM\_TRIANGLE generates a symmetric PWM waveform using an up-down counter. The counter counts from the frame period value down to 1, then from 1 back up to the frame period value, creating a triangle wave pattern.

### Counter Behavior

```
Frame = 4

Counter:  4 → 3 → 2 → 1      (count down)
          1 → 2 → 3 → 4      (count up)
          → repeat

PWM Period = 2 × Frame Period × Base Period
```

### Output Logic

At each base period:

- If Y[15:0] >= counter → output HIGH
- If Y[15:0] < counter → output LOW

### Configuration

| Register | Field        | Purpose                                                    |
|----------|--------------|------------------------------------------------------------|
| X[15:0]  | Base period  | Clock cycles per counter update (1-65536; 0 selects 65536) |
| X[31:16] | Frame period | Counter range (1-65536; a field value of 0 selects 65536)  |
| Y[15:0]  | Duty value   | 0 (always low) to frame period (always high)               |

### Timing Formulas

```
PWM frequency = sysclk / (2 × X[31:16] × X[15:0])

PWM period = 2 × X[31:16] × X[15:0] / sysclk

Duty cycle = Y[15:0] / X[31:16] × 100%
```

## Worked Example

### 1 kHz triangle PWM at 50% duty with 200 MHz sysclk:

Target: 1 kHz PWM, 50% duty  
 PWM period =  $1/1000 = 1 \text{ ms} = 200,000 \text{ clocks}$

Choose: Base period = 1, Frame period = 100,000  
 $\rightarrow \text{PWM period} = 2 \times 100,000 \times 1 = 200,000 \text{ clocks} \checkmark$

Duty = 50% =  $50,000 / 100,000$   
 $\rightarrow Y = 50,000$

## Configuration Sequence

### Spin2:

```
CON
  _clkfreq = 200_000_000
  PWM_PIN = 20

PUB triangle_pwm(freq_hz, duty_percent) | frame, y_val
  ' Calculate frame period for desired frequency
  frame := _clkfreq / (2 * freq_hz)
  y_val := frame * duty_percent / 100

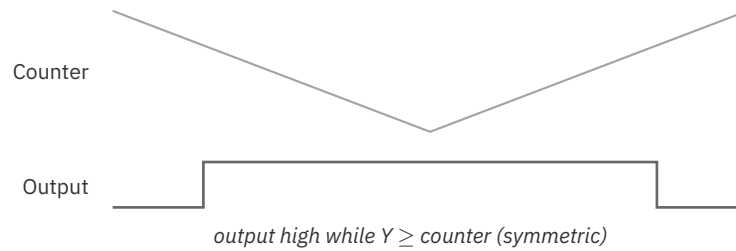
  PINFLOAT(PWM_PIN)
  WRPIN(PWM_PIN, P_PWM_TRIANGLE | P_OE)
  WXPIN(PWM_PIN, 1 | (frame << 16))      ' Base=1, frame period
  WYPIN(PWM_PIN, y_val)
  PINLOW(PWM_PIN)
```

### PASM2:

```
DIRL      #PWM_PIN
WRPIN     ##(P_PWM_TRIANGLE | P_OE), #PWM_PIN
WXPIN     x_val, #PWM_PIN
DIRH      #PWM_PIN
WYPIN     y_val, #PWM_PIN
```

## Duty Cycle Waveform

For frame period = 8, duty value = 6:



**Figure 9.2.** PWM triangle duty

The symmetric counting creates equal rise and fall times.

## 9.3 P\_PWM\_SAWTOOTH Mode (%01001)

### Function

P\_PWM\_SAWTOOTH generates an asymmetric PWM waveform using an up-only counter. The counter counts from 1 up to the frame period value, then resets to 1.

### Counter Behavior

```
Frame = 4

Counter:  1 → 2 → 3 → 4      (count up)
          1 → 2 → 3 → 4      (count up, repeat)

PWM Period = Frame Period × Base Period
```

### Key Difference from Triangle

| Aspect              | P_PWM_TRIANGLE                             | P_PWM_SAWTOOTH                             |
|---------------------|--------------------------------------------|--------------------------------------------|
| Counter pattern     | Up-down                                    | Up only                                    |
| PWM period          | $2 \times \text{frame} \times \text{base}$ | $1 \times \text{frame} \times \text{base}$ |
| Frequency at same X | Half                                       | Full                                       |
| Edges per cycle     | 2 symmetric                                | 2 asymmetric                               |

## Configuration

| Register | Field        | Purpose                                                   |
|----------|--------------|-----------------------------------------------------------|
| X[15:0]  | Base period  | Clock cycles per counter update                           |
| X[31:16] | Frame period | Counter range (1-65536; a field value of 0 selects 65536) |
| Y[15:0]  | Duty value   | 0 (always low) to frame period (always high)              |

## Timing Formulas

$$\text{PWM frequency} = \text{sysclk} / (\text{X}[31:16] \times \text{X}[15:0])$$

$$\text{PWM period} = \text{X}[31:16] \times \text{X}[15:0] / \text{sysclk}$$

$$\text{Duty cycle} = \text{Y}[15:0] / \text{X}[31:16] \times 100\%$$

## Worked Example

### 10 kHz sawtooth PWM at 25% duty with 200 MHz sysclk:

Target: 10 kHz PWM, 25% duty  
 PWM period =  $1/10,000 = 100 \mu\text{s} = 20,000$  clocks

Choose: Base period = 1, Frame period = 20,000  
 $\rightarrow \text{PWM period} = 20,000 \times 1 = 20,000$  clocks ✓

Duty = 25% =  $5,000 / 20,000$   
 $\rightarrow Y = 5,000$

## Configuration Sequence

### Spin2:

```
CON
  _clkfreq = 200_000_000
  PWM_PIN = 20

PUB sawtooth_pwm(freq_hz, duty_percent) | frame, y_val
  ' Calculate frame period for desired frequency
  frame := _clkfreq / freq_hz
  y_val := frame * duty_percent / 100

  PINFLOAT(PWM_PIN)
  WRPIN(PWM_PIN, P_PWM_SAWTOOTH | P_OE)
  WXPIN(PWM_PIN, 1 | (frame << 16))
```

*continues on next page →*

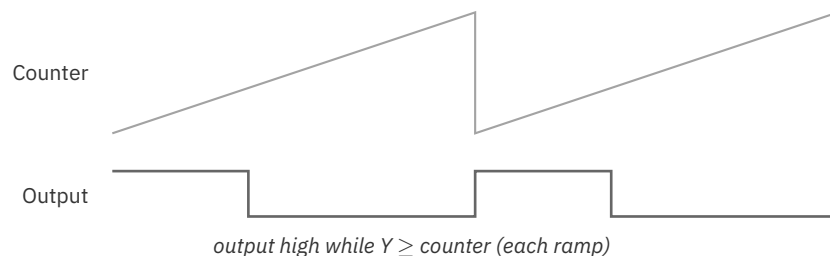
```
WYPIN(PWM_PIN, y_val)
PINLOW(PWM_PIN)
```

**PASM2:**

```
DIRL      #PWM_PIN
WRPIN     ##(P_PWM_SAWTOOTH | P_OE), #PWM_PIN
WXPIN     x_val, #PWM_PIN
DIRH      #PWM_PIN
WYPIN     y_val, #PWM_PIN
```

**Duty Cycle Waveform**

For frame period = 8, duty value = 3:



**Figure 9.3.** PWM sawtooth duty

The sawtooth pattern creates a fast rising edge and slow falling edge in the output.

**9.4 P\_PWM\_SMPS Mode (%01010)****Function**

P\_PWM\_SMPS generates PWM output for switch-mode power supply control with voltage and current feedback. This mode extends sawtooth PWM with two feedback inputs that control cycle initiation and output cutoff.

**Feedback Inputs**

| Input   | Function         | Source                      |
|---------|------------------|-----------------------------|
| A-input | Voltage feedback | Low = start new cycle       |
| B-input | Current limit    | High = immediate output low |

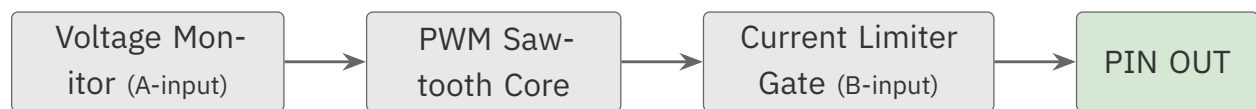


## Operation Sequence

1. Counter runs sawtooth pattern (1 to frame period)
2. At frame end, wait for A-input to go low (voltage sag)
3. When A goes low, start new cycle, capture Y, raise IN
4. During cycle, if B-input goes high, force output low for remainder

The **IN flag rising marks the cycle boundary** — the instant a fresh Y is captured for the new cycle. That makes IN the synchronization cue for software: wait on (or poll) IN before writing the next duty value with WYPIN, and the update lands cleanly on the upcoming cycle instead of mid-pulse.

## Block Diagram



**Figure 9.4.** SMPS mode block diagram

## Configuration

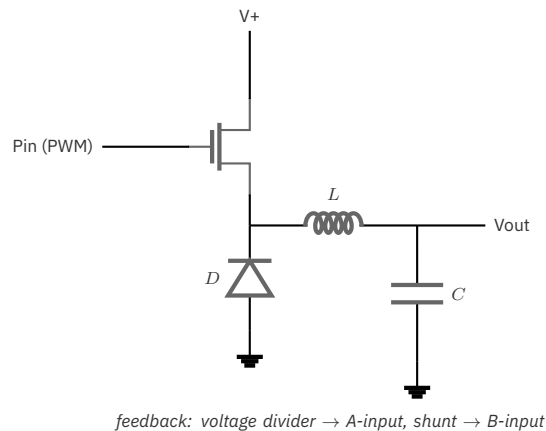
| Register | Field        | Purpose                         |
|----------|--------------|---------------------------------|
| X[15:0]  | Base period  | Clock cycles per counter update |
| X[31:16] | Frame period | Maximum PWM pulse width         |
| Y[15:0]  | Duty value   | PWM threshold (can be set once) |

## Input Selection

Use mode field bits to select A and B input sources:

| Constant   | Effect             |
|------------|--------------------|
| P_PLUS1_A  | A-input from pin+1 |
| P_MINUS1_A | A-input from pin-1 |
| P_PLUS1_B  | B-input from pin+1 |
| P_MINUS1_B | B-input from pin-1 |

## Typical SMPS Circuit



**Figure 9.5.** Typical SMPS (buck) circuit

## Configuration Sequence

### Spin2:

```
CON
  _clkfreq = 200_000_000
  SMPS_PIN = 20      ' FET gate
  V_SENSE = 21       ' Voltage feedback (A-input)
  I_SENSE = 19       ' Current sense (B-input)

PUB smps_controller(duty_percent, voltage_threshold, current_limit) ...
  | mode, frame, y_val
  ' Configure voltage comparator
  WRPIN(V_SENSE, P_COMPARE_AB)
  WXPIN(V_SENSE, voltage_threshold)
  PINH(V_SENSE)

  ' Configure current comparator
  WRPIN(I_SENSE, P_COMPARE_AB)
  WXPIN(I_SENSE, current_limit)
  PINH(I_SENSE)

  ' Configure SMPS controller
  frame := 256      ' 256 steps
  y_val := frame * duty_percent / 100
  mode := P_PWM_SMPS | P_OE | P_PLUS1_A | P_MINUS1_B

  PINFLOAT(SMPS_PIN)
  WRPIN(SMPS_PIN, mode)
  WXPIN(SMPS_PIN, 25 | (frame << 16))  ' 25 clocks base
  WYPIN(SMPS_PIN, y_val)
  PINLOW(SMPS_PIN)
```

**PASM2:**

```

DIRL      #SMPS_PIN
MOV        smps_cfg, ##(P_PWM_SMPS | P_OE)
OR         smps_cfg, ##(P_PLUS1_A | P_MINUS1_B)
WRPIN     smps_cfg, #SMPS_PIN
WXPIN     x_val, #SMPS_PIN
DIRH      #SMPS_PIN
WYPIN     y_val, #SMPS_PIN    ' Set once, runs autonomously

```

**Set-and-Forget Operation**

P\_PWM\_SMPS is designed for autonomous operation. After initial configuration with WYPIN, the smart pin:

- Monitors voltage via A-input
- Initiates pulses when voltage sags
- Limits current via B-input
- Requires no software intervention

**9.5 Dynamic Duty Cycle Updates****Updating Y Register**

Triangle and sawtooth PWM modes capture Y[15:0] at the start of each frame. (SMPS mode is event-driven: it captures Y[15:0] after the frame period completes *and* the A-input goes low — not at a fixed clock boundary.) To change duty cycle:

**Spin2:**

```
WYPIN(PWM_PIN, new_duty_value)
```

**PASM2:**

```
WYPIN      new_duty, #PWM_PIN
```

The new value takes effect at the next frame boundary, preventing glitches.

**Glitch-Free Updates**

The Y capture mechanism ensures:

- Mid-cycle writes do not affect current cycle

- New duty applies at next frame start
- No partial pulses or timing artifacts

## Update Timing

For smooth transitions, update rate should be much slower than PWM frequency:

| Application   | PWM Frequency | Update Rate |
|---------------|---------------|-------------|
| LED dimming   | 500 Hz        | 50-100 Hz   |
| Motor control | 20 kHz        | 1-5 kHz     |
| Audio         | 100 kHz       | 44.1 kHz    |

## 9.6 PWM Resolution and Frequency Tradeoffs

### Resolution vs Frequency

PWM resolution depends on frame period ( $X[31:16]$ ):

| Frame Period | Resolution | Max Frequency, triangle (200 MHz) |
|--------------|------------|-----------------------------------|
| 256          | 8-bit      | 390.6 kHz                         |
| 512          | 9-bit      | 195.3 kHz                         |
| 1024         | 10-bit     | 97.7 kHz                          |
| 4096         | 12-bit     | 24.4 kHz                          |
| 65535        | 16-bit     | 1.5 kHz                           |

These are triangle-mode maxima ( $\text{sysclk} / (2 * \text{frame})$ ). Sawtooth uses the full frame as one period, so its maximum frequency is double each value.

### Choosing Parameters

**For motor control (20 kHz, 10-bit resolution):**

```
Frame period = 200_000_000 / 20_000 = 10,000
Actual resolution = log2(10,000) ≈ 13.3 bits
Y range: 0 to 10,000
```

**For LED dimming (500 Hz, 12-bit resolution):**

```

Frame period = 200_000_000 / 500 = 400,000
Must limit to 65535 max, use base period
Base = 7, Frame = 57,143
Y range: 0 to 57,143

```

## 9.7 Complete Examples

### Example 1: LED Brightness Control

```

CON
  _clkfreq = 200_000_000
  LED_PIN = 56
  PWM_FREQ = 500                                ' 500 Hz (no flicker)
  ' 500 Hz sawtooth: period = 200 MHz / 500 = 400,000 clocks. That exceeds the
  ' 16-bit frame field, so split it across base and frame: base x frame = period.
  BASE_PERIOD = 8
  FRAME_PERIOD = 50000                          ' 8 x 50,000 = 400,000 -> 500 Hz

PUB led_control() | brightness
  PINFLOAT(LED_PIN)
  WRPIN(LED_PIN, P_PWM_SAWTOOTH | P_OE)
  WXPIN(LED_PIN, BASE_PERIOD | (FRAME_PERIOD << 16))
  WYPIN(LED_PIN, 0)                             ' Start at 0%
  PINLOW(LED_PIN)

  ' Fade up
  repeat brightness from 0 to FRAME_PERIOD step FRAME_PERIOD/100
    WYPIN(LED_PIN, brightness)
    WAITMS(20)

  ' Fade down
  repeat brightness from FRAME_PERIOD to 0 step FRAME_PERIOD/100
    WYPIN(LED_PIN, brightness)
    WAITMS(20)

```

ch09-pwm-led-fade.spin2

### Example 2: Servo Motor Control

Standard hobby servos expect 50 Hz PWM with 1-2 ms pulse width:

```

CON
  _clkfreq = 200_000_000
  SERVO_PIN = 20

```

continues on next page →

```

' 50 Hz PWM = 20 ms period = 4,000,000 clocks
' Use base=64 to fit in 16-bit frame
BASE_PERIOD = 64
FRAME_PERIOD = 62500                                ' 64 × 62500 = 4,000,000

' Servo pulse: 1 ms = 200,000 clocks = 3125 frame units
'               2 ms = 400,000 clocks = 6250 frame units
SERVO_MIN = 3125                                     ' 0° position
SERVO_MAX = 6250                                     ' 180° position

PUB servo_control()
  PINFLOAT(SERVO_PIN)
  WRPIN(SERVO_PIN, P_PWM_SAWTOOTH | P_OE)
  WXPIN(SERVO_PIN, BASE_PERIOD | (FRAME_PERIOD << 16))
  WYPIN(SERVO_PIN, (SERVO_MIN + SERVO_MAX) / 2) ' Center
  PINLOW(SERVO_PIN)

PUB set_servo_angle(degrees) | pulse
  ' Map 0-180° to SERVO_MIN-SERVO_MAX
  pulse := SERVO_MIN + (SERVO_MAX - SERVO_MIN) * degrees / 180
  WYPIN(SERVO_PIN, pulse)

```

### Example 3: Motor Speed Control with Acceleration

```

CON
  _clkfreq = 200_000_000
  MOTOR_PIN = 16
  PWM_FREQ = 20_000                                ' 20 kHz (inaudible)

VAR
  LONG current_speed
  LONG target_speed
  LONG frame_period

PUB motor_init()
  frame_period := _clkfreq / (2 * PWM_FREQ) ' triangle period = 2 × frame

  PINFLOAT(MOTOR_PIN)
  WRPIN(MOTOR_PIN, P_PWM_TRIANGLE | P_OE) ' Triangle for smooth drive
  WXPIN(MOTOR_PIN, 1 | (frame_period << 16))
  WYPIN(MOTOR_PIN, 0)
  PINLOW(MOTOR_PIN)

  current_speed := 0
  target_speed := 0

PUB set_motor_speed(percent)
  target_speed := frame_period * percent / 100

PUB motor_update() | delta
  ' Call periodically for acceleration control

```

*→ continued from previous page*

```

if current_speed < target_speed
    delta := (target_speed - current_speed) / 10 + 1
    current_speed += delta
elseif current_speed > target_speed
    delta := (current_speed - target_speed) / 10 + 1
    current_speed -= delta

WYPIN(MOTOR_PIN, current_speed)

```

## Example 4: PASM2 High-Frequency PWM

```

CON
    _clkfreq = 200_000_000

DAT          ORG

' 100 kHz PWM with 10-bit resolution
    DIRL      #PWM_PIN
    WRPIN     ##(P_PWM_SAWTOOTH | P_OE), #PWM_PIN
    WXPIN     ##$07D0_0001, #PWM_PIN      ' Frame=2000, Base=1
    DIRH      #PWM_PIN
    WYPIN     ##1000, #PWM_PIN  ' 50% duty (1000 of 2000)

' Update duty in real-time
pwm_loop
    RDLONG    new_duty, duty_ptr
    WYPIN     new_duty, #PWM_PIN
    WAITX     delay
    JMP       #pwm_loop

PWM_PIN      LONG    20
duty_ptr     LONG    0                ' Hub address for duty
new_duty     LONG    0
delay        LONG    20_000          ' Update rate

```

## 9.8 Quick Reference

### P\_PWM\_TRIANGLE Configuration

| Parameter     | Register | Formula                                                      |
|---------------|----------|--------------------------------------------------------------|
| Base period   | X[15:0]  | Clock cycles per update                                      |
| Frame period  | X[31:16] | Counter range                                                |
| Duty value    | Y[15:0]  | 0 to frame period                                            |
| PWM frequency | -        | $\text{sysclk} / (2 \times \text{frame} \times \text{base})$ |
| Duty cycle    | -        | $Y / \text{frame} \times 100\%$                              |

### P\_PWM\_SAWTOOTH Configuration

| Parameter     | Register | Formula                                             |
|---------------|----------|-----------------------------------------------------|
| Base period   | X[15:0]  | Clock cycles per update                             |
| Frame period  | X[31:16] | Counter range                                       |
| Duty value    | Y[15:0]  | 0 to frame period                                   |
| PWM frequency | -        | $\text{sysclk} / (\text{frame} \times \text{base})$ |
| Duty cycle    | -        | $Y / \text{frame} \times 100\%$                     |

### P\_PWM\_SMPS Configuration

| Parameter    | Register  | Purpose                 |
|--------------|-----------|-------------------------|
| Base period  | X[15:0]   | Clock cycles per update |
| Frame period | X[31:16]  | Maximum pulse width     |
| Duty value   | Y[15:0]   | PWM threshold           |
| A-input      | Mode bits | Voltage feedback        |
| B-input      | Mode bits | Current limit           |



## Mode Constants

| Constant        | Value  | Purpose            |
|-----------------|--------|--------------------|
| P_PWM_TRIANGLE  | %01000 | Symmetric PWM      |
| P_PWM_SAWTOOTH  | %01001 | Asymmetric PWM     |
| P_PWM_SMPS      | %01010 | SMPS with feedback |
| P_OE            | -      | Enable output      |
| P_INVERT_OUTPUT | -      | Invert PWM signal  |
| P_PLUS1_A       | -      | A from pin+1       |
| P_MINUS1_A      | -      | A from pin-1       |
| P_PLUS1_B       | -      | B from pin+1       |
| P_MINUS1_B      | -      | B from pin-1       |

## Reset State (DIR=0)

All PWM modes:

- IN = low
- Output = low
- Y[15:0] = captured (ready for DIR=1)

*This chapter covered PWM output modes. For DAC-based analog output, see Chapter 10. For serial transmission modes, see Chapter 11.*

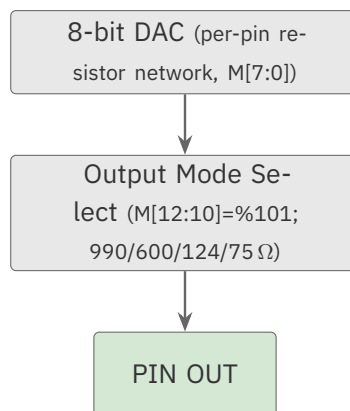
# Chapter 10: DAC Output

This chapter covers digital-to-analog conversion using the P2's built-in DAC capabilities. Topics include the resistor DAC output options, 8-bit direct DAC control, and 16-bit dithered DAC modes: **P\_DAC\_DITHER\_RND** (%00010) and **P\_DAC\_DITHER\_PWM** (%00011).

## 10.1 DAC Architecture Overview

### P2 DAC Structure

Each P2 I/O pin includes analog output capability through a resistive DAC network. The DAC operates at 8-bit resolution natively, with dithering modes available to achieve effective 16-bit resolution.



**Figure 10.1.** DAC structure

### DAC Mode Enable

The DAC output requires  $M[12:10] = \%101$  in the pin configuration. This is automatically set by the **P\_DAC\_\*** constants:

| Constant      | Resistance | Voltage Range | Current Capability |
|---------------|------------|---------------|--------------------|
| P_DAC_990R_3V | 990Ω       | 0 to 3.3V     | ~3.3 mA max        |
| P_DAC_600R_2V | 600Ω       | 0 to 2.0V     | ~3.3 mA max        |
| P_DAC_124R_3V | 124Ω       | 0 to 3.3V     | ~27 mA max         |
| P_DAC_75R_2V  | 75Ω        | 0 to 2.0V     | ~27 mA max         |

## Resolution Options

| Mode            | Resolution | Update Rate   | Best For        |
|-----------------|------------|---------------|-----------------|
| Direct (M[7:0]) | 8-bit      | Every clock   | Fast signals    |
| Dithered PRNG   | 16-bit     | Sample period | Control signals |
| Dithered PWM    | 16-bit     | Sample period | Audio           |

## 10.2 Resistor DAC Options

### Understanding the DAC Network

The P2 uses a resistor-weighted DAC that switches between voltage rails. The resistance values determine both the output impedance and the voltage swing.

#### P\_DAC\_990R\_3V

High impedance, full voltage range.

##### Specifications:

- Output impedance: 990Ω
- Voltage range: 0V to 3.3V
- Bit weight:  $3.3\text{V} / 256 = 12.9\text{ mV/LSB}$
- Drive current: Limited (~3.3 mA at full scale)

##### Best for:

- High-impedance loads
- Voltage references
- Signals to op-amp inputs
- Low-power applications

##### Spin2:

```
WRPIN(pin, P_DAC_990R_3V | P_OE)
PINH(pin)
```

#### P\_DAC\_600R\_2V

Lower impedance, reduced voltage range.

##### Specifications:

- Output impedance:  $600\Omega$
- Voltage range: 0V to 2.0V
- Bit weight:  $2.0V / 256 = 7.8 \text{ mV/LSB}$
- Drive current: Moderate ( $\sim 3.3 \text{ mA}$  at full scale)

**Best for:**

- Interface to 2V systems
- Better load driving than  $990\Omega$
- Moderate current requirements

## **P\_DAC\_124R\_3V**

Low impedance, full voltage range.

**Specifications:**

- Output impedance:  $124\Omega$
- Voltage range: 0V to 3.3V
- Bit weight:  $3.3V / 256 = 12.9 \text{ mV/LSB}$
- Drive current: High ( $\sim 27 \text{ mA}$  at full scale)

**Best for:**

- Driving cables
- Direct speaker drive
- LED brightness control
- Low-impedance loads

## **P\_DAC\_75R\_2V**

Lowest impedance, reduced voltage range.

**Specifications:**

- Output impedance:  $75\Omega$
- Voltage range: 0V to 2.0V
- Bit weight:  $2.0V / 256 = 7.8 \text{ mV/LSB}$
- Drive current: Highest ( $\sim 27 \text{ mA}$  at full scale)

**Best for:**

- $75\Omega$  cable termination
- Video signals (though limited to 2V)
- Maximum current drive

## Selection Guide

| Application    | Recommended   | Reason                     |
|----------------|---------------|----------------------------|
| Op-amp input   | P_DAC_990R_3V | High impedance acceptable  |
| Audio to amp   | P_DAC_600R_2V | Balance of drive and range |
| Direct speaker | P_DAC_124R_3V | Current drive needed       |
| LED control    | P_DAC_124R_3V | Current source capability  |
| Coax cable     | P_DAC_75R_2V  | Impedance matching         |

## 10.3 Direct 8-bit DAC Control

### Using WRPIN for Static DAC

For simple DAC output without smart pin modes, write the value directly:

#### Spin2:

```
CON
    DAC_PIN = 20

PUB set_voltage_8bit(value) | mode
    ' Configure for 8-bit DAC output
    ' Value in M[7:0] of WRPIN D operand
    mode := (value << 8) | P_DAC_124R_3V
    WRPIN(DAC_PIN, mode)
    PINH(DAC_PIN)

PUB voltage_to_dac(millivolts) : dac_value
    ' Convert millivolts to 8-bit DAC value (3.3V range)
    dac_value := millivolts * 256 / 3300
    dac_value := 0 #> dac_value <# 255
```

#### PASM2:

```
' Set DAC to mid-scale (128)
MOV     dac_mode, ##($80 << 8) | P_DAC_990R_3V
WRPIN   dac_mode, #DAC_PIN
DIRH    #DAC_PIN
```

### Updating DAC Value

To change the DAC output, issue a new WRPIN with the updated M[7:0] field:

**Spin2:**

```
PUB update_dac(pin, value) | current_mode
  ' Read current mode, update value bits
  current_mode := (value << 8) | P_DAC_124R_3V
  WRPIN(pin, current_mode)
```

**BIT\_DAC Mode**

When OUT controls the pin (not a smart pin mode), M[7:4] and M[3:0] define two DAC levels:

- OUT=1: M[7:4] duplicated as {M[7:4], M[7:4]}
- OUT=0: M[3:0] duplicated as {M[3:0], M[3:0]}

This creates a simple 2-level DAC controlled by the OUT bit.

**10.4 16-bit Dithered DAC Modes****Dithering Concept**

The P2 achieves 16-bit DAC resolution using 8-bit hardware plus temporal dithering. By rapidly switching between adjacent 8-bit values in a precise pattern, the time-averaged output achieves 16-bit resolution.

```
Target: $8040 (16-bit)
Upper BYTE: $80 (128)
Lower BYTE: $40 (64 of 256)
```

```
Output pattern: 75% at $80, 25% at $81
Average = 0.75 × 128 + 0.25 × 129 = 128.25 ≈ $80.40
```

**“16-bit” here is nominal — a *temporal-averaging* resolution, not absolute accuracy.**

The hardware DAC is 8-bit (256 levels); dithering trades time for amplitude resolution, so the effective bits realized depend on the low-pass filtering and settling of whatever the pin drives. Treat 16-bit as the averaged-over-time ceiling, not a guaranteed per-sample precision. (For pseudo-random *noise* output — mode %00001 — see §18.3.)

**P\_DAC\_DITHER\_RND (%00010)**

Uses pseudo-random dithering for smooth 16-bit output.

**Characteristics:**

- Random switching between adjacent levels

- Uniform spectral distribution of dither noise
- No periodic artifacts
- Suitable for control signals

**Configuration:**

| Register | Purpose                                        |
|----------|------------------------------------------------|
| X[15:0]  | Sample period in clocks (1 = immediate update) |
| Y[15:0]  | 16-bit DAC value                               |
| Z        | ADC accumulation (if OUT=1)                    |

**Spin2:**

```

CON
    _clkfreq = 200_000_000
    DAC_PIN = 48

PUB dithered_dac_prng(value16) | mode
    ' Setup 16-bit PRNG dithered DAC
    mode := P_DAC_DITHER_RND | P_DAC_124R_3V | P_OE

    PINFLOAT(DAC_PIN)
    WRPIN(DAC_PIN, mode)
    WXPIN(DAC_PIN, 1)                ' Immediate updates
    WYPIN(DAC_PIN, value16)
    PINLOW(DAC_PIN)

PUB update_value(value16)
    WYPIN(DAC_PIN, value16)        ' Takes effect immediately

```

**PASM2:**

```

DIRL      #DAC_PIN
WRPIN ##(P_DAC_DITHER_RND | P_DAC_124R_3V | P_OE), #DAC_PIN
WXPIN     #1, #DAC_PIN          ' Immediate mode
DIRH      #DAC_PIN
WYPIN     value16, #DAC_PIN

```

**P\_DAC\_DITHER\_PWM (%00011)**

Uses PWM dithering for better dynamic range.

**Characteristics:**

- Maximum 2 transitions per 256 clocks
- Lower switching noise than PRNG

- Fclock/256 component at -48 dB
- Suitable for audio applications

**Configuration:**

| Register | Purpose                                 |
|----------|-----------------------------------------|
| X[15:0]  | Sample period (must be multiple of 256) |
| Y[15:0]  | 16-bit DAC value                        |
| Z        | ADC accumulation (if OUT=1)             |

**Spin2:**

```

CON
    _clkfreq = 200_000_000
    DAC_PIN = 48
    SAMPLE_PERIOD = 256                ' Minimum (256 clocks)

PUB dithered_dac_pwm(value16) | mode
    ' Setup 16-bit PWM dithered DAC
    mode := P_DAC_DITHER_PWM | P_DAC_600R_2V | P_OE

    PINFLOAT(DAC_PIN)
    WRPIN(DAC_PIN, mode)
    WXPIN(DAC_PIN, SAMPLE_PERIOD)      ' Must be multiple of 256
    WYPIN(DAC_PIN, value16)
    PINLOW(DAC_PIN)

PUB update_value_sync(value16)
    ' Wait for sample complete before update
    repeat until PINREAD(DAC_PIN)
    WYPIN(DAC_PIN, value16)

```

**PASM2:**

```

                DIRL      #DAC_PIN
                WRPIN    ##(P_DAC_DITHER_PWM | P_DAC_600R_2V | P_OE), #DAC_PIN
                WXPIN    ##256, #DAC_PIN    ' Sample period
                DIRH      #DAC_PIN
                WYPIN     value16, #DAC_PIN

.wait          TESTP     #DAC_PIN wc      ' Wait for IN flag
              if_nc     JMP      #.wait
                WYPIN     new_value, #DAC_PIN

```



## Comparing Dithering Methods

| Aspect        | PRNG Dither        | PWM Dither                |
|---------------|--------------------|---------------------------|
| Transitions   | Random (many)      | Max 2 per 256 clocks      |
| Spectrum      | White noise floor  | Fclock/256 tone at -48 dB |
| Dynamic range | Good               | Better                    |
| Best for      | Control signals    | Audio                     |
| Sample period | Any value $\geq 1$ | Multiple of 256           |

## 10.5 DAC with Other Modes

### NCO + DAC for Waveform Generation

Combine NCO frequency generation with DAC output:

**Spin2:**

```
CON
  _clkfreq = 200_000_000
  WAVE_PIN = 20

PUB nco_dac_wave(freq_hz) | mode, y_val
  ' NCO square wave through DAC for filtered sine output
  mode := P_NCO_FREQ | P_DAC_990R_3V | P_OE
  y_val := freq_hz FRAC _clkfreq

  PINFLOAT(WAVE_PIN)
  WRPIN(WAVE_PIN, mode)
  WXPIN(WAVE_PIN, 1)
  WYPIN(WAVE_PIN, y_val)
  PINLOW(WAVE_PIN)
```

The NCO generates a square wave, and with external RC filtering, approximates a sine wave.

### PWM + DAC Integration

PWM modes can combine with DAC for analog PWM output:

```
' PWM triangle through DAC for smooth analog output
mode := P_PWM_TRIANGLE | P_DAC_600R_2V | P_OE
```

## 10.6 ADC Feedback

### Monitoring DAC Loading

Dithered DAC modes support ADC feedback to measure pin loading:

**Spin2:**

```
PUB read_dac_loading(pin) : loading | mode
  ' Enable ADC feedback (OUT=1)
  PINWRITE(pin, 1)

  ' Wait for accumulation
  WAITUS(100)

  ' Read ADC value
  loading := RDPIN(pin)
```

The ADC accumulates samples during the sample period. The result indicates how the DAC output is being loaded by external circuitry.

### Load Detection Applications

- Verify expected load impedance
- Detect open/short conditions
- Implement current limiting
- Calibrate DAC output

## 10.7 Voltage Calculation

### 8-bit DAC Voltage

```
Voltage = (DAC_value / 256) × Full_Scale_Voltage
```

```
For P_DAC_990R_3V OR P_DAC_124R_3V:
  Voltage = (DAC_value / 256) × 3.3V
```

```
For P_DAC_600R_2V OR P_DAC_75R_2V:
  Voltage = (DAC_value / 256) × 2.0V
```

## 16-bit DAC Voltage

$$\text{Voltage} = (\text{DAC\_value} / 65536) \times \text{Full\_Scale\_Voltage}$$

For 3.3V range:

$$\begin{aligned}\text{Voltage} &= (\text{DAC\_value} / 65536) \times 3.3\text{V} \\ \text{Resolution} &= 3.3\text{V} / 65536 = 50.4 \mu\text{V/LSB}\end{aligned}$$

For 2.0V range:

$$\begin{aligned}\text{Voltage} &= (\text{DAC\_value} / 65536) \times 2.0\text{V} \\ \text{Resolution} &= 2.0\text{V} / 65536 = 30.5 \mu\text{V/LSB}\end{aligned}$$

## Voltage to DAC Value

Spin2:

```
PUB millivolts_to_dac16(mv, full_scale_mv) : dac16
  ' Convert millivolts to 16-bit DAC value
  dac16 := (mv * 65536) / full_scale_mv
  dac16 := 0 #> dac16 <# 65535

PUB set_voltage_mv(pin, mv)
  ' Set DAC to specific voltage (3.3V full scale)
  WYPIN(pin, millivolts_to_dac16(mv, 3300))
```

## 10.8 Complete Examples

### Example 1: Simple Voltage Reference

```
CON
  _clkfreq = 200_000_000
  REF_PIN = 20

PUB voltage_reference(millivolts) | mode, dac_val
  ' Create stable voltage reference
  ' Using 16-bit PWM dithered DAC for precision

  mode := P_DAC_DITHER_PWM | P_DAC_990R_3V | P_OE
  dac_val := (millivolts * 65536) / 3300

  PINFLOAT(REF_PIN)
  WRPIN(REF_PIN, mode)
  WXPIN(REF_PIN, 256)           ' Minimum sample period
  WYPIN(REF_PIN, dac_val)
  PINLOW(REF_PIN)
```

*continues on next page →*

```
PUB set_2v5_reference()  
  voltage_reference(2500)          ' 2.5V output
```

## Example 2: Audio Waveform Generator

```
CON  
  _clkfreq = 200_000_000  
  AUDIO_PIN = 48  
  SAMPLE_RATE = 44100  
  
VAR  
  LONG phase  
  LONG phase_inc  
  
PUB audio_init()  
  ' Initialize audio DAC at 44.1 kHz  
  PINFLOAT(AUDIO_PIN)  
  WRPIN(AUDIO_PIN, P_DAC_DITHER_PWM | P_DAC_600R_2V | P_OE)  
  WXPIN(AUDIO_PIN, _clkfreq / SAMPLE_RATE / 256 * 256) ' Round to 256  
  WYPIN(AUDIO_PIN, $8000) ' Start at mid-scale  
  PINLOW(AUDIO_PIN)  
  
  phase := 0  
  
PUB set_frequency(hz)  
  ' Set sine wave frequency  
  phase_inc := hz FRAC SAMPLE_RATE  
  
PUB audio_sample() : sample | sine_val  
  ' Generate next audio sample  
  phase += phase_inc  
  
  ' Get sine value (-32767 to +32767) using CORDIC (length, angle, twist)  
  sine_val := QSIN(32767, phase, 0)  
  
  ' Convert to 16-bit unsigned (0 to 65535)  
  sample := sine_val + $8000  
  
PUB audio_output()  
  ' Output audio sample  
  repeat until PINREAD(AUDIO_PIN)  
  WYPIN(AUDIO_PIN, audio_sample())
```

ch10-audio-dac.spin2

### Example 3: DC Motor Speed Control

```

CON
    _clkfreq = 200_000_000
    MOTOR_PIN = 16

VAR
    LONG current_speed
    LONG target_speed

PUB motor_init()
    ' Initialize motor control DAC
    PINFLOAT(MOTOR_PIN)
    WRPIN(MOTOR_PIN, P_DAC_DITHER_RND | P_DAC_124R_3V | P_OE)
    WXPIN(MOTOR_PIN, 1)           ' Fast updates
    WYPIN(MOTOR_PIN, 0)          ' Start stopped
    PINLOW(MOTOR_PIN)

    current_speed := 0
    target_speed := 0

PUB set_motor_speed(percent)
    ' Set target speed (0-100%)
    target_speed := (percent * 65535) / 100

PUB motor_ramp_update()
    ' Smooth acceleration/deceleration
    if current_speed < target_speed
        current_speed += (target_speed - current_speed) / 10 + 1
    elseif current_speed > target_speed
        current_speed -= (current_speed - target_speed) / 10 + 1

    WYPIN(MOTOR_PIN, current_speed)

```

### Example 4: PASM2 Function Generator

```

CON
    _clkfreq = 200_000_000

DAT            ORG

    ' Initialize 16-bit dithered DAC
        DIRL      #DAC_PIN
        WRPIN ##(P_DAC_DITHER_PWM | P_DAC_990R_3V | P_OE), #DAC_PIN
        WXPIN     ##512, #DAC_PIN      ' 512 clock sample period
        DIRH      #DAC_PIN

    ' Generate sawtooth wave
saw_loop
        WYPIN     value16, #DAC_PIN
        ADD       value16, step_size

```

*continues on next page →*

→ continued from previous page

```

                WAITX    delay
                JMP      #saw_loop

' Generate triangle wave
tri_loop
                WYPIN    value16, #DAC_PIN
                ADD      value16, direction
                CMP      value16, ##$FFFF wz
            if_z NEG      direction
                CMP      value16, #0 wz
            if_z NEG      direction
                WAITX    delay
                JMP      #tri_loop

DAC_PIN        LONG      20
value16         LONG      0
step_size       LONG      256          ' Increment per sample
direction       LONG      256
delay           LONG      2000         ' Sample interval

```

## 10.9 Design Considerations

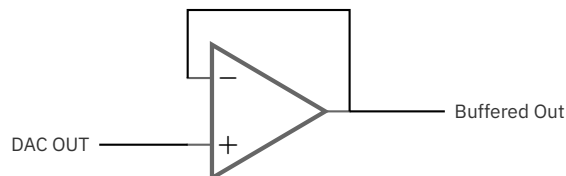
### Output Impedance and Loading

The DAC output impedance determines load driving capability:

| DAC Type      | Output Z | Max Load | Voltage Drop at 1mA |
|---------------|----------|----------|---------------------|
| P_DAC_990R_3V | 990Ω     | >10kΩ    | 0.99V               |
| P_DAC_600R_2V | 600Ω     | >6kΩ     | 0.60V               |
| P_DAC_124R_3V | 124Ω     | >1.2kΩ   | 0.12V               |
| P_DAC_75R_2V  | 75Ω      | >750Ω    | 0.08V               |

### External Buffering

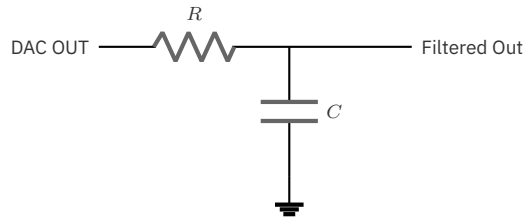
For driving low-impedance loads or cables, add an external buffer:



**Figure 10.2.** Op-amp output buffer (unity-gain follower)

## Filtering Dither Noise

For clean analog output, add an RC low-pass filter:



**Figure 10.3.** RC dither filter

Cutoff frequency:  $f_c = 1 / (2\pi \times R \times C)$

## Power Supply Considerations

- DAC output is relative to pin ground
- Ensure clean power supply for best performance
- Consider decoupling near the pin
- Load current affects power dissipation

## 10.10 Quick Reference

### Resistor DAC Constants

| Constant      | Resistance | Voltage | Mode Bits     |
|---------------|------------|---------|---------------|
| P_DAC_990R_3V | 990Ω       | 3.3V    | M[12:10]=%101 |
| P_DAC_600R_2V | 600Ω       | 2.0V    | M[12:10]=%101 |
| P_DAC_124R_3V | 124Ω       | 3.3V    | M[12:10]=%101 |
| P_DAC_75R_2V  | 75Ω        | 2.0V    | M[12:10]=%101 |

### Dithered DAC Modes

| Mode        | Constant                  | Resolution | Sample Period   |
|-------------|---------------------------|------------|-----------------|
| PRNG Dither | P_DAC_DITHER_RND (%00010) | 16-bit     | Any ≥1          |
| PWM Dither  | P_DAC_DITHER_PWM (%00011) | 16-bit     | Multiple of 256 |

## Voltage Formulas

8-bit:  $V = (\text{DAC\_value} / 256) \times V_{\text{full\_scale}}$

16-bit:  $V = (\text{DAC\_value} / 65536) \times V_{\text{full\_scale}}$

$\text{DAC\_value} = (V_{\text{target}} / V_{\text{full\_scale}}) \times \text{resolution}$

## Reset State (DIR=0)

All DAC modes:

- IN = low
- Y[15:0] = captured (ready for DIR=1)
- Output = low (0V)

*This chapter covered DAC analog output. For serial transmission modes, see Chapter 11. For input modes, see Part III.*



# Chapter 11: Serial Transmission

This chapter covers the serial transmission modes: **P\_ASYNC\_TX** (%11110) for asynchronous UART-style transmission and **P\_SYNC\_TX** (%11100) for synchronous SPI-style transmission.

## 11.1 Serial Transmission Overview

### Asynchronous vs Synchronous

| Aspect   | Asynchronous (UART)  | Synchronous (SPI)     |
|----------|----------------------|-----------------------|
| Clock    | Implicit (baud rate) | Explicit (clock line) |
| Framing  | Start/stop bits      | None                  |
| Pins     | 1 (TX)               | 2 (Data + Clock)      |
| Timing   | Self-synchronizing   | Clock-synchronized    |
| Use case | Point-to-point       | Bus communication     |

### P2 Smart Pin Serial Features

- Hardware-generated timing and framing
- 1 to 32 bits per frame
- Fractional baud rate for precise timing
- Double-buffered transmission
- IN flag indicates ready for next data

## 11.2 P\_ASYNC\_TX Mode (%11110)

### Function

P\_ASYNC\_TX transmits asynchronous serial data with automatic start and stop bit generation. The output idles high, transitions low for the start bit, transmits data bits LSB first, then returns high for the stop bit.

## Frame Format

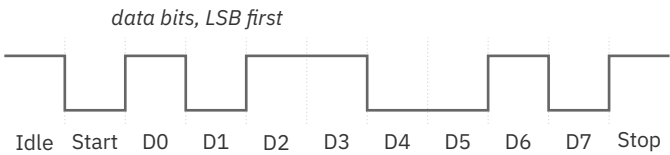


Figure 11.1. UART transmit frame

## Configuration

| Register | Field      | Purpose                                                   |
|----------|------------|-----------------------------------------------------------|
| X[31:16] | Bit period | System clocks per bit (integer part)                      |
| X[15:10] | Fractional | Base-2 fractional clocks (1/64 increments)                |
| X[4:0]   | Bit count  | Word size minus 1 (write 7 for 8-bit; supports 1-32 bits) |
| Y[31:0]  | Data       | Transmit data (LSB first)                                 |

## Baud Rate Calculation

### Basic formula (integer only):

```
X[31:16] = sysclk / baud_rate
```

Example: 200 MHz, 115200 baud

```
X[31:16] = 200,000,000 / 115,200 = 1736
```

**With fractional precision:**

```
bit_period = (sysclk / baud_rate) × 65536
X[31:10] = bit_period & $FFFFFFC00
```

Example: 200 MHz, 115200 baud

```
bit_period = (200,000,000 / 115,200) × 65536 = 113,777,778
```

```
X[31:10] = 113,777,778 & $FFFFFFC00 = $06C8_1C00
```

**Common Baud Rates at 200 MHz**

| Baud Rate | X[31:16] (integer) | X (with fractional) | Error |
|-----------|--------------------|---------------------|-------|
| 9600      | 20833              | \$5161_5400         | 0.00% |
| 19200     | 10417              | \$28B0_A800         | 0.00% |
| 38400     | 5208               | \$1458_5400         | 0.01% |
| 57600     | 3472               | \$0D90_3800         | 0.01% |
| 115200    | 1736               | \$06C8_1C00         | 0.01% |
| 230400    | 868                | \$0364_0C00         | 0.01% |
| 460800    | 434                | \$01B2_0400         | 0.01% |
| 921600    | 217                | \$00D9_0000         | 0.01% |
| 1000000   | 200                | \$00C8_0000         | 0.00% |

**Configuration Sequence****Spin2:**

```
CON
  _clkfreq = 200_000_000
  TX_PIN = 20
  BAUD = 115200

PUB uart_tx_init() | bit_period
  ' Calculate bit period with fractional precision
  bit_period := (_clkfreq / BAUD) << 16

  PINFLOAT(TX_PIN)
  WRPIN(TX_PIN, P_ASYNC_TX | P_OE)
  WXPIN(TX_PIN, bit_period | 7)      ' 8 data bits (X[4:0] = N - 1)
  PINLOW(TX_PIN)

PUB tx_byte(value)
  ' Wait until transmitter ready
```

*continues on next page →*

*↪ continued from previous page*

```
repeat until PINREAD(TX_PIN)
WYPIN(TX_PIN, value)
```

**PASM2:**

```

MOV      bit_period, ##(200_000_000 / 115200) << 16
OR       bit_period, #7      ' 8 data bits (X[4:0] = N - 1)

DIRL     #TX_PIN
WRPIN    ##(P_ASYNC_TX | P_OE), #TX_PIN
WXPIN    bit_period, #TX_PIN
DIRH     #TX_PIN

.wait    TESTP    #TX_PIN wc      ' Check IN flag
if_nc    JMP      #.wait
WYPIN    data, #TX_PIN          ' Transmit byte
```

**RS-232 Polarity**

Standard RS-232 uses inverted logic. Use `P_INVERT_OUTPUT` to match:

```
mode := P_ASYNC_TX | P_OE | P_INVERT_OUTPUT
```

**Parity Implementation**

P2 does not generate parity in hardware. Calculate and include parity as an extra bit:

```

PUB tx_byte_with_parity(value) | parity, data9
  ' Calculate even parity
  parity := value
  parity ^= parity >> 4
  parity ^= parity >> 2
  parity ^= parity >> 1
  parity &= 1

  ' Combine 8 data bits + parity
  data9 := value | (parity << 8)

  ' Configure for 9 bits (X[4:0] = N - 1 = 8)
  WXPIN(TX_PIN, (bit_period & $FFFF0000) | 8)

  repeat until PINREAD(TX_PIN)
  WYPIN(TX_PIN, data9)
```

## 11.3 P\_SYNC\_TX Mode (%11100)

### Function

P\_SYNC\_TX transmits synchronous serial data clocked by an external or companion clock signal. Data shifts out on clock edges, with no start or stop bits. This mode is suitable for SPI master transmission, shift register control, and other synchronous protocols.

### Critical: Clock Routing

By default, the B-input reads from the local pin, which is useless for synchronous transmission where the clock is on a different pin.

**Required:** Add a pin-selection constant to route the clock:

| Constant   | Clock Source |
|------------|--------------|
| P_PLUS1_B  | Pin + 1      |
| P_MINUS1_B | Pin - 1      |
| P_PLUS2_B  | Pin + 2      |
| P_MINUS2_B | Pin - 2      |
| P_PLUS3_B  | Pin + 3      |
| P_MINUS3_B | Pin - 3      |

### Wrong:

```
mode := P_SYNC_TX | P_OE          ' NO CLOCK ROUTING!
```

### Correct:

```
mode := P_SYNC_TX | P_OE | P_PLUS1_B    ' Clock from pin+1
```

### Configuration

| Register | Field     | Purpose                                     |
|----------|-----------|---------------------------------------------|
| X[5]     | Mode      | 0 = Continuous, 1 = Start-stop              |
| X[4:0]   | Bit count | Number of bits minus 1 (0-31 for 1-32 bits) |
| Y[31:0]  | Data      | Transmit data (LSB first)                   |

## Transmission Modes

### Continuous Mode (X[5] = 0):

- Double-buffered for gapless transmission
- Prime shifter with first data before enabling
- Buffer automatically loads to shifter after transmission
- IN flag indicates buffer empty

### Start-Stop Mode (X[5] = 1):

- Data can be modified before clock starts
- Suitable for non-continuous transmissions
- More control over timing

## Clock Edge Selection

| Clock Edge         | Configuration         |
|--------------------|-----------------------|
| Positive (rising)  | Default (no modifier) |
| Negative (falling) | Add P_INVERT_A        |

## Configuration Sequence

### Spin2 (SPI Master TX, Positive Edge):

```

CON
    TX_PIN = 41
    CLK_PIN = 40

PUB spi_master_init() | tx_mode, clk_mode
    ' Configure data pin
    tx_mode := P_SYNC_TX | P_OE | P_MINUS1_B ' Clock from CLK_PIN

    PINFLOAT(TX_PIN)
    WRPIN(TX_PIN, tx_mode)
    WXPIN(TX_PIN, %1_00111) ' Start-stop, 8 bits
    PINLOW(TX_PIN)

    ' Configure clock using transition mode
    clk_mode := P_TRANSITION | P_OE

    PINFLOAT(CLK_PIN)
    WRPIN(CLK_PIN, clk_mode)
    WXPIN(CLK_PIN, $1000) ' Clock period
    PINLOW(CLK_PIN)

```

continues on next page →

↪ continued from previous page

```

PUB spi_tx_byte(value)
  WYPIN(TX_PIN, value)          ' Load data
  WYPIN(CLK_PIN, 16)            ' 16 transitions = 8 clocks

```

**PASM2:**

```

' Setup sync serial TX (positive edge)
DIRL    #TX_PIN
WRPIN   ##(P_SYNC_TX | P_OE | P_MINUS1_B), #TX_PIN
WXPIN   #%1_00111, #TX_PIN    ' Start-stop, 8 bits
DIRH    #TX_PIN

' Setup clock generator
DIRL    #CLK_PIN
WRPIN   ##(P_TRANSITION | P_OE), #CLK_PIN
WXPIN   ##$1000, #CLK_PIN     ' Clock period
DIRH    #CLK_PIN

' Transmit byte
WYPIN   data, #TX_PIN         ' Load data
WYPIN   #16, #CLK_PIN        ' Generate 8 clock cycles

```

**MSB-First Transmission**

P\_SYNC\_TX transmits LSB first. For MSB-first protocols (like most SPI):

**Spin2:**

```

PUB spi_tx_msb_first(value) | reversed
  ' reverse the data bits for MSB-first
  reversed := value REV 8          ' Reverse 8 bits

  WYPIN(TX_PIN, reversed)
  WYPIN(CLK_PIN, 16)

```

**PASM2:**

```

REV      data          ' reverse the data bits, MSB-first
WYPIN    data, #TX_PIN
WYPIN    #16, #CLK_PIN

```

**Continuous Streaming**

For continuous data streams without gaps:

```

PUB continuous_stream()
  ' Prime the shifter before enabling
  WYPIN(TX_PIN, first_byte)

  ' Enable pin
  PINLOW(TX_PIN)

  ' Load second byte into buffer
  WYPIN(TX_PIN, second_byte)

  ' Continuous transmission loop
  repeat
    if PINREAD(TX_PIN)          ' IN raised = buffer ready
      WYPIN(TX_PIN, get_next_byte()) ' Load next

```

## 11.4 Clock Generation

### Using P\_TRANSITION for SPI Clock

The P\_TRANSITION mode generates clock signals for synchronous transmission:

```

CON
  CLK_PIN = 40

PUB clock_setup(period)
  PINFLOAT(CLK_PIN)
  WRPIN(CLK_PIN, P_TRANSITION | P_OE)
  WXPIN(CLK_PIN, period)          ' Clocks per half-period
  PINLOW(CLK_PIN)

PUB send_clocks(count)
  WYPIN(CLK_PIN, count * 2)      ' 2 transitions per clock

```

### Clock Polarity (CPOL)

| CPOL | Idle State | Configuration                    |
|------|------------|----------------------------------|
| 0    | Low        | Default                          |
| 1    | High       | Add P_INVERT_OUTPUT on clock pin |



## Clock Phase (CPHA)

| CPHA | Data Sample Edge  | Data Change Edge  |
|------|-------------------|-------------------|
| 0    | Leading (first)   | Trailing (second) |
| 1    | Trailing (second) | Leading (first)   |

For CPHA=1, use P\_INVERT\_A on the data pin.

## 11.5 Complete Examples

### Example 1: UART Debug Console

```

CON
    _clkfreq = 200_000_000
    TX_PIN = 62
    BAUD = 115200

VAR
    LONG bit_period

PUB start()
    bit_period := (_clkfreq / BAUD) << 16

    PINFLOAT(TX_PIN)
    WRPIN(TX_PIN, P_ASYNC_TX | P_OE)
    WXPIN(TX_PIN, bit_period | 7)          ' 8 data bits (X[4:0] = N - 1)
    PINLOW(TX_PIN)

PUB tx(c)
    repeat until PINREAD(TX_PIN)
    WYPIN(TX_PIN, c)

PUB str(s) | c
    repeat
        c := BYTE[s++]
        if c == 0
            quit
        tx(c)

PUB dec(value) | digits[12], count
    count := 0
    if value < 0
        tx("-")
        value := -value
    repeat
        digits[count++] := value // 10 + "0"
        value /= 10
    until value == 0

```

*continues on next page →*

*↪ continued from previous page*

```

repeat count
  tx(digits[--count])

PUB hex(value, digits)
  repeat digits
    digits--
    tx(lookupz((value >> (digits * 4)) & $F : "0".."9", "A".."F"))

PUB newline()
  tx(13)
  tx(10)

```

## Example 2: SPI Master (Mode 0)

```

CON
  _clkfreq = 200_000_000
  MOSI_PIN = 41
  CLK_PIN = 40
  CS_PIN = 39
  SPI_PERIOD = 50                                ' 2 MHz at 200 MHz sysclk

PUB spi_init()
  ' Configure MOSI
  PINFLOAT(MOSI_PIN)
  WRPIN(MOSI_PIN, P_SYNC_TX | P_OE | P_MINUS1_B)
  WXPIN(MOSI_PIN, %1_00111)                      ' Start-stop, 8 bits
  PINLOW(MOSI_PIN)

  ' Configure CLK
  PINFLOAT(CLK_PIN)
  WRPIN(CLK_PIN, P_TRANSITION | P_OE)
  WXPIN(CLK_PIN, SPI_PERIOD)
  PINLOW(CLK_PIN)

  ' Configure CS (active low)
  PINHIGH(CS_PIN)

PUB spi_select()
  PINLOW(CS_PIN)

PUB spi_deselect()
  PINHIGH(CS_PIN)

PUB spi_tx_byte(value) | reversed
  ' MSB first
  reversed := value REV 8                        ' reverse the data bits for MSB-first

  WYPIN(MOSI_PIN, reversed)
  WYPIN(CLK_PIN, 16)                             ' 8 clock cycles

  ' Wait for completion

```

*continues on next page →*

*↪ continued from previous page*

```

repeat until PINREAD(MOSI_PIN)

PUB spi_write_register(addr, value)
  spi_select()
  spi_tx_byte(addr)
  spi_tx_byte(value)
  spi_deselect()

```

ch11-spi-master.spin2

### Example 3: Multi-Byte UART Transmission

```

CON
  _clkfreq = 200_000_000
  TX_PIN = 20
  BAUD = 1_000_000 ' 1 Mbps

PUB fast_uart_init() | bit_period
  bit_period := (_clkfreq / BAUD) << 16

  PINFLOAT(TX_PIN)
  WRPIN(TX_PIN, P_ASYNC_TX | P_OE)
  WXPIN(TX_PIN, bit_period | 7) ' 8 data bits (X[4:0] = N - 1)
  PINLOW(TX_PIN)

PUB tx_buffer(ptr, count) | i
  ' Transmit buffer as fast as possible
  repeat i from 0 to count - 1
    repeat until PINREAD(TX_PIN)
      WYPIN(TX_PIN, BYTE[ptr + i])

```

### Example 4: PASM2 High-Speed Serial

```

CON
  _clkfreq = 200_000_000

DAT          ORG

' Initialize async TX
  MOV      x_val, ##(200_000_000 / 115200) << 16
  OR       x_val, #8

  DIRL     #TX_PIN
  WRPIN    ##(P_ASYNC_TX | P_OE), #TX_PIN
  WXPIN    x_val, #TX_PIN
  DIRH     #TX_PIN

' Transmit message

```

*continues on next page →*

→ continued from previous page

```

    tx_loop      MOV      ptra, ##message

                RDBYTE    data, ptra++
                CMP       data, #0 wz
    if_z         JMP      #done

    .wait        TESTP    #TX_PIN wc
    if_nc        JMP      #.wait

                WYPIN     data, #TX_PIN
                JMP       #tx_loop

done            JMP      #$.

TX_PIN          LONG     20
x_val           LONG     0
data            LONG     0
message         BYTE     "Hello, PASM2!", 13, 10, 0
                ALIGNL

```

## 11.6 Baud Rate Error Analysis

### Error Calculation

```

actual_baud = sysclk / round(sysclk / target_baud)
error = ABS(actual_baud - target_baud) / target_baud × 100%

```

### Maximum Allowable Error

UART receivers typically tolerate  $\pm 2\text{--}3\%$  baud rate error. At 10 bits per frame (start + 8 data + stop), cumulative error must not exceed half a bit period.

### Fractional Timing Benefits

| Method        | Precision  | Error at 115200 baud |
|---------------|------------|----------------------|
| Integer only  | 1 clock    | 0.02%                |
| With X[15:10] | 1/64 clock | <0.001%              |

## 11.7 Quick Reference

### P\_ASYNC\_TX Configuration

| Parameter  | Register | Notes                                                     |
|------------|----------|-----------------------------------------------------------|
| Bit period | X[31:16] | sysclk / baud                                             |
| Fractional | X[15:10] | 1/64 clock precision                                      |
| Data bits  | X[4:0]   | Word size minus 1 (write 7 for 8-bit; supports 1-32 bits) |
| Data       | Y        | LSB first                                                 |
| Ready flag | IN       | High when ready                                           |

### P\_SYNC\_TX Configuration

| Parameter    | Register | Notes                      |
|--------------|----------|----------------------------|
| Mode         | X[5]     | 0=continuous, 1=start-stop |
| Bit count    | X[4:0]   | N-1 for N bits             |
| Data         | Y        | LSB first                  |
| Buffer empty | IN       | High when empty            |
| Clock source | Mode     | Must add P_PLUS1_B etc.    |

### Mode Constants

| Constant        | Value  | Purpose                |
|-----------------|--------|------------------------|
| P_ASYNC_TX      | %11110 | Async serial transmit  |
| P_SYNC_TX       | %11100 | Sync serial transmit   |
| P_OE            | -      | Enable output          |
| P_INVERT_OUTPUT | -      | Invert signal (RS-232) |
| P_PLUS1_B       | -      | Clock from pin+1       |
| P_MINUS1_B      | -      | Clock from pin-1       |

## Baud Rate Formula

```
X = (sysclk / baud) << 16 | data_bits
```

With fractional:

```
X = ((sysclk * 65536 / baud) & $FFFFFFC00) | data_bits
```

## Reset State (DIR=0)

- P\_ASYNC\_TX: Output high (idle state)
- P\_SYNC\_TX: Output low, data can be primed

*This chapter covered serial transmission modes. For serial reception modes, see Chapter 17. For other input modes, see Part III.*

## Part III: Input Modes

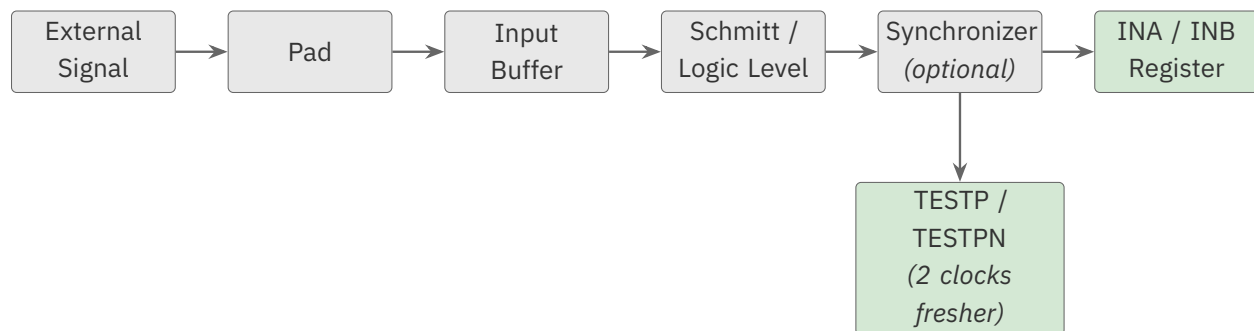
### Chapter 12: Digital Input

This chapter covers reading digital signals, from basic direct I/O through enhanced input conditioning. Topics include INA/INB registers, TESTP instruction, Schmitt trigger inputs, level comparison, and pull-up/pull-down resistors.

#### 12.1 Input Architecture

##### P2 Input Path

Every P2 I/O pin includes a complete input path with multiple conditioning options:



**Figure 12.1.** Input signal path

##### Input Timing

| Path             | Latency  | Data Freshness |
|------------------|----------|----------------|
| INA/INB register | 3 clocks | Older          |
| TESTP/TESTPN     | 2 clocks | Fresher        |

##### Default Input Mode

With no configuration (WRPIN = 0 or P\_NORMAL), pins operate as standard CMOS inputs with approximately 1.65V threshold.

## 12.2 Reading Input State

### Spin2 Methods

**PINREAD(pin)** - Read single pin state:

```
value := PINREAD(pin)           ' Returns 0 or 1
```

**PINR(pin)** - Alias for PINREAD:

```
value := PINR(pin)
```

**Reading via INA/INB:**

```
value := INA           ' All 32 bits of P0-P31
value := INB           ' All 32 bits of P32-P63
bit := (INA >> pin) & 1 ' Single bit extraction
```

### PASM2 Instructions

**TESTP** - Read pin to C or Z flag:

```
TESTP    #pin wc      ' Pin state → C flag
TESTP    #pin wz      ' Pin state → Z flag
if_c JMP  #pin_high   ' Branch if high
if_z JMP  #pin_low    ' Branch if zero (low)
```

**TESTPN** - Read inverted pin state:

```
TESTPN   #pin wc      ' Inverted state → C
if_c JMP  #pin_low    ' C=1 means pin was low
```

**TESTP Flag Operations:**

```
TESTP    #pin andc    ' C = C AND pin_state
TESTP    #pin orc     ' C = C OR pin_state
TESTP    #pin xor     ' C = C XOR pin_state
TESTP    #pin andz    ' Z = Z AND pin_state
TESTP    #pin orz     ' Z = Z OR pin_state
TESTP    #pin xorz    ' Z = Z XOR pin_state
```



**Reading INA/INB directly:**

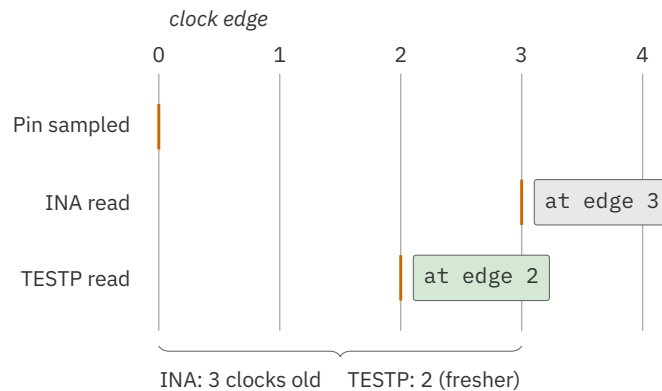
```

MOV    value, ina      ' Read P0-P31
MOV    value, inb      ' Read P32-P63
TEST   ina, mask wz    ' Test specific bits

```

**TESTP vs INA Timing**

TESTP reaches the pin a clock sooner than the INA/INB register path (the latencies are tabulated in §12.1; see also Ch1, Input Timing). For time-critical input sampling, prefer TESTP.



**Figure 12.2.** TESTP vs INA read latency

**12.3 Input Conditioning Options****P\_LOGIC\_A and P\_LOGIC\_B\_FB**

Standard CMOS logic input with ~1.65V threshold:

```

WRPIN(pin, P_LOGIC_A)      ' Default logic input
WRPIN(pin, P_LOGIC_B_FB)   ' Same, different internal routing

```

**P\_SCHMITT\_A**

Schmitt trigger input — its hysteresis (separate rising and falling thresholds) adds noise immunity and produces clean edges on slow or noisy signals. For how a Schmitt trigger works, see Ch2 §2.3.

```
WRPIN(pin, P_SCHMITT_A)
```

**Use when:**

- Input signal has slow edges
- Signal travels through noisy environment
- Preventing oscillation on threshold crossing

## TTL Threshold (via P\_LEVEL\_A)

There is no dedicated TTL-threshold constant. To detect a ~1.4V TTL crossing, use the programmable level comparator with a level value of 108 ( $1.4V \div 3.3V \times 256 \approx 108$ ):

```
WRPIN(pin, P_LEVEL_A | (108 << 8))    ' ~1.4V threshold (TTL)
PINFLOAT(pin)
```

### Use when:

- Interfacing with TTL logic
- Legacy 5V logic with reduced swing
- Signals that don't reach full CMOS levels

## P\_LEVEL\_A

Programmable level comparator input:

```
' Compare against 8-bit level value
' Level in M[7:0] (shifted into WRPIN value)
level := 128                                ' Mid-scale (approx 1.65V)
WRPIN(pin, P_LEVEL_A | (level << 8))
```

### Level calculation:

```
threshold_voltage = (level / 256) × 3.3V
```

| Level | Voltage |
|-------|---------|
| 0     | 0.0V    |
| 64    | 0.83V   |
| 128   | 1.65V   |
| 192   | 2.48V   |
| 255   | 3.28V   |

### Use when:

- Custom threshold required

- Detecting specific voltage levels
- Analog signal digitization

## 12.4 Pull-Up and Pull-Down Resistors

Pull resistors and when to use them are covered in Ch2 §2.2; this section gives the smart-pin constants and how to apply them to inputs.

### Available Options

| Constant    | Resistance | Current at 3.3V |
|-------------|------------|-----------------|
| P_HIGH_15K  | 15kΩ       | 220 μA          |
| P_HIGH_150K | 150kΩ      | 22 μA           |
| P_LOW_15K   | 15kΩ       | 220 μA          |
| P_LOW_150K  | 150kΩ      | 22 μA           |

### Configuration

#### Pull-up (for active-low buttons):

```
' 15kΩ pull-up to VDD
WRPIN(pin, P_HIGH_15K)
PINFLOAT(pin)                                ' Input mode
```

#### Pull-down (for active-high buttons):

```
' 15kΩ pull-down to GND
WRPIN(pin, P_LOW_15K)
PINFLOAT(pin)
```

#### Combined with input conditioning:

```
' Schmitt trigger input with pull-up
WRPIN(pin, P_SCHMITT_A | P_HIGH_15K)
PINFLOAT(pin)
```

## Choosing Resistance

| Resistance    | Advantages                 | Disadvantages                       |
|---------------|----------------------------|-------------------------------------|
| 15k $\Omega$  | Stronger pull, faster rise | Higher current draw                 |
| 150k $\Omega$ | Lower power                | Slower rise, more noise susceptible |

### 15k $\Omega$ recommended for:

- Mechanical switches and buttons
- Long wire runs
- Noisy environments

### 150k $\Omega$ suitable for:

- Battery-powered systems
- Short PCB traces
- Low-speed signals

## 12.5 Floating Input Behavior

### Why Inputs Float

When an input pin has no connection and no pull resistor:

- Input buffer amplifies internal noise
- State oscillates unpredictably
- High-speed transitions increase power consumption
- Can cause false triggering

### Detecting Floating Inputs

Floating inputs exhibit rapid state changes:

```
PUB detect_float(pin) : is_floating | count, i
  ' Count transitions in short period
  count := 0
  repeat i from 0 to 1000
    if PINREAD(pin) <> PINREAD(pin)
      count++

  is_floating := (count > 100)
```

## Preventing Float

**Always configure unused pins:**

```
' Option 1: Drive low
PINLOW(unused_pin)

' Option 2: Pull-down
WRPIN(unused_pin, P_LOW_150K)
PINFLOAT(unused_pin)

' Option 3: Pull-up
WRPIN(unused_pin, P_HIGH_150K)
PINFLOAT(unused_pin)
```

## 12.6 Multi-Pin Input Patterns

### Reading Pin Groups

**Spin2:**

```
' Read 8 pins starting at base_pin
pins_value := PINREAD(base_pin ADDPINS 7)

' Read specific pin range
value := INA.[base_pin + 7..base_pin]
```

**PASM2:**

```
' Read bits from INA
MOV      value, ina
SHR      value, #base_pin
AND      value, #$FF      ' Mask to 8 bits
```

### Atomic Multi-Pin Read

INA/INB provide atomic snapshot of all 32 pins:

```
' All pins read at same instant
snapshot_a := INA
snapshot_b := INB

' Extract fields
lower_byte := snapshot_a & $FF
upper_nibble := (snapshot_a >> 28) & $F
```

## Pin Field Extraction

### Spin2 pin field syntax:

```
' pins 8-11 (4 bits)
value := PINREAD(8 ADDPINS 3)

' Or using INA range
value := INA.[11..8]
```

## 12.7 Software Debouncing

### Why Debounce?

Mechanical switches and buttons bounce for 1-50ms after contact, causing multiple false transitions.

### Simple Delay Debounce

```
PUB read_button_debounced(pin) : state
  ' Wait for stable state
  state := PINREAD(pin)
  WAITMS(20)                ' Typical bounce period
  return PINREAD(pin)
```

### Integration Debounce

```
VAR
  LONG button_acc[8]          ' Accumulator per button

PUB update_buttons() | i, sample
  repeat i from 0 to 7
    sample := PINREAD(button_pins[i])
    if sample
      button_acc[i] := (button_acc[i] + 1) <# 10 ' Saturate at 10
    else
      button_acc[i] := (button_acc[i] - 1) #> 0  ' Floor at 0

PUB is_button_pressed(idx) : pressed
  pressed := (button_acc[idx] >= 8)          ' Threshold
```

## State Machine Debounce

```

CON
    DEBOUNCE_MS = 50

VAR
    LONG last_state
    LONG last_change_ms

PUB debounced_read(pin) : stable_state
    if PINREAD(pin) <> last_state
        if (GETMS() - last_change_ms) > DEBOUNCE_MS
            last_state := PINREAD(pin)
            last_change_ms := GETMS()
    stable_state := last_state

```

## 12.8 Active-Low Signals

### Understanding Active-Low

Many buttons and sensors use active-low signaling:

- Idle/released: Logic high (VDD through pull-up)
- Active/pressed: Logic low (grounded)

### Configuration

```

CON
    BUTTON_PIN = 20

PUB button_init()
    WRPIN(BUTTON_PIN, P_HIGH_15K)      ' Internal pull-up
    PINFLOAT(BUTTON_PIN)                ' Input mode

PUB is_pressed() : pressed
    pressed := NOT PINREAD(BUTTON_PIN)  ' Invert for natural sense

```

### Using TESTPN

PASM2 TESTPN provides inverted read:

```

        TESTPN    #BUTTON_PIN wc      ' C=1 when pin is LOW
if_c JMP          #button_pressed

```

## 12.9 Complete Examples

### Example 1: Button with LED

```

CON
    _clkfreq = 200_000_000
    LED_PIN = 56
    BUTTON_PIN = 57

PUB main()
    ' Configure LED as output
    PINLOW(LED_PIN)

    ' Configure button with pull-up and Schmitt trigger
    WRPIN(BUTTON_PIN, P_SCHMITT_A | P_HIGH_15K)
    PINFLOAT(BUTTON_PIN)

    ' Main loop
    repeat
        if NOT PINREAD(BUTTON_PIN)          ' Button pressed (active low)
            PINHIGH(LED_PIN)
        else
            PINLOW(LED_PIN)

ch12-button-schmitt-led.spin2

```

### Example 2: Multiple Button Input

```

CON
    _clkfreq = 200_000_000
    BUTTON_BASE = 20          ' Buttons on pins 20-23

PUB main() | buttons, last_buttons, i
    ' Configure 4 buttons with pull-ups
    repeat i from 0 to 3
        WRPIN(BUTTON_BASE + i, P_SCHMITT_A | P_HIGH_15K)
        PINFLOAT(BUTTON_BASE + i)

    last_buttons := 0

    repeat
        buttons := PINREAD(BUTTON_BASE ADDPINS 3)
        buttons := buttons XOR $F          ' Invert for active-low

        if buttons <> last_buttons
            process_buttons(buttons, last_buttons)
            last_buttons := buttons

    WAITMS(10)                  ' Debounce delay

```

*continues on next page →*



*↔ continued from previous page*

```

PUB process_buttons(current, previous) | i, pressed, released
  repeat i from 0 to 3
    pressed := (current.[i]) AND NOT (previous.[i])
    released := NOT (current.[i]) AND (previous.[i])

    if pressed
      DEBUG("Button ", i, " pressed")
    if released
      DEBUG("Button ", i, " released")

```

### Example 3: PASM2 Pin Polling

```

CON
  _clkfreq = 200_000_000

DAT          ORG

' Configure input pin
      MOV      pin, #BUTTON_PIN
      WRPIN    ##P_SCHMITT_A | P_HIGH_15K, pin
      DIRL     pin                    ' Input mode

' Wait for button press
wait_press
      TESTPN   pin wc                ' C=1 if pin low (pressed)
      if_nc    JMP      #wait_press

' Wait for release
wait_release
      TESTP    pin wc                ' C=1 if pin high (released)
      if_nc    JMP      #wait_release

      JMP      #wait_press          ' Wait for next press

BUTTON_PIN  LONG    20
pin          RES     1

```

### Example 4: Voltage Level Detection

```

CON
  _clkfreq = 200_000_000
  ANALOG_PIN = 10

PUB detect_voltage_ranges() : range | level, threshold
  ' Configure level comparator
  ' Test against multiple thresholds

  ' Test for >2.5V

```

*continues on next page →*

```

threshold := (250 * 256) / 330          ' 193
WRPIN(ANALOG_PIN, P_LEVEL_A | (threshold << 8))
PINFLOAT(ANALOG_PIN)
WAITUS(10)
if PINREAD(ANALOG_PIN)
    return 3                            ' Above 2.5V

' Test for >1.65V
threshold := 128                        ' Mid-scale
WRPIN(ANALOG_PIN, P_LEVEL_A | (threshold << 8))
WAITUS(10)
if PINREAD(ANALOG_PIN)
    return 2                            ' 1.65V to 2.5V

' Test for >0.83V
threshold := 64
WRPIN(ANALOG_PIN, P_LEVEL_A | (threshold << 8))
WAITUS(10)
if PINREAD(ANALOG_PIN)
    return 1                            ' 0.83V to 1.65V

return 0                                ' Below 0.83V

```

## 12.10 Input Timing Analysis

### Propagation Delay

From external signal to INA/INB register:

- Input buffer: ~2ns
- Synchronizer: ~1-2 clock cycles
- Register: 1 clock cycle
- Total: 3 clock cycles typical

At 200 MHz (5ns clock):

- 3 clocks = 15ns minimum
- Add external filter/conditioning time

### Sampling Considerations

For high-speed sampling:

- Use TESTP for 2-clock path (10ns at 200 MHz) — the fastest input path; the input synchronizer latency is inherent and cannot be bypassed
- Account for metastability in async signals

## Maximum Input Frequency

Theoretical maximum depends on sampling method:

- With 2-clock TESTP path: Up to  $\text{sysclk}/4$  (50 MHz at 200 MHz)
- Practical limit with noise margin:  $\text{sysclk}/8$  to  $\text{sysclk}/10$

## 12.11 Quick Reference

### Input Reading

| Method     | Spin2                   | PASM2         | Latency  |
|------------|-------------------------|---------------|----------|
| Single pin | PINREAD(pin)            | TESTP #pin wc | 2 clocks |
| Multi-pin  | PINREAD(base ADDPINS n) | mov val,ina   | 3 clocks |
| Register   | INA, INB                | ina, inb      | 3 clocks |

### Input Conditioning

| Constant    | Function                                                              |
|-------------|-----------------------------------------------------------------------|
| P_NORMAL    | Default CMOS input                                                    |
| P_LOGIC_A   | Logic input, OUT feedback                                             |
| P_SCHMITT_A | Schmitt trigger (adds input hysteresis)                               |
| P_LEVEL_A   | Programmable level comparator (use level=108 for ~1.4V TTL threshold) |

### Pull Resistors

| Constant    | Value                   | Use                     |
|-------------|-------------------------|-------------------------|
| P_HIGH_15K  | 15k $\Omega$ pull-up    | Buttons, noisy signals  |
| P_HIGH_150K | 150k $\Omega$ pull-up   | Low power, short traces |
| P_LOW_15K   | 15k $\Omega$ pull-down  | Active-high inputs      |
| P_LOW_150K  | 150k $\Omega$ pull-down | Low power               |

## Timing Summary

For input/output path latencies (INA/INB vs TESTP), see §12.1 and Ch1, Input Timing.

*This chapter covered basic digital input. For signal measurement modes (timing, counting), see Chapter 13. For serial reception, see Chapter 17.*

# Chapter 13: Timing Measurement

This chapter covers smart pin modes for measuring time intervals: **P\_STATE\_TICKS** (%10000) for timing both high and low states, **P\_HIGH\_TICKS** (%10001) for timing high states only, and **P\_EVENTS\_TICKS** (%10010) for event timing and timeout detection.

## 13.1 Timing Measurement Overview

### P2 Timing Capabilities

The P2 smart pin timing modes provide hardware-based time measurement with clock-cycle resolution. All measurements are in system clock cycles.

| Mode           | Function                    | Trigger                |
|----------------|-----------------------------|------------------------|
| P_STATE_TICKS  | Both high and low durations | Every transition       |
| P_HIGH_TICKS   | High state duration only    | High-to-low transition |
| P_EVENTS_TICKS | Time N events or timeout    | Event count or timeout |

### Resolution and Range

At 200 MHz sysclk:

- Resolution: 5 ns (1 clock cycle)
- Maximum measurement: 800000000 clocks = 10.74 seconds
- Overflow behavior: Z saturates at 800000000

### Divide-by-Zero-Safe Preload

On reset (DIR=0), **all three timing modes preload Z to 0000\_0001, not 0**. Software that reads Z before the first measurement completes therefore gets 1, never zero — which keeps a naive  $\text{period} / Z$  calculation from dividing by zero on the first window.

### Common Applications

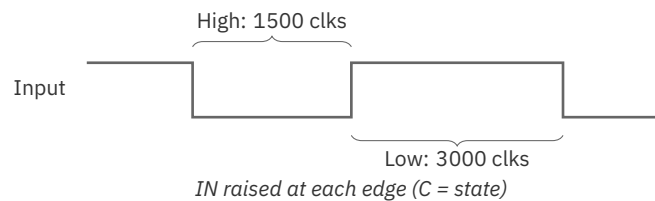
- PWM duty cycle analysis
- Pulse width measurement
- Frequency measurement
- Protocol timing verification
- Timeout/watchdog monitoring

## 13.2 P\_STATE\_TICKS Mode (%10000)

### Function

P\_STATE\_TICKS continuously measures the duration of each logic state (both high and low). On every transition, the previous state's duration is captured in Z and the state type is stored in the C flag.

### Operation



**Figure 13.1.** Pulse-width / state-time measurement

### Configuration

| Register | Purpose                                |
|----------|----------------------------------------|
| X        | Not used                               |
| Y        | Not used                               |
| Z        | Duration of previous state (clocks)    |
| C flag   | Previous state (1=was high, 0=was low) |
| IN flag  | Raised on every transition             |

On reset (DIR=0), IN is low and Z preloads to 1 — see §13.1.

**Bit 31 means different things across these modes.** In P\_STATE\_TICKS it is the captured C/s-tate flag (1 = the timed state was high); in P\_HIGH\_TICKS and P\_EVENTS\_TICKS it is the saturation/overflow indicator. The same & \$7FFF\_FFFF mask clears it in every mode, but read bit 31 according to the mode configured.

## Reading Measurements

### Spin2:

```

CON
    _clkfreq = 200_000_000
    INPUT_PIN = 20

PUB measure_states() | duration, was_high
    PINFLOAT(INPUT_PIN)
    WRPIN(INPUT_PIN, P_STATE_TICKS)
    PINLOW(INPUT_PIN)                ' Enable

    repeat
        repeat until PINREAD(INPUT_PIN)    ' Wait for transition
        duration := RDPIN(INPUT_PIN)

        ' Check C flag (bit 31 of RDPIN result indicates C)
        was_high := (duration >> 31) & 1
        duration &= $7FFFFFFF            ' Mask off C flag

    if was_high
        DEBUG("High time: ", UDEC_(duration), " clocks")
    else
        DEBUG("Low time: ", UDEC_(duration), " clocks")

```

**Enabling the smart pin.** The `PINLOW(INPUT_PIN)` ' Enable line just sets `DIR = 1` — `PINLOW`, `PINHIGH`, and `DIRH` all do this, and the `OUT` level is irrelevant in input modes (see Ch3 §3.4).

**PINREAD here means the IN flag.** In the wait loop, `PINREAD(INPUT_PIN)` returns the smart pin's IN flag (measurement-ready), *not* the pin's logic level as it did in Ch12 §12.2 — the same `CALL` carries two meanings depending on whether the pin is in smart-pin mode (see Ch3 §3.3 and Ch5 §5.1).

### PASM2:

```

                DIRL      #INPUT_PIN
                WRPIN     ##P_STATE_TICKS, #INPUT_PIN
                DIRH      #INPUT_PIN

.loop          TESTP     #INPUT_PIN wc      ' Wait for IN flag
              if_nc JMP  #.loop

                RDPIN     duration, #INPUT_PIN wc  ' Read duration, C=state
              if_c  MOV   high_time, duration
              if_nc MOV   low_time, duration

                JMP       #.loop

```

## PWM Analysis Example

```

CON
  _clkfreq = 200_000_000
  PWM_PIN = 20

VAR
  LONG high_time, low_time

PUB analyze_pwm() : frequency, duty_percent
  PINFLOAT(PWM_PIN)
  WRPIN(PWM_PIN, P_STATE_TICKS | P_SCHMITT_A)
  PINLOW(PWM_PIN)

  ' Get one complete cycle
  repeat 2
    repeat until PINREAD(PWM_PIN)
    if RDPIN(PWM_PIN) & $8000_0000      ' C flag = was high
      high_time := RDPIN(PWM_PIN) & $7FFF_FFFF
    else
      low_time := RDPIN(PWM_PIN) & $7FFF_FFFF

  ' Calculate results
  frequency := _clkfreq / (high_time + low_time)
  duty_percent := MULDIV64(high_time, 100, high_time + low_time)

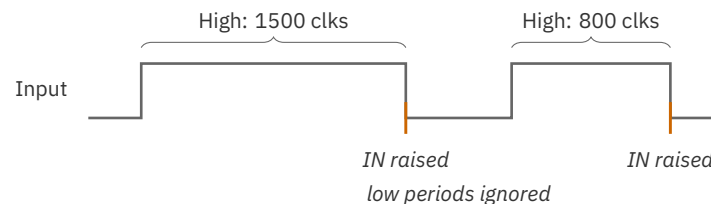
```

## 13.3 P\_HIGH\_TICKS Mode (%10001)

### Function

P\_HIGH\_TICKS measures only the duration of high states. On each high-to-low transition, the high time is captured in Z and IN is raised. Low periods are ignored.

### Operation



**Figure 13.2.** P\_HIGH\_TICKS high-time measurement



## Configuration

| Register | Purpose                                  |
|----------|------------------------------------------|
| X        | Not used                                 |
| Y        | Not used                                 |
| Z        | Duration of previous high state (clocks) |
| IN flag  | Raised on high-to-low transition         |

On reset (DIR=0), IN is low and Z preloads to 1 (see §13.1). Z saturates at \$8000\_0000; in this mode bit 31 is the overflow indicator, which is why the read examples mask with \$7FFF\_FFFF.

## Pulse Width Measurement

### Spin2:

```

CON
  _clkfreq = 200_000_000
  PULSE_PIN = 20

PUB measure_pulse_width() : width_us | clocks
  PINFLOAT(PULSE_PIN)
  WRPIN(PULSE_PIN, P_HIGH_TICKS)
  PINLOW(PULSE_PIN)

  ' Wait for pulse to complete
  repeat until PINREAD(PULSE_PIN)

  clocks := RDPIN(PULSE_PIN) & $7FFF_FFFF
  width_us := clocks / (_clkfreq / 1_000_000)

```

### PASM2:

```

                DIRL      #PULSE_PIN
                WRPIN     ##P_HIGH_TICKS, #PULSE_PIN
                DIRH      #PULSE_PIN

.wait          TESTP     #PULSE_PIN wc
              if_nc JMP   #.wait

                RDPIN     pulse_width, #PULSE_PIN
                AND       pulse_width, ##$7FFFFFFF

```

## Servo Pulse Measurement

Hobby servos use 1-2ms pulses at 50 Hz:

```
CON
    _clkfreq = 200_000_000
    SERVO_PIN = 20

PUB read_servo_pulse() : position_us | clocks
    PINFLOAT(SERVO_PIN)
    WRPIN(SERVO_PIN, P_HIGH_TICKS | P_SCHMITT_A)
    PINLOW(SERVO_PIN)

    repeat until PINREAD(SERVO_PIN)

    clocks := RDPIN(SERVO_PIN) & $7FFF_FFFF
    position_us := clocks / (_clkfreq / 1_000_000)

    ' Expected range: 1000-2000 µs
    ' 1000 µs = 0°, 1500 µs = 90°, 2000 µs = 180°
```

## Measuring Low Periods

Use P\_INVERT\_A to measure low periods instead:

```
' Measure low time by inverting input
WRPIN(pin, P_HIGH_TICKS | P_INVERT_A)
```

## 13.4 P\_EVENTS\_TICKS Mode (%10010)

### Function

P\_EVENTS\_TICKS operates in two modes controlled by Y[2]:

- **Event timing (Y[2]=0):** Measures time for X events to occur
- **Timeout detection (Y[2]=1):** Detects when no event occurs within X clocks

On reset (DIR=0), IN is low and Z preloads to 1 in both sub-modes (see §13.1).

## Event Type Selection

Y[1:0] selects what constitutes an event:

| Y[1:0] | Event Type           |
|--------|----------------------|
| %00    | A-input high (level) |
| %01    | A-input rising edge  |
| %1x    | A-input any edge     |

## Event Timing Mode (Y[2]=0)

Measures time until X events occur:

**Spin2:**

```

CON
  _clkfreq = 200_000_000
  FREQ_PIN = 20

PUB measure_frequency() : freq_hz | clocks, events
  events := 100                                ' Count 100 edges

  PINFLOAT(FREQ_PIN)
  WRPIN(FREQ_PIN, P_EVENTS_TICKS)
  WXPIN(FREQ_PIN, events)                      ' X = event count
  WYPIN(FREQ_PIN, %01)                         ' Y[1:0] = rising edge, Y[2]=0
  PINLOW(FREQ_PIN)

  repeat until PINREAD(FREQ_PIN)               ' Wait for N events

  clocks := RDPIN(FREQ_PIN) & $7FFF_FFFF
  freq_hz := (_clkfreq * events) / clocks

```

**PASM2:**

```

DIRL      #FREQ_PIN
WRPIN     ##P_EVENTS_TICKS, #FREQ_PIN
WXPIN     #100, #FREQ_PIN      ' 100 events
WYPIN     ##%01, #FREQ_PIN    ' Rising edges
DIRH      #FREQ_PIN

.wait     TESTP      #FREQ_PIN wc
        if_nc JMP     #.wait

        RDPIN      period, #FREQ_PIN
        AND        period, ##$7FFFFFFF

        ' frequency = MULDIV64(sysclk, 100, period)

```

**Timeout Detection Mode (Y[2]=1)**

Detects missing events (communication watchdog):

**Spin2:**

```

CON
  _clkfreq = 200_000_000
  COMM_PIN = 20
  TIMEOUT_MS = 100                                ' 100ms timeout

PUB comm_watchdog() | timeout_clocks, elapsed
  timeout_clocks := (_clkfreq / 1000) * TIMEOUT_MS

  PINFLOAT(COMM_PIN)
  WRPIN(COMM_PIN, P_EVENTS_TICKS)
  WXPIN(COMM_PIN, timeout_clocks)                  ' X = timeout clocks
  WYPIN(COMM_PIN, %101)                            ' Y[2]=1 (timeout), Y[1:0]=01 (rise)
  PINLOW(COMM_PIN)

  repeat
    if PINREAD(COMM_PIN)                          ' IN flag = timeout occurred
      elapsed := RDPIN(COMM_PIN) & $7FFF_FFFF
      DEBUG("Comm timeout! ", UDEC(elapsed), " clocks since last")
      handle_timeout()

    WAITMS(10)                                     ' Check periodically

PUB handle_timeout()
  ' Application-specific timeout response -
  ' flash an LED, reset peripheral, etc.

```

**PASM2:**

```

DIRL      #COMM_PIN
WRPIN     ##P_EVENTS_TICKS, #COMM_PIN
WXPIN     ##20_000_000, #COMM_PIN ' 100ms at 200MHz
WYPIN     #%101, #COMM_PIN ' Timeout on missing edge
DIRH      #COMM_PIN

.monitor  TESTP      #COMM_PIN wc      ' Check for timeout
        if_c  CALL    #timeout_handler
        JMP    #.monitor

timeout_handler
        RDPIN      elapsed, #COMM_PIN ' Clocks since last edge
        RET

```

## Continuous vs Retriggering

In timeout mode:

- Event resets timer and Z to 1
- Timeout raises IN and restarts timer
- Z always contains clocks since last event

In event-timing mode (Y[2]=0), reading the result with **RDPIN** acknowledges the measurement and **auto-restarts** it — the next RDPIN returns the interval to the following event, so no explicit re-arm is needed for back-to-back measurements. **RQPIN** is only a quiet peek: it returns the current value *without* acknowledging, so it does not clear IN and does not restart the measurement (see Ch15 §15.3).

## 13.5 Input Signal Routing

### Using Adjacent Pin Inputs

For signals on adjacent pins, use input routing constants:

| Constant   | Source  |
|------------|---------|
| P_PLUS1_A  | Pin + 1 |
| P_MINUS1_A | Pin - 1 |
| P_PLUS2_A  | Pin + 2 |
| P_MINUS2_A | Pin - 2 |
| P_PLUS3_A  | Pin + 3 |
| P_MINUS3_A | Pin - 3 |

**Example:**

```
' Measure signal on pin 21 using smart pin on pin 20
WRPIN(20, P_HIGH_TICKS | P_PLUS1_A)
```

**Input Conditioning for Timing**

Always use input conditioning for reliable timing:

```
' Add Schmitt trigger for clean edges
WRPIN(pin, P_STATE_TICKS | P_SCHMITT_A)

' Add filtering for noisy signals
WRPIN(pin, P_HIGH_TICKS | P_FILT1_AB)
```

**13.6 Accuracy Analysis****Measurement Resolution**

| sysclk  | Resolution | Max Measurable |
|---------|------------|----------------|
| 100 MHz | 10 ns      | 21.47 s        |
| 180 MHz | 5.56 ns    | 11.93 s        |
| 250 MHz | 4 ns       | 8.59 s         |
| 350 MHz | 2.86 ns    | 6.14 s         |

**Error Sources****Quantization error:**

- $\pm 1$  clock cycle inherent uncertainty
- Relative error decreases with longer measurements

**For frequency measurement:**

```
error = 1 / (events * measured_period)
```

Example: 100 edges, 10 kHz signal

period = 10,000 clocks per edge

total = 1,000,000 clocks

error = 1 / 1,000,000 = 0.0001% = 1 ppm

**Averaging for Accuracy**

Measure multiple periods and average:

```
PUB measure_frequency_averaged(events, samples) : freq | total, i
  total := 0
  repeat i from 0 to samples - 1
    total += measure_single(events)

  freq := (_clkfreq * events * samples) / total
```

**13.7 Complete Examples****Example 1: Complete PWM Analyzer**

```
CON
  _clkfreq = 200_000_000
  PWM_PIN = 20

VAR
  LONG frequency
  LONG duty_percent
  LONG high_us
  LONG low_us

PUB pwm_analyzer() | h_clocks, l_clocks, got_high, got_low
  PINFLOAT(PWM_PIN)
  WRPIN(PWM_PIN, P_STATE_TICKS | P_SCHMITT_A)
  PINLOW(PWM_PIN)

  got_high := false
  got_low := false

  ' Capture one complete cycle
  repeat until got_high AND got_low
    repeat until PINREAD(PWM_PIN)

  if RDPIN(PWM_PIN) & $8000_0000
```

*continues on next page →*

*→ continued from previous page*

```

    h_clocks := RDPIN(PWM_PIN) & $7FFF_FFFF
    got_high := true
else
    l_clocks := RDPIN(PWM_PIN) & $7FFF_FFFF
    got_low := true

' Calculate results
frequency := _clkfreq / (h_clocks + l_clocks)
duty_percent := MULDIV64(h_clocks, 100, h_clocks + l_clocks)
high_us := h_clocks / (_clkfreq / 1_000_000)
low_us := l_clocks / (_clkfreq / 1_000_000)

DEBUG("Frequency: ", UDEC_(frequency), " Hz")
DEBUG("Duty: ", UDEC_(duty_percent), "%")
DEBUG("High: ", UDEC_(high_us), " µs")
DEBUG("Low: ", UDEC_(low_us), " µs")

```

## Example 2: Ultrasonic Distance Measurement

```

CON
    _clkfreq = 200_000_000
    TRIG_PIN = 20
    ECHO_PIN = 21

PUB measure_distance_cm() : distance | echo_us
    ' Configure echo pin for pulse timing
    PINFLOAT(ECHO_PIN)
    WRPIN(ECHO_PIN, P_HIGH_TICKS | P_SCHMITT_A)
    PINLOW(ECHO_PIN)

    ' Send 10µs trigger pulse
    PINHIGH(TRIG_PIN)
    WAITUS(10)
    PINLOW(TRIG_PIN)

    ' Wait for echo pulse to complete
    repeat until PINREAD(ECHO_PIN)

    echo_us := (RDPIN(ECHO_PIN) & $7FFF_FFFF) / (_clkfreq / 1_000_000)

    ' Distance = (echo_time / 2) / 29.1 µs/cm
    distance := echo_us / 58

```

ch13-ultrasonic-distance.spin2



### Example 3: PASM2 High-Speed Frequency Counter

```

CON
    _clkfreq = 200_000_000

DAT          ORG

' Initialize frequency measurement
    DIRL      #FREQ_PIN
    WRPIN     ##P_EVENTS_TICKS, #FREQ_PIN
    WXPIN     ##1000, #FREQ_PIN    ' 1000 edges
    WYPIN     #%11, #FREQ_PIN     ' Any edge
    DIRH      #FREQ_PIN

' Measure loop
freq_loop
.wait        TESTP    #FREQ_PIN wc
            if_nc JMP  #.wait

            RDPIN     period, #FREQ_PIN
            AND       period, ##$7FFFFFFF

            ' Calculate frequency = MULDIV64(sysclk, 1000, period)
            ' Store for main cog to read
            WRLONG    period, #period_hub

            ' Auto-restarts on read
            JMP       #freq_loop

FREQ_PIN     LONG      20
period       LONG      0
period_hub   LONG      0

```

### Example 4: Communication Watchdog

```

CON
    _clkfreq = 200_000_000
    RX_PIN = 63
    TIMEOUT_MS = 500

VAR
    LONG comm_ok
    LONG last_timeout

PUB comm_monitor() | timeout_clocks
    timeout_clocks := (_clkfreq / 1000) * TIMEOUT_MS

    PINFLOAT(RX_PIN)
    WRPIN(RX_PIN, P_EVENTS_TICKS | P_SCHMITT_A)
    WXPIN(RX_PIN, timeout_clocks)
    WYPIN(RX_PIN, %111)          ' Timeout on any edge

```

*continues on next page →*

```

PINLOW(RX_PIN)

comm_ok := true

repeat
    if PINREAD(RX_PIN)                                ' Timeout occurred
        comm_ok := false
        last_timeout := GETMS()
        DEBUG("Communication lost!")
    elseif NOT comm_ok
        comm_ok := true
        DEBUG("Communication restored")

WAITMS(50)

```

## 13.8 Quick Reference

### Timing Mode Summary

| Mode   | Constant       | Measures            | Trigger      |
|--------|----------------|---------------------|--------------|
| %10000 | P_STATE_TICKS  | High and low times  | Every edge   |
| %10001 | P_HIGH_TICKS   | High time only      | High→low     |
| %10010 | P_EVENTS_TICKS | N events or timeout | Configurable |

### P\_EVENTS\_TICKS Y Register

| Y Value | Mode        | Event Type  |
|---------|-------------|-------------|
| %000    | Time events | High level  |
| %001    | Time events | Rising edge |
| %01x    | Time events | Any edge    |
| %100    | Timeout     | High level  |
| %101    | Timeout     | Rising edge |
| %11x    | Timeout     | Any edge    |

## Time Calculations

```
frequency = sysclk / period_clocks
period_us = clocks / (sysclk / 1,000,000)
period_ms = clocks / (sysclk / 1,000)
duty_percent = high_clocks * 100 / (high_clocks + low_clocks)
```

## Common Input Modifiers

| Constant    | Effect                |
|-------------|-----------------------|
| P_SCHMITT_A | Schmitt trigger input |
| P_INVERT_A  | Invert input polarity |
| P_FILT1_AB  | Add input filtering   |
| P_PLUS1_A   | Input from pin+1      |

## Limits

- Maximum measurement: 800000000 clocks
- At 200 MHz: 10.74 seconds
- Overflow behavior: Saturates at max value

*This chapter covered timing measurement modes. For counting modes, see Chapter 14. For period measurement with more options, see Chapter 15.*

# Chapter 14: Counting Modes

This chapter covers smart pin counting modes: **P\_REG\_UP** (%01100) for gated edge counting, **P\_REG\_UP\_DOWN** (%01101) for accumulator up/down, **P\_COUNT\_RISES** (%01110) for edge counting with direction, **P\_COUNT\_HIGHS** (%01111) for high-time counting, and **P\_QUADRATURE** (%01011) for quadrature encoder decoding.

## 14.1 Counting Mode Overview

### Available Counting Modes

| Mode   | Constant      | Function                                 |
|--------|---------------|------------------------------------------|
| %01011 | P_QUADRATURE  | Quadrature encoder decoding              |
| %01100 | P_REG_UP      | Count A edges when B high (gated)        |
| %01101 | P_REG_UP_DOWN | Accumulate A edges, B controls direction |
| %01110 | P_COUNT_RISES | Count edges with optional up/down        |
| %01111 | P_COUNT_HIGHS | Count clocks while input high            |

### Common Features

All counting modes share these characteristics:

- 32-bit counter range
- Continuous or periodic measurement
- X register controls measurement period
- Z register holds count value
- IN flag indicates period completion

### Continuous vs Periodic Mode

#### Continuous (X=0):

- Counter runs indefinitely
- Read current value anytime with RDPIN/RQPIN
- No IN flag generation
- Suitable for position tracking

#### Periodic (X>0):

- Counts for X clock cycles

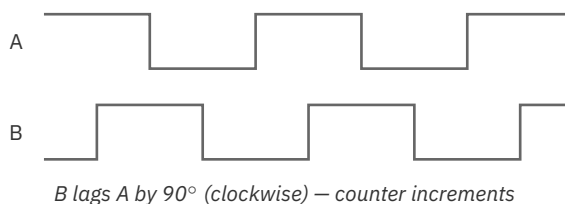
- Result placed in Z at period end
- IN flag raised at each period
- Counter continues with residual value
- Suitable for rate/velocity measurement

## 14.2 P\_QUADRATURE Mode (%01011)

### Function

P\_QUADRATURE decodes standard quadrature encoder signals (A/B phase with 90° offset). The counter increments or decrements based on rotation direction, providing position tracking with 4× resolution.

### Quadrature Signal Pattern



**Figure 14.1.** Quadrature A/B signals

### Configuration

| Register | Purpose                               |
|----------|---------------------------------------|
| X        | Period (0=continuous, >0=periodic)    |
| Y        | Not used                              |
| Z        | Quadrature step count (signed 32-bit) |

## Position Tracking (Continuous Mode)

### Spin2:

```

CON
    _clkfreq = 200_000_000
    ENC_A = 20                ' Encoder A signal
    ENC_B = 21                ' Encoder B signal (must be A+1)

PUB encoder_init()
    ' Configure quadrature decoder (uses A and B inputs)
    PINFLOAT(ENC_A)
    WRPIN(ENC_A, P_QUADRATURE | P_PLUS1_B) ' B from adjacent pin
    WXPIN(ENC_A, 0)                ' Continuous mode
    PINLOW(ENC_A)

PUB read_position() : position
    position := RDPIN(ENC_A)        ' Signed 32-bit position

PUB zero_position()
    PINFLOAT(ENC_A)                ' Pulse DIR low
    PINLOW(ENC_A)                  ' Re-enable

```

### PASM2:

```

                DIRL      #ENC_A
                WRPIN     ##P_QUADRATURE | P_PLUS1_B, #ENC_A
                WXPIN     #0, #ENC_A                ' Continuous
                DIRH      #ENC_A

.read          RDPIN     position, #ENC_A          ' Get position

```

**The raw count is 4× the detent count.** A quadrature encoder produces four transitions per detent (two on A, two on B), and this mode counts all of them — so the signed Z value advances by  $\pm 4$  per click. Reading Z directly (as `read_position()` does) gives the full 4×-resolution count, which is what fine positioning requires. When *detents* are needed, divide by four while preserving the sign with an arithmetic shift right by 2 — `RDPIN(ENC_A) ~> 2` in Spin2, or `SAR position, #2` in PASM2 — not a plain unsigned shift, which would corrupt negative (reverse) counts.

## Velocity Measurement (Periodic Mode)

```

CON
    _clkfreq = 200_000_000
    ENC_A = 20
    PERIOD_MS = 100          ' 100ms measurement

```

*continues on next page →*

*↪ continued from previous page*

```

PUB encoder_velocity_init() | period_clocks
    period_clocks := (_clkfreq / 1000) * PERIOD_MS

    PINFLOAT(ENC_A)
    WRPIN(ENC_A, P_QUADRATURE | P_PLUS1_B)
    WXPIN(ENC_A, period_clocks)          ' Periodic mode
    PINLOW(ENC_A)

PUB read_velocity() : steps_per_period
    repeat until PINREAD(ENC_A)          ' Wait for period
    steps_per_period := RDPIN(ENC_A)      ' Signed value

```

## Dual Encoder Setup

Use two smart pins for position and velocity simultaneously:

```

CON
    _clkfreq = 200_000_000
    POS_PIN = 20          ' Position tracking
    VEL_PIN = 22          ' Velocity measurement

PUB dual_encoder_init()
    ' Position on pin 20 (continuous)
    WRPIN(POS_PIN, P_QUADRATURE | P_PLUS1_B)
    WXPIN(POS_PIN, 0)
    PINLOW(POS_PIN)

    ' Velocity on pin 22 (periodic, same encoder signals)
    WRPIN(VEL_PIN, P_QUADRATURE | P_MINUS1_B)
    WXPIN(VEL_PIN, _clkfreq / 10)          ' 100ms period
    PINLOW(VEL_PIN)

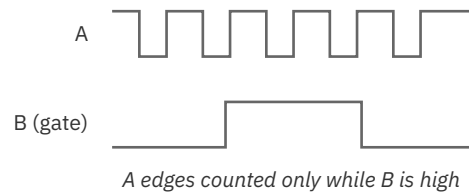
```

## 14.3 P\_REG\_UP Mode (%01100)

### Function

P\_REG\_UP counts positive edges on A-input, but only when B-input is high. This provides gated counting for frequency measurement and event counting with enable control.

## Operation



**Figure 14.2.** Gated edge counter

## Configuration

| Register | Purpose                            |
|----------|------------------------------------|
| X        | Period (0=continuous, >0=periodic) |
| Y        | Not used                           |
| Z        | Edge count                         |

## Gated Frequency Counter

```

CON
    _clkfreq = 200_000_000
    SIGNAL_PIN = 20
    GATE_PIN = 21
    GATE_TIME_MS = 1000                ' 1 second gate

PUB frequency_counter() : freq_hz | period_clocks
    period_clocks := (_clkfreq / 1000) * GATE_TIME_MS

    ' Configure gated counter
    PINFLOAT(SIGNAL_PIN)
    WRPIN(SIGNAL_PIN, P_REG_UP | P_PLUS1_B)
    WXPIN(SIGNAL_PIN, period_clocks)
    PINLOW(SIGNAL_PIN)

    ' Gate is controlled by B-input (pin 21)
    ' For hardware gate: connect gate signal to pin 21
    ' For software gate: drive pin 21 high to enable counting

    PINHIGH(GATE_PIN)                ' Enable counting

    repeat until PINREAD(SIGNAL_PIN)  ' Wait for period
    freq_hz := RDPIN(SIGNAL_PIN)      ' Edges in gate period
  
```



## Software-Controlled Gate

```
PUB gated_count_between(enable_pin, signal_pin) : count
  ' Count events while enable_pin is high
  PINFLOAT(signal_pin)
  WRPIN(signal_pin, P_REG_UP | P_PLUS1_B)
  WXPIN(signal_pin, 0)          ' Continuous
  PINLOW(signal_pin)

  PINHIGH(enable_pin)           ' Start counting
  WAITMS(1000)                  ' Count for 1 second
  PINLOW(enable_pin)             ' Stop counting

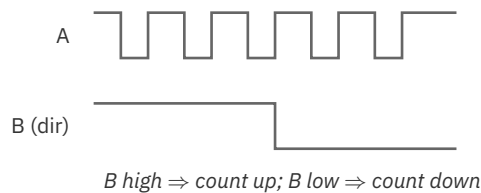
  count := RDPIN(signal_pin)
```

## 14.4 P\_REG\_UP\_DOWN Mode (%01101)

### Function

P\_REG\_UP\_DOWN accumulates A-input positive edges with direction controlled by B-input. When B is high, edges increment the counter. When B is low, edges decrement the counter.

### Operation



**Figure 14.3.** Up/down counter

### Configuration

| Register | Purpose                              |
|----------|--------------------------------------|
| X        | Period (0=continuous, >0=periodic)   |
| Y        | Not used                             |
| Z        | Signed count (increments/decrements) |

## Up/Down Counter

```

CON
  COUNT_PIN = 20
  DIR_PIN = 21                                ' High=up, Low=down

PUB updown_counter_init()
  PINFLOAT(COUNT_PIN)
  WRPIN(COUNT_PIN, P_REG_UP_DOWN | P_PLUS1_B)
  WXPIN(COUNT_PIN, 0)                        ' Continuous
  PINLOW(COUNT_PIN)

PUB read_count() : value
  value := RDPIN(COUNT_PIN)                  ' Signed result

PUB count_up()
  PINHIGH(DIR_PIN)                          ' Next edges increment

PUB count_down()
  PINLOW(DIR_PIN)                          ' Next edges decrement

```

## 14.5 P\_COUNT\_RISES Mode (%01110)

### Function

P\_COUNT\_RISES has two sub-modes controlled by Y[0]:

- Y[0]=0: Count A-input positive edges only
- Y[0]=1: Increment on A-input edge, decrement on B-input edge

### Single-Input Mode (Y[0]=0)

Simple edge counter on A-input:

```

CON
  PULSE_PIN = 20

PUB edge_counter_init()
  PINFLOAT(PULSE_PIN)
  WRPIN(PULSE_PIN, P_COUNT_RISES)
  WXPIN(PULSE_PIN, 0)                        ' Continuous
  WYPIN(PULSE_PIN, 0)                        ' Y[0]=0: A edges only
  PINLOW(PULSE_PIN)

PUB read_edge_count() : count
  count := RDPIN(PULSE_PIN)

```

## Dual-Input Mode (Y[0]=1)

Independent up/down on two signals:

```

CON
  UP_PIN = 20                ' A-input
  DOWN_PIN = 21              ' B-input

PUB dual_counter_init()
  PINFLOAT(UP_PIN)
  WRPIN(UP_PIN, P_COUNT_RISES | P_PLUS1_B)
  WXPIN(UP_PIN, 0)           ' Continuous
  WYPIN(UP_PIN, 1)           ' Y[0]=1: A up, B down
  PINLOW(UP_PIN)

PUB read_net_count() : value
  value := RDPIN(UP_PIN)     ' Net difference

```

## Periodic Rate Measurement

```

CON
  _clkfreq = 200_000_000
  EVENT_PIN = 20
  PERIOD_MS = 100

PUB event_rate() : events_per_period | period
  period := (_clkfreq / 1000) * PERIOD_MS

  PINFLOAT(EVENT_PIN)
  WRPIN(EVENT_PIN, P_COUNT_RISES)
  WXPIN(EVENT_PIN, period)
  WYPIN(EVENT_PIN, 0)
  PINLOW(EVENT_PIN)

  repeat until PINREAD(EVENT_PIN)
  events_per_period := RDPIN(EVENT_PIN)

```

## 14.6 P\_COUNT\_HIGHS Mode (%01111)

### Function

P\_COUNT\_HIGHS counts system clock cycles while input is in a particular state. Two sub-modes controlled by Y[0]:

- Y[0]=0: Count clocks while A-input high
- Y[0]=1: Increment clocks while A high, decrement while B high

## Duty Cycle Integration

```

CON
    _clkfreq = 200_000_000
    PWM_PIN = 20
    PERIOD_MS = 100

PUB measure_duty_cycle() : duty_percent | high_clocks, period_clocks
    period_clocks := (_clkfreq / 1000) * PERIOD_MS

    PINFLOAT(PWM_PIN)
    WRPIN(PWM_PIN, P_COUNT_HIGHS)
    WXPIN(PWM_PIN, period_clocks)
    WYPIN(PWM_PIN, 0)                ' Count A-high clocks
    PINLOW(PWM_PIN)

    repeat until PINREAD(PWM_PIN)
    high_clocks := RDPIN(PWM_PIN)

    duty_percent := MULDIV64(high_clocks, 100, period_clocks)

```

## Differential High-Time

Using Y[0]=1 for differential measurement:

```

CON
    _clkfreq = 200_000_000
    SIGNAL_A = 20
    SIGNAL_B = 21

PUB differential_high_time() : net_clocks | period
    period := _clkfreq / 10          ' 100ms

    PINFLOAT(SIGNAL_A)
    WRPIN(SIGNAL_A, P_COUNT_HIGHS | P_PLUS1_B)
    WXPIN(SIGNAL_A, period)
    WYPIN(SIGNAL_A, 1)                ' A increments, B decrements
    PINLOW(SIGNAL_A)

    repeat until PINREAD(SIGNAL_A)
    net_clocks := RDPIN(SIGNAL_A)    ' Signed difference

```

## 14.7 Input Signal Routing

### Adjacent Pin Selection

For modes using two inputs (A and B):

| Constant   | B-Input Source                           |
|------------|------------------------------------------|
| P_LOCAL_B  | Same pin (default)                       |
| P_PLUS1_B  | Pin + 1                                  |
| P_MINUS1_B | Pin - 1                                  |
| P_PLUS2_B  | Pin + 2                                  |
| P_MINUS2_B | Pin - 2                                  |
| P_PLUS3_B  | Pin + 3                                  |
| P_MINUS3_B | Pin - 3                                  |
| P_OUTBIT_B | This pin's own OUT bit (software-driven) |

P\_OUTBIT\_B (and the matching P\_OUTBIT\_A) routes the input from the pin's **OUT register bit** rather than a physical pin — so a cog can gate or step the counter purely in software, by writing OUT, with no external signal and no adjacent pin tied up. Useful for a software-controlled gate (e.g. enabling P\_REG\_UP counting for a measured interval) or for self-test.

### Input Conditioning

Add conditioning for reliable counting:

```
' Schmitt trigger for noisy signals
WRPIN(pin, P_COUNT_RISES | P_SCHMITT_A)

' Filter to reduce glitches
WRPIN(pin, P_QUADRATURE | P_FILT1_AB | P_PLUS1_B)

' Invert input polarity
WRPIN(pin, P_REG_UP | P_INVERT_A)
```

## 14.8 Counter Overflow and Range

### 32-Bit Counter Range

All counting modes use 32-bit counters:

- Unsigned modes: 0 to 4,294,967,295
- Signed modes: -2,147,483,648 to +2,147,483,647

### Overflow Behavior

Counters wrap on overflow:

```
Unsigned: $FFFFFFFF + 1 → $00000000
Signed:   $7FFFFFFF + 1 → $80000000
```

### Detecting Overflow

For high-count applications:

```
VAR
    LONG total_count
    LONG last_reading

PUB update_extended_count() | current, delta
    current := RDPIN(COUNT_PIN)
    delta := current - last_reading      ' Handles wrap
    total_count += delta
    last_reading := current
```

## 14.9 Mode Selection Guide

### Choosing the Right Mode

| Application          | Mode          | Configuration    |
|----------------------|---------------|------------------|
| Rotary encoder       | P_QUADRATURE  | X=0 for position |
| Frequency counter    | P_REG_UP      | X=gate_period    |
| Event counter        | P_COUNT_RISES | X=0, Y=0         |
| Up/down buttons      | P_COUNT_RISES | X=0, Y=1         |
| Step/direction motor | P_REG_UP_DOWN | X=0              |
| PWM duty cycle       | P_COUNT_HIGHS | X=period, Y=0    |
| Differential time    | P_COUNT_HIGHS | X=period, Y=1    |

## 14.10 Complete Examples

### Example 1: Frequency Counter

```

CON
  _clkfreq = 200_000_000
  FREQ_PIN = 20
  GATE_MS = 1000

PUB frequency_counter() : freq | period, count
  period := (_clkfreq / 1000) * GATE_MS

  PINFLOAT(FREQ_PIN)
  WRPIN(FREQ_PIN, P_COUNT_RISES | P_SCHMITT_A)
  WXPIN(FREQ_PIN, period)
  WYPIN(FREQ_PIN, 0)
  PINLOW(FREQ_PIN)

  repeat
    repeat until PINREAD(FREQ_PIN)
    count := RDPIN(FREQ_PIN)
    freq := count * (1000 / GATE_MS)      ' Scale to Hz

  DEBUG("Frequency: ", UDEC_(freq), " Hz")

```

## Example 2: Motor Position Control

```
CON
    _clkfreq = 200_000_000
    ENC_A = 20
    ENC_B = 21
    MOTOR_PWM = 30

VAR
    LONG target_position
    LONG current_position

PUB motor_control()
    ' Initialize encoder
    WRPIN(ENC_A, P_QUADRATURE | P_PLUS1_B | P_SCHMITT_A)
    WXPIN(ENC_A, 0)
    PINLOW(ENC_A)

    target_position := 0

    repeat
        current_position := RDPIN(ENC_A)
        adjust_motor(target_position - current_position)
        WAITMS(10)

PUB goto_position(pos)
    target_position := pos

PRI adjust_motor(error)
    ' Simple proportional control
    if error > 10
        motor_forward()
    elseif error < -10
        motor_reverse()
    else
        motor_stop()

PRI motor_forward()
    ' Application-specific: drive PWM/H-bridge high for forward direction
    PINH(MOTOR_PWM)

PRI motor_reverse()
    ' Application-specific: drive PWM/H-bridge for reverse direction
    PINL(MOTOR_PWM)

PRI motor_stop()
    ' Application-specific: stop motor (PWM=0 or coast)
    PINL(MOTOR_PWM)
```



### Example 3: PASM2 Event Counter

```

CON
    _clkfreq = 200_000_000

DAT          ORG

' Initialize edge counter
    DIRL      #EVENT_PIN
    WRPIN     ##P_COUNT_RISES, #EVENT_PIN
    WXPIN     #0, #EVENT_PIN      ' Continuous
    WYPIN     #0, #EVENT_PIN      ' A edges only
    DIRH      #EVENT_PIN

' Count loop
count_loop
    RDPIN     count, #EVENT_PIN    ' Read current count
    WRLONG    count, #count_hub    ' Store for main cog

    WAITX     ##200_000            ' 1ms update rate
    JMP       #count_loop

EVENT_PIN    LONG    20
count        LONG    0
count_hub    LONG    0

```

### Example 4: RPM Measurement

```

CON
    _clkfreq = 200_000_000
    TACH_PIN = 20
    PULSES_PER_REV = 1          ' Hall sensor
    SAMPLE_MS = 100

PUB measure_rpm() : rpm | period, pulses
    period := (_clkfreq / 1000) * SAMPLE_MS

    PINFLOAT(TACH_PIN)
    WRPIN(TACH_PIN, P_COUNT_RISES | P_SCHMITT_A)
    WXPIN(TACH_PIN, period)
    WYPIN(TACH_PIN, 0)
    PINLOW(TACH_PIN)

    repeat
        repeat until PINREAD(TACH_PIN)
        pulses := RDPIN(TACH_PIN)

        ' RPM = (pulses / pulses_per_rev) * (60000 / sample_ms)
        rpm := (pulses * 60000) / (PULSES_PER_REV * SAMPLE_MS)

    DEBUG("RPM: ", UDEC_(rpm))

```

*continues on next page →*

## 14.11 Quick Reference

### Mode Summary

| Mode          | Binary | A-Input   | B-Input      | Output        |
|---------------|--------|-----------|--------------|---------------|
| P_QUADRATURE  | %01011 | Phase A   | Phase B      | Position      |
| P_REG_UP      | %01100 | Events    | Gate         | Gated count   |
| P_REG_UP_DOWN | %01101 | Events    | Direction    | Up/down count |
| P_COUNT_RISES | %01110 | Up events | Down events* | Net count     |
| P_COUNT_HIGHS | %01111 | Time high | Time high*   | Clock count   |

\*When Y[0]=1

### Common Configurations

```
' Continuous position tracking
WXPIN(pin, 0)                                ' X=0 for continuous

' 100ms periodic measurement at 200 MHz
WXPIN(pin, 20_000_000)                       ' X = sysclk/10

' 1 second gate at 200 MHz
WXPIN(pin, 200_000_000)                      ' X = sysclk
```

### B-Input Routing

| Need       | Configuration     |
|------------|-------------------|
| B on pin+1 | mode   P_PLUS1_B  |
| B on pin-1 | mode   P_MINUS1_B |
| Invert B   | mode   P_INVERT_B |

## Reset Behavior

All counting modes when DIR=0:

- IN = low
- Z = initial adder value: 0 or +1 for unidirectional counters; bidirectional modes (quadrature, up/down) can also load -1, accounting for any edge coincident with reset
- Counter ready to start on DIR=1

*This chapter covered counting modes. For period measurement modes, see Chapter 15. For quadrature encoder details, see the P\_QUADRATURE section above.*

# Chapter 15: Frequency Measurement

## Periods, Duty & Reciprocal Counting

This chapter covers smart pin modes for measuring signal periods and calculating frequency. Two approaches are available: measuring over a fixed number of periods, or measuring over a fixed time window. Used together, these modes enable precise frequency and duty cycle determination.

### 15.1 Measurement Philosophy

#### Which Chapter and Mode?

Several smart-pin modes across Chapters 13–15 measure time-domain signal properties. Use this map to pick the right starting point:

| To measure...                           | Recommended mode(s)                                            | Where            |
|-----------------------------------------|----------------------------------------------------------------|------------------|
| Pulse width / high or low duration      | P_HIGH_TICKS, P_STATE_TICKS                                    | Ch13             |
| Time between events / timeout           | P_EVENTS_TICKS                                                 | Ch13             |
| Edge or event count                     | P_COUNT_RISES (and other counting modes)                       | Ch14             |
| Period (precise, frequency range known) | P_PERIODS_TICKS                                                | Ch15 §15.2       |
| Frequency (unknown or variable)         | P_COUNTER_PERIODS                                              | Ch15 §15.3       |
| Duty cycle                              | P_PERIODS_HIGHS + P_PERIODS_TICKS<br>(or the time-window pair) | Ch15 §15.2/§15.4 |

#### Two Approaches to Period Measurement

| Approach     | Modes             | Method                                           | Best For                                          |
|--------------|-------------------|--------------------------------------------------|---------------------------------------------------|
| Period-based | %10011,<br>%10100 | Count time or states over X periods              | Known frequency range, precise period measurement |
| Time-based   | %10101-<br>%10111 | Count time, states, or periods in X clock window | Unknown frequency, consistent update rate         |

## Why Multiple Concurrent Measurements?

The silicon documentation states: “At least two of these measurements must be made concurrently to get useful results.”

For frequency calculation:

```
frequency = periods / time
```

For duty cycle calculation:

```
duty_cycle = high_time / total_time
```

A single measurement provides either a count or a time, but calculating frequency or duty requires both.

**Compute these ratios with MULDIV64, not `*` and `/`.** Frequency and duty combine large values: `periods * sysclk` overflows a 32-bit long for any real signal — 100 periods times 200 MHz is already 20 billion, past the 4.29-billion limit — so a plain `(periods * sysclk) / time` silently returns a wrong number. Spin2’s `MULDIV64(a, b, divisor)` forms the `a * b` product in a 64-bit intermediate, then divides, so the result stays exact. Every frequency and duty calculation in this chapter uses it; so should yours.

## Trigger Sensitivity

All period measurement modes use Y[1:0] to select A/B input trigger combinations:

| Y[1:0] | Trigger          | Description                                            |
|--------|------------------|--------------------------------------------------------|
| %00    | A-rise to B-rise | Standard period: rising edge to rising edge            |
| %01    | A-rise to B-edge | A rising to any B transition                           |
| %10    | A-edge to B-rise | Any A transition to B rising                           |
| %11    | A-edge to B-edge | Any transition to any transition (maximum sensitivity) |

**Note:** The B-input reads the same pin as the A-input *by default* (when no `P_PLUSn_B` / `P_MINUSn_B` routing modifier is applied) — exactly what single-pin cycle measurement needs. No special constant is required.

## 15.2 Period-Based Modes (Measure X Periods)

### Mode %10011: P\_PERIODS\_TICKS

**Purpose:** Measure total time for X complete signal periods.

**Operation:**

1. Configure X register with number of periods to measure
2. Smart pin counts clock cycles from first trigger to completion of X periods
3. IN flag raised when measurement complete
4. RDPIN returns total clock cycles

**Registers:**

| Register | Function                         |
|----------|----------------------------------|
| X        | Number of periods to measure     |
| Y[1:0]   | Trigger sensitivity              |
| Z        | Total clock cycles for X periods |

**Configuration:**

```
' Measure time for 100 periods
PINSTART(pin, P_PERIODS_TICKS, 100, %00)
```

**Period Calculation:**

```
period_clocks = RDPIN(pin)           ' Total for X periods
single_period = period_clocks / X     ' Average period
frequency = sysclk / single_period    ' In Hz
```

### Mode %10100: P\_PERIODS\_HIGHS

**Purpose:** Measure total high-state time across X periods.

**Operation:**

1. Configure X register with number of periods to measure
2. Smart pin accumulates clock cycles when A-input is HIGH
3. IN flag raised when X periods complete
4. RDPIN returns total high-time clock cycles

**Registers:**

| Register | Function                                                        |
|----------|-----------------------------------------------------------------|
| X        | Number of periods to measure                                    |
| Y[1:0]   | Trigger sensitivity                                             |
| Z        | Total clock cycles A was HIGH across X periods (max \$80000000) |

**Configuration:**

```
' Measure high time across 100 periods
PINSTART(pin, P_PERIODS_HIGHS, 100, %00)
```

**Duty Cycle with Both Modes:**

```
CON
  _clkfreq = 200_000_000
  SIG_PIN = 20
  PERIODS = 100

PUB measure_duty() | total_time, high_time, duty_percent
  ' Start both measurements
  PINSTART(SIG_PIN, P_PERIODS_TICKS, PERIODS, %00)
  PINSTART(SIG_PIN+1, P_PERIODS_HIGHS, PERIODS, %00)

  ' Wait for completion
  REPEAT UNTIL PINREAD(SIG_PIN)

  total_time := RDPIN(SIG_PIN)           ' Total period time
  high_time := RDPIN(SIG_PIN+1)         ' Total high time

  duty_percent := MULDIV64(high_time, 100, total_time)
  DEBUG("Duty cycle: ", UDEC_(duty_percent), "%")
```

The loop waits on only SIG\_PIN, yet reads both pins. That is safe because both smart pins watch the same signal for the same number of periods, so they finish on the same edge — once SIG\_PIN's IN flag rises, SIG\_PIN+1's result is already latched and ready to read.

## 15.3 Time-Based Modes (Measure in X Clocks)

### Mode %10101: P\_COUNTER\_TICKS

**Purpose:** Measure total period time within a minimum X-clock window.

**Operation:**

1. Configure X register with minimum measurement window (clock cycles)

2. Smart pin measures until X clocks elapse AND current period completes
3. Accumulates total period time (clock cycles)
4. IN flag raised when measurement complete

**Registers:**

| Register | Function                                         |
|----------|--------------------------------------------------|
| X        | Minimum measurement window (clock cycles)        |
| Y[1:0]   | Trigger sensitivity                              |
| Z        | Total clock cycles for all periods within window |

**Key Difference from %10011:**

- %10011: “Measure time for exactly X periods”
- %10101: “Measure time for all periods within X clocks”

Because the window stretches to the end of the period already in progress, **Z reports the actual elapsed clocks — always  $\geq X$ , never exactly X**. Use Z, not the nominal X, as the time term in the math. That is also what makes concurrent measurement exact: run %10101, %10110, and %10111 together on the same signal with the same X, and because all three close on the same period-aligned window, frequency (periods / Z) and duty (high / Z) stay mutually consistent (see §15.4).

**Configuration:**

```
' Measure periods within 100ms window
window_clocks := _clkfreq / 10           ' 100ms
PINSTART(pin, P_COUNTER_TICKS, window_clocks, %00)
```

**Mode %10110: P\_COUNTER\_HIGHS**

**Purpose:** Measure total high-state time within a minimum X-clock window.

**Operation:**

1. Configure X register with minimum measurement window
2. Smart pin accumulates clock cycles when A-input is HIGH
3. Measurement continues until X clocks AND period completion
4. IN flag raised, RDPIN returns accumulated high time



**Registers:**

| Register | Function                                    |
|----------|---------------------------------------------|
| X        | Minimum measurement window (clock cycles)   |
| Y[1:0]   | Trigger sensitivity                         |
| Z        | Total clock cycles A was HIGH within window |

**Configuration:**

```
' Measure high time within 1-second window
PINSTART(pin, P_COUNTER_HIGHS, _clkfreq, %00)
```

**Mode %10111: P\_COUNTER\_PERIODS**

**Purpose:** Count complete periods within a minimum X-clock window.

**Operation:**

1. Configure X register with minimum measurement window
2. Smart pin counts complete periods
3. Measurement continues until X clocks AND period completion
4. IN flag raised, RDPIN returns period count

**Registers:**

| Register | Function                                    |
|----------|---------------------------------------------|
| X        | Minimum measurement window (clock cycles)   |
| Y[1:0]   | Trigger sensitivity                         |
| Z        | Number of complete periods (max \$80000000) |

**Configuration:**

```
' Count periods in 1-second window
PINSTART(pin, P_COUNTER_PERIODS, _clkfreq, %00)
```

## Frequency Calculation:

```
REPEAT UNTIL PINREAD(pin)
period_count := RDPIN(pin)
' For 1-second window, period_count = frequency in Hz
frequency := period_count
```

## Restart and Acknowledge

These modes restart automatically: a new measurement begins on the next trigger after the window completes, so they are not re-armed by hand. How the result is *read* decides whether the IN flag is cleared:

- **RDPIN** reads Z **and acknowledges** — it clears IN, so the next completed window can raise it again. Use RDPIN as the once-per-window read.
- **RQPIN** reads Z **quietly** — it does *not* clear IN. Use it to peek mid-stream without disturbing the IN-driven cadence; the matching RDPIN still does the acknowledge.

Reading with RDPIN each time IN rises gives exactly one fresh result per window, in lock-step with the hardware.

## 15.4 Combined Measurements

### Frequency and Duty Cycle Measurement

Using three pins simultaneously for complete signal characterization:

```
CON
_clkfreq = 200_000_000
PIN_TIME = 20                ' Measures total time
PIN_HIGH = 21                ' Measures high time
PIN_PERIODS = 22             ' Counts periods
WINDOW_MS = 100              ' 100ms measurement window

PUB measure_signal() | window, time_clks, high_clks, periods, freq, duty
window := (_clkfreq / 1000) * WINDOW_MS

' Configure all three measurements
PINSTART(PIN_TIME, P_COUNTER_TICKS, window, %00)
PINSTART(PIN_HIGH, P_COUNTER_HIGHS, window, %00)
PINSTART(PIN_PERIODS, P_COUNTER_PERIODS, window, %00)

REPEAT
' Wait for all measurements to complete
REPEAT UNTIL PINREAD(PIN_TIME) AND PINREAD(PIN_HIGH) ...
AND PINREAD(PIN_PERIODS)
```

*continues on next page →*

*↪ continued from previous page*

```

time_clks := RDPIN(PIN_TIME)           ' Actual measurement time
high_clks := RDPIN(PIN_HIGH)           ' Total high time
periods := RDPIN(PIN_PERIODS)          ' Period count

' Calculate frequency: periods / time
freq := MULDIV64(periods, _clkfreq, time_clks)

' Calculate duty: high_time / total_time
duty := MULDIV64(high_clks, 100, time_clks)

DEBUG("Frequency: ", UDEC_(freq), " Hz")
DEBUG("Duty cycle: ", UDEC_(duty), "%")
DEBUG("Periods: ", UDEC_(periods))
DEBUG("----")

```

**All three cells must watch the same signal.** Each PINSTART above measures the pin named, so the signal has to reach PIN\_TIME, PIN\_HIGH, and PIN\_PERIODS. Rather than wiring it to three pins, leave it on one and aim the other two cells at that pin with input routing. These period-aligned modes measure from an A-input rise to a B-input rise (Y = %00), so route **both** inputs to the observed pin: P\_MINUS1\_A | P\_MINUS1\_B and P\_MINUS2\_A | P\_MINUS2\_B make a cell take its A *and* B input from the pin one or two below it — so with the signal on PIN\_TIME, start PIN\_HIGH with P\_COUNTER\_HIGHS | P\_MINUS1\_A | P\_MINUS1\_B and PIN\_PERIODS with P\_COUNTER\_PERIODS | P\_MINUS2\_A | P\_MINUS2\_B. A cell watching a neighbor does not consume that pin; the observed pin stays free for its own use. (Route **both** inputs — verified on P2 silicon: with only A routed, each neighbor's B-input stays on its own idle pin, which never rises, so the period-aligned window never closes, the cell's IN flag stays low, and the REPEAT UNTIL never exits.)

## Why Three Measurements?

The actual measurement time extends beyond X clocks to complete the final period. Using P\_COUNTER\_TICKS provides the **actual** measurement duration, enabling precise calculations:

```

actual_frequency = MULDIV64(periods, sysclk, time_clks)
actual_duty = MULDIV64(high_clks, 100, time_clks)    ' percent

```

Without knowing the actual elapsed time, calculations would have error due to the period completion extension.

## 15.5 PASM2 Implementation

### Period Measurement

```

CON
    _clkfreq = 200_000_000
    SIG_PIN = 20
    PERIODS_TO_MEASURE = 1000

DAT          ORG

                ' Configure period measurement
DIRL          #SIG_PIN          ' Reset smart pin
WRPIN         ##P_PERIODS_TICKS, #SIG_PIN
WXPIN         ##PERIODS_TO_MEASURE, #SIG_PIN
WYPIN         #%00, #SIG_PIN    ' Rise to rise
DIRH          #SIG_PIN          ' Start measurement

.wait_done
    TESTP      #SIG_PIN wc      ' Check IN flag
    if_nc JMP   #.wait_done      ' Wait for completion

    RDPIN      total_time, #SIG_PIN ' Get total clock cycles

    ' Calculate single period time
    MOV        period_time, total_time
    QDIV        period_time, ##PERIODS_TO_MEASURE
    GETQX       period_time      ' Average period in clocks

    ' Calculate frequency: sysclk / period
    MOV        freq, ##_clkfreq
    QDIV        freq, period_time
    GETQX       freq             ' Frequency in Hz

    JMP        #.wait_done      ' Continuous measurement

total_time    RES    1
period_time   RES    1
freq          RES    1

```

### Time-Window Frequency Counter

```

CON
    _clkfreq = 200_000_000
    SIG_PIN = 20

DAT          ORG

                ' Configure 1-second window period counter
DIRL          #SIG_PIN

```

*continues on next page →*

*→ continued from previous page*

```

        WRPIN    ##P_COUNTER_PERIODS, #SIG_PIN
        WXPIN    ##_clkfreq, #SIG_PIN      ' 1-second window
        WYPIN    ##%00, #SIG_PIN
        DIRH     #SIG_PIN

.measure_loop
        TESTP    #SIG_PIN wc
        if_nc JMP    #.measure_loop

        RDPIN    frequency, #SIG_PIN      ' periods/sec = Hz

        ' frequency now contains Hz value
        ' Process or display...

        JMP      #.measure_loop

frequency RES    1

```

## 15.6 Application Examples

### Example 1: Simple Frequency Counter

```

CON
    _clkfreq = 200_000_000
    INPUT_PIN = 20
    GATE_TIME_MS = 1000                ' 1 second gate

PUB frequency_counter() | freq
    ' Count periods in 1-second window
    PINSTART(INPUT_PIN, P_COUNTER_PERIODS, _clkfreq, %00)

    DEBUG("Frequency Counter - 1 second gate")

    REPEAT
        REPEAT UNTIL PINREAD(INPUT_PIN)
        freq := RDPIN(INPUT_PIN)
        DEBUG("Frequency: ", UDEC_(freq), " Hz")

```

### Example 2: RPM Measurement

```

CON
    _clkfreq = 200_000_000
    TACH_PIN = 20
    PULSES_PER_REV = 2                ' 2 magnets on wheel

PUB measure_rpm() | periods, rpm, window
    ' 100ms measurement window

```

*continues on next page →*

```

window := _clkfreq / 10
PINSTART(TACH_PIN, P_COUNTER_PERIODS, window, %00)

REPEAT
  REPEAT UNTIL PINREAD(TACH_PIN)
  periods := RDPIN(TACH_PIN)

  ' Convert to RPM
  ' periods in 100ms = periods * 10 per second
  ' RPM = (periods * 10 * 60) / PULSES_PER_REV
  rpm := (periods * 600) / PULSES_PER_REV

  DEBUG("RPM: ", UDEC_(rpm))

```

### Example 3: PWM Analyzer

```

CON
  _clkfreq = 200_000_000
  PWM_PIN = 20
  NUM_PERIODS = 50 ' Average over 50 periods

PUB pwm_analyzer() | total_time, high_time, freq, duty, period_ns
  ' Use period-based measurement for PWM analysis
  PINSTART(PWM_PIN, P_PERIODS_TICKS, NUM_PERIODS, %00)
  PINSTART(PWM_PIN+1, P_PERIODS_HIGHS, NUM_PERIODS, %00)

  DEBUG("PWM Analyzer - averaging ", UDEC_(NUM_PERIODS), " periods")

  REPEAT
    REPEAT UNTIL PINREAD(PWM_PIN)

    total_time := RDPIN(PWM_PIN)
    high_time := RDPIN(PWM_PIN+1)

    ' Calculate frequency
    freq := MULDIV64(NUM_PERIODS, _clkfreq, total_time)

    ' Calculate duty cycle
    duty := MULDIV64(high_time, 1000, total_time) ' 0.1% resolution

    ' Calculate period in nanoseconds
    period_ns := MULDIV64(total_time, 1000, ...
      NUM_PERIODS * (_clkfreq / 1_000_000))

    DEBUG("Frequency: ", UDEC_(freq), " Hz")
    DEBUG("Duty cycle: ", UDEC_(duty/10), ".", UDEC_(duty//10), "%")
    DEBUG("Period: ", UDEC_(period_ns), " ns")
    DEBUG("----")

```

## Example 4: Precision Oscillator Calibration

```

CON
  _clkfreq = 200_000_000
  REF_PIN = 20                                ' Reference signal input
  TARGET_FREQ = 10_000_000                     ' 10 MHz target

PUB oscillator_calibration() | measured, error_ppm, periods
  ' Use many periods for high precision
  periods := 10000
  PINSTART(REF_PIN, P_PERIODS_TICKS, periods, %00)

  DEBUG("Oscillator Calibration")
  DEBUG("Target: ", UDEC_(TARGET_FREQ), " Hz")

  REPEAT
    REPEAT UNTIL PINREAD(REF_PIN)

    measured := RDPIN(REF_PIN)

    ' Expected clocks for TARGET_FREQ over periods cycles
    ' expected = periods * (sysclk / TARGET_FREQ)
    ' error_ppm = ((measured - expected) * 1_000_000) / expected

    ' Simplified: calculate measured frequency
    measured := MULDIV64(periods, _clkfreq, measured)

    ' Calculate error in ppm
    if measured >= TARGET_FREQ
      error_ppm := ((measured - TARGET_FREQ) * 1_000_000) / TARGET_FREQ
      DEBUG("Measured: ", UDEC_(measured), ...
        " Hz (+", UDEC_(error_ppm), " ppm)")
    else
      error_ppm := ((TARGET_FREQ - measured) * 1_000_000) / TARGET_FREQ
      DEBUG("Measured: ", UDEC_(measured), ...
        " Hz (-", UDEC_(error_ppm), " ppm)")

```

ch15-oscillator-calibration.spin2

## 15.7 Precision Considerations

### Measurement Resolution

| Mode              | Resolution    | Accuracy                  |
|-------------------|---------------|---------------------------|
| P_PERIODS_TICKS   | 1 clock cycle | $\pm 1$ clock per period  |
| P_COUNTER_PERIODS | 1 period      | $\pm 1$ period per window |

#### Improving Precision:

- Increase measurement periods (X) for period-based modes
- Increase time window for time-based modes
- Use higher sysclk frequency

### Error Sources

| Source           | Effect              | Mitigation                |
|------------------|---------------------|---------------------------|
| Quantization     | $\pm 1$ clock cycle | Measure more periods      |
| Trigger jitter   | Random error        | Use Schmitt trigger input |
| Clock accuracy   | Systematic error    | Use calibrated crystal    |
| Period variation | Averaged out        | Measure multiple periods  |

### Gate Time vs Resolution

| Gate Time | Resolution at 1 kHz | Resolution at 1 MHz |
|-----------|---------------------|---------------------|
| 10 ms     | 10 Hz (1%)          | 10 kHz (1%)         |
| 100 ms    | 1 Hz (0.1%)         | 1 kHz (0.1%)        |
| 1 second  | 0.1 Hz (0.01%)      | 100 Hz (0.01%)      |

## 15.8 Mode Selection Guide

### Choose P\_PERIODS\_TICKS (%10011) When:

- Signal frequency is approximately known
- Precise period measurement needed
- Consistent number of samples required



- Measuring periodic signals (clocks, PWM)

### Choose **P\_PERIODS\_HIGHS (%10100)** When:

- Duty cycle measurement needed
- Averaging duty over multiple periods
- Signal quality analysis required

### Choose **P\_COUNTER\_PERIODS (%10111)** When:

- Frequency is unknown or variable
- Need consistent update rate
- Simple frequency counting application
- RPM or event rate measurement

### Choose **P\_COUNTER\_TICKS (%10101)** When:

- Need actual measurement duration
- Combining with P\_COUNTER\_PERIODS for precision
- Time-windowed period analysis

### Choose **P\_COUNTER\_HIGHS (%10110)** When:

- Duty cycle in time window needed
- Combining with other time-window modes
- Variable frequency duty analysis

## 15.9 Quick Reference

### Mode Constants

| Mode   | Constant          | Description                        |
|--------|-------------------|------------------------------------|
| %10011 | P_PERIODS_TICKS   | For X periods, count clock cycles  |
| %10100 | P_PERIODS_HIGHS   | For X periods, count A-high cycles |
| %10101 | P_COUNTER_TICKS   | In X clocks, count period time     |
| %10110 | P_COUNTER_HIGHS   | In X clocks, count A-high time     |
| %10111 | P_COUNTER_PERIODS | In X clocks, count periods         |

## Trigger Sensitivity (Y[1:0])

| Value | Trigger          |
|-------|------------------|
| %00   | A-rise to B-rise |
| %01   | A-rise to B-edge |
| %10   | A-edge to B-rise |
| %11   | A-edge to B-edge |

## Common Modifiers

| Modifier   | Function                                           |
|------------|----------------------------------------------------|
| (default)  | B reads the same pin as A — single-pin measurement |
| P_PLUS1_B  | Use next pin as B-input                            |
| P_MINUS1_B | Use previous pin as B-input                        |
| P_FILT1_AB | Add input filtering                                |

## Frequency Formulas

### From period measurement (P\_PERIODS\_TICKS):

```
frequency = MULDIV64(num_periods, sysclk, rdpin_value)
```

### From period count (P\_COUNTER\_PERIODS):

```
frequency = MULDIV64(rdpin_value, sysclk, window_clocks)
' Or for 1-second window:
frequency = rdpin_value ' Direct Hz reading
```

## Duty Cycle Formulas

### From period-based modes:

```
duty_percent = MULDIV64(high_time, 100, total_time)
```

Where:

- high\_time = RDPIN from P\_PERIODS\_HIGHS
- total\_time = RDPIN from P\_PERIODS\_TICKS

---

*This chapter covered period and frequency measurement modes. For ADC input, see Chapter 16. For serial reception, see Chapter 17.*

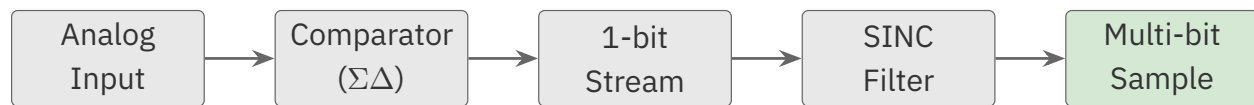
# Chapter 16: ADC (Analog Input)

This chapter covers the P2's analog-to-digital conversion capabilities using smart pin modes P\_ADC (%11000), P\_ADC\_EXT (%11001), and P\_ADC\_SCOPE (%11010). Topics include internal/external clocking, SINC filtering, gain settings, and triggered acquisition.

## 16.1 ADC Architecture

### Overview

The P2 includes a sigma-delta ADC on every I/O pin. Unlike traditional SAR or flash ADCs, sigma-delta ADCs oversample a single bit and use digital filtering to achieve multi-bit resolution. The smart pin modes provide hardware filtering with optional software post-processing.



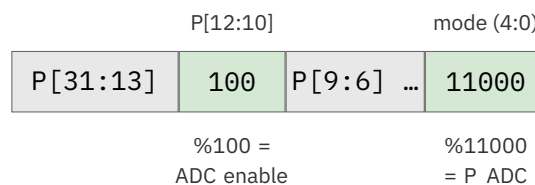
**Figure 16.1.** Sigma-delta ADC chain

### ADC Modes

| Mode   | Constant    | Description                                    |
|--------|-------------|------------------------------------------------|
| %11000 | P_ADC       | Internal clock ADC with filtering              |
| %11001 | P_ADC_EXT   | External clock ADC for delta-sigma integration |
| %11010 | P_ADC_SCOPE | Triggered oscilloscope-style capture           |

### Pin Configuration

ADC operation requires specific pin mode bits. Set P[12:10] = %100 in the WRPIN value:



**Figure 16.2.** ADC pin-configuration fields

## 16.2 ADC Input Modes

### Input Configuration Options

| Constant    | P[9:7] | Description       | Input Range  |
|-------------|--------|-------------------|--------------|
| P_ADC_GIO   | %000   | Ground-referenced | 0V to 3.3V   |
| P_ADC_VIO   | %001   | VIO-referenced    | VIO-relative |
| P_ADC_FLOAT | %010   | Floating input    | Self-biased  |
| P_ADC_1X    | %011   | 1x gain           | 0V to 3.3V   |
| P_ADC_3X    | %100   | 3.16x gain        | 0V to ~1.04V |
| P_ADC_10X   | %101   | 10x gain          | 0V to 330mV  |
| P_ADC_30X   | %110   | 31.6x gain        | 0V to ~104mV |
| P_ADC_100X  | %111   | 100x gain         | 0V to 33mV   |

### Choosing an Input Mode

#### P\_ADC\_GIO (Ground-referenced):

- Most common mode for general-purpose ADC
- Full 0V to 3.3V range
- Best for sensors and potentiometers

#### P\_ADC\_1X through P\_ADC\_100X (Gain modes):

- Amplify small signals before conversion
- Reduce noise by using more of the ADC range
- Higher gain = smaller input range

#### Example: Gain Selection

```
' For a 0-100mV sensor, use 30x gain
' 100mV × 31.6 = 3.16V (uses most of ADC range)
WRPIN(pin, P_ADC_30X | P_ADC)
```

## 16.3 Mode %11000: P\_ADC (Internal Clock)

### Operation

Samples the analog input at the system clock rate and applies SINC filtering to produce multi-bit samples. The filter type and sample period determine resolution and update rate.

### X Register Configuration

X[5:4]: Filter mode  
X[3:0]: Sample period =  $2^{(X[3:0])}$  clocks

#### Filter Modes:

| X[5:4] | Mode              | Description                              |
|--------|-------------------|------------------------------------------|
| %00    | SINC2 Sampling    | Complete conversion in hardware          |
| %01    | SINC2 Filtering   | Requires software difference computation |
| %10    | SINC3 Filtering   | Requires software multi-stage difference |
| %11    | Bitstream Capture | Raw bits (LSB = oldest)                  |

### Resolution and Sample Rate

| X[3:0] | Sample Period | SINC2 Sample | SINC2 Filter | SINC3 Filter | Bitstream   |
|--------|---------------|--------------|--------------|--------------|-------------|
| %0001  | 2 clocks      | 2 bits       | -            | -            | 2 new bits  |
| %0011  | 8 clocks      | 4 bits       | 4 ENOB       | -            | 8 new bits  |
| %0101  | 32 clocks     | 6 bits       | 6 ENOB       | 10 ENOB      | 32 new bits |
| %0111  | 128 clocks    | 8 bits       | 8 ENOB       | 14 ENOB      | overflow    |
| %1001  | 512 clocks    | 10 bits      | 10 ENOB      | 18 ENOB      | overflow    |
| %1011  | 2048 clocks   | 12 bits      | 12 ENOB      | overflow     | overflow    |
| %1101  | 8192 clocks   | 14 bits      | 14 ENOB      | overflow     | overflow    |

*ENOB = Effective Number of Bits*

**Beyond 14 bits — the instrumentation ceiling.** The table stops at 14 bits because that is the single-conversion SINC2 limit. Reaching further is possible by running SINC2 *filtering* mode fast and **summing many per-period differentials** over a long integration window (optionally with input gain ahead of it): each doubling of the accumulated sample count buys roughly another half-bit, and long integrations PUSH

into **16–17-bit / microvolt territory**. This is a *mechanism*, not a guaranteed specification — the absolute resolution actually achieved depends on the board, the source impedance, the VIO supply, and temperature (see §16.8 Accuracy Considerations, and the ratiometric method later in this section). Treat any specific ENOB figure as a bench result for a *particular* rig, not a datasheet value.

## Sample Rate Calculation

```
sample_rate = sysclk / 2^(X[3:0])
```

At 200 MHz with X[3:0] = %0111 (128 clocks):

```
sample_rate = 200_000_000 / 128 = 1,562,500 samples/sec
```

## SINC2 Sampling Mode (%00)

### Advantages:

- Complete conversion in hardware
- Just read RDPIN for latest sample
- Power-of-2 sample periods only

### Configuration:

```
CON
    _clkfreq = 200_000_000
    ADC_PIN = 46

PUB adc_init()
    ' Configure ADC with 8-bit SINC2 sampling
    WRPIN(ADC_PIN, P_ADC_GIO | P_ADC)
    WXPIN(ADC_PIN, %00_0111)           ' SINC2 sampling, 128 clocks
    PINH(ADC_PIN)                     ' Enable smart pin

PUB read_adc() : value
    value := RDPIN(ADC_PIN)           ' Get latest sample
```

## SINC2 Filtering Mode (%01)

Requires software post-processing to compute the difference between consecutive accumulator readings.

**Configuration:**

```

PUB sinc2_init()
  WRPIN(ADC_PIN, P_ADC_GIO | P_ADC)
  WXPIN(ADC_PIN, %01_0111)      ' SINC2 filtering, 128 clocks
  PINH(ADC_PIN)

PUB sinc2_read() : sample | acc
  REPEAT UNTIL PINREAD(ADC_PIN)  ' Wait for new sample
  acc := RDPIN(ADC_PIN)          ' Get accumulator
  sample := acc - last_acc       ' Compute difference
  last_acc := acc                ' Save for next time

```

**PASM2 Implementation:**

```

RDPIN    x, #ADC_PIN      ' Get SINC2 accumulator
SHL      x, #5            ' Prescale 27-bit to 32-bit
SUB      x, diff          ' Compute sample
ADD      diff, x          ' Update diff value
' x now contains the sample

```

**SINC3 Filtering Mode (%10)**

SINC3 provides better dynamic response than SINC2, doubling the effective bits for fast-changing signals. Limited to 512 samples per period due to 27-bit accumulator.

**Post-processing:**

```

RDPIN    x, #ADC_PIN      ' Get SINC3 accumulator
SHL      x, #5            ' Prescale to 32-bit
SUB      x, diff1         ' First difference
ADD      diff1, x
SUB      x, diff2         ' Second difference
ADD      diff2, x
SUB      x, diff3         ' Third difference
ADD      diff3, x
' x now contains the sample

```

**Two things the post-processing must get right.**

- **Warm-up.** The difference math depends on a valid prior accumulator state, so the filter is only accurate **from the second period for SINC2, and from the third period for SINC3**. Discard the first reading (SINC2) or the first two (SINC3) after starting.
- **Normalization.** To right-justify the differenced result, apply a final right-shift sized to the sample count:  $\text{LOG2}(\text{samples}) - 1$  bits for SINC2,  $\text{LOG2}(\text{samples})$  bits



for SINC3 (e.g. 128 samples → 6 for SINC2). The shift tracks the sample period, so it changes whenever X changes.

**Startup warm-up and source-switch flush are two different discards.** The warm-up above is a *one-time* settling of the differencing filter when the smart pin first starts. A **separate** discard applies every time the input source **changes** (for example GIO → VIO → pin in the instrumentation method below): switching the ADC's reference contaminates the **first 3 samples** — two for the SINC filter to decimate the step through, plus one for the analog front end to settle — so the **4th sample after a source switch is the first clean one**. A steady single-source reading pays only the startup warm-up, once; a method that rotates among sources pays the 3-sample flush on every switch.

## Bitstream Capture Mode (%11)

Captures raw ADC bitstream for custom processing algorithms.

```
PUB bitstream_init()
  WRPIN(ADC_PIN, P_ADC_GIO | P_ADC)
  WXPIN(ADC_PIN, %11_0101)           ' Bitstream, 32 bits
  PINH(ADC_PIN)

PUB read_bitstream() : bits
  REPEAT UNTIL PINREAD(ADC_PIN)
  bits := RDPIN(ADC_PIN)             ' 32 bits, LSB = oldest
```

## Ratiometric Absolute-Voltage Instrumentation

The gain and filter modes above turn the pin reading into a *number*, but that number is relative to the ADC's own internal references — which themselves drift with supply and temperature. To recover an **absolute** voltage in microvolts, measure the pin against the chip's two internal references and scale ratiometrically. This is the foundation of single-pin instrumentation measurement on the P2; the complete, runnable builds live in the P2AN001 application note, so the sketch here stays minimal.

**Read all three sources.** The ADC input can be switched among the internal ground reference (P\_ADC\_GIO), the internal supply reference (P\_ADC\_VIO), and the external pin. Absolute voltage needs **all three** — the shortcut of reading only P\_ADC\_FLOAT and the pin is far noisier, because the float point only *approximately* sits mid-supply. Read each reference from the same pin in turn, then place the pin between them:

```
uV = (pin - GIO) / (VIO - GIO) × 3,300,000
```

```

PUB read_microvolts() : uv | gio, vio, pin
  ' Read each reference from the same pin, in turn.
  gio := read_source(P_ADC_GIO)           ' internal ground reference
  vio := read_source(P_ADC_VIO)           ' internal supply reference
  pin := read_source(P_ADC_1X)            ' the external pin
  ' Ratiometric: where does the pin sit between GIO and VIO?
  ' muldiv64 keeps the (pin - gio) x 3_300_000 product at 64 bits.
  uv := muldiv64(pin - gio, 3_300_000, vio - gio)

PRI read_source(input_mode) : sample | acc, last
  WRPIN(ADC_PIN, input_mode | P_ADC)
  WXPIN(ADC_PIN, %01_0111)                ' SINC2 filtering, 128 clocks
  PINH(ADC_PIN)
  ' Switching the source contaminates the first 3 samples; the 4th is
  ' the first clean one (see the source-switch flush note above).
  last := RDPIN(ADC_PIN)
  REPEAT 4
    REPEAT UNTIL PINREAD(ADC_PIN)
    acc := RDPIN(ADC_PIN)
    sample := acc - last
    last := acc

```

**The references are local to the pin's power group.** The P2 powers its I/O pins in **isolated groups of four** — pins 0–3, 4–7, 8–11, ..., 60–63 — and each group shares a single VIO/GIO supply pair (P2 datasheet, pin descriptions). When a pin's ADC selects P\_ADC\_GIO or P\_ADC\_VIO, it measures *its own group's* ground and supply rails. This is what makes the single-pin ratiometric reading absolute: pin, GIO, and VIO are all referenced to the same local domain, so the supply and temperature drift common to all three divides out. It also carries a layout rule for multi-pin work (§16.6): pins tied together for one measurement should sit **within a single group**, so they share one reference domain — straddling a group boundary mixes supply domains and degrades the result.

**Handle the out-of-band cases.** Both edges of the formula are legitimate readings, not errors:

- **Below ground** (pin < GIO): pin - GIO is negative, so uv is negative — the signal sits below the ground reference (below 0 V).
- **Over-range** (pin > VIO): pin - GIO exceeds VIO - GIO, so uv exceeds 3,300,000 μV — the signal is above the supply reference. Clamp or flag these as the application requires.

How close the absolute number lands depends on the front-end limits in §16.8 — most importantly the matched-resistor absolute-error floor, which no amount of averaging removes.

## 16.4 Mode %11001: P\_ADC\_EXT (External Clock)

### Purpose

For interfacing with external delta-sigma ADC chips. Samples A-input data on B-input rising edges, allowing the P2 to apply SINC filtering to external ADC bitstreams.

### Configuration

```
CON
    DATA_PIN = 20                ' A-input: ADC data
    CLOCK_PIN = 21                ' B-input: ADC clock

PUB external_adc_init()
    ' External ADC with SINC2 sampling
    WRPIN(DATA_PIN, P_ADC_EXT | P_PLUS1_B) ' Use next pin as clock
    WXPIN(DATA_PIN, %00_0111)             ' SINC2, 8-bit
    PINH(DATA_PIN)
```

### Custom Sample Periods

Use WYPIN to override the power-of-2 period from X[3:0] with an arbitrary value in **Y[13:0]**:

The WYPIN override only applies when **X[5:4] > %00** — i.e. in SINC2 Filtering, SINC3 Filtering, or Bitstream modes. In **SINC2 Sampling (X[5:4] = %00)** the period is fixed by X[3:0] and WYPIN has no effect, so a non-power-of-2 rate there requires one of the filtering modes instead.

```
WRPIN(ADC_PIN, P_ADC_EXT | P_PLUS1_B)
WXPIN(ADC_PIN, %10_0111)                ' SINC3 base
WYPIN(ADC_PIN, 320)                     ' Override: 320 clock period
PINH(ADC_PIN)
```

### Accumulator Limits

| Filter | Max Period    | Why                              |
|--------|---------------|----------------------------------|
| SINC2  | 11,585 clocks | 27-bit accumulator: $2^{(27/2)}$ |
| SINC3  | 512 clocks    | 27-bit accumulator: $2^{(27/3)}$ |

## 16.5 Mode %11010: P\_ADC\_SCOPE (Triggered Capture)

### Purpose

Oscilloscope-style triggered acquisition for capturing signal events. Supports four simultaneous ADC channels with hysteretic triggering.

### Four-Channel Architecture

The scope mode captures from four consecutive pins simultaneously. Pin numbers must be multiples of 4 (0, 4, 8, ..., 60).

```
Pin group starting at 52:
  Pin 52: Channel 0 (AND trigger source)
  Pin 53: Channel 1
  Pin 54: Channel 2
  Pin 55: Channel 3
```

### Configuration

```
CON
  SCOPE_BASE = 52                                ' Must be multiple of 4

PUB scope_init(trigger_config)
  ' Configure 4 consecutive pins for scope mode
  WRPIN(SCOPE_BASE, P_ADC_GPIO | P_ADC_SCOPE)
  WXPIN(SCOPE_BASE, trigger_config)
  PINH(SCOPE_BASE)
```

### X Register: Trigger Configuration

```
X[15:10]: B (trigger) value, 6-bit MSB-justified (0-252, step 4)
X[7:2]:   A (arm) value, 6-bit MSB-justified (0-252, step 4)
X[1:0]:   Filter: %00 = 68-tap Tukey, %01 = 45-tap Tukey, %1x = 28-tap Hann
```

The hysteretic trigger works as follows:

1. Signal must cross arm level to arm the trigger
2. Signal must then cross trigger level to fire
3. Data capture begins after trigger fires

## Reading Scope Data

```

GETSCP    combined          ' Read all 4 channels (32-bit)
' combined = [ch3][ch2][ch1][ch0], 8 bits each

' Or read individual pins:
RDPIN     ch0, #SCOPE_BASE
RDPIN     ch1, #SCOPE_BASE+1
RDPIN     ch2, #SCOPE_BASE+2
RDPIN     ch3, #SCOPE_BASE+3

```

## 16.6 Multi-Channel ADC

**Power-domain layout.** The P2 powers its pins in isolated groups of four (§16.3) — pins 0–3, 4–7, ..., 60–63, each group on its own VIO/GIO pair. This shapes multi-channel layout two ways: each pin’s P\_ADC\_GIO/P\_ADC\_VIO references *its own* group, so an independent channel is self-consistent wherever it sits; but any pins tied to a *shared* node (as in a constant-impedance multi-pin instrument) must sit within one group to share a reference domain. The example below spans pins 40–47 — two full groups (40–43, 44–47) — which is fine because the channels are independent.

## Configuring Multiple Pins

Configure each pin individually:

```

CON
  ADC_BASE = 40
  NUM_CHANNELS = 8

PUB multi_adc_init() | ch
  REPEAT ch FROM 0 TO NUM_CHANNELS-1
    WRPIN(ADC_BASE + ch, P_ADC_GIO | P_ADC)
    WXPIN(ADC_BASE + ch, %00_0111)          ' 8-bit SINC2
    PINH(ADC_BASE + ch)

PUB read_all_channels(ptr) | ch
  REPEAT ch FROM 0 TO NUM_CHANNELS-1
    LONG[ptr][ch] := RDPIN(ADC_BASE + ch)

```

## Simultaneous Configuration

Configure multiple pins with a single WRPIN using pin group encoding:

```

' Configure pins 16-23 simultaneously
' Pin group: bits [10:6] = additional pins (7)
' Base pin: bits [5:0] = starting pin (16)
MOV      pinaddr, #%00111_010000    ' 8 pins starting at 16
WRPIN    adc_mode, pinaddr
WXPIN    #%00_0111, pinaddr
DIRH     pinaddr

```

## 16.7 Practical Examples

### Example 1: Simple Potentiometer Reading

```

CON
    _clkfreq = 200_000_000
    POT_PIN = 46
    LED_BASE = 56

PUB main() | adc_value, led_bits, i
    ' Initialize ADC - 8-bit, ~1.5 MHz sample rate
    WRPIN(POT_PIN, P_ADC_GIO | P_ADC)
    WXPIN(POT_PIN, %00_0111)
    PINH(POT_PIN)

    ' Initialize LED outputs
    REPEAT i FROM 0 TO 7
        PINLOW(LED_BASE + i)

    REPEAT
        adc_value := RDPIN(POT_PIN)

        ' Display value on 8 LEDs
        REPEAT i FROM 0 TO 7
            IF adc_value.[i]
                PINHIGH(LED_BASE + i)
            ELSE
                PINLOW(LED_BASE + i)

        WAITMS(50)

```

### Example 2: Audio Sampling

```

CON
    _clkfreq = 200_000_000
    AUDIO_PIN = 46
    SAMPLE_RATE = 44100
    BUFFER_SIZE = 1024

```

*continues on next page →*

→ continued from previous page

```

VAR
  LONG audio_buffer[BUFFER_SIZE]
  LONG buffer_index

PUB main() | sample_period
  ' Calculate sample period for 44.1 kHz
  sample_period := _clkfreq / SAMPLE_RATE      ' ~4535 clocks

  ' Configure ADC with SINC2 sampling
  ' Use period override for exact rate
  WRPIN(AUDIO_PIN, P_ADC_GPIO | P_ADC)
  WXPIN(AUDIO_PIN, %01_1100)                  ' SINC2 filter, base period
  WYPIN(AUDIO_PIN, sample_period)              ' Override period
  PINH(AUDIO_PIN)

  REPEAT
    capture_buffer()
    process_audio()

PRI capture_buffer() | i, last_acc, acc
  last_acc := RDPIN(AUDIO_PIN)

  REPEAT i FROM 0 TO BUFFER_SIZE-1
    REPEAT UNTIL PINREAD(AUDIO_PIN)
    acc := RDPIN(AUDIO_PIN)
    audio_buffer[i] := acc - last_acc          ' SINC2 difference
    last_acc := acc

PRI process_audio()
  ' Application-specific audio processing of audio_buffer[]

```

ch16-adc-audio-capture.spin2

**Feeding a microphone: no bias network needed.** This example uses P\_ADC\_GPIO (ground-referenced). For an AC source such as an electret microphone, switch to P\_ADC\_FLOAT: the floating input **self-biases to roughly mid-supply**, so the mic couples straight in through a single series capacitor — no external bias-divider resistors. That self-bias point is only *approximately*  $V_{IO}/2$ , so for absolute-voltage work use the ratiometric three-reference method in §16.3; for audio (AC, where only the changes matter) the approximate midpoint is exactly what audio needs.

### Example 3: High-Resolution DC Measurement

```

CON
  _clkfreq = 200_000_000
  SENSOR_PIN = 46

PUB measure_voltage() : millivolts | sample, last_acc, acc, ack
  ' 14-bit resolution with SINC2 (8192 clocks)

```

continues on next page →

*↪ continued from previous page*

```

WRPIN(SENSOR_PIN, P_ADC_1X | P_ADC)
WXPIN(SENSOR_PIN, %01_1101)          ' SINC2 filter, 14-bit
PINH(SENSOR_PIN)

' Discard the first reading (SINC2 is valid from the 2nd period)
REPEAT UNTIL PINREAD(SENSOR_PIN)
ack := RDPIN(SENSOR_PIN)              ' discard warm-up sample

' Get actual measurement
REPEAT UNTIL PINREAD(SENSOR_PIN)
last_acc := RDPIN(SENSOR_PIN)
REPEAT UNTIL PINREAD(SENSOR_PIN)
acc := RDPIN(SENSOR_PIN)

sample := (acc - last_acc) & $3FFF    ' 14-bit value

' Convert to millivolts (0-3300mV for 0-16383)
millivolts := (sample * 3300) / 16383

```

## Example 4: Small Signal with Gain

```

CON
  _clkfreq = 200_000_000
  THERMOCOUPLE_PIN = 46              ' Range ~0-50mV depending on type

PUB read_thermocouple() : microvolts | sample
  ' Use 100x gain: 33mV max input → full ADC range
  WRPIN(THERMOCOUPLE_PIN, P_ADC_100X | P_ADC)
  WXPIN(THERMOCOUPLE_PIN, %00_1001)  ' SINC2 sampling, 10-bit
  PINH(THERMOCOUPLE_PIN)

  WAITMS(1)                          ' Let filter stabilize

  sample := RDPIN(THERMOCOUPLE_PIN)

  ' Convert: 0-1023 → 0-33000 μV (0-33mV at 100x gain)
  microvolts := (sample * 33000) / 1023

```

## Example 5: PASM2 ADC with Event Detection

```

CON
  _clkfreq = 200_000_000
  ADC_PIN = 46

DAT          ORG

              ' Initialize ADC
DIRL         #ADC_PIN

```

*continues on next page →*



```

WRPIN    ##P_ADC_GIO | P_ADC, #ADC_PIN
WXPIN    ##00_0111, #ADC_PIN    ' 8-bit SINC2
DIRH      #ADC_PIN

' Set up event detection for IN flag
SETSE1    ##001<<6 + ADC_PIN    ' Event on IN high

.loop

WAITSE1                                ' Wait for sample ready
RDPIN     sample, #ADC_PIN          ' Read sample

' Process sample...
CMP        sample, threshold wc    ' Compare to threshold
if_c CALL   #below_threshold
if_nc CALL   #above_threshold

JMP        #.loop

sample     RES        1
threshold  LONG       128          ' Mid-scale threshold

```

## 16.8 Accuracy Considerations

### Noise Sources

| Source            | Effect              | Mitigation                     |
|-------------------|---------------------|--------------------------------|
| Supply noise      | Adds to conversion  | Clean power supply, decoupling |
| Digital crosstalk | Couples into analog | Separate analog from digital   |
| Input impedance   | Source loading      | Low-impedance source           |
| Temperature       | Offset drift        | Periodic calibration           |

### Hardware Limits

Some bounds come from the analog front end itself and **cannot be averaged away** — know them before promising absolute accuracy:

- **Input impedance  $\approx 500\text{ k}\Omega$**  (on the  $1\times$  range). A low-impedance source loads this lightly, but a high-impedance source — or a large external series resistor — forms a divider that shifts the reading. Buffer high-Z sources, or account for the divider.
- **Absolute-error floor  $\approx 15\text{ mV}$** . The GIO, VIO, and pin paths use three *separate* matched on-chip resistors that do not match perfectly, so different pins can read up to about 15 mV apart in absolute terms. This is a design limit, not noise — more averaging will not remove it. Where absolute accuracy matters, self-calibrate by driving the pin to each rail and measuring the result, or characterize the per-pin offset once.

- **Supply and temperature sensitivity.** The internal references track the VIO supply, so a noisy switch-mode VIO degrades precision — feed VIO from a clean LDO for instrumentation work. GIO and VIO also drift with temperature (VIO is the more stable of the two), giving each chip a per-pin fingerprint; periodic re-referencing handles the slow drift.
- **Power-of-2 sample period.** In SINC2 sampling mode the period must be a power of two ( $2^X[3:0]$ ) and cannot be freely dithered (§16.3, Resolution and Sample Rate).

## Improving Accuracy

### Averaging:

```
PUB averaged_reading(num_samples) : average | sum, i
  sum := 0
  REPEAT num_samples
    REPEAT UNTIL PINREAD(ADC_PIN)
      sum += RDPIN(ADC_PIN)
  average := sum / num_samples
```

**Oversampling for Extra Bits:** Each 4x oversampling adds approximately 1 bit of resolution.

**Calibration:**

```

VAR
    LONG adc_offset           ' Zero offset
    LONG adc_scale           ' Gain factor

PUB calibrate()
    ' Connect input to ground
    adc_offset := read_averaged(100)

    ' Connect input to known voltage (e.g., 2.5V)
    ' Expected value for 8-bit: (2.5/3.3) × 255 = 193
    adc_scale := (193 * 256) / (read_averaged(100) - adc_offset)

PUB calibrated_read() : value
    value := RDPIN(ADC_PIN)
    value := ((value - adc_offset) * adc_scale) >> 8

```

**Resolution vs Speed Trade-off**

| Resolution | Sample Period | Sample Rate at 200 MHz |
|------------|---------------|------------------------|
| 8 bits     | 128 clocks    | 1.56 MHz               |
| 10 bits    | 512 clocks    | 390 kHz                |
| 12 bits    | 2048 clocks   | 97.6 kHz               |
| 14 bits    | 8192 clocks   | 24.4 kHz               |

**16.9 Quick Reference****Mode Constants**

| Constant    | Mode   | Description             |
|-------------|--------|-------------------------|
| P_ADC       | %11000 | Internal clock ADC      |
| P_ADC_EXT   | %11001 | External clock ADC      |
| P_ADC_SCOPE | %11010 | Triggered scope capture |

## Input Mode Constants

| Constant    | Function                |
|-------------|-------------------------|
| P_ADC_GIO   | Ground-referenced input |
| P_ADC_VIO   | VIO-referenced input    |
| P_ADC_FLOAT | Floating input          |
| P_ADC_1X    | 1x gain (0-3.3V)        |
| P_ADC_3X    | 3.16x gain (0-1.04V)    |
| P_ADC_10X   | 10x gain (0-330mV)      |
| P_ADC_30X   | 31.6x gain (0-104mV)    |
| P_ADC_100X  | 100x gain (0-33mV)      |

## Filter Mode Summary

| X[5:4] | Mode            | Post-Processing            |
|--------|-----------------|----------------------------|
| %00    | SINC2 Sampling  | None (hardware complete)   |
| %01    | SINC2 Filtering | Software difference        |
| %10    | SINC3 Filtering | Software triple difference |
| %11    | Bitstream       | Custom processing          |

## Sample Rate Formula

```
sample_rate = sysclk / 2^(X[3:0])
```

Or with WYPIN override:

```
sample_rate = sysclk / WYPIN_value
```

## Voltage Conversion

For P\_ADC\_1X (0-3.3V range):

```
voltage_mv = (sample × 3300) / full_scale
```

Where full\_scale depends on resolution (255 for 8-bit, 16383 for 14-bit).

---

*This chapter covered analog-to-digital conversion. For serial reception, see Chapter 17. For USB, see Chapter 19.*

# Chapter 17: Serial Receive

This chapter covers receiving serial data using smart pin modes P\_SYNC\_RX (%11101) for synchronous (SPI-style) reception and P\_ASYNC\_RX (%11111) for asynchronous (UART-style) reception. Topics include baud rate configuration, clock routing, data formatting, and error handling.

## 17.1 Serial Receive Modes Overview

### Available Modes

| Mode   | Constant   | Description                                        |
|--------|------------|----------------------------------------------------|
| %11101 | P_SYNC_RX  | Synchronous serial receive (data + external clock) |
| %11111 | P_ASYNC_RX | Asynchronous serial receive (UART/RS-232 style)    |

### Mode Selection

#### Choose P\_SYNC\_RX when:

- External clock signal available (SPI slave, shift register)
- Clock provided by transmitting device
- Precise bit timing controlled externally

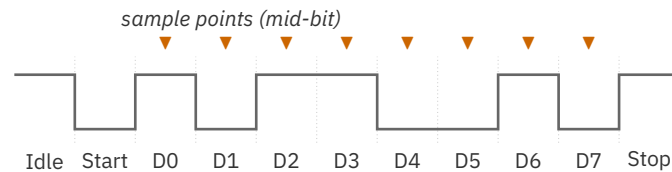
#### Choose P\_ASYNC\_RX when:

- No external clock (UART, RS-232)
- Both ends agree on baud rate
- Standard serial communication

## 17.2 Mode %11111: P\_ASYNC\_RX (Asynchronous Receive)

### Operation

Receives serial data asynchronously with automatic start bit detection. The smart pin monitors for a high-to-low transition (start bit), samples data bits at mid-bit timing, and validates the stop bit.



**Figure 17.1.** UART receive frame

### X Register Configuration

X[31:16]: System clock periods per bit (integer part)  
 X[15:10]: Fractional clock periods (1/64th increments)  
 X[9:5]: Reserved  
 X[4:0]: Number of data bits minus 1 (0-31 for 1-32 bits)

### Baud Rate Calculation

#### Basic formula:

```
bit_period = sysclk / baud
X_value = (bit_period << 16) | (data_bits - 1)
```

#### With fractional precision:

```
bit_period_frac := MULDIV64(_clkfreq, 65536, baud)
X_value := (bit_period_frac & $FFFFFFC00) | (data_bits - 1)
```

**Common baud rates at 200 MHz:**

| Baud   | Clocks/bit | X[31:16] Value |
|--------|------------|----------------|
| 9600   | 20833      | \$5161         |
| 19200  | 10417      | \$28B1         |
| 38400  | 5208       | \$1458         |
| 57600  | 3472       | \$0D90         |
| 115200 | 1736       | \$06C8         |
| 230400 | 868        | \$0364         |
| 460800 | 434        | \$01B2         |
| 921600 | 217        | \$00D9         |

**Data Justification**

RDPIN and RQPIN return the received word **MSB-justified** at Z[31]. For an N-bit word, the data occupies Z[31:32-N]; right-shift by **32 - N** to LSB-justify. Failing to shift produces incorrect values (a received 8-bit byte appears as a 32-bit value with the byte in the upper 8 bits and the low byte always zero).

| Data bits (N) | Z occupancy | Right shift                        |
|---------------|-------------|------------------------------------|
| 8             | Z[31:24]    | >> 24 (Spin2) / SHR D, #24 (PASM2) |
| 9             | Z[31:23]    | >> 23                              |
| 16            | Z[31:16]    | >> 16                              |
| 32            | Z[31:0]     | none                               |

This applies equally to async and sync RX modes.

**Basic UART Reception**

```

CON
    _clkfreq = 200_000_000
    RX_PIN = 21
    BAUD = 115_200

PUB uart_init() | bit_period
    bit_period := _clkfreq / BAUD
    PINSTART(RX_PIN, P_ASYNC_RX, (bit_period << 16) | 7, 0) ' 8 data bits

PUB receive_byte() : value

```

*continues on next page →*



*↪ continued from previous page*

```

REPEAT UNTIL PINREAD(RX_PIN)           ' Wait for data
value := RDPIN(RX_PIN) >> 24           ' LSB-justify 8-bit byte (32-8)

```

## Reception with Timeout

```

PUB receive_with_timeout(timeout_ms) : value | deadline
  deadline := GETMS() + timeout_ms

  REPEAT
    IF PINREAD(RX_PIN)
      RETURN RDPIN(RX_PIN) >> 24      ' Data received (LSB-justify 8-bit)

    IF GETMS() >= deadline
      RETURN -1                        ' Timeout

PUB has_data() : available
  available := PINREAD(RX_PIN) <> 0

```

## PASM2 UART Reception

```

CON
  _clkfreq = 200_000_000
  RX_PIN = 21
  BAUD = 115_200

DAT          ORG

              ' Calculate bit period
MOV          bit_period, ##_clkfreq / BAUD
SHL          bit_period, #16
OR           bit_period, #7              ' 8 data bits

              ' Configure UART receive
DIRL         #RX_PIN
WRPIN        ##P_ASYNC_RX, #RX_PIN
WXPIN        bit_period, #RX_PIN
DIRH         #RX_PIN

.receive_loop
  TESTP      #RX_PIN wc                ' Check IN flag
  if_nc JMP   #.receive_loop          ' Wait for data

  RDPIN      rx_data, #RX_PIN          ' Read MSB-justified word
  SHR        rx_data, #24              ' LSB-justify 8-bit byte (32-8)
  ' Process rx_data...

  JMP        #.receive_loop

```

*continues on next page →*

↔ continued from previous page

```

bit_period    RES    1
rx_data       RES    1

```

## 17.3 Mode %11101: P\_SYNC\_RX (Synchronous Receive)

### Operation

Receives serial data synchronized to an external clock signal. Data is sampled on clock edges, with the A-input carrying data and B-input carrying the clock.

### Critical: Clock Routing

**The B-input defaults to the local pin, which is useless for SPI.** A pin-selection constant **MUST** be added:

```

' WRONG - no clock routing:
mode := P_SYNC_RX                                ' Will not work!

' CORRECT - clock from adjacent pin:
mode := P_SYNC_RX | P_PLUS1_B                    ' Clock on pin+1

```

### Pin Selection Constants

| Constant   | Clock Source         |
|------------|----------------------|
| P_PLUS1_B  | Next pin (pin+1)     |
| P_MINUS1_B | Previous pin (pin-1) |
| P_PLUS2_B  | Pin+2                |
| P_PLUS3_B  | Pin+3                |
| P_MINUS2_B | Pin-2                |
| P_MINUS3_B | Pin-3                |

### X Register Configuration

```

X[5]:  Sample timing (0=before edge, 1=on edge)
X[4:0]: Number of bits minus 1

```

### Sample Timing:

- X[5]=0 (before edge): More tolerant, works with any transmitter
- X[5]=1 (on edge): Fast P2-to-P2 transfers, requires 2-clock hold time

## Data Justification

Data justification is identical to async receive: the word arrives MSB-justified at Z[31], so right-shift by  $32 - N$  to align an N-bit value. See §17.2 for the full table.

## Basic SPI Slave Receive

```
CON
    _clkfreq = 200_000_000
    MISO_PIN = 30                ' Data input
    SCK_PIN = 31                 ' Clock input (MISO_PIN + 1)

PUB spi_slave_init()
    ' 8-bit receive, clock on next pin, sample before edge
    PINSTART(MISO_PIN, P_SYNC_RX | P_PLUS1_B, %0_00111, 0)

PUB receive_byte() : value
    REPEAT UNTIL PINREAD(MISO_PIN)      ' Wait for 8 bits
    value := RDPIN(MISO_PIN) >> 24      ' Read and right-justify
```

## MSB-First Reception

Standard sync receive is LSB-first. For MSB-first protocols, reverse the bits:

```
PUB receive_msb_first() : value
    REPEAT UNTIL PINREAD(MISO_PIN)
    value := RDPIN(MISO_PIN)
    value := value REV 32          ' Reverse all 32 bits
    value := value & $FF          ' Mask to 8 bits
```

### PASM2:

```
    RDPIN    data, #MISO_PIN
    REV      data                ' Reverse all 32 bits
    ZEROX    data, #7           ' Keep only bits 0-7
```

## Clock Polarity

For inverted clock (sample on falling edge):

```
PINSTART(DATA_PIN, P_SYNC_RX | P_PLUS1_B | P_INVERT_B, %0_00111, 0)
```

## PASM2 SPI Slave

```
CON
    _clkfreq = 200_000_000
    DATA_PIN = 30
    CLK_PIN = 31

DAT          ORG

                ' Configure sync receive, clock from pin+1
    DIRL      #DATA_PIN
    WRPIN     ##P_SYNC_RX | P_PLUS1_B, #DATA_PIN
    WXPIN     ##0_00111, #DATA_PIN      ' Pre-edge, 8 bits
    DIRH      #DATA_PIN

.receive_loop
    TESTP     #DATA_PIN wc              ' Check IN flag
    if_nc JMP  #.receive_loop

                RDPIN    rx_byte, #DATA_PIN      ' Read data
                SHR      rx_byte, #24           ' Right-justify

                ' Process rx_byte...
                JMP      #.receive_loop

rx_byte      RES      1
```

## 17.4 Full-Duplex UART

### Transmit and Receive Configuration

```
CON
    _clkfreq = 200_000_000
    TX_PIN = 20
    RX_PIN = 21
    BAUD = 115_200

VAR
    LONG tx_ready
    LONG rx_data

PUB serial_init() | bit_period
    bit_period := _clkfreq / BAUD

    ' Configure TX
```

*continues on next page →*

*↪ continued from previous page*

```

    PINSTART(TX_PIN, P_ASYNC_TX | P_OE, (bit_period << 16) | 7, 0)

    ' Configure RX
    PINSTART(RX_PIN, P_ASYNC_RX, (bit_period << 16) | 7, 0)

PUB send_byte(value)
    REPEAT UNTIL PINREAD(TX_PIN)          ' Wait for TX ready
    WYPIN(TX_PIN, value)

PUB receive_byte() : value
    REPEAT UNTIL PINREAD(RX_PIN)
    value := RDPIN(RX_PIN) >> 24          ' LSB-justify 8-bit byte

PUB echo_test()
    ' Echo received bytes back to sender
    REPEAT
        IF PINREAD(RX_PIN)
            send_byte(RDPIN(RX_PIN) >> 24) ' LSB-justify before resending

```

## Half-Duplex Coordination

For half-duplex protocols (RS-485, single-wire):

```

CON
    DATA_PIN = 20
    DIR_PIN = 21                                ' Direction control
    TX_PIN = DATA_PIN                          ' Single-wire: TX and RX share DATA_PIN
    RX_PIN = DATA_PIN

PUB send_message(ptr, len) | i
    PINHIGH(DIR_PIN)                            ' Enable transmitter
    WAITUS(10)                                  ' Allow turnaround

    REPEAT i FROM 0 TO len-1
        send_byte(BYTE[ptr][i])

    REPEAT UNTIL PINREAD(TX_PIN)                ' Wait for last byte
    WAITUS(100)                                 ' Bit time + margin
    PINLOW(DIR_PIN)                             ' Enable receiver

PUB receive_message(ptr, max_len, timeout_ms) : count | deadline, b
    count := 0
    deadline := GETMS() + timeout_ms

    REPEAT WHILE count < max_len
        IF PINREAD(RX_PIN)
            b := RDPIN(RX_PIN) >> 24            ' LSB-justify 8-bit byte
            BYTE[ptr][count++] := b
            deadline := GETMS() + timeout_ms    ' Reset timeout
        ELSEIF GETMS() >= deadline
            QUIT                                ' Timeout - end of message

```

*continues on next page →*

```
PRI send_byte(b)
  ' Application-specific: TX byte b via DATA_PIN (configured for TX)
```

## 17.5 Buffered Reception

### Circular Buffer

```
CON
  RX_PIN = 63
  BUFFER_SIZE = 256                                ' Must be power of 2
  BUFFER_MASK = BUFFER_SIZE - 1

VAR
  BYTE rx_buffer[BUFFER_SIZE]
  LONG head, tail

PUB buffer_init()
  head := 0
  tail := 0

PUB poll_rx()
  ' Call frequently to move data from pin to buffer
  IF PINREAD(RX_PIN)
    rx_buffer[head] := RDPIN(RX_PIN) >> 24          ' LSB-justify 8-bit byte
    head := (head + 1) & BUFFER_MASK
    ' Note: overwrites old data if buffer full

PUB available() : count
  count := (head - tail) & BUFFER_MASK

PUB read_byte() : value
  IF head == tail
    RETURN -1                                         ' Buffer empty

  value := rx_buffer[tail]
  tail := (tail + 1) & BUFFER_MASK
```

### Cog-Based Receiver

For continuous high-speed reception, use a dedicated cog:

```
VAR
  LONG rx_cog
  LONG rx_stack[64]
  BYTE rx_buffer[1024]
  LONG rx_head
```

*→ continued from previous page*

```

PUB start_receiver()
  rx_head := 0
  rx_cog := COGSPIN(NEWCOG, receiver_loop(), @rx_stack)

PRI receiver_loop()
  uart_init()

  REPEAT
    IF PINREAD(RX_PIN)
      rx_buffer[rx_head++] := RDPIN(RX_PIN) >> 24 ' LSB-justify byte
      IF rx_head >= 1024
        rx_head := 0 ' Wrap around

PUB get_rx_head() : pos
  pos := rx_head

```

## 17.6 Error Detection

### Framing Error

A framing error occurs when the stop bit is not high. The P2 does not automatically flag this; framing errors are detected by examining received data:

```

PUB receive_with_check() : value, error | raw
  ' Requires PINSTART configured for 9 data bits
  ' (X[4:0] = 8) to capture stop bit.
  REPEAT UNTIL PINREAD(RX_PIN)
    raw := RDPIN(RX_PIN) >> 23 ' 9 bits, shift by 32-9

  ' After the shift: bits 7:0 are the data byte,
  ' bit 8 is the captured stop bit
  value := raw & $FF
  IF (raw & $100) == 0
    error := TRUE ' Missing stop bit
  ELSE
    error := FALSE

  value := raw & $FF

```

### Overrun Detection

Overrun occurs when new data arrives before previous data is read:

```

VAR
  LONG last_read_time
  LONG overrun_count

```

*continues on next page →*

```
PUB check_overrun() : overrun
  ' Track time between reads
  ' If the interval is too long, data is lost
  overrun := FALSE
  ' Application-specific logic based on baud rate
```

## Break Detection

A break is a prolonged low condition (longer than one frame):

```
PUB detect_break(pin) : is_break | start_time
  IF PINREAD(pin) == 0                    ' Input is low
    start_time := GETMS()
    REPEAT WHILE PINREAD(pin) == 0
      IF (GETMS() - start_time) > 20      ' > frame time
        is_break := TRUE
      RETURN
  is_break := FALSE
```

## 17.7 RS-232 Signal Inversion

RS-232 uses inverted logic (mark=-3V to -15V, space=+3V to +15V). After level conversion to 3.3V logic, the signal is still inverted.

### Using P\_INVERT\_IN

```
' For RS-232 with external level shifter
PINSTART(RX_PIN, P_ASYNC_RX | P_INVERT_IN, (bit_period << 16) | 7, 0)
```

## 17.8 Multi-Drop Networks

### RS-485 Reception

```
CON
  _clkgfreq = 200_000_000
  RX_PIN = 20
  TX_PIN = 21
  DE_PIN = 22                    ' Driver Enable
  BAUD = 9600
  MY_ADDRESS = $05

PUB rs485_init() | bp
```



*↪ continued from previous page*

```

bp := _clkfreq / BAUD
PINSTART(RX_PIN, P_ASYNC_RX, (bp << 16) | 7, 0)
PINSTART(TX_PIN, P_ASYNC_TX | P_OE, (bp << 16) | 7, 0)
PINLOW(DE_PIN)                                ' Receiver enabled

PUB listen_for_address() : addressed | addr
REPEAT
  IF PINREAD(RX_PIN)
    addr := RDPIN(RX_PIN) >> 24                ' LSB-justify 8-bit byte
    IF addr == MY_ADDRESS
      RETURN TRUE
    IF addr == $FF                             ' Broadcast
      RETURN TRUE
  RETURN FALSE

```

## 17.9 Application Examples

### Example 1: GPS NMEA Receiver

```

CON
  _clkfreq = 200_000_000
  GPS_PIN = 20
  GPS_BAUD = 9600

VAR
  BYTE nmea_buffer[100]
  LONG buffer_idx

PUB gps_receiver() | ch
  ' Initialize 8N1 at 9600 baud
  PINSTART(GPS_PIN, P_ASYNC_RX, ((_clkfreq / GPS_BAUD) << 16) | 7, 0)

  buffer_idx := 0

  REPEAT
    IF PINREAD(GPS_PIN)
      ch := RDPIN(GPS_PIN) >> 24                ' LSB-justify 8-bit byte

      IF ch == "$"                             ' Start of sentence
        buffer_idx := 0

        nmea_buffer[buffer_idx++] := ch

        IF ch == 10                            ' End of line
          nmea_buffer[buffer_idx] := 0
          process_nmea(@nmea_buffer)
          buffer_idx := 0

        IF buffer_idx >= 99
          buffer_idx := 0                        ' Overflow protection

```

*continues on next page →*

```
PRI process_nmea(ptr)
  ' Application-specific: parse NMEA sentence starting at ptr
```

## Example 2: SPI Sensor Read

```
CON
  _clkfreq = 200_000_000
  MOSI_PIN = 30
  MISO_PIN = 31
  SCK_PIN = 32
  CS_PIN = 33

PUB read_sensor_register(reg_addr) : value
  ' Configure SPI
  PINSTART(MISO_PIN, P_SYNC_RX | P_PLUS1_B, %0_00111, 0) ' 8 bits

  PINLOW(CS_PIN) ' Select device

  ' Send register address (would use TX pin)
  send_spi_byte(reg_addr | $80) ' Read flag

  ' Receive data
  REPEAT UNTIL PINREAD(MISO_PIN)
  value := RDPIN(MISO_PIN) >> 24

  PINHIGH(CS_PIN) ' Deselect

PRI send_spi_byte(b)
  ' Application-specific: clock byte b out via SPI TX pin
```

## Example 3: Command Parser

```
CON
  _clkfreq = 200_000_000
  RX_PIN = 21
  BAUD = 115_200

VAR
  BYTE cmd_buffer[64]
  LONG cmd_len

PUB command_loop() | ch
  PINSTART(RX_PIN, P_ASYNC_RX, ((_clkfreq / BAUD) << 16) | 7, 0)
  cmd_len := 0

  REPEAT
    IF PINREAD(RX_PIN)
      ch := RDPIN(RX_PIN) >> 24 ' LSB-justify 8-bit byte
```

```

CASE ch
  13:                                     ' Enter
    cmd_buffer[cmd_len] := 0
    execute_command(@cmd_buffer)
    cmd_len := 0

    8, 127:                             ' Backspace/Delete
      IF cmd_len > 0
        cmd_len--

    32..126:                             ' Printable
      IF cmd_len < 63
        cmd_buffer[cmd_len++] := ch

PRI execute_command(ptr)
  IF STRCOMP(ptr, STRING("help"))
    send_string(STRING("Commands: help, status, reset"))
  ELSEIF STRCOMP(ptr, STRING("status"))
    send_string(STRING("System OK"))
  ' ... more commands

PRI send_string(ptr)
  ' Application-specific: TX null-terminated string at ptr

```

ch17-uart-command-loop.spin2

## 17.10 Quick Reference

### Mode Constants

| Constant   | Mode   | Description                 |
|------------|--------|-----------------------------|
| P_ASYNC_RX | %11111 | Asynchronous serial receive |
| P_SYNC_RX  | %11101 | Synchronous serial receive  |

### P\_ASYNC\_RX Configuration

#### X Register:

```

X[31:16]: Clocks per bit (sysclk / baud)
X[15:10]: Fractional clocks (precision)
X[4:0]:   Data bits - 1 (7 for 8-bit)

```

**Baud calculation:**

```
X_value := ((sysclk / baud) << 16) | (bits - 1)
```

**P\_SYNC\_RX Configuration****X Register:**

```
X[5]: 0=sample before edge, 1=sample on edge
X[4:0]: Data bits - 1
```

**Required modifier:** P\_PLUS1\_B (or similar) for clock routing

**Data justification:** MSB-justified, SHR #(32-bits) to right-justify

**Related Modifiers**

| Modifier    | Function              |
|-------------|-----------------------|
| P_INVERT_IN | Invert input (RS-232) |
| P_INVERT_B  | Invert B-input clock  |
| P_PLUS1_B   | Clock on pin+1        |
| P_MINUS1_B  | Clock on pin-1        |

**Common Patterns****Blocking receive (8-bit example; shift by 32 - N for other widths):**

```
REPEAT UNTIL PINREAD(pin)
value := RDPIN(pin) >> 24 ' LSB-justify 8-bit byte
```

**Polled receive:**

```
IF PINREAD(pin)
value := RDPIN(pin) >> 24 ' LSB-justify 8-bit byte
```

**With timeout:**

```
deadline := GETMS() + timeout_ms  
REPEAT UNTIL PINREAD(pin) OR (GETMS() >= deadline)
```

*This chapter covered serial reception. For special modes like USB, see Chapter 19. For inter-cog data sharing, see Chapter 18.*

## Part IV: Special Modes

# Chapter 18: Repository

## Inter-Cog Data Sharing

This chapter covers the repository modes (%00001-%00011) that serve dual purposes: inter-cog data sharing via the long repository function, and high-resolution DAC output with dithering. These modes provide hardware-arbitrated data transfer without lock contention.

### 18.1 Repository Concept

#### Dual-Purpose Modes

Modes %00001-%00011 behave differently based on pin configuration:

| Condition                | Function               |
|--------------------------|------------------------|
| NOT DAC_MODE             | 32-bit long repository |
| DAC_MODE (P[12:10]=%101) | DAC with dithering     |

#### Repository Function

When not configured for DAC output, these modes create a shared data register:

- **WXPIN** writes a 32-bit long to the repository
- **RDPIN/RQPIN** reads the stored long
- **IN flag** indicates when new data has been written

This enables lock-free data sharing between cogs through dedicated pin hardware.

## Mode Variants

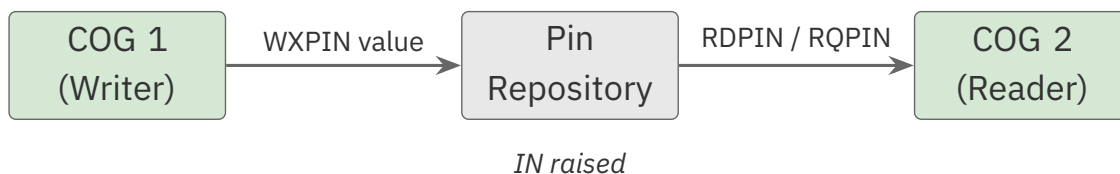
| Mode   | Constant         | Repository | DAC Function         |
|--------|------------------|------------|----------------------|
| %00001 | P_REPOSITORY     | Yes        | Noise output         |
| %00010 | P_DAC_DITHER_RND | Yes        | PRNG-dithered 16-bit |
| %00011 | P_DAC_DITHER_PWM | Yes        | PWM-dithered 16-bit  |

## 18.2 Long Repository (Non-DAC Mode)

### Purpose

The repository provides a hardware-arbitrated communication channel between cogs. Unlike hub RAM which may require locks for atomic access, the repository guarantees atomic 32-bit reads and writes.

### Operation



**Figure 18.1.** Pin repository handoff

### Configuration

```

CON
  REPO_PIN = 48

PUB setup_repository()
  WRPIN(REPO_PIN, P_REPOSITORY)      ' Mode %00001
  PINH(REPO_PIN)                     ' Enable

PUB write_value(value)
  WXPIN(REPO_PIN, value)              ' Store 32-bit value

PUB read_value() : value
  value := RQPIN(REPO_PIN)            ' Read without clearing IN
  
```

## PASM2 Repository Access

```

DAT          ORG

        ' Configure repository
DIRL        #REPO_PIN
WRPIN       ##P_REPOSITORY, #REPO_PIN
DIRH        #REPO_PIN

        ' Write value
WXPIN       ##$DEADBEEF, #REPO_PIN  ' Store value

        ' Read value
RQPIN       data, #REPO_PIN          ' Get stored value

data        RES        1

```

## Multi-Cog Sharing

### Writer cog:

```

PUB sensor_cog() | reading
    setup_repository()

    REPEAT
        reading := read_sensor()
        WXPIN(REPO_PIN, reading)          ' Share with other Cogs
        WAITMS(10)

```

### Reader cogs:

```

PUB display_cog()
    REPEAT
        IF PINREAD(REPO_PIN)              ' New data available?
            display_value(RQPIN(REPO_PIN))
        WAITMS(100)

PUB logger_cog()
    REPEAT
        log_value(RQPIN(REPO_PIN))        ' Read current value
        WAITMS(1000)

```



## 18.3 Mode %00001: DAC Noise

### Purpose

When configured for DAC output, mode %00001 generates pseudo-random noise on the 8-bit DAC. Each pin produces a unique random pattern.

P\_REPOSITORY and P\_DAC\_NOISE name the same %00001 mode — the DAC\_MODE bits (P[12:10]=%101) decide whether the pin acts as a long repository or a noise DAC.

### Configuration

```
CON
    NOISE_PIN = 20

PUB setup_noise_dac()
    ' P[12:10] = %101 enables DAC output
    WRPIN(NOISE_PIN, P_DAC_NOISE | P_DAC_124R_3V | P_OE)
    WXPIN(NOISE_PIN, 0)                ' No sample period
    PINH(NOISE_PIN)
```

### X Register: Sample Period

| X[15:0] | Behavior                              |
|---------|---------------------------------------|
| 0       | 65,536 clocks (longest sample period) |
| N       | IN raised every N clocks              |

**Note:** The DAC outputs noise continuously regardless of sample period. The sample period only affects when IN is raised.

### Voltage Range

The noise spans the full scale of the selected DAC range. See Chapter 10 §10.2 for the resistor-DAC voltage options.

### Example: White Noise Generator

```
CON
    _clkfreq = 200_000_000
    AUDIO_PIN = 20

PUB white_noise()
    ' Configure for 3.3V peak noise output
    WRPIN(AUDIO_PIN, P_DAC_NOISE | P_DAC_124R_3V | P_OE)
```

*continues on next page →*

*↪ continued from previous page*

```

WXPIN(AUDIO_PIN, 0)                ' Max period (low power)
PINH(AUDIO_PIN)

' Noise runs continuously - just wait
REPEAT
  WAITMS(1000)

```

## 18.4 Mode %00010: DAC PRNG Dither

### Purpose

Provides 16-bit DAC resolution using pseudo-random dithering of the 8-bit DAC. The dithering randomly toggles between adjacent DAC levels to achieve higher effective resolution when averaged over time.

### Operation

- Y[15:0] sets the desired 16-bit output value
- Hardware randomly dithers between adjacent 8-bit levels
- Averaging over time yields 16-bit effective resolution

**The “16-bit” figure is nominal — a temporal-averaging ceiling, not sample-by-sample accuracy** (the hardware DAC is 8-bit). See Chapter 10 §10.4 for the full dithering-resolution treatment.

### Configuration

```

CON
  DAC_PIN = 20

PUB setup_dither_dac() | mode
  mode := P_DAC_DITHER_RND | P_DAC_124R_3V | P_OE
  WRPIN(DAC_PIN, mode)
  WXPIN(DAC_PIN, 1)                ' Update immediately
  PINH(DAC_PIN)

PUB set_voltage(value_16bit)
  WYPIN(DAC_PIN, value_16bit)      ' 16-bit value

```

## X Register: Sample Period

| X[15:0] | Behavior                             |
|---------|--------------------------------------|
| 1       | Update immediately (IN stays high)   |
| N       | Y captured every N clocks, IN raised |

For audio waveforms, set sample period to match sample rate:

```
sample_period := _clkfreq / sample_rate
WXPIN(DAC_PIN, sample_period)
```

## Voltage Calculation

```
voltage = (Y[15:0] / 65536) × DAC_max_voltage
```

For P\_DAC\_124R\_3V:

```
voltage = (Y[15:0] / 65536) × 3.3V
```

## Example: 16-bit Audio DAC

```
CON
    _clkfreq = 200_000_000
    DAC_PIN = 20
    SAMPLE_RATE = 44100

PUB audio_dac() | sample_period
    sample_period := _clkfreq / SAMPLE_RATE      ' ~4535 clocks

    ' Configure 16-bit dithered DAC
    WRPIN(DAC_PIN, P_DAC_DITHER_RND | P_DAC_124R_3V | P_OE)
    WXPIN(DAC_PIN, sample_period)
    PINH(DAC_PIN)

    ' Output audio samples
    REPEAT
        REPEAT UNTIL PINREAD(DAC_PIN)            ' Wait for sample period
        WYPIN(DAC_PIN, get_next_sample())

PRI get_next_sample() : sample
    ' Application-specific: return next 16-bit audio sample
    sample := $8000
```

## ADC Readback

When OUT is high, the pin's ADC is enabled and RDPIN returns the 16-bit ADC accumulation (useful for measuring DAC loading). See Chapter 10 §10.6 for the ADC-feedback pattern.

## 18.5 Mode %00011: DAC PWM Dither

### Purpose

Provides 16-bit DAC resolution using PWM dithering. PWM dithering is more deterministic than PRNG dithering and provides better dynamic range, but introduces a fixed-frequency component.

### Operation

- Y[15:0] sets the desired 16-bit output value
- Hardware PWM-dithers between adjacent 8-bit levels
- Sample period must be multiple of 256 for proper operation

### Key Difference from PRNG Dither

| Aspect                     | PRNG Dither (%00010) | PWM Dither (%00011) |
|----------------------------|----------------------|---------------------|
| Transition pattern         | Random               | Deterministic       |
| Transitions per 256 clocks | Up to one per clock  | At most two         |
| Noise floor                | Higher               | Lower               |
| Spurious tones             | None                 | One at Fclock/256   |
| Dynamic range              | Good                 | Better (-48dB spur) |

The “at most two transitions per 256 clocks” is what gives PWM dither its lower noise floor and lower switching activity: where the PRNG mode can flip the DAC on any clock, the PWM mode confines all of a period's switching to two edges.

### Configuration

```
CON
  DAC_PIN = 20

PUB setup_pwm_dither_dac() | mode, period
  mode := P_DAC_DITHER_PWM | P_DAC_124R_3V | P_OE

  ' Period MUST be multiple of 256
  period := 256 * 16                                ' 4096 clocks
```

*continues on next page →*

```
WRPIN(DAC_PIN, mode)
WXPIN(DAC_PIN, period)
PINH(DAC_PIN)
```

## Sample Period Constraint

**X[15:0] must have X[7:0] = 0** (multiple of 256):

| Period | Valid? | Notes                 |
|--------|--------|-----------------------|
| 256    | Yes    | Minimum (fast update) |
| 512    | Yes    |                       |
| 4096   | Yes    | $256 \times 16$       |
| 1000   | No     | $X[7:0] \neq 0$       |

## Example: High-Quality Audio

```
CON
    _clkfreq = 200_000_000
    DAC_PIN = 20
    SAMPLE_RATE = 48000

PUB hq_audio_dac() | period, samples_per_period
    ' Calculate period as multiple of 256
    ' At 48 kHz: period = 200_000_000 / 48000 = 4166.67
    ' Nearest 256 multiple: 4096 = 256 × 16
    period := 4096

    WRPIN(DAC_PIN, P_DAC_DITHER_PWM | P_DAC_124R_3V | P_OE)
    WXPIN(DAC_PIN, period)
    PINH(DAC_PIN)

    ' Actual sample rate: 200 MHz / 4096 = 48,828 Hz
    ' Close enough for most applications

REPEAT
    REPEAT UNTIL PINREAD(DAC_PIN)
    WYPIN(DAC_PIN, get_next_sample())

PRI get_next_sample() : sample
    ' Application-specific: return next 16-bit audio sample
    sample := $8000
```

## 18.6 Comparison with Other Inter-Cog Mechanisms

### Hub RAM

| Aspect      | Hub RAM            | Repository      |
|-------------|--------------------|-----------------|
| Capacity    | 512 KB             | 32 bits per pin |
| Access      | May need LOCK      | Atomic          |
| Speed       | 8-15 clocks/access | 1 instruction   |
| Flexibility | High               | Limited         |
| Best for    | Large data         | Flags, status   |

### LOCK Bits

| Aspect     | LOCK Bits         | Repository      |
|------------|-------------------|-----------------|
| Capacity   | 16 locks total    | 32 bits per pin |
| Function   | Mutex only        | Data + flag     |
| Complexity | TRY/REL pattern   | Read/Write      |
| Best for   | Critical sections | Data sharing    |

### Repository Advantages

1. **No contention:** Hardware arbitration, no lock waits
2. **Atomic updates:** Guaranteed 32-bit coherence
3. **Flag included:** IN indicates new data
4. **Non-blocking reads:** RQPIN doesn't clear IN

### When to Use Repository

- Sharing single sensor reading across multiple cogs
- Status flags and state indicators
- Real-time data where latest value is sufficient
- Simple producer-consumer patterns

## 18.7 Application Examples

### Example 1: Shared Sensor Reading

```
CON
    _clkfreq = 200_000_000
    REPO_PIN = 48
    TEMP_SENSOR = 20

VAR
    LONG sensor_stack[64]

PUB main()
    ' Start sensor reading Cog
    COGSPIN(NEWCOG, sensor_cog(), @sensor_stack)

    ' This Cog reads the shared value
    setup_repository_reader()

    REPEAT
        display_temperature(RQPIN(REPO_PIN))
        WAITMS(500)

PRI setup_repository_reader()
    ' Just need to read - writer sets up the pin
    ' Repository is already configured by sensor_cog

PRI sensor_cog() | temp
    ' Configure repository
    WRPIN(REPO_PIN, P_REPOSITORY)
    PINH(REPO_PIN)

    REPEAT
        temp := read_temperature_sensor()
        WXPIN(REPO_PIN, temp)           ' Share reading
        WAITMS(100)

PRI display_temperature(t)
    ' Application-specific: render temperature value t

PRI read_temperature_sensor() : t
    ' Application-specific: return current temperature reading
    t := 25
```

ch18-repository-multicog.spin2

## Example 2: Multi-Cog Status Flags

```

CON
    STATUS_PIN = 48
    FLAG_RUNNING = $0001
    FLAG_ERROR = $0002
    FLAG_COMPLETE = $0004

PUB set_flag(flag)
    WXPIN(STATUS_PIN, RQPIN(STATUS_PIN) | flag)

PUB clear_flag(flag)
    WXPIN(STATUS_PIN, RQPIN(STATUS_PIN) & !flag)

PUB test_flag(flag) : set
    set := (RQPIN(STATUS_PIN) & flag) <> 0

```

**Caution — set/clear-flag is a read-modify-write, not an atomic update.** `WXPIN(STATUS_PIN, RQPIN(STATUS_PIN) | flag)` reads, modifies, then writes back. The 32-bit *store* is atomic (§18.6), but the read-modify-write spanning those three steps is not: if two cogs each set a different flag at the same time, one update can be lost. This pattern is safe only when a **single cog owns all writes** to the repository. For multiple writers, guard the update with a lock or give each writer its own repository pin.

## Example 3: Stereo Audio with Dithered DAC

```

CON
    _clkfreq = 200_000_000
    LEFT_PIN = 20
    RIGHT_PIN = 21
    SAMPLE_RATE = 44100

PUB stereo_audio() | period
    period := _clkfreq / SAMPLE_RATE

    ' Configure both channels for PRNG dithering
    WRPIN(LEFT_PIN, P_DAC_DITHER_RND | P_DAC_124R_3V | P_OE)
    WRPIN(RIGHT_PIN, P_DAC_DITHER_RND | P_DAC_124R_3V | P_OE)
    WXPIN(LEFT_PIN, period)
    WXPIN(RIGHT_PIN, period)
    PINH(LEFT_PIN)
    PINH(RIGHT_PIN)

    REPEAT
        REPEAT UNTIL PINREAD(LEFT_PIN)          ' Wait for sample time
        WYPIN(LEFT_PIN, get_left_sample())
        WYPIN(RIGHT_PIN, get_right_sample())

PRI get_left_sample() : sample

```

*continues on next page →*



```

    ' Application-specific: return next 16-bit left-channel sample
    sample := $8000

PRI get_right_sample() : sample
    ' Application-specific: return next 16-bit right-channel sample
    sample := $8000

```

## Example 4: Function Generator with PWM DAC

```

CON
    _clkfreq = 200_000_000
    DAC_PIN = 20

VAR
    WORD sine_table[256]

PUB function_generator(frequency) | period, phase, increment
    ' Build sine table (0-65535 range)
    build_sine_table()

    ' Calculate DDS parameters
    ' phase accumulator increments per sample
    period := 4096                                ' PWM dither requirement
    increment := (frequency * 256 * 65536) / (_clkfreq / period)

    WRPIN(DAC_PIN, P_DAC_DITHER_PWM | P_DAC_124R_3V | P_OE)
    WXPIN(DAC_PIN, period)
    PINH(DAC_PIN)

    phase := 0
    REPEAT
        REPEAT UNTIL PINREAD(DAC_PIN)
        WYPIN(DAC_PIN, sine_table[phase >> 8])    ' Output current phase
        phase += increment                          ' Advance phase

PRI build_sine_table() | i
    ' Fill sine_table[0..255] with sine values 0..65535
    REPEAT i FROM 0 TO 255
        sine_table[i] := $8000 + QSIN(32767, i << 24, 0)

```

## 18.8 Quick Reference

### Mode Constants

| Constant         | Mode   | Function (Non-DAC) | Function (DAC)       |
|------------------|--------|--------------------|----------------------|
| P_REPOSITORY     | %00001 | 32-bit repository  | Noise output         |
| P_DAC_DITHER_RND | %00010 | 32-bit repository  | PRNG-dithered 16-bit |
| P_DAC_DITHER_PWM | %00011 | 32-bit repository  | PWM-dithered 16-bit  |

### DAC Mode Enable

Add to WRPIN value: P\_DAC\_xxR\_yV | P\_OE

- P[12:10] = %101 for DAC output
- P\_OE sets the output-enable flag (drives the pin)

### Register Usage

#### Repository Mode:

| Register                              | Write       | Read           |
|---------------------------------------|-------------|----------------|
| X via WXPIN                           | Store value | -              |
| Y                                     | Not used    | -              |
| Z via RQPIN (or RDPIN to acknowledge) | -           | Retrieve value |

#### DAC Dither Modes:

| Register | Function                                  |
|----------|-------------------------------------------|
| X[15:0]  | Sample period (PWM must be $\times 256$ ) |
| Y[15:0]  | 16-bit DAC value                          |
| Z        | ADC readback (if OUT=1)                   |

### Key Points

- **Repository:** WXPIN writes, RQPIN reads without clearing IN
- **DAC Noise:** Random 8-bit values every clock
- **PRNG Dither:** Random toggle between adjacent levels
- **PWM Dither:** Deterministic dither, period must be  $\times 256$
- **All modes:** IN raised when sample period completes

---

*This chapter covered repository and dithered DAC modes. For USB host/device, see Chapter 19. For a complete mode reference, see Appendix F.*

# Chapter 19: USB Host/Device

This chapter covers the USB smart pin mode P\_USB\_PAIR (%11011). The P2 provides hardware-assisted USB through smart pins, handling the differential signaling and timing while software manages the USB protocol stack.

## 19.1 USB Overview

### What the Smart Pin Provides

The USB mode handles the physical layer:

- Differential signaling (D+/D-)
- USB line state detection (J, K, SE0, SE1)
- Bit-level timing for USB 1.1 speeds
- Output driver control

### What Software Must Provide

The USB protocol stack:

- Packet formatting and parsing
- Endpoint management
- Device enumeration
- Class-specific protocols (HID, CDC, Mass Storage, etc.)

### Complexity Warning

USB is a complex protocol. Building a complete USB stack from scratch requires:

- Deep understanding of USB specification
- Significant development time
- Extensive testing with various devices

**Recommendation:** Use existing USB libraries from Parallax or the P2 community rather than implementing from scratch.

## 19.2 Pin Configuration

### Pin Pair Requirement

USB requires an even/odd consecutive pin pair:

| Pin             | Function |
|-----------------|----------|
| Even (e.g., 56) | DM (D-)  |
| Odd (e.g., 57)  | DP (D+)  |

Valid pairs: 0/1, 2/3, 4/5, ..., 56/57, 58/59, 60/61, 62/63

### Basic Configuration

```
CON
    USB_DM = 56                ' D- on even pin
    USB_DP = 57                ' D+ on odd pin (DM+1)

PUB configure_usb_pins() | baud
    ' Configure as USB pair with output enabled
    WRPIN(USB_DM, P_USB_PAIR | P_OE)
    ' WXPIN on the LOWER pin sets USB mode + baud:
    '   D[15]=1 host / 0 device, D[14]=1 full-speed / 0 low-speed
    '   D[13:0]=baud fraction
    baud := 12_000_000 / (clkfreq / $10000) ' full-speed 12 Mbps
    WXPIN(USB_DM, $4000 | baud)              ' device, full-speed
    PINHIGH(USB_DM)
    PINHIGH(USB_DP)
```

### Output Control

| WRPIN D Bit    | Function                                |
|----------------|-----------------------------------------|
| D = %1_11011_0 | Output drive enabled (normal operation) |
| D = %0_11011_0 | Output drive disabled (sniffer mode)    |

**Sniffer Mode:** Disables output drive to passively monitor USB traffic without affecting the bus.

## 19.3 USB Signaling

### Line States

USB uses differential signaling with specific line states:

| State | D+   | D-   | Meaning                          |
|-------|------|------|----------------------------------|
| J     | High | Low  | Idle (Full Speed), Data 0        |
| K     | Low  | High | Data 1 (Full Speed)              |
| SE0   | Low  | Low  | End of Packet, Reset, Disconnect |
| SE1   | High | High | Invalid (error condition)        |

**Note:** J and K meanings swap for Low Speed vs Full Speed, because USB uses complementary (mirrored) line signaling. (This is a signaling-polarity difference tied to the speed setting, not a way to reassign the physical DP/DM pins.)

### USB Speeds

| Speed           | Bit Rate | Supported |
|-----------------|----------|-----------|
| Low Speed (LS)  | 1.5 Mbps | Yes       |
| Full Speed (FS) | 12 Mbps  | Yes       |
| High Speed (HS) | 480 Mbps | No        |

The P2 USB mode supports USB 1.1 speeds only.

## 19.4 Register Usage

### Overview

The USB mode uses the smart pin registers for configuration and data:

| Register | Function                                                                                                                                                                                              |
|----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| X        | USB configuration, set via WXPIN on the lower (even) pin: D[15]=1 host / 0 device, D[14]=1 full-speed / 0 low-speed, D[13:0]=baud rate as a 16-bit fraction of sysclk (target_Hz / clkfreq × \$10000) |
| Y        | Line-state and packet output, set via WYPIN on the lower pin (see table below)                                                                                                                        |
| Z        | Receiver data + 16-bit status word, read via RDPIN/RQPIN on the lower pin (see bit layout below)                                                                                                      |

**All smart pin access happens on the lower (even/DM) pin** — WXPIN, WYPIN, and RDPIN/RQPIN are all issued there. The upper (odd/DP) pin takes no WXPIN/WYPIN; software only reads its IN flag (with TESTP). WXPIN **must** be issued on the lower pin to establish host/device, speed, and baud rate *before* raising DIR. (Source: *Parallax Propeller 2 Documentation v35 - Rev B/C*, USB host/device mode.)

### Baud Rate — Worked Example

The baud fraction's top two bits must be zero, so the baud rate must stay below  $\frac{1}{4}$  of sysclk. For 12 Mbps (full-speed) on an 80 MHz clock:

```
baud_fraction = 12,000,000 / 80,000,000 × $10000 = $2666
```

Selecting host + full-speed (D[15]=1, D[14]=1, i.e. \$C000) gives a WXPIN value of **\$E666** (\$C000 | \$2666).

### Y Register — Line States and Packet Output (WYPIN)

Write these with WYPIN on the lower pin to drive a line state or start a packet:

| WYPIN D | Action                                                                        |
|---------|-------------------------------------------------------------------------------|
| 0       | Output IDLE (float, except an optional resistor to 3.3 V / GND)               |
| 1       | Output SE0 (drive both DP and DM low)                                         |
| 2       | Output K (drive the K state)                                                  |
| 3       | Output J (J state — like IDLE, but actively driven)                           |
| 4       | Output EOP (end-of-packet: SE0, SE0, J, then IDLE)                            |
| \$80    | SOP — start-of-packet, then bytes, with automatic EOP when the buffer empties |

## Z Register — Receiver Status Word

RDPIN/RQPIN on the lower pin returns a 16-bit status word (with RDPIN . . . WC, the error flag also lands in C):

| Bit(s) | Meaning                                                                                          |
|--------|--------------------------------------------------------------------------------------------------|
| [15:8] | Last byte received                                                                               |
| [7]    | Byte toggle — cleared on SOP, toggles on each byte received (use it to detect a <i>new</i> byte) |
| [6]    | Error — cleared on SOP; set on bit-unstuff error, EOP with SE0 > 3 bits, or SE1                  |
| [5]    | EOP received                                                                                     |
| [4]    | SOP received                                                                                     |
| [3]    | SE1 in (illegal)                                                                                 |
| [2]    | SE0 in (RESET)                                                                                   |
| [1]    | K in (RESUME)                                                                                    |
| [0]    | J in (IDLE)                                                                                      |

## IN Flag — Per-Pin Semantics

The two pins carry **different** IN meanings:

- **Upper (odd / DP) pin** — IN rises whenever the **output buffer empties**, signalling that the next output byte may be written (via WYPIN to the lower pin). Read it with TESTP.
- **Lower (even / DM) pin** — IN rises on any **change in receiver status**; read the 16-bit status word with RDPIN/RQPIN.

After a lower-pin status change, IN will not rise again until acknowledged with one of WRPIN/WX-PIN/WYPIN/RDPIN/AKPIN — so always acknowledge before waiting for the next event, or the event is missed.

## Sending a Packet

1. WYPIN #80 on the lower pin to emit SOP.
2. After each **IN rise on the upper pin**, WYPIN BYTE on the lower pin to buffer the next byte.
3. Stop sending bytes and the transmitter appends EOP automatically.

Always confirm the upper pin's IN rose after each WYPIN before issuing the next one — even for a state change — because all output is paced by the baud generator and the buffer only empties at the next bit period.



## Transmitter and Receiver Are Independent

TX and RX have separate state machines; only the baud generator is shared. Note that the **receiver also sees all local transmit output** — the pin's own transmitted bytes appear in the RX status stream, so software must account for that loopback.

### CAUTION

**On an FPGA-emulated P2 the USB signaling resistors must be fitted externally** — the built-in resistors this mode relies on exist only on the ASIC. See Appendix G (FPGA Board Differences) for the specifics.

### CAUTION

**Transmit pacing tightens as the system clock rises.** Beyond the basic IN-flag handshake above, community USB drivers report that at higher `clkfreq` the transmit buffer must not be re-fed too soon, or bit-stuffed bits can be dropped — the safe inter-byte spacing scales with the system clock. Both the host driver (OBEX #4198) and the device driver (OBEX #4727) insert `sysclk`-proportional delays between output bytes to stay reliable. This is a community-observed behaviour; the exact mechanism is not described in the current silicon documentation, so tune the per-clock delay against the actual clock rather than treating any single value as a published figure.

## 19.5 Host vs Device Mode

### Device Mode

As a USB device, the P2:

- Waits for host to initiate communication
- Responds to USB requests
- Provides endpoints for data transfer
- Simpler to implement than host mode

#### Common device classes:

- CDC (Virtual COM Port)
- HID (Keyboard, Mouse, Gamepad)
- Mass Storage

### Host Mode

As a USB host, the P2:

- Provides bus power (5V)
- Initiates all communication
- Enumerates and configures devices
- Must handle all connected device types

**Host implementation is significantly more complex than device mode.**

## 19.6 Using USB Libraries

### Recommended Approach

Rather than implementing USB from scratch, use existing libraries:

**Parallax OBEX (Object Exchange)** — two community drivers are the natural starting points, one for each role:

- **USBnew** (OBEX #4198, by Wuerfel\_21) — a USB **host** / HID-input driver: with the P2 acting as host, it reads keyboards, mice, and gamepads.
- **USB Human-Interface-Device Driver** (OBEX #4727, by Chris Gadd) — a USB **device** driver: the P2 presents itself as a HID peripheral to a host.

These are the community implementations to study and build from; review each against the application's requirements and test it before relying on it.

### P2 Forums:

- Community-developed USB stacks
- Example implementations
- Troubleshooting support

### Example: Using a USB Library

```
' This is pseudocode showing typical USB library usage
' Actual syntax depends on the specific library

OBJ
  usb : "usb_cdc"                                ' USB CDC library

PUB main()
  usb.start(USB_DM, USB_DP)                        ' Initialize USB

  REPEAT
    IF usb.rx_check()                             ' Data available?
      process_data(usb.rx())                       ' Read and process

    IF have_data_to_send()
      usb.tx(output_data)                          ' Send data
```

## 19.7 Basic USB Configuration Example

### Pin Setup

```

CON
    _clkfreq = 200_000_000
    USB_DM = 56
    USB_DP = 57

PUB configure_usb() | baud
    ' Reset both pins
    PINFLOAT(USB_DM)
    PINFLOAT(USB_DP)

    ' Configure USB mode with output enabled
    WRPIN(USB_DM, P_USB_PAIR | P_OE)

    ' Set USB mode + baud on lower pin (D[15]=0 device, D[14]=1 full-speed)
    baud := 12_000_000 / (clkfreq / $100000)
    WXPIN(USB_DM, $4000 | baud)

    ' Enable the USB pair
    PINHIGH(USB_DM)
    PINHIGH(USB_DP)

```

ch19-usb-device-config.spin2

### PASM2 Configuration

```

DAT          ORG

    ' Reset USB pins
    DIRL      #USB_DM
    DIRL      #USB_DP

    ' Configure USB mode + baud (lower pin)
    WRPIN     usb_mode, #USB_DM
    WXPIN     usb_cfg, #USB_DM

    ' Enable USB pair
    DIRH      #USB_DM
    DIRH      #USB_DP

    ' Monitor for USB events
    .loop
        TESTP  #USB_DM wc          ' Check IN flag
        if_c   CALL  #handle_usb_event
        JMP    #.loop

```

*continues on next page →*

```

handle_usb_event
    RDPIN    usb_data, #USB_DM    ' Read USB data/status
    ' Process USB event...
    RET

usb_mode    LONG    P_USB_PAIR | P_OE
usb_cfg     LONG    $4000 | ($10000 * 12 / 200) ' device FS @ 200MHz
usb_data    RES     1

```

## 19.8 Hardware Considerations

### External Components

USB host mode requires:

- 5V power supply for VBUS
- Current limiting for protection
- Pull-up/pull-down resistors for speed identification

USB device mode requires:

- Pull-up resistor on D+ (Full Speed) or D- (Low Speed)
- No level shifting needed on D+/D-: the P2's I/O is 3.3V, which matches USB low/full-speed signaling (only VBUS is 5V)

### Pin Selection

Choose USB pins based on:

- Physical proximity to USB connector
- Trace length matching for differential pair
- Isolation from noisy digital signals

## 19.9 Limitations

### What P2 USB Cannot Do

- USB 2.0 High Speed (480 Mbps)
- USB 3.x SuperSpeed
- Isochronous transfers with guaranteed timing (challenging)

## Software Requirements

Implementing USB requires:

- Precise timing for packet handling
- State machine for USB protocol
- Buffer management for endpoints
- Error handling and recovery

**Development time:** Expect weeks to months for a robust USB implementation.

## 19.10 Quick Reference

### Mode Constant

| Constant   | Value  | Description                |
|------------|--------|----------------------------|
| P_USB_PAIR | %11011 | USB differential pair mode |

### Pin Requirements

- Even/odd consecutive pins
- Even pin = DM (D-)
- Odd pin = DP (D+)
- Both pins must be enabled (DIRH)

### Configuration Pattern

```
WRPIN(even_pin, P_USB_PAIR | P_OE)           ' Configure with output
WXPIN(even_pin, $4000 | (12_000_000 / (clkfreq / $10000))) ' full-speed
PINHIGH(even_pin)                             ' Enable DM
PINHIGH(even_pin+1)                           ' Enable DP
```

### Key Points

- Smart pin handles physical layer signaling
- Software must implement full USB protocol stack
- Use existing libraries when possible
- Supports USB 1.1 Full Speed and Low Speed only
- OUT signals are overridden by USB mode
- Limited official documentation - community resources essential

## Resources

- Parallax P2 Forums: [forums.parallax.com](http://forums.parallax.com)
- Parallax OBEX: [obex.parallax.com](http://obex.parallax.com)
- USB Specification: [usb.org](http://usb.org)
- P2 USB implementation examples from community members

*This chapter covered the USB smart pin mode. For a complete mode reference, see Appendix F.*

## Part V: Appendices

### Appendix A: Intent Index

This appendix provides task-oriented navigation. Find the task to accomplish, then follow the reference to the appropriate chapter and mode.

#### Generate Signals

*Producing output — clocks, frequencies, PWM, analog, and serial.*

| I want to...                 | Go to | Primary mode                           | Also consider             |
|------------------------------|-------|----------------------------------------|---------------------------|
| Generate a clock signal      | Ch 7  | P_TRANSITION (%00101)                  | P_NCO_FREQ                |
| Generate a fixed frequency   | Ch 8  | P_NCO_FREQ (%00110)                    | P_TRANSITION              |
| Generate PWM (motor control) | Ch 9  | P_PWM_SAWTOOTH (%01001)                | P_PWM_SMPS                |
| Generate PWM (LED dimming)   | Ch 9  | P_PWM_TRIANGLE (%01000)                | P_NCO_DUTY                |
| Generate audio tones         | Ch 8  | P_NCO_FREQ (%00110)                    | DAC modes                 |
| Generate arbitrary waveforms | Ch 10 | P_DAC_DITHER_RND /<br>P_DAC_DITHER_PWM | NCO + lookup table        |
| Output analog voltage        | Ch 10 | P_DAC_990R_3V (8-bit)                  | P_DAC_DITHER_RND (16-bit) |
| Transmit serial — UART       | Ch 11 | P_ASYNC_TX (%11110)                    | P_INVERT_OUT (RS-232)     |
| Transmit serial — SPI        | Ch 11 | P_SYNC_TX (%11100)                     | clock polarity/phase      |
| Generate precise pulses      | Ch 7  | P_PULSE (%00100)                       | NCO modes                 |

## Measure Signals

*Reading signal characteristics — width, frequency, period, counts, analog.*

| I want to...                 | Go to | Primary mode                         | Also consider                          |
|------------------------------|-------|--------------------------------------|----------------------------------------|
| Measure pulse width          | Ch 13 | P_HIGH_TICKS (%10001)                | P_STATE_TICKS (%10000) for both states |
| Measure signal frequency     | Ch 15 | P_COUNTER_PERIODS (%10111), 1-s gate | P_PERIODS_TICKS                        |
| Measure signal period        | Ch 15 | P_PERIODS_TICKS (%10011)             | P_COUNTER_TICKS                        |
| Measure duty cycle           | Ch 15 | P_PERIODS_TICKS + P_PERIODS_HIGHS    | P_COUNTER_TICKS + P_COUNTER_HIGHS      |
| Measure time between events  | Ch 13 | P_EVENTS_TICKS (%10010)              | timeout detection                      |
| Count events                 | Ch 14 | P_COUNT_RISES / P_COUNT_HIGHS        | P_QUADRATURE (bidirectional)           |
| Measure analog voltage (ADC) | Ch 16 | P_ADC (%11000), SINC2                | gain P_ADC_1X...P_ADC_100X             |
| Receive serial — UART        | Ch 17 | P_ASYNC_RX (%11111)                  | P_INVERT_IN (RS-232)                   |
| Receive serial — SPI         | Ch 17 | P_SYNC_RX (%11101)                   | clock routing P_PLUS1_B                |

## Control Outputs

*Driving actuators and indicators.*

| I want to...                 | Go to | Primary mode                 | Also consider          |
|------------------------------|-------|------------------------------|------------------------|
| Turn a pin on or off         | Ch 6  | PINHIGH / PINLOW / PINTOGGLE | PINWRITE (value-based) |
| Control LED brightness       | Ch 9  | P_PWM_TRIANGLE               | P_NCO_DUTY             |
| Control motor speed          | Ch 9  | P_PWM_SAWTOOTH               | P_PWM_SMPS (H-bridge)  |
| Control servo position       | Ch 7  | P_PULSE (1–2 ms)             | PWM at 50 Hz           |
| Output precise analog levels | Ch 10 | P_DAC_DITHER_PWM             | external DAC           |



## Read Inputs

*Sensing the outside world.*

| <b>I want to...</b>        | <b>Go to</b> | <b>Primary mode</b>            | <b>Also consider</b>          |
|----------------------------|--------------|--------------------------------|-------------------------------|
| Read a button or switch    | Ch 12        | PINREAD + pull-up (P_HIGH_15K) | P_SCHMITT_A for noisy signals |
| Read a digital sensor      | Ch 12        | TESTP (fast flag read)         | input conditioning options    |
| Read a rotary encoder      | Ch 14        | P_QUADRATURE (%01011)          | velocity from position deltas |
| Read an analog sensor      | Ch 16        | P_ADC + gain                   | averaging for noise           |
| Read multiple pins at once | Ch 12        | INA / INB registers            | PINREAD with ADDPINS          |

## Communicate

*Talking to other devices — UART, SPI, I<sup>2</sup>C, USB.*

| <b>I want to...</b>        | <b>Go to</b>  | <b>Primary mode</b>                | <b>Also consider</b>               |
|----------------------------|---------------|------------------------------------|------------------------------------|
| UART / RS-232              | Ch 11 & Ch 17 | P_ASYNC_TX + P_ASYNC_RX            | P_INVERT_IN / P_INVERT_OUT         |
| Be an SPI master           | Ch 11 & Ch 17 | P_SYNC_TX + separate clock pin     | NCO for clock generation           |
| Be an SPI slave            | Ch 17         | P_SYNC_RX (%11101) + clock routing | left-justified data                |
| Implement I <sup>2</sup> C | Ch 6          | open-drain + clock stretching      | existing I <sup>2</sup> C library  |
| Use USB                    | Ch 19         | P_USB_PAIR (%11011), even/odd pair | existing USB library (recommended) |

## Coordinate and Synchronize

*Timing, events, and inter-cog coordination.*

| I want to...                     | Go to | Primary mode                        | Also consider                |
|----------------------------------|-------|-------------------------------------|------------------------------|
| Synchronize multiple pin outputs | Ch 7  | SETSE1 / WAITSE1 events             | shared X base period         |
| Share data between Cogs          | Ch 18 | P_REPOSITORY (%00001, non-DAC)      | RQPIN for non-blocking reads |
| Precise timing control           | Ch 1  | 3-clock output/input latency (§1.2) | TESTP for 2-clock input path |
| Generate synchronized waveforms  | Ch 8  | multiple NCO pins, related freqs    | common base period for phase |

## Quick Mode Lookup

**Table A.2.**

| Mode          | Constant                   | Primary Use                |
|---------------|----------------------------|----------------------------|
| %00001        | P_REPOSITORY / P_DAC_NOISE | Inter-Cog data / Noise     |
| %00010        | P_DAC_DITHER_RND           | 16-bit DAC (random dither) |
| %00011        | P_DAC_DITHER_PWM           | 16-bit DAC (PWM dither)    |
| %00100        | P_PULSE                    | Pulse generation           |
| %00101        | P_TRANSITION               | Clock/transition output    |
| %00110        | P_NCO_FREQ                 | Frequency synthesis        |
| %00111        | P_NCO_DUTY                 | Duty cycle control         |
| %01000        | P_PWM_TRIANGLE             | Triangle PWM               |
| %01001        | P_PWM_SAWTOOTH             | Sawtooth PWM               |
| %01010        | P_PWM_SMPS                 | SMPS PWM                   |
| %01011        | P_QUADRATURE               | Quadrature encoder         |
| %01100-%01111 | P_COUNT_*                  | Counting modes             |
| %10000        | P_STATE_TICKS              | Measure both states        |
| %10001        | P_HIGH_TICKS               | Measure high time          |
| %10010        | P_EVENTS_TICKS             | Event timing/timeout       |
| %10011        | P_PERIODS_TICKS            | Measure X periods          |

Continued on next page

Table A.2. (Continued)

| Mode   | Constant          | Primary Use               |
|--------|-------------------|---------------------------|
| %10100 | P_PERIODS_HIGHS   | Sum highs in X periods    |
| %10101 | P_COUNTER_TICKS   | Time in X-clock window    |
| %10110 | P_COUNTER_HIGHS   | Highs in X-clock window   |
| %10111 | P_COUNTER_PERIODS | Count periods in X clocks |
| %11000 | P_ADC             | Internal clock ADC        |
| %11001 | P_ADC_EXT         | External clock ADC        |
| %11010 | P_ADC_SCOPE       | Triggered scope ADC       |
| %11011 | P_USB_PAIR        | USB differential pair     |
| %11100 | P_SYNC_TX         | Synchronous serial TX     |
| %11101 | P_SYNC_RX         | Synchronous serial RX     |
| %11110 | P_ASYNC_TX        | Asynchronous serial TX    |
| %11111 | P_ASYNC_RX        | Asynchronous serial RX    |

*For detailed mode descriptions, see the relevant chapter. For P\_ constant values, see Appendix B.*

# Appendix B: P\_ Constants Quick Reference

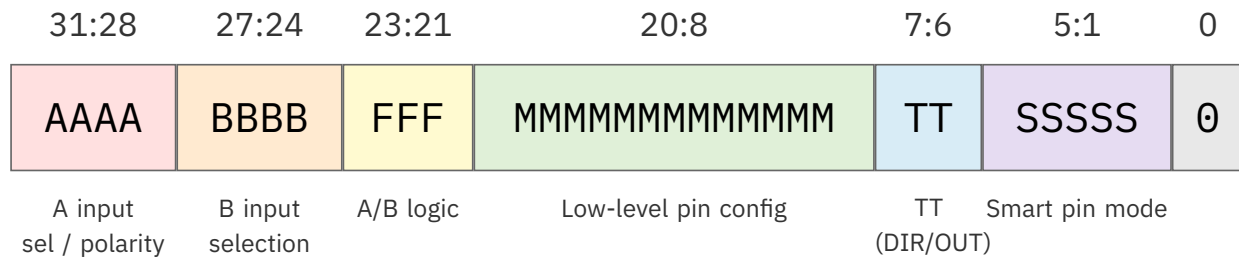
This appendix provides a complete reference for all P\_ constants used in smart pin configuration.

## How to Use P\_ Constants

Constants are combined using the OR operator to build a complete WRPIN configuration:

```
mode := P_ASYNC_TX | P_OE | P_INVERT_OUT
WRPIN(pin, mode)
```

### Structure of a P\_ constant value:



**Figure B.1.** Structure of a P\_ constant value

## Smart Pin Modes (pick one)

**Table B.3.**

| Constant         | Value  | Mode    | Description                 | Chapter |
|------------------|--------|---------|-----------------------------|---------|
| P_NORMAL         | %00000 | Default | Normal mode (not smart pin) | 3       |
| P_REPOSITORY     | %00001 | Non-DAC | Long repository             | 18      |
| P_DAC_NOISE      | %00001 | DAC     | DAC noise output            | 10      |
| P_DAC_DITHER_RND | %00010 | DAC     | 16-bit random dither DAC    | 10      |
| P_DAC_DITHER_PWM | %00011 | DAC     | 16-bit PWM dither DAC       | 10      |
| P_PULSE          | %00100 | -       | Pulse/cycle output          | 7       |
| P_TRANSITION     | %00101 | -       | Transition output           | 7       |

Continued on next page

Table B.3. (Continued)

| Constant          | Value  | Mode | Description               | Chapter |
|-------------------|--------|------|---------------------------|---------|
| P_NCO_FREQ        | %00110 | -    | NCO frequency output      | 8       |
| P_NCO_DUTY        | %00111 | -    | NCO duty output           | 8       |
| P_PWM_TRIANGLE    | %01000 | -    | PWM triangle              | 9       |
| P_PWM_SAWTOOTH    | %01001 | -    | PWM sawtooth              | 9       |
| P_PWM_SMPS        | %01010 | -    | PWM SMPS I/O              | 9       |
| P_QUADRATURE      | %01011 | -    | Quadrature encoder        | 14      |
| P_REG_UP          | %01100 | -    | Inc on A-rise when B-high | 14      |
| P_REG_UP_DOWN     | %01101 | -    | Inc/dec gated counter     | 14      |
| P_COUNT_RISES     | %01110 | -    | Count A-rises             | 14      |
| P_COUNT_HIGHS     | %01111 | -    | Count A-highs             | 14      |
| P_STATE_TICKS     | %10000 | -    | Time A-low and A-high     | 13      |
| P_HIGH_TICKS      | %10001 | -    | Time A-high states        | 13      |
| P_EVENTS_TICKS    | %10010 | -    | Time X events / timeout   | 13      |
| P_PERIODS_TICKS   | %10011 | -    | Measure X periods         | 15      |
| P_PERIODS_HIGHS   | %10100 | -    | Sum highs in X periods    | 15      |
| P_COUNTER_TICKS   | %10101 | -    | Time in X-clock window    | 15      |
| P_COUNTER_HIGHS   | %10110 | -    | Highs in X-clock window   | 15      |
| P_COUNTER_PERIODS | %10111 | -    | Count periods in X clocks | 15      |
| P_ADC             | %11000 | -    | ADC internal clock        | 16      |
| P_ADC_EXT         | %11001 | -    | ADC external clock        | 16      |
| P_ADC_SCOPE       | %11010 | -    | ADC scope with trigger    | 16      |
| P_USB_PAIR        | %11011 | -    | USB differential pair     | 19      |
| P_SYNC_TX         | %11100 | -    | Synchronous serial TX     | 11      |
| P_SYNC_RX         | %11101 | -    | Synchronous serial RX     | 17      |
| P_ASYNC_TX        | %11110 | -    | Asynchronous serial TX    | 11      |
| P_ASYNC_RX        | %11111 | -    | Asynchronous serial RX    | 17      |

## A Input Selection (pick one)

| Constant   | Bits [31:28] | Description                      |
|------------|--------------|----------------------------------|
| P_TRUE_A   | %0000        | True A input (default)           |
| P_INVERT_A | %1000        | Invert A input                   |
| P_LOCAL_A  | %0000        | Select local pin for A (default) |
| P_PLUS1_A  | %0001        | Select pin+1 for A               |
| P_PLUS2_A  | %0010        | Select pin+2 for A               |
| P_PLUS3_A  | %0011        | Select pin+3 for A               |
| P_OUTBIT_A | %0100        | Select OUT bit for A             |
| P_MINUS3_A | %0101        | Select pin-3 for A               |
| P_MINUS2_A | %0110        | Select pin-2 for A               |
| P_MINUS1_A | %0111        | Select pin-1 for A               |

## B Input Selection (pick one)

| Constant   | Bits [27:24] | Description                      |
|------------|--------------|----------------------------------|
| P_TRUE_B   | %0000        | True B input (default)           |
| P_INVERT_B | %1000        | Invert B input                   |
| P_LOCAL_B  | %0000        | Select local pin for B (default) |
| P_PLUS1_B  | %0001        | Select pin+1 for B               |
| P_PLUS2_B  | %0010        | Select pin+2 for B               |
| P_PLUS3_B  | %0011        | Select pin+3 for B               |
| P_OUTBIT_B | %0100        | Select OUT bit for B             |
| P_MINUS3_B | %0101        | Select pin-3 for B               |
| P_MINUS2_B | %0110        | Select pin-2 for B               |
| P_MINUS1_B | %0111        | Select pin-1 for B               |

## A/B Input Logic (pick one)

| Constant   | Bits [23:21] | Description                   |
|------------|--------------|-------------------------------|
| P_PASS_AB  | %000         | Pass A, B unchanged (default) |
| P_AND_AB   | %001         | A AND B, B                    |
| P_OR_AB    | %010         | A OR B, B                     |
| P_XOR_AB   | %011         | A XOR B, B                    |
| P_FILT0_AB | %100         | FILT0 settings for A, B       |
| P_FILT1_AB | %101         | FILT1 settings for A, B       |
| P_FILT2_AB | %110         | FILT2 settings for A, B       |
| P_FILT3_AB | %111         | FILT3 settings for A, B       |

## Input Conditioning Modes (pick one)

### Logic/Schmitt/Comparator Modes

| Constant        | Description                               | Chapter |
|-----------------|-------------------------------------------|---------|
| P_LOGIC_A       | Logic level A to IN, output OUT (default) | 12      |
| P_LOGIC_A_FB    | Logic level A to IN, output feedback      | 12      |
| P_LOGIC_B_FB    | Logic level B to IN, output feedback      | 12      |
| P_SCHMITT_A     | Schmitt trigger A to IN, output OUT       | 12      |
| P_SCHMITT_A_FB  | Schmitt trigger A to IN, output feedback  | 12      |
| P_SCHMITT_B_FB  | Schmitt trigger B to IN, output feedback  | 12      |
| P_COMPARE_AB    | A > B to IN, output OUT                   | 12      |
| P_COMPARE_AB_FB | A > B to IN, output feedback              | 12      |

### Level Comparison Modes

| Constant      | Description                        | Chapter |
|---------------|------------------------------------|---------|
| P_LEVEL_A     | A > Level to IN, output OUT        | 12      |
| P_LEVEL_A_FBN | A > Level to IN, negative feedback | 12      |
| P_LEVEL_B_FBP | B > Level to IN, positive feedback | 12      |
| P_LEVEL_B_FBN | B > Level to IN, negative feedback | 12      |

## ADC Input Modes (pick one)

| Constant    | Gain  | Input Range | Description             |
|-------------|-------|-------------|-------------------------|
| P_ADC_GIO   | -     | -           | Ground-referenced input |
| P_ADC_VIO   | -     | -           | VIO-referenced input    |
| P_ADC_FLOAT | -     | -           | Floating input          |
| P_ADC_1X    | 1x    | 0-3.3V      | Unity gain              |
| P_ADC_3X    | 3.16x | 0-1.04V     | 3.16x gain              |
| P_ADC_10X   | 10x   | 0-330mV     | 10x gain                |
| P_ADC_30X   | 31.6x | 0-104mV     | 31.6x gain              |
| P_ADC_100X  | 100x  | 0-33mV      | 100x gain               |

## DAC Output Modes (pick one)

| Constant      | Resistance   | Voltage   | Description      |
|---------------|--------------|-----------|------------------|
| P_DAC_990R_3V | 990 $\Omega$ | 3.3V peak | Standard DAC     |
| P_DAC_600R_2V | 600 $\Omega$ | 2.0V peak | Lower impedance  |
| P_DAC_124R_3V | 124 $\Omega$ | 3.3V peak | Low impedance    |
| P_DAC_75R_2V  | 75 $\Omega$  | 2.0V peak | Lowest impedance |

## Sync/Async I/O (pick one)

| Constant   | Description                |
|------------|----------------------------|
| P_ASYNC_IO | Asynchronous I/O (default) |
| P_SYNC_IO  | Synchronous I/O            |



## IN/OUT Polarity (pick one each)

### IN Polarity

| Constant    | Description           |
|-------------|-----------------------|
| P_TRUE_IN   | True IN bit (default) |
| P_INVERT_IN | Invert IN bit         |

### Output Polarity

| Constant        | Description               |
|-----------------|---------------------------|
| P_TRUE_OUTPUT   | True output (default)     |
| P_TRUE_OUT      | Alias for P_TRUE_OUTPUT   |
| P_INVERT_OUTPUT | Inverted output           |
| P_INVERT_OUT    | Alias for P_INVERT_OUTPUT |

## Drive Strength - High (pick one)

| Constant     | Drive         | Description               |
|--------------|---------------|---------------------------|
| P_HIGH_FAST  | 30mA          | Fast drive high (default) |
| P_HIGH_1K5   | 1.5k $\Omega$ | Resistive pull-up         |
| P_HIGH_15K   | 15k $\Omega$  | Weak pull-up              |
| P_HIGH_150K  | 150k $\Omega$ | Very weak pull-up         |
| P_HIGH_1MA   | 1mA           | Current source            |
| P_HIGH_100UA | 100uA         | Weak current source       |
| P_HIGH_10UA  | 10uA          | Very weak current source  |
| P_HIGH_FLOAT | -             | Float high (tri-state)    |

## Drive Strength - Low (pick one)

| Constant    | Drive         | Description              |
|-------------|---------------|--------------------------|
| P_LOW_FAST  | 30mA          | Fast drive low (default) |
| P_LOW_1K5   | 1.5k $\Omega$ | Resistive pull-down      |
| P_LOW_15K   | 15k $\Omega$  | Weak pull-down           |
| P_LOW_150K  | 150k $\Omega$ | Very weak pull-down      |
| P_LOW_1MA   | 1mA           | Current sink             |
| P_LOW_100UA | 100uA         | Weak current sink        |
| P_LOW_10UA  | 10uA          | Very weak current sink   |
| P_LOW_FLOAT | -             | Float low (tri-state)    |

## TT Bits / DIR-OUT Control (pick one)

| Constant  | TT Value | Description                        |
|-----------|----------|------------------------------------|
| P_TT_00   | %00      | Default TT setting                 |
| P_TT_01   | %01      | TT = 01                            |
| P_TT_10   | %10      | TT = 10                            |
| P_TT_11   | %11      | TT = 11                            |
| P_OE      | %01      | Enable output in smart pin mode    |
| P_CHANNEL | %01      | Enable DAC channel (non-smart pin) |
| P_BITDAC  | %10      | Enable BITDAC (non-smart pin)      |

## Common Combinations

### UART Transmit

|                                  |                        |
|----------------------------------|------------------------|
| P_ASYNC_TX   P_OE                | ' Basic UART TX        |
| P_ASYNC_TX   P_OE   P_INVERT_OUT | ' RS-232 TX (inverted) |

## UART Receive

|                          |                        |
|--------------------------|------------------------|
| P_ASYNC_RX               | ' Basic UART RX        |
| P_ASYNC_RX   P_INVERT_IN | ' RS-232 RX (inverted) |

## SPI Master TX

|                  |                |
|------------------|----------------|
| P_SYNC_TX   P_OE | ' SPI data out |
|------------------|----------------|

## SPI Slave RX

|                       |                                    |
|-----------------------|------------------------------------|
| P_SYNC_RX   P_PLUS1_B | ' SPI data in, clock from next pin |
|-----------------------|------------------------------------|

## PWM Output

|                       |                |
|-----------------------|----------------|
| P_PWM_SAWTOOTH   P_OE | ' Standard PWM |
| P_PWM_TRIANGLE   P_OE | ' Triangle PWM |

## DAC Output

|                                         |                       |
|-----------------------------------------|-----------------------|
| P_DAC_DITHER_RND   P_DAC_124R_3V   P_OE | ' 16-bit dithered DAC |
| P_DAC_990R_3V   P_OE                    | ' Basic 8-bit DAC     |

## ADC Input

|                   |                         |
|-------------------|-------------------------|
| P_ADC_GIO   P_ADC | ' Ground-referenced ADC |
| P_ADC_10X   P_ADC | ' 10x gain ADC          |

## Button Input with Pull-up

|                          |                                 |
|--------------------------|---------------------------------|
| P_SCHMITT_A   P_HIGH_15K | ' Schmitt trigger + 15k pull-up |
|--------------------------|---------------------------------|

## Open-Drain Output

P\_HIGH\_FLOAT | P\_LOW\_FAST

' Open-drain (for I2C, etc.)

*For mode-specific usage, see the relevant chapter. For task-based lookup, see Appendix A.*

# Appendix C: Formulas Reference

This appendix collects all mathematical formulas from the P2 I/O & Smart Pins User Guide in a single quick-reference location.

## NCO Frequency Generation

### Output Frequency from Y Value

#### Formula:

$$\text{frequency} = (Y \times \text{sysclk}) / (X[15:0] \times 2^{32})$$

For  $X[15:0] = 1$  (maximum update rate):

$$\text{frequency} = (Y \times \text{sysclk}) / 2^{32}$$

#### Variables:

- **frequency:** Output frequency in Hz
- **Y:** NCO frequency control word (32-bit)
- **sysclk:** System clock frequency in Hz
- **X[15:0]:** Base period (1 for maximum resolution)

#### Worked Example (1 kHz at 200 MHz):

```
frequency = 1000 Hz
sysclk = 200,000,000 Hz
Y = (1000 × 4,294,967,296) / 200,000,000
Y = 21,475
```

**Note:** Frequency resolution is  $\text{sysclk} / 2^{32}$  (~0.047 Hz at 200 MHz).

### Y Value from Desired Frequency

#### Formula:

$$Y = (\text{frequency} \times 2^{32}) / \text{sysclk}$$

In Spin2:

```
y_value := frequency FRAC _clkfreq
```

**Worked Example (10 kHz at 200 MHz):**

$$Y = (10,000 \times 4,294,967,296) / 200,000,000$$

$$Y = 214,748$$

## PWM Output

### PWM Frequency (Triangle Mode)

**Formula:**

$$\text{PWM\_frequency} = \text{sysclk} / (2 \times X[31:16] \times X[15:0])$$

**Variables:**

- $X[31:16]$ : Frame period (counter range)
- $X[15:0]$ : Base period (clocks per counter update)

**Worked Example (1 kHz PWM at 200 MHz):**

$$\text{PWM\_frequency} = 200,000,000 / (2 \times 100,000 \times 1) = 1000 \text{ Hz}$$

### PWM Frequency (Sawtooth Mode)

**Formula:**

$$\text{PWM\_frequency} = \text{sysclk} / (X[31:16] \times X[15:0])$$

**Worked Example (20 kHz PWM at 200 MHz):**

$$X[31:16] = 200,000,000 / 20,000 = 10,000$$

$$\text{PWM\_frequency} = 200,000,000 / (10,000 \times 1) = 20,000 \text{ Hz}$$

## PWM Duty Cycle

**Formula:**

$$\text{duty\_percent} = (Y[15:0] / X[31:16]) \times 100\%$$

**Worked Example (50% duty with frame=10,000):**

$$Y = 10,000 \times 50 / 100 = 5,000$$

$$\text{duty\_percent} = (5,000 / 10,000) \times 100\% = 50\%$$

## PWM Resolution (Bits)

### Formula:

```
resolution_bits = log2(X[31:16])
```

| Frame Period | Resolution |
|--------------|------------|
| 256          | 8 bits     |
| 1024         | 10 bits    |
| 4096         | 12 bits    |
| 65535        | 16 bits    |

## Serial Communication (UART)

### Baud Rate Timing

#### Formula (Basic):

```
X[31:16] = sysclk / baud_rate
```

#### Formula (With Fractional Precision):

```
X = ((sysclk × 65536 / baud_rate) & $FFFFFFC00) | data_bits
```

#### Variables:

- X[31:16]: Integer bit period in clocks
- X[15:10]: Fractional adjustment (1/64 clock)
- X[4:0]: Data-bit count, encoded as N-1 (write 0-31 to select 1-32 bits; e.g. write 7 for an 8-bit word)
- baud\_rate: Desired baud rate in bits/second

#### Worked Example (115200 baud at 200 MHz):

```
X[31:16] = 200,000,000 / 115,200 = 1736 clocks/bit
bit_period = 1736 × 65536 = 113,770,496
X = ($06C8_0000 | 7) = $06C8_0007 (8 data bits: field = N-1 = 7)
```

## Baud Rate Error

### Formula:

```
actual_baud = sysclk / round(sysclk / target_baud)
error_percent = ABS(actual_baud - target_baud) / target_baud × 100%
```

**Note:** UART typically tolerates  $\pm 2\text{-}3\%$  baud rate error.

## ADC (Analog Input)

### ADC Sample Rate

#### Formula:

```
sample_rate = sysclk / 2^(X[3:0])
```

#### Variables:

- X[3:0]: Sample period exponent (1-15)

#### Worked Example (8-bit SINC2 at 200 MHz):

```
X[3:0] = 7 (128 clocks)
sample_rate = 200,000,000 / 128 = 1,562,500 Hz
```

| Resolution | X[3:0] | Period      | Sample Rate at 200 MHz |
|------------|--------|-------------|------------------------|
| 8 bits     | %0111  | 128 clocks  | 1.56 MHz               |
| 10 bits    | %1001  | 512 clocks  | 390 kHz                |
| 12 bits    | %1011  | 2048 clocks | 97.6 kHz               |
| 14 bits    | %1101  | 8192 clocks | 24.4 kHz               |

## ADC Voltage Conversion

### Formula:

```
voltage_mv = (sample × 3300) / full_scale
```

#### Variables:

- sample: Raw ADC reading
- full\_scale: Maximum ADC value (depends on resolution)
- For 8-bit: full\_scale = 255



- For 14-bit:  $\text{full\_scale} = 16383$

Use  $\text{full\_scale} = 2^{\text{bits}} - 1$  for the resolution actually configured. See Chapter 16 for how mode and sample period set the bit depth.

#### Worked Example (8-bit ADC reading 128):

$$\text{voltage\_mv} = (128 \times 3300) / 255 = 1656 \text{ mV}$$

## ADC with Gain

#### Effective Input Range:

$$\text{input\_max} = 3300\text{mV} / \text{gain\_factor}$$

| Gain Mode  | Gain Factor | Input Range |
|------------|-------------|-------------|
| P_ADC_1X   | 1           | 0-3.3V      |
| P_ADC_3X   | 3.16        | 0-1.04V     |
| P_ADC_10X  | 10          | 0-330mV     |
| P_ADC_30X  | 31.6        | 0-104mV     |
| P_ADC_100X | 100         | 0-33mV      |

## DAC (Analog Output)

### 8-bit DAC Voltage

#### Formula:

$$\text{voltage} = (\text{DAC\_value} / 256) \times \text{V\_full\_scale}$$

#### Worked Example (DAC value 128, 3.3V range):

$$\text{voltage} = (128 / 256) \times 3.3\text{V} = 1.65\text{V}$$

### 16-bit DAC Voltage

#### Formula:

$$\text{voltage} = (\text{DAC\_value} / 65536) \times \text{V\_full\_scale}$$

**Resolution:**

- 3.3V range:  $3.3\text{V} / 65536 = 50.4 \mu\text{V/LSB}$
- 2.0V range:  $2.0\text{V} / 65536 = 30.5 \mu\text{V/LSB}$

**Worked Example (DAC value 32768, 3.3V range):**

```
voltage = (32768 / 65536) × 3.3V = 1.65V
```

**The 16-bit DAC is nominal — a temporal-averaging figure, not absolute accuracy.**

The hardware DAC is 8-bit (256 levels  $\approx 12.9 \text{ mV/step}$  at 3.3 V); the dithered modes reach 16-bit *averaged over time*, so the  $\mu\text{V/LSB}$  values above are the ideal step size, not guaranteed accuracy. Real effective bits depend on output low-pass filtering and load (see §18.4).

## Voltage to DAC Value

**8-bit:**

```
dac8 := (millivolts * 256) / 3300
```

**16-bit:**

```
dac16 := (millivolts * 65536) / 3300
```

## Timing Measurement

### Frequency from Period

**Formula:**

```
frequency = sysclk / period_clocks
```

**Worked Example (period of 200,000 clocks at 200 MHz):**

```
frequency = 200,000,000 / 200,000 = 1000 Hz
```

## Period in Microseconds

### Formula:

```
period_us = clocks / (sysclk / 1,000,000)
```

### Worked Example (1000 clocks at 200 MHz):

```
period_us = 1000 / (200,000,000 / 1,000,000) = 1000 / 200 = 5 μs
```

## Duty Cycle

### Formula:

```
duty_percent = (high_clocks × 100) / (high_clocks + low_clocks)
```

### Worked Example (high=3000, low=7000 clocks):

```
duty_percent = (3000 × 100) / (3000 + 7000) = 30%
```

## Period/Frequency Measurement

### Frequency from Period Ticks

#### Formula (P\_PERIODS\_TICKS):

```
frequency = (num_periods × sysclk) / rdpin_value
```

#### Variables:

- num\_periods: X register value (periods measured)
- rdpin\_value: Total clocks for all periods

### Worked Example (100 periods, 2,000,000 clocks at 200 MHz):

```
frequency = (100 × 200,000,000) / 2,000,000 = 10,000 Hz
```

### Frequency from Period Count

#### Formula (P\_COUNTER\_PERIODS with 1-second window):

```
frequency = rdpin_value (direct Hz reading)
```

**General Formula:**

```
frequency = (rdpin_value × sysclk) / window_clocks
```

**Duty Cycle from Period Modes****Using P\_PERIODS\_TICKS and P\_PERIODS\_HIGHS:**

```
duty_percent = (high_time × 100) / total_time
```

Where:

- high\_time = RDPIN from P\_PERIODS\_HIGHS
- total\_time = RDPIN from P\_PERIODS\_TICKS

**Quadrature Encoder****Position to Degrees****Formula:**

```
degrees = (position × 360) / counts_per_revolution
```

**Variables:**

- position: Quadrature count from RDPIN
- counts\_per\_revolution: 4 × encoder lines per revolution

**Worked Example (1000 line encoder, position 1000):**

```
counts_per_revolution = 4 × 1000 = 4000  
degrees = (1000 × 360) / 4000 = 90°
```

**Velocity (Steps per Period)****Formula:**

```
rpm = (steps_per_period × 60 × (1000 / period_ms)) / counts_per_revolution
```

**Worked Example (500 steps in 100ms, 4000 counts/rev):**

```
rpm = (500 × 60 × 10) / 4000 = 75 RPM
```

## Common sysclk Values

### Pre-Calculated Values at 200 MHz

| Frequency | NCO Y Value | PWM Frame (Sawtooth) |
|-----------|-------------|----------------------|
| 100 Hz    | 2,147       | 2,000,000            |
| 1 kHz     | 21,475      | 200,000              |
| 10 kHz    | 214,748     | 20,000               |
| 100 kHz   | 2,147,484   | 2,000                |
| 1 MHz     | 21,474,836  | 200                  |

### Common Baud Rates at 200 MHz

| Baud Rate | X[31:16] | Error |
|-----------|----------|-------|
| 9600      | 20833    | 0.00% |
| 19200     | 10417    | 0.00% |
| 38400     | 5208     | 0.01% |
| 57600     | 3472     | 0.01% |
| 115200    | 1736     | 0.01% |
| 230400    | 868      | 0.01% |
| 460800    | 434      | 0.01% |
| 921600    | 217      | 0.01% |
| 1000000   | 200      | 0.00% |

### ADC Resolution vs Speed at 200 MHz

See the **ADC Sample Rate** table above (resolution, X[3:0], period, and rate in one place).

## Pre-Calculated Values at 250 MHz

| Frequency | NCO Y Value | PWM Frame (Sawtooth) |
|-----------|-------------|----------------------|
| 100 Hz    | 1,718       | 2,500,000            |
| 1 kHz     | 17,180      | 250,000              |
| 10 kHz    | 171,799     | 25,000               |
| 100 kHz   | 1,717,987   | 2,500                |
| 1 MHz     | 17,179,869  | 250                  |

## Common Baud Rates at 250 MHz

| Baud Rate | X[31:16] | Error |
|-----------|----------|-------|
| 9600      | 26042    | 0.00% |
| 19200     | 13021    | 0.00% |
| 38400     | 6510     | 0.01% |
| 57600     | 4340     | 0.01% |
| 115200    | 2170     | 0.01% |
| 230400    | 1085     | 0.01% |
| 460800    | 543      | 0.09% |
| 921600    | 271      | 0.10% |
| 1000000   | 250      | 0.00% |

## Pre-Calculated Values at 300 MHz

| Frequency | NCO Y Value | PWM Frame (Sawtooth) |
|-----------|-------------|----------------------|
| 100 Hz    | 1,432       | 3,000,000            |
| 1 kHz     | 14,317      | 300,000              |
| 10 kHz    | 143,165     | 30,000               |
| 100 kHz   | 1,431,656   | 3,000                |
| 1 MHz     | 14,316,558  | 300                  |

## Common Baud Rates at 300 MHz

| Baud Rate | X[31:16] | Error |
|-----------|----------|-------|
| 9600      | 31250    | 0.00% |
| 19200     | 15625    | 0.00% |
| 38400     | 7813     | 0.01% |
| 57600     | 5208     | 0.01% |
| 115200    | 2604     | 0.01% |
| 230400    | 1302     | 0.01% |
| 460800    | 651      | 0.01% |
| 921600    | 326      | 0.15% |
| 1000000   | 300      | 0.00% |

## Pre-Calculated Values at 350 MHz

| Frequency | NCO Y Value | PWM Frame (Sawtooth) |
|-----------|-------------|----------------------|
| 100 Hz    | 1,227       | 3,500,000            |
| 1 kHz     | 12,271      | 350,000              |
| 10 kHz    | 122,713     | 35,000               |
| 100 kHz   | 1,227,134   | 3,500                |
| 1 MHz     | 12,271,335  | 350                  |

## Common Baud Rates at 350 MHz

| Baud Rate | X[31:16] | Error |
|-----------|----------|-------|
| 9600      | 36458    | 0.00% |
| 19200     | 18229    | 0.00% |
| 38400     | 9115     | 0.00% |
| 57600     | 6076     | 0.01% |
| 115200    | 3038     | 0.01% |
| 230400    | 1519     | 0.01% |
| 460800    | 760      | 0.06% |
| 921600    | 380      | 0.06% |
| 1000000   | 350      | 0.00% |

## Accuracy Notes

### NCO Frequency

- Resolution:  $\text{sysclk} / 2^{32}$
- At 200 MHz: ~0.047 Hz resolution
- Maximum frequency:  $\text{sysclk} / 2$  (Nyquist limit)

### PWM

- Resolution determined by frame period
- Maximum useful resolution: 16 bits (frame = 65535)
- Duty cycle error:  $1/\text{frame} \times 100\%$

### UART Baud

- Error should be <3% for reliable communication
- Fractional timing (X[15:10]) provides <0.01% error
- Both transmitter and receiver errors accumulate

### ADC

- SINC2 sampling provides power-of-2 sample periods only
- SINC3 limited to 512 clocks maximum period



- Oversampling 4× provides ~1 additional bit resolution

## DAC

- 8-bit native resolution (256 levels)
- 16-bit dithered resolution requires low-pass filtering
- Output accuracy depends on power supply and loading

*This appendix provides formula reference. For  $P_{\_}$  constants, see Appendix B. For application examples, see Appendix D.*

# Appendix D: Mode Comparison Charts

This appendix provides comparison matrices to help select the appropriate smart pin mode for the application.

## Output Mode Comparison

### All Output Modes at a Glance

| Mode         | Constant       | Freq Range         | Resolution    | Duty Control    | Continuous | Complexity | Primary Use             |
|--------------|----------------|--------------------|---------------|-----------------|------------|------------|-------------------------|
| Digital      | -              | DC only            | 1-bit         | N/A             | Yes        | Low        | On/off control          |
| Pulse        | P_PULSE        | DC to MHz          | 16-bit timing | Fixed per pulse | One-shot   | Low        | Single pulses, triggers |
| Transition   | P_TRANSITION   | DC to 100 MHz      | 16-bit period | 50% fixed       | Counted    | Low        | Clock generation        |
| NCO Freq     | P_NCO_FREQ     | 0.05 Hz to 100 MHz | 32-bit        | 50% fixed       | Yes        | Low        | Frequency synthesis     |
| NCO Duty     | P_NCO_DUTY     | 0.05 Hz to 100 MHz | 32-bit        | Variable        | Yes        | Medium     | Variable duty waves     |
| PWM Triangle | P_PWM_TRIANGLE | 1 Hz to 390 kHz    | 16-bit        | Full range      | Yes        | Low        | Motor, LED dimming      |
| PWM Sawtooth | P_PWM_SAWTOOTH | 1 Hz to 780 kHz    | 16-bit        | Full range      | Yes        | Low        | Motor, audio            |
| PWM SMPS     | P_PWM_SMPS     | Variable           | 16-bit        | Feedback        | Autonomous | High       | Power supply            |
| DAC 8-bit    | P_DAC_xxx      | DC                 | 8-bit         | N/A             | Yes        | Low        | Voltage reference       |
| DAC 16-bit   | P_DAC_DITHER_* | DC to audio        | 16-bit        | N/A             | Yes        | Medium     | Audio, precision        |
| Sync TX      | P_SYNC_TX      | Clock rate         | 1-32 bits     | N/A             | Clocked    | Medium     | SPI master              |
| Async TX     | P_ASYNC_TX     | 300 to 1M+ baud    | 1-32 bits     | N/A             | Per-byte   | Low        | UART                    |

# Input Mode Comparison

## All Input Modes at a Glance

| Mode            | Constant          | Measurement            | Resolution | Speed        | Autonomous | Complexity | Primary Use         |
|-----------------|-------------------|------------------------|------------|--------------|------------|------------|---------------------|
| Digital         | -                 | Logic state            | 1-bit      | Instant      | No         | Low        | Button, sensor      |
| State Ticks     | P_STATE_TICKS     | High/low time          | 1 clock    | Every edge   | Yes        | Low        | PWM analysis        |
| High Ticks      | P_HIGH_TICKS      | Pulse width            | 1 clock    | Per pulse    | Yes        | Low        | Servo, pulse        |
| Events Ticks    | P_EVENTS_TICKS    | N events / timeout     | 1 clock    | Configurable | Yes        | Medium     | Frequency, watchdog |
| Quadrature      | P_QUADRATURE      | Position/velocity      | 4x encoder | Every edge   | Yes        | Low        | Encoder             |
| Count Highs     | P_COUNT_HIGHS     | Gated edges            | 32-bit     | Configurable | Yes        | Low        | Freq counter        |
| Count Up/Down   | P_REG_UP_DOWN     | Up/down by B direction | 32-bit     | Configurable | Yes        | Low        | Step/direction      |
| Count Edges     | P_COUNT_RISES     | Edge/rise count        | 32-bit     | Configurable | Yes        | Low        | Event counter       |
| High Clocks     | P_COUNT_HIGHS     | High time sum          | 32-bit     | Configurable | Yes        | Low        | Duty cycle          |
| Periods Ticks   | P_PERIODS_TICKS   | N period time          | 1 clock    | N periods    | Yes        | Medium     | Precision freq      |
| Periods Highs   | P_PERIODS_HIGHS   | N period high          | 1 clock    | N periods    | Yes        | Medium     | Duty cycle          |
| Counter Ticks   | P_COUNTER_TICKS   | Time in window         | 1 clock    | Time window  | Yes        | Medium     | Freq measurement    |
| Counter Highs   | P_COUNTER_HIGHS   | High time in window    | 1 clock    | Time window  | Yes        | Medium     | Duty in window      |
| Counter Periods | P_COUNTER_PERIODS | Periods in window      | 1 period   | Time window  | Yes        | Low        | Freq counter        |
| ADC             | P_ADC             | Voltage                | 8-14 bits  | kHz to MHz   | Yes        | Medium     | Analog sensor       |
| ADC Scope       | P_ADC_SCOPE       | 4-ch capture           | 8 bits     | Triggered    | Triggered  | High       | Oscilloscope        |
| Sync RX         | P_SYNC_RX         | Serial data            | 1-32 bits  | Clock rate   | Yes        | Medium     | SPI slave           |
| Async RX        | P_ASYNC_RX        | Serial data            | 1-32 bits  | Baud rate    | Yes        | Low        | UART                |

## Frequency Generation Comparison

### When to Use Each Mode

| Application           | Best Mode                  | Why                                 |
|-----------------------|----------------------------|-------------------------------------|
| Fixed frequency clock | P_TRANSITION or P_NCO_FREQ | Clean 50% duty, precise frequency   |
| Variable frequency    | P_NCO_FREQ                 | 32-bit resolution, instant updates  |
| Audio tone            | P_NCO_FREQ                 | Sub-Hz resolution for musical notes |
| Motor PWM             | P_PWM_SAWTOOTH             | Fast switching, full duty range     |
| LED dimming           | P_PWM_TRIANGLE             | Smooth transitions, no flicker      |
| Servo control         | P_PULSE                    | Precise 1-2ms pulses at 50 Hz       |
| Analog waveform       | P_DAC_DITHER_PWM           | True analog output, 16-bit          |
| SMPS control          | P_PWM_SMPS                 | Built-in voltage/current feedback   |

### Frequency Range by Mode

| Mode           | Minimum | Maximum | Resolution       |
|----------------|---------|---------|------------------|
| P_NCO_FREQ     | 0.05 Hz | 100 MHz | 0.05 Hz (32-bit) |
| P_TRANSITION   | DC      | 100 MHz | 1 clock          |
| P_PWM_TRIANGLE | ~1 Hz   | 390 kHz | 1/frame          |
| P_PWM_SAWTOOTH | ~1 Hz   | 780 kHz | 1/frame          |
| P_PULSE        | Single  | MHz     | 1 clock          |

### Duty Cycle Capability

| Mode           | Duty Range  | Control Method  |
|----------------|-------------|-----------------|
| P_NCO_FREQ     | 50% fixed   | None            |
| P_NCO_DUTY     | 0-100%      | Y value         |
| P_PWM_TRIANGLE | 0-100%      | Y / frame       |
| P_PWM_SAWTOOTH | 0-100%      | Y / frame       |
| P_TRANSITION   | 50% fixed   | None            |
| P_PULSE        | Pulse width | Explicit clocks |

## Counting Mode Comparison

### Choosing the Right Counter

| Scenario                | Best Mode     | Configuration  |
|-------------------------|---------------|----------------|
| Simple event count      | P_COUNT_RISES | X=0, Y=0       |
| Gated frequency counter | P_COUNT_HIGHS | X=gate_period  |
| Step/direction motor    | P_COUNT_RISES | X=0            |
| Up/down buttons         | P_COUNT_RISES | X=0, Y=1       |
| Rotary encoder          | P_QUADRATURE  | X=0 (position) |
| Encoder velocity        | P_QUADRATURE  | X=period       |
| PWM duty integration    | P_HIGH_TICKS  | X=period       |
| Differential timing     | P_HIGH_TICKS  | X=period, Y=1  |

### Counter Features Matrix

| Mode          | A-Input | B-Input    | Direction | Gating | Signed |
|---------------|---------|------------|-----------|--------|--------|
| P_COUNT_RISES | Count   | Down (Y=1) | Y[0]      | No     | Yes    |
| P_REG_UP      | Count   | Gate       | No        | Yes    | No     |
| P_REG_UP_DOWN | Count   | Direction  | B level   | No     | Yes    |
| P_COUNT_HIGHS | Time    | Time (Y=1) | Y[0]      | Level  | Yes    |
| P_QUADRATURE  | Phase A | Phase B    | Automatic | No     | Yes    |

## Period/Frequency Measurement Comparison

### Mode Selection Guide

| Need                   | Best Mode                         | Why                            |
|------------------------|-----------------------------------|--------------------------------|
| Simple frequency count | P_COUNTER_PERIODS                 | Direct Hz reading with 1s gate |
| Precise period         | P_PERIODS_TICKS                   | Clock-accurate over N periods  |
| Unknown frequency      | P_COUNTER_PERIODS                 | Time-windowed, consistent rate |
| Duty cycle             | P_PERIODS_HIGHS + P_PERIODS_TICKS | Both measurements needed       |
| RPM measurement        | P_COUNTER_PERIODS                 | 100ms-1s gate time             |
| Oscillator calibration | P_PERIODS_TICKS                   | Many periods for ppm accuracy  |

### Measurement Resolution

| Mode              | What's Measured         | Resolution     | Precision Improves With |
|-------------------|-------------------------|----------------|-------------------------|
| P_PERIODS_TICKS   | Time for X periods      | $\pm 1$ clock  | More periods            |
| P_PERIODS_HIGHS   | High time for X periods | $\pm 1$ clock  | More periods            |
| P_COUNTER_TICKS   | Period time in window   | $\pm 1$ clock  | Longer window           |
| P_COUNTER_HIGHS   | High time in window     | $\pm 1$ clock  | Longer window           |
| P_COUNTER_PERIODS | Periods in window       | $\pm 1$ period | Longer window           |

# Serial Mode Comparison

## Transmit Modes

| Aspect          | P_ASYNC_TX (UART) | P_SYNC_TX (SPI)      |
|-----------------|-------------------|----------------------|
| Clock           | Implicit (baud)   | Explicit (B-input)   |
| Framing         | Start/stop bits   | None                 |
| Pins            | 1 (TX)            | 2 (Data + Clock)     |
| Bits per frame  | 1-32 + start/stop | 1-32                 |
| Update rate     | Baud / (bits + 2) | Clock / bits         |
| Double buffered | Yes               | Yes                  |
| Best for        | Point-to-point    | Bus, shift registers |

## Receive Modes

| Aspect             | P_ASYNC_RX (UART)   | P_SYNC_RX (SPI)               |
|--------------------|---------------------|-------------------------------|
| Clock              | Implicit (baud)     | External (B-input)            |
| Framing            | Auto-detects start  | None                          |
| Pins               | 1 (RX)              | 1 (Data), clock from adjacent |
| Bits per frame     | 1-32                | 1-32                          |
| Clock routing      | N/A                 | P_PLUS1_B etc. required       |
| Data justification | Left (MSB at Z[31]) | Left (MSB at Z[31])           |
| Best for           | RS-232, debug       | SPI slave, shift in           |

## Serial Speed Comparison

| Protocol    | Max Speed | Typical Use    |
|-------------|-----------|----------------|
| UART 115200 | 115 kbps  | Debug, GPS     |
| UART 1 Mbps | 1 Mbps    | Fast serial    |
| SPI 1 MHz   | 1 Mbps    | Sensors        |
| SPI 10 MHz  | 10 Mbps   | Flash, display |
| SPI 25 MHz  | 25 Mbps   | High-speed ADC |

# ADC Mode Comparison

## ADC Modes

| Mode        | Clock    | Triggering   | Channels | Best For           |
|-------------|----------|--------------|----------|--------------------|
| P_ADC       | Internal | Continuous   | 1        | General sensors    |
| P_ADC_EXT   | External | B-input edge | 1        | External ADC chips |
| P_ADC_SCOPE | Internal | Hysteretic   | 4        | Signal capture     |

## Filter Modes (X[5:4])

| X[5:4] | Mode            | Post-Processing  | Resolution  | Speed   |
|--------|-----------------|------------------|-------------|---------|
| %00    | SINC2 Sampling  | None             | 8-14 bits   | Fast    |
| %01    | SINC2 Filtering | Software diff    | 8-14 ENOB   | Medium  |
| %10    | SINC3 Filtering | Software 3x diff | 10-18 ENOB  | Slow    |
| %11    | Bitstream       | Custom           | 1 bit/clock | Fastest |

## Gain Selection Guide

| Signal Level | Best Gain  | Input Range | Use Case           |
|--------------|------------|-------------|--------------------|
| 0-3.3V       | P_ADC_1X   | Full        | Pot, sensor        |
| 0-1V         | P_ADC_3X   | 0-1.04V     | Low-voltage sensor |
| 0-300mV      | P_ADC_10X  | 0-330mV     | Thermocouple       |
| 0-100mV      | P_ADC_30X  | 0-104mV     | Strain gauge       |
| 0-30mV       | P_ADC_100X | 0-33mV      | Microphone         |



## DAC Mode Comparison

### Resistor Options

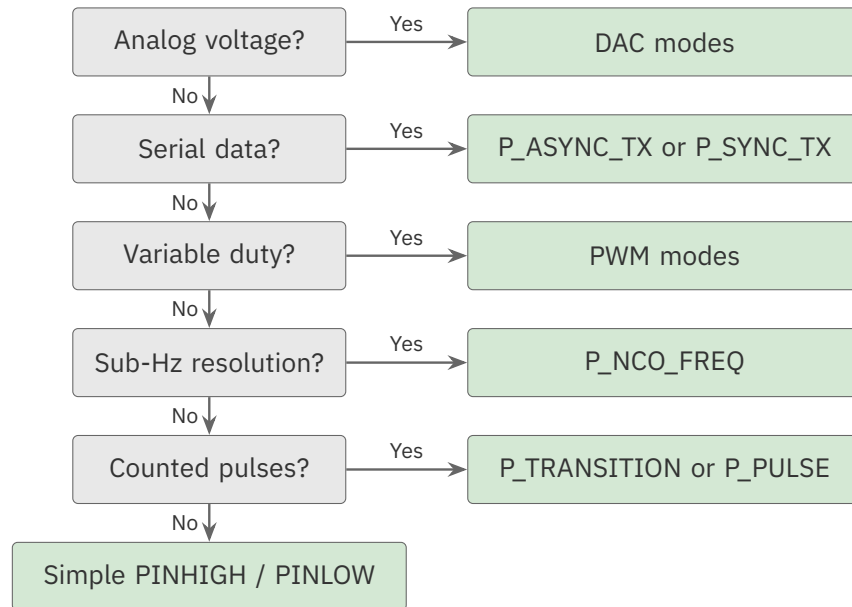
| Mode          | Resistance   | Voltage | Current | Best For     |
|---------------|--------------|---------|---------|--------------|
| P_DAC_990R_3V | 990 $\Omega$ | 0-3.3V  | ~3.3 mA | Op-amp input |
| P_DAC_600R_2V | 600 $\Omega$ | 0-2.0V  | ~3.3 mA | Medium load  |
| P_DAC_124R_3V | 124 $\Omega$ | 0-3.3V  | ~27 mA  | LED, speaker |
| P_DAC_75R_2V  | 75 $\Omega$  | 0-2.0V  | ~27 mA  | Coax cable   |

### Dithering Comparison

| Aspect        | P_DAC_DITHER_RND  | P_DAC_DITHER_PWM    |
|---------------|-------------------|---------------------|
| Pattern       | Random            | Deterministic       |
| Transitions   | Many              | Max 2 per 256 clk   |
| Spectrum      | White noise floor | Fclock/256 at -48dB |
| Dynamic range | Good              | Better              |
| Sample period | Any $\geq 1$      | Multiple of 256     |
| Best for      | Control signals   | Audio               |

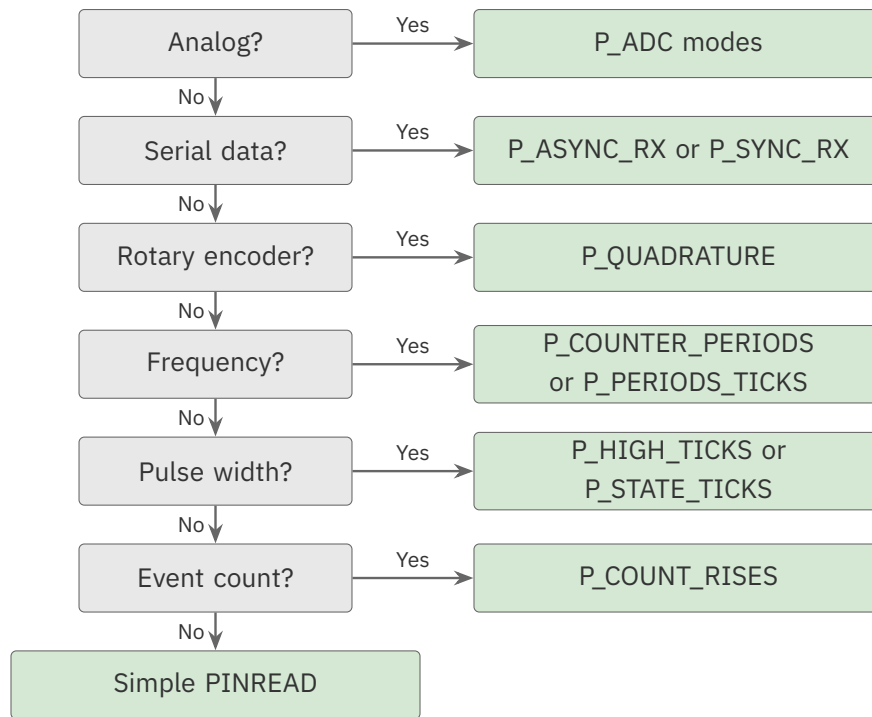
## Quick Selection Trees

### “I need to generate a signal”



**Figure D.1.** Output mode selection

## “I need to measure a signal”



**Figure D.2.** Input mode selection

For *P\_* constant values, see Appendix B. For formulas, see Appendix C. For troubleshooting, see Appendix E.

# Appendix E: Troubleshooting

This appendix provides problem/solution guidance for common smart pin issues organized by symptom.

## Pin Not Responding

### Symptom

Pin appears completely inactive. No output changes, no IN flag, no measurements.

### Likely Causes

1. **DIR not set** - Smart pin not enabled
2. **WRPIN not executed** - Mode not configured
3. **Wrong pin number** - Configuration applied to different pin
4. **Pin used by another cog** - Conflicting configurations

### Diagnostic Steps

1. Read pin state:

```
DEBUG("DIR: ", UDEC_(PINREAD(pin) >> 31))
DEBUG("OUT: ", UDEC_((INA >> pin) & 1))
```

2. Verify mode was written:

```
' Reset and reconfigure
PINL(pin)
WRPIN(pin, your_mode)
PINH(pin)
DEBUG("Mode applied")
```

### Solutions

#### Enable the pin:

```
' After WRPIN, MUST set DIR
WRPIN(pin, P_NCO_FREQ | P_OE)
PINH(pin)                                ' THIS IS REQUIRED
```

**For output modes, also set output:**

```
PINLOW(pin)           ' Sets DIR=1, starts smart pin
' or
PINH(pin)             ' Alternative
```

**Check for typos in pin number:**

```
CON
  MY_PIN = 20           ' Define constant

PUB setup(mode)
  ' Use constant, not magic numbers
  WRPIN(MY_PIN, mode)   ' Correct
  PINH(MY_PIN)
```

## No Output Visible

### Symptom

Smart pin configured for output, but oscilloscope shows no signal or wrong level.

### Likely Causes

1. **P\_OE not set** - Output enable missing
2. **Wrong drive strength** - Signal too weak
3. **Output routing mismatch** - Signal going elsewhere
4. **Hardware issue** - Shorted pin, wrong connection

### Diagnostic Steps

1. Check mode includes P\_OE:

```
mode := P_NCO_FREQ | P_OE           ' P_OE is REQUIRED for output
```

2. TEST with maximum drive:

```
mode := P_NCO_FREQ | P_OE | P_HIGH_FAST | P_LOW_FAST
```

3. Verify basic output works:

```
' Test pin with simple on/off
PINHIGH(pin)
WAITMS(1000)
PINLOW(pin)
WAITMS(1000)
```

## Solutions

### Add P\_OE to mode:

```
' WRONG - no output
WRPIN(pin, P_PWM_SAWTOOTH)

' CORRECT - output enabled
WRPIN(pin, P_PWM_SAWTOOTH | P_OE)
```

### Check inverted output:

```
' If signal appears inverted
mode := P_NCO_FREQ | P_OE | P_INVERT_OUTPUT
```

### For weak signals, increase drive:

```
' Default is P_HIGH_FAST | P_LOW_FAST
' For high-impedance loads, this should work
' For capacitive loads, ensure adequate drive
```

## Wrong Frequency or Timing

### Symptom

Output frequency or timing does not match expected value.

### Likely Causes

1. **sysclk assumption wrong** - Using wrong clock frequency
2. **Formula error** - Incorrect calculation
3. **Integer overflow** - Calculation exceeds 32 bits
4. **X register not set** - Default value being used

## Diagnostic Steps

1. Verify sysclk:

```
DEBUG("sysclk: ", UDEC(_clkfreq))
```

2. Check calculated values:

```
y_val := frequency FRAC _clkfreq
DEBUG("Y value: ", UHEX(y_val))
```

3. Verify X register was written:

```
WXPIN(pin, x_value)
DEBUG("X written: ", UHEX(x_value))
```

## Solutions

### Use correct sysclk:

```
CON
    _clkfreq = 200_000_000          ' Verify this matches actual clock

PUB calc_frequency(hz) : y_val
    y_val := hz FRAC _clkfreq
```

### Use the FRAC operator for NCO Y calculation:

```
' CORRECT - FRAC = (frequency * 2^32) / _clkfreq (32-bit)
' without manual 33-bit constant arithmetic.
y_val := frequency FRAC _clkfreq
```

Use FRAC, not a hand-rolled  $\text{frequency} * \$1\_0000\_0000 / \_clkfreq$  (the  $\$1\_0000\_0000$  literal exceeds the 32-bit constant range). See Chapter 8 for how FRAC derives the NCO Y value.

### For NCO, remember X[15:0] affects frequency:

```
' With X[15:0] = 1 (default)
frequency = Y * sysclk / 2^32

' With X[15:0] = 10
frequency = Y * sysclk / (10 * 2^32)
```

# Noisy or Unstable Signal

## Symptom

Input measurements fluctuate, outputs have jitter, counts are erratic.

## Likely Causes

1. **No input conditioning** - Raw input picking up noise
2. **Poor grounding** - Ground loops or high impedance
3. **Inadequate filtering** - High-frequency noise passing through
4. **Edge detection on slow signals** - Multiple triggers per transition

## Diagnostic Steps

1. Check input waveform on oscilloscope
2. Verify ground connections
3. TEST with Schmitt trigger enabled

## Solutions

### Add Schmitt trigger:

```
' WRONG - raw input
mode := P_COUNT_RISES

' CORRECT - Schmitt trigger for clean edges
mode := P_COUNT_RISES | P_SCHMITT_A
```

### Add input filtering:

```
' For noisy signals, add filter
mode := P_HIGH_TICKS | P_SCHMITT_A | P_FILT1_AB
```

### Use higher sample count for averaging:

```
' Measure over more periods to average noise
WXPIN(pin, 1000)           ' 1000 periods instead of 10
```



**For ADC, increase sample period:**

```
' More samples = more filtering
WXPIN(adc_pin, %00_1001)           ' 512 clocks instead of 128
```

## Serial Not Working

### Symptom

UART or SPI communication fails. No data received or garbled data.

### Likely Causes

1. **Baud rate mismatch** - TX and RX at different speeds
2. **Wrong polarity** - Signal inverted (RS-232 vs TTL)
3. **Bit count mismatch** - Wrong number of data bits
4. **Missing clock routing** - For sync modes

### Diagnostic Steps

1. Verify baud calculation:

```
bit_period := _clkfreq / BAUD
DEBUG("Bit period: ", UDEC_(bit_period))
```

2. Check with loopback:

```
' Connect TX to RX and verify echo
```

3. Use oscilloscope to measure actual baud rate

## Solutions

### Match TX and RX configuration:

```
' TRANSMIT
tx_x := (_clkfreq / BAUD) << 16 | 7      ' 8 data bits
WRPIN(TX_PIN, P_ASYNC_TX | P_OE)
WXPIN(TX_PIN, tx_x)

' RECEIVE - must match exactly
rx_x := (_clkfreq / BAUD) << 16 | 7      ' Same baud and bits
WRPIN(RX_PIN, P_ASYNC_RX)
WXPIN(RX_PIN, rx_x)
```

### For RS-232, add inversion:

```
' RS-232 uses inverted logic
WRPIN(TX_PIN, P_ASYNC_TX | P_OE | P_INVERT_OUTPUT)
WRPIN(RX_PIN, P_ASYNC_RX | P_INVERT_IN)
```

### For P\_SYNC\_TX/RX, add clock routing:

```
' WRONG - no clock source specified
mode := P_SYNC_TX | P_OE

' CORRECT - clock from adjacent pin
mode := P_SYNC_TX | P_OE | P_PLUS1_B      ' Clock from pin+1
```

## ADC Readings Wrong

### Symptom

ADC returns unexpected values, zero, or maximum.

### Likely Causes

1. **Wrong gain setting** - Signal outside input range
2. **Missing reference** - Floating input
3. **Wrong filter mode** - Post-processing not applied
4. **Sample period too short** - Not enough resolution

## Diagnostic Steps

1. Read raw ADC value:

```
raw := RDPIN(adc_pin)
DEBUG("Raw ADC: ", UHEX_(raw))
```

2. Verify input is within range
3. Check for saturation (stuck at 0 or max)

## Solutions

### Match gain to signal level:

```
' For 0-3.3V signal
mode := P_ADC_1X | P_ADC

' For 0-100mV signal
mode := P_ADC_30X | P_ADC
```

### Use ground reference for single-ended:

```
mode := P_ADC_GIO | P_ADC          ' Ground-referenced input
```

### For SINC2 filtering, compute difference:

```
' SINC2 filter mode requires difference
REPEAT UNTIL PINREAD(adc_pin)
acc := RDPIN(adc_pin)
sample := acc - last_acc          ' THIS IS REQUIRED
last_acc := acc
```

### Increase sample period for more bits:

```
' 8-bit resolution
WXPIN(adc_pin, %00_0111)          ' 128 clocks

' 12-bit resolution
WXPIN(adc_pin, %00_1011)          ' 2048 clocks
```

# Encoder Counts Incorrect

## Symptom

Quadrature encoder reports wrong position or skips counts.

## Likely Causes

1. **A/B wiring swapped** - Direction reversed
2. **Noise causing false edges** - Missing conditioning
3. **B-input not routed** - Using wrong pin
4. **Speed too fast** - Missing transitions

## Diagnostic Steps

1. Verify count direction:

```
' Rotate slowly, check count increases/decreases correctly
pos := RDPIN(enc_pin)
DEBUG("Position: ", SDEC_(pos))
```

2. Check for noise by holding still:

```
' Stationary encoder should give stable count
```

## Solutions

### Route B-input correctly:

```
' Encoder A on pin 20, B on pin 21
mode := P_QUADRATURE | P_PLUS1_B      ' B from pin+1
WRPIN(20, mode)
```

### Add Schmitt trigger for noisy signals:

```
mode := P_QUADRATURE | P_PLUS1_B | P_SCHMITT_A
```

### If direction reversed, swap wiring or invert:

```
mode := P_QUADRATURE | P_PLUS1_B | P_INVERT_A
```

**For high-speed encoders, verify no missed edges:**

```
' Maximum encoder speed depends on edge separation
' At 200 MHz, minimum detectable pulse is ~5ns
' For 1000 line encoder at 10,000 RPM:
' edges/sec = 1000 * 4 * 10000/60 = 666,667
' Period = 1.5 μs - well within capability
```

## Mode Stops Working

### Symptom

Smart pin works initially then stops. No more IN flags or output changes.

### Likely Causes

1. **IN flag not acknowledged** - Accumulator overflow
2. **Measurement complete, not restarted** - One-shot mode
3. **Counter overflow** - 32-bit limit reached

### Diagnostic Steps

1. Check if IN flag is being cleared:

```
' RDPIN clears IN, RQPIN does not
value := RDPIN(pin)           ' Clears IN
' vs
value := RQPIN(pin)           ' Does NOT clear IN
```

2. Verify mode auto-restarts

## Solutions

**Read with RDPIN to acknowledge:**

```
REPEAT
  REPEAT UNTIL PINREAD(pin)      ' Wait for IN
  value := RDPIN(pin)            ' READ AND CLEAR - restarts
```

**For continuous measurement, verify X value:**

```
' X=0 means continuous, no IN flag
WXPIN(pin, 0)                                ' Continuous - read anytime

' X>0 means periodic, IN raised each period
WXPIN(pin, period)                            ' Periodic - must read to restart
```

**Pulse DIR to reset if stuck:**

```
PINL(pin)                                     ' Disable
PINH(pin)                                     ' Re-enable from fresh state
```

## Interference Between Pins

### Symptom

Configuring one pin affects behavior of another.

### Likely Causes

1. **Input routing overlap** - Same signal feeding multiple pins
2. **Shared resources** - Adjacent pin interactions
3. **Ground bounce** - High-current switching affecting nearby signals

### Diagnostic Steps

1. TEST pins in isolation
2. Check for input routing to adjacent pins
3. Monitor with oscilloscope for crosstalk

### Solutions

**Verify no accidental routing:**

```
' Check A or B is not routed from affected pin
' P_PLUS1_A, P_MINUS1_A, etc. share inputs

' If pin 20 has issues when pin 21 is used:
' Check if pin 21 mode includes P_MINUS1_A
```

**Isolate pin configurations:**

```
' Configure pins one at a time, test each
WRPIN(pin1, mode1)
PINH(pin1)
' Test pin1 works

WRPIN(pin2, mode2)
PINH(pin2)
' Test pin2 works AND pin1 still works
```

**For high-current outputs, add slew limiting:**

```
' Reduce switching speed to minimize ground bounce
' Use slower drive if timing permits
```

## Debugging Techniques

### Using RDPIN to Inspect State

```
' Read Z register contents
z_value := RDPIN(pin)
DEBUG("Z: ", UHEX_(z_value))

' Read without clearing IN
z_value := RQPIN(pin)
DEBUG("Z (no clear): ", UHEX_(z_value))

' Check IN flag
in_flag := PINREAD(pin)
DEBUG("IN: ", UDEC_(in_flag))
```

### Incremental Configuration Testing

```
' Step 1: Verify pin can output
PINHIGH(pin)
WAITMS(100)
PINLOW(pin)
' Confirm on scope

' Step 2: Add smart pin mode
PINL(pin)
WRPIN(pin, P_NCO_FREQ | P_OE)
WXPIN(pin, 1)
WYPIN(pin, 21475)           ' 1 kHz
```

*continues on next page →*

```
PINH(pin)
' Check for 1 kHz

' Step 3: Add full configuration
' ...continue adding complexity
```

## Logic Analyzer Protocol Decoding

For serial protocols:

1. Capture raw waveform
2. Verify timing matches expected baud/clock
3. Decode data and compare to expected
4. Check for framing errors

## Oscilloscope Measurements

### For PWM:

- Measure frequency
- Measure duty cycle
- Check for glitches at transitions

### For ADC:

- Measure input voltage
- Verify within expected range
- Check for noise on input

### For Serial:

- Measure bit period
- Verify logic levels
- Check start/stop bits (async)

## Common Debug Patterns

### Blink test:

```
' Simplest test - does the pin toggle?
REPEAT
  PINTOGGLE(pin)
  WAITMS(500)
```



**Counter verification:**

```
' Verify counter is incrementing
REPEAT 10
  DEBUG("Count: ", UDEC_(RDPIN(pin)))
  WAITMS(100)
```

**Mode echo test:**

```
' Loopback test for serial
WRPIN(TX_PIN, P_ASYNC_TX | P_OE)
WRPIN(RX_PIN, P_ASYNC_RX)
' Wire TX_PIN to RX_PIN
WYPIN(TX_PIN, $55)
WAITMS(1)
received := RDPIN(RX_PIN)
DEBUG("Sent: $55, Received: ", UHEX_(received))
```

## Quick Diagnostic Checklist

### Output Not Working

- ☐ WRPIN executed with correct mode?
- ☐ Mode includes P\_OE?
- ☐ DIRH or PINLOW called?
- ☐ X and Y registers set correctly?
- ☐ Pin number correct?

### Input Not Working

- ☐ WRPIN executed?
- ☐ DIRH called?
- ☐ Waiting for IN flag when required?
- ☐ Using RDPIN (not RQPIN) to restart?
- ☐ Input conditioning appropriate?

### Serial Not Working

- ☐ Baud rate calculation correct?
- ☐ TX and RX configured identically?
- ☐ Polarity matches (P\_INVERT\_\*)?
- ☐ Bit count matches?
- ☐ For sync, clock routing added?

## ADC Not Working

- ☐ Input mode (P\_ADC\_GIO, etc.) appropriate?
- ☐ Gain matches input level?
- ☐ Filter mode understood?
- ☐ Sample period set?
- ☐ For SINC2/3, difference computed?

*For mode details, see relevant chapter. For P\_ constants, see Appendix B. For formulas, see Appendix C.*

# Appendix F: Complete Mode Reference

Quick reference for all 32 smart pin modes, organized by mode number.

## Mode Number Cross-Reference

| Mode   | Constant                   | Description                |
|--------|----------------------------|----------------------------|
| %00000 | P_NORMAL                   | Normal I/O (not smart pin) |
| %00001 | P_REPOSITORY / P_DAC_NOISE | Repository or DAC noise    |
| %00010 | P_DAC_DITHER_RND           | 16-bit PRNG dithered DAC   |
| %00011 | P_DAC_DITHER_PWM           | 16-bit PWM dithered DAC    |
| %00100 | P_PULSE                    | Pulse/cycle output         |
| %00101 | P_TRANSITION               | Transition output          |
| %00110 | P_NCO_FREQ                 | NCO frequency (50% duty)   |
| %00111 | P_NCO_DUTY                 | NCO with variable duty     |
| %01000 | P_PWM_TRIANGLE             | Triangle-wave PWM          |
| %01001 | P_PWM_SAWTOOTH             | Sawtooth-wave PWM          |
| %01010 | P_PWM_SMPS                 | SMPS PWM with feedback     |
| %01011 | P_QUADRATURE               | Quadrature encoder         |
| %01100 | P_REG_UP                   | Gated increment counter    |
| %01101 | P_REG_UP_DOWN              | Up/down gated counter      |
| %01110 | P_COUNT_RISES              | Count A-input rises        |
| %01111 | P_COUNT_HIGHS              | Count A-input high states  |
| %10000 | P_STATE_TICKS              | Time high and low states   |
| %10001 | P_HIGH_TICKS               | Time high states only      |
| %10010 | P_EVENTS_TICKS             | Time N events or timeout   |
| %10011 | P_PERIODS_TICKS            | Time X periods             |
| %10100 | P_PERIODS_HIGHS            | High time for X periods    |
| %10101 | P_COUNTER_TICKS            | Period time in X clocks    |
| %10110 | P_COUNTER_HIGHS            | High time in X clocks      |
| %10111 | P_COUNTER_PERIODS          | Count periods in X clocks  |
| %11000 | P_ADC                      | ADC internal clock         |
| %11001 | P_ADC_EXT                  | ADC external clock         |
| %11010 | P_ADC_SCOPE                | ADC triggered scope        |
| %11011 | P_USB_PAIR                 | USB differential pair      |
| %11100 | P_SYNC_TX                  | Synchronous serial TX      |
| %11101 | P_SYNC_RX                  | Synchronous serial RX      |
| %11110 | P_ASYNC_TX                 | Asynchronous serial TX     |
| %11111 | P_ASYNC_RX                 | Asynchronous serial RX     |

**Mode %00000: P\_NORMAL****Normal I/O (not a smart pin mode)**

Default mode. Pin operates as standard digital I/O without smart pin functionality.

**Register Usage**

| Register | Function      |
|----------|---------------|
| DIR      | Output enable |
| OUT      | Output value  |
| IN       | Input value   |

**Key Constants**

None required for normal I/O.

**Quick Example**

```
PINHIGH(pin)           ' Set output high
PINLOW(pin)             ' Set output low
state := PINREAD(pin)   ' Read input
```

**Reference**

Chapter 6: Digital Output, Chapter 12: Digital Input

### Mode %00001: P\_REPOSITORY / P\_DAC\_NOISE

#### Inter-cog data sharing or DAC noise generator

Dual-purpose mode. Without DAC enable: 32-bit repository for data sharing between cogs.  
With DAC enable: pseudo-random noise output.

### Register Usage

| Register    | Repository       | DAC Noise       |
|-------------|------------------|-----------------|
| X[15:0]     | Not used         | Sample period   |
| Y via WXPIN | Value to store   | Not used        |
| Z via RDPIN | Stored value     | Not used        |
| IN          | New data written | Period complete |

### Key Constants

|                      |                        |
|----------------------|------------------------|
| P_REPOSITORY         | ' Mode constant        |
| P_DAC_990R_3V   P_OE | ' For DAC noise output |

### Quick Example

```
' Repository mode
WRPIN(pin, P_REPOSITORY)
PINH(pin)
WXPIN(pin, value)           ' Write value
data := RQPIN(pin)          ' Read value

' DAC noise mode
WRPIN(pin, P_DAC_NOISE | P_DAC_124R_3V | P_OE)
PINH(pin)
```

### Reference

Chapter 18: Repository and Inter-Cog Data Sharing, Chapter 10: DAC Output

**Mode %00010: P\_DAC\_DITHER\_RND**

**16-bit PRNG dithered DAC**

Provides nominal 16-bit DAC resolution (averaged over time) using pseudo-random dithering between adjacent 8-bit levels. The hardware DAC is 8-bit; real precision depends on output filtering – see §18.4.

**Register Usage**

| Register | Function                      |
|----------|-------------------------------|
| X[15:0]  | Sample period (1 = immediate) |
| Y[15:0]  | 16-bit DAC value              |
| Z        | ADC accumulation (if OUT=1)   |
| IN       | Sample period complete        |

**Key Constants**

P\_DAC\_DITHER\_RND | P\_DAC\_124R\_3V | P\_OE

**Quick Example**

```
WRPIN(pin, P_DAC_DITHER_RND | P_DAC_124R_3V | P_OE)
WXPIN(pin, 1)                                     ' Immediate updates
WYPIN(pin, $8000)                                  ' Mid-scale output
PINH(pin)
```

**Reference**

Chapter 10: DAC Output

**Mode %00011: P\_DAC\_DITHER\_PWM**

**16-bit PWM dithered DAC**

Provides 16-bit DAC resolution using PWM dithering. Better dynamic range than PRNG dithering.

**Register Usage**

| Register | Function                                |
|----------|-----------------------------------------|
| X[15:0]  | Sample period (must be multiple of 256) |
| Y[15:0]  | 16-bit DAC value                        |
| Z        | ADC accumulation (if OUT=1)             |
| IN       | Sample period complete                  |

**Key Constants**

```
P_DAC_DITHER_PWM | P_DAC_600R_2V | P_OE
```

**Quick Example**

```
WRPIN(pin, P_DAC_DITHER_PWM | P_DAC_600R_2V | P_OE)
WXPIN(pin, 256)                                     ' Period must be 256×N
WYPIN(pin, $8000)
PINH(pin)
```

**Reference**

Chapter 10: DAC Output

**Mode %00100: P\_PULSE**

**Pulse/cycle output**

Generates precise timed pulses. Output a specified number of transitions with configurable timing.

**Register Usage**

| Register | Function                                                         |
|----------|------------------------------------------------------------------|
| X[15:0]  | Base period (clocks per pulse)                                   |
| X[31:16] | Compare value (output high while the base-period counter > this) |
| Y[31:0]  | Pulse/cycle count (decrements to 0)                              |
| IN       | Pulses complete                                                  |

**Key Constants**

P\_PULSE | P\_OE

**Quick Example**

```
WRPIN(pin, P_PULSE | P_OE)
WXPIN(pin, 16 | (8 << 16))    ' period 16, high above 8 (50%)
WYPIN(pin, 5)                 ' 5 pulses
PINH(pin)
```

**Reference**

Chapter 7: Transition and Pulse Output



**Mode %00101: P\_TRANSITION**

**Transition output**

Generates a specified number of output transitions with precise timing. Creates square waves or counted pulses.

**Register Usage**

| Register | Function                                                          |
|----------|-------------------------------------------------------------------|
| X[15:0]  | Base period (clocks per transition)                               |
| Y[31:0]  | Transition count (0 = idle; use NCO %00110/%00111 for continuous) |
| IN       | Transitions complete                                              |

**Key Constants**

P\_TRANSITION | P\_OE

**Quick Example**

```
WRPIN(pin, P_TRANSITION | P_OE)
WXPIN(pin, 100)                ' 100 clocks per transition
WYPIN(pin, 20)                  ' 20 transitions (10 cycles)
PINH(pin)
```

**Reference**

Chapter 7: Transition and Pulse Output

**Mode %00110: P\_NCO\_FREQ**

**NCO frequency generator (50% duty)**

Numerically Controlled Oscillator for precise frequency synthesis. Output is Z[31], creating 50% duty cycle square wave.

**Register Usage**

| Register | Function                               |
|----------|----------------------------------------|
| X[15:0]  | Base period (1 for maximum resolution) |
| X[31:16] | Initial phase                          |
| Y[31:0]  | Frequency control word                 |
| Z[31:0]  | Phase accumulator                      |
| IN       | Z overflow                             |

**Key Constants**

P\_NCO\_FREQ | P\_OE

**Quick Example**

```
' 1 kHz at 200 MHz sysclk
y_val := 1000 FRAC 200_000_000
WRPIN(pin, P_NCO_FREQ | P_OE)
WXPIN(pin, 1)
WYPIN(pin, y_val)
PINLOW(pin)
```

**Reference**

Chapter 8: Frequency Generation (NCO)

**Mode %00111: P\_NCO\_DUTY**

**NCO with variable duty cycle**

NCO frequency generator with duty cycle control. Output reflects Z overflow state.

**Register Usage**

| Register | Function                               |
|----------|----------------------------------------|
| X[15:0]  | Base period (1 for maximum resolution) |
| X[31:16] | Initial phase                          |
| Y[31:0]  | Frequency × duty control               |
| Z[31:0]  | Phase accumulator                      |
| IN       | Z overflow                             |

**Key Constants**

P\_NCO\_DUTY | P\_OE

**Quick Example**

```
WRPIN(pin, P_NCO_DUTY | P_OE)
WXPIN(pin, 1)
WYPIN(pin, $8000_0000)      ' 50% duty
PINLOW(pin)
```

**Reference**

Chapter 8: Frequency Generation (NCO)

**Mode %01000: P\_PWM\_TRIANGLE**

**Triangle-wave PWM**

PWM with up-down counter for symmetric output. Creates smooth PWM transitions.

**Register Usage**

| Register | Function                       |
|----------|--------------------------------|
| X[15:0]  | Base period (clocks per count) |
| X[31:16] | Frame period (counter range)   |
| Y[15:0]  | Duty value (0 to frame)        |
| IN       | Frame complete                 |

**Key Constants**

P\_PWM\_TRIANGLE | P\_OE

**Quick Example**

```
' 1 kHz PWM at 50% duty, 200 MHz sysclk
WRPIN(pin, P_PWM_TRIANGLE | P_OE)
WXPIN(pin, 1 | (100_000 << 16))      ' Frame=100000
WYPIN(pin, 50_000)                   ' 50% duty
PINLOW(pin)
```

**Reference**

Chapter 9: PWM Output

**Mode %01001: P\_PWM\_SAWTOOTH**

**Sawtooth-wave PWM**

PWM with up-only counter. Twice the frequency of triangle mode for same X values.

**Register Usage**

| Register | Function                       |
|----------|--------------------------------|
| X[15:0]  | Base period (clocks per count) |
| X[31:16] | Frame period (counter range)   |
| Y[15:0]  | Duty value (0 to frame)        |
| IN       | Frame complete                 |

**Key Constants**

P\_PWM\_SAWTOOTH | P\_OE

**Quick Example**

```
' 20 kHz motor PWM, 200 MHz sysclk
WRPIN(pin, P_PWM_SAWTOOTH | P_OE)
WXPIN(pin, 1 | (10_000 << 16))      ' Frame=10000
WYPIN(pin, 2500)                    ' 25% duty
PINLOW(pin)
```

**Reference**

Chapter 9: PWM Output

**Mode %01010: P\_PWM\_SMPS**

**SMPS PWM with feedback**

Switch-mode power supply controller with voltage and current feedback inputs.

**Register Usage**

| Register | Function                           |
|----------|------------------------------------|
| X[15:0]  | Base period                        |
| X[31:16] | Frame period (max pulse)           |
| Y[15:0]  | Duty value                         |
| A-input  | Voltage feedback (low = new cycle) |
| B-input  | Current limit (high = cut off)     |
| IN       | Cycle start                        |

**Key Constants**

P\_PWM\_SMPS | P\_OE | P\_PLUS1\_A | P\_MINUS1\_B

**Quick Example**

```
mode := P_PWM_SMPS | P_OE | P_PLUS1_A | P_MINUS1_B
WRPIN(pin, mode)
WXPIN(pin, 25 | (256 << 16))
WYPIN(pin, 128)           ' Set once, runs autonomous
PINLOW(pin)
```

**Reference**

Chapter 9: PWM Output

## Mode %01011: P\_QUADRATURE

### Quadrature encoder decoder

Decodes A/B quadrature signals for position tracking with 4× resolution.

## Register Usage

| Register | Function                           |
|----------|------------------------------------|
| X        | Period (0=continuous, >0=periodic) |
| Y        | Not used                           |
| Z        | Signed position/velocity count     |
| A-input  | Encoder phase A                    |
| B-input  | Encoder phase B                    |
| IN       | Period complete (if X>0)           |

## Key Constants

P\_QUADRATURE | P\_PLUS1\_B | P\_SCHMITT\_A

## Quick Example

```
' Encoder A on pin 20, B on pin 21
WRPIN(20, P_QUADRATURE | P_PLUS1_B | P_SCHMITT_A)
WXPIN(20, 0)                                ' Continuous mode
PINLOW(20)
position := RDPIN(20)                        ' Read position
```

## Reference

Chapter 14: Counting Modes

**Mode %01100: P\_REG\_UP**

**Gated increment counter**

Counts A-input rising edges, but only when B-input is high.

**Register Usage**

| Register | Function                           |
|----------|------------------------------------|
| X        | Period (0=continuous, >0=periodic) |
| Y        | Not used                           |
| Z        | Edge count                         |
| A-input  | Count signal                       |
| B-input  | Gate enable                        |
| IN       | Period complete                    |

**Key Constants**

P\_REG\_UP | P\_PLUS1\_B

**Quick Example**

```
WRPIN(pin, P_REG_UP | P_PLUS1_B)
WXPIN(pin, 0)                    ' Continuous
PINH(pin)
count := RDPIN(pin)
```

**Reference**

Chapter 14: Counting Modes



**Mode %01101: P\_REG\_UP\_DOWN****Up/down gated counter**

Counts A-input edges. B-input controls direction: high=increment, low=decrement.

**Register Usage**

| Register | Function                           |
|----------|------------------------------------|
| X        | Period (0=continuous, >0=periodic) |
| Y        | Not used                           |
| Z        | Signed count                       |
| A-input  | Count signal                       |
| B-input  | Direction (high=up)                |
| IN       | Period complete                    |

**Key Constants**

P\_REG\_UP\_DOWN | P\_PLUS1\_B

**Quick Example**

```
WRPIN(pin, P_REG_UP_DOWN | P_PLUS1_B)
WXPIN(pin, 0)
PINH(pin)
count := RDPIN(pin)           ' Signed result
```

**Reference**

Chapter 14: Counting Modes

**Mode %01110: P\_COUNT\_RISES**

**Count A-input rising edges**

Simple edge counter. Y[0] controls mode: 0=A edges only, 1=A up/B down.

**Register Usage**

| Register | Function                           |
|----------|------------------------------------|
| X        | Period (0=continuous, >0=periodic) |
| Y[0]     | Mode: 0=A only, 1=A up/B down      |
| Z        | Edge count                         |
| IN       | Period complete                    |

**Key Constants**

P\_COUNT\_RISES | P\_SCHMITT\_A

**Quick Example**

```
WRPIN(pin, P_COUNT_RISES | P_SCHMITT_A)
WXPIN(pin, 0)                               ' Continuous
WYPIN(pin, 0)                               ' A edges only
PINH(pin)
count := RDPIN(pin)
```

**Reference**

Chapter 14: Counting Modes

**Mode %01111: P\_COUNT\_HIGHS**

**Count A-input high clocks**

Counts system clocks while A-input is high. Y[0] controls mode.

**Register Usage**

| Register | Function                              |
|----------|---------------------------------------|
| X        | Period (0=continuous, >0=periodic)    |
| Y[0]     | Mode: 0=A high, 1=A high minus B high |
| Z        | Clock count                           |
| IN       | Period complete                       |

**Key Constants**

P\_COUNT\_HIGHS

**Quick Example**

```
WRPIN(pin, P_COUNT_HIGHS)
WXPIN(pin, _clkfreq)           ' 1 second period
WYPIN(pin, 0)
PINH(pin)
REPEAT UNTIL PINREAD(pin)
high_clocks := RDPIN(pin)
```

**Reference**

Chapter 14: Counting Modes

**Mode %10000: P\_STATE\_TICKS**

**Time high and low states**

Measures duration of each state. IN raised on every transition with previous state duration.

**Register Usage**

| Register | Function                            |
|----------|-------------------------------------|
| X        | Not used                            |
| Y        | Not used                            |
| Z        | Duration of previous state (clocks) |
| C flag   | Previous state (1=was high)         |
| IN       | Every transition                    |

On reset (DIR=0), Z starts at **\$0000\_0001** (not 0), and Z saturates at **\$8000\_0000**.

**Key Constants**

P\_STATE\_TICKS | P\_SCHMITT\_A

**Quick Example**

```
WRPIN(pin, P_STATE_TICKS | P_SCHMITT_A)
PINH(pin)
REPEAT UNTIL PINREAD(pin)
duration := RDPIN(pin) wc           ' C=1 if was high
```

**Reference**

Chapter 13: Timing Measurement

**Mode %10001: P\_HIGH\_TICKS**

**Time high states only**

Measures duration of high pulses. IN raised on high-to-low transition.

**Register Usage**

| Register | Function                           |
|----------|------------------------------------|
| X        | Not used                           |
| Y        | Not used                           |
| Z        | Duration of previous high (clocks) |
| IN       | High-to-low transition             |

On reset (DIR=0), Z starts at **\$0000\_0001** (not 0). Z saturates at \$8000\_0000, and bit 31 doubles as the overflow flag — which is why the example masks the result with \$7FFF\_FFFF.

**Key Constants**

```
P_HIGH_TICKS | P_SCHMITT_A
P_HIGH_TICKS | P_INVERT_A      ' To measure low pulses
```

**Quick Example**

```
WRPIN(pin, P_HIGH_TICKS | P_SCHMITT_A)
PINH(pin)
REPEAT UNTIL PINREAD(pin)
pulse_width := RDPIN(pin) & $7FFF_FFFF
```

**Reference**

Chapter 13: Timing Measurement

**Mode %10010: P\_EVENTS\_TICKS**

**Time N events or detect timeout**

Two modes: measure time for X events (Y[2]=0), or detect timeout without events (Y[2]=1).

**Register Usage**

| Register | Function                                        |
|----------|-------------------------------------------------|
| X        | Event count (Y[2]=0) or timeout clocks (Y[2]=1) |
| Y[1:0]   | Event type: %00=high, %01=rise, %1x=edge        |
| Y[2]     | Mode: 0=events, 1=timeout                       |
| Z        | Elapsed clocks                                  |
| IN       | Events complete or timeout                      |

**Key Constants**

P\_EVENTS\_TICKS

**Quick Example**

```
' Measure time for 100 rising edges
WRPIN(pin, P_EVENTS_TICKS)
WXPIN(pin, 100)
WYPIN(pin, %01)           ' Rising edges, event mode
PINH(pin)
REPEAT UNTIL PINREAD(pin)
clocks := RDPIN(pin)
```

**Reference**

Chapter 13: Timing Measurement

## Mode %10011: P\_PERIODS\_TICKS

### Time X complete periods

Measures total clock cycles for X signal periods.

## Register Usage

| Register | Function                                                                                           |
|----------|----------------------------------------------------------------------------------------------------|
| X        | Number of periods to measure                                                                       |
| Y[1:0]   | A→B event pair: %00 = A-rise→B-rise, %01 = A-rise→B-edge, %10 = A-edge→B-rise, %11 = A-edge→B-edge |
| Z        | Total clocks for all periods                                                                       |
| IN       | Measurement complete                                                                               |

This is a **two-input** mode — each period is measured from an A-input event to a B-input event, so B-input routing is required (set B to the same pin as A for single-pin cycle measurement). On reset (DIR=0), Z starts at **\$0000\_0000** — note that the period-counting modes reset Z to 0, unlike the state/timing modes (%10000–%10010), which reset to \$0000\_0001. Z saturates at \$8000\_0000.

## Key Constants

P\_PERIODS\_TICKS

## Quick Example

```
WRPIN(pin, P_PERIODS_TICKS)
WXPIN(pin, 100)           ' Measure 100 periods
WYPIN(pin, %00)          ' Rise to rise
PINH(pin)
REPEAT UNTIL PINREAD(pin)
total_clocks := RDPIN(pin)
freq := (100 * _clkfreq) / total_clocks
```

## Reference

Chapter 15: Period and Frequency Measurement

**Mode %10100: P\_PERIODS\_HIGHS**

**High time for X periods**

Accumulates high-state time across X periods for duty cycle measurement.

**Register Usage**

| Register | Function                         |
|----------|----------------------------------|
| X        | Number of periods                |
| Y[1:0]   | Trigger type                     |
| Z        | Total high clocks across periods |
| IN       | Measurement complete             |

**Key Constants**

P\_PERIODS\_HIGHS

**Quick Example**

```
WRPIN(pin, P_PERIODS_HIGHS)
WXPIN(pin, 100)
WYPIN(pin, %00)
PINH(pin)
REPEAT UNTIL PINREAD(pin)
high_clocks := RDPIN(pin)
```

**Reference**

Chapter 15: Period and Frequency Measurement



**Mode %10101: P\_COUNTER\_TICKS**

**Period time in X clock window**

Measures total period time within a minimum X-clock window.

**Register Usage**

| Register | Function                |
|----------|-------------------------|
| X        | Minimum window (clocks) |
| Y[1:0]   | Trigger type            |
| Z        | Actual elapsed clocks   |
| IN       | Window complete         |

**Key Constants**

P\_COUNTER\_TICKS

**Quick Example**

```
WRPIN(pin, P_COUNTER_TICKS)
WXPIN(pin, _clkfreq)           ' 1 second window
WYPIN(pin, %00)
PINH(pin)
REPEAT UNTIL PINREAD(pin)
actual_time := RDPIN(pin)
```

**Reference**

Chapter 15: Period and Frequency Measurement

**Mode %10110: P\_COUNTER\_HIGHS**

**High time in X clock window**

Accumulates high-state time within a minimum X-clock window.

**Register Usage**

| Register | Function                    |
|----------|-----------------------------|
| X        | Minimum window (clocks)     |
| Y[1:0]   | Trigger type                |
| Z        | Total high clocks in window |
| IN       | Window complete             |

**Key Constants**

P\_COUNTER\_HIGHS

**Quick Example**

```
WRPIN(pin, P_COUNTER_HIGHS)
WXPIN(pin, _clkfreq / 10)           ' 100ms window
WYPIN(pin, %00)
PINH(pin)
REPEAT UNTIL PINREAD(pin)
high_time := RDPIN(pin)
```

**Reference**

Chapter 15: Period and Frequency Measurement

**Mode %10111: P\_COUNTER\_PERIODS**

**Count periods in X clock window**

Counts complete periods within a minimum X-clock window. Simple frequency counter.

**Register Usage**

| Register | Function                |
|----------|-------------------------|
| X        | Minimum window (clocks) |
| Y[1:0]   | Trigger type            |
| Z        | Period count            |
| IN       | Window complete         |

**Key Constants**

|                   |
|-------------------|
| P_COUNTER_PERIODS |
|-------------------|

**Quick Example**

```
' Direct Hz reading with 1-second gate
WRPIN(pin, P_COUNTER_PERIODS)
WXPIN(pin, _clkfreq)           ' 1 second
WYPIN(pin, %00)
PINH(pin)
REPEAT UNTIL PINREAD(pin)
frequency_hz := RDPIN(pin)
```

**Reference**

Chapter 15: Period and Frequency Measurement

**Mode %11000: P\_ADC**

**ADC with internal clock**

Sigma-delta ADC with SINC filtering. 8-14 bit resolution depending on sample period.

**Register Usage**

| Register | Function                              |
|----------|---------------------------------------|
| X[5:4]   | Filter mode                           |
| X[3:0]   | Sample period = 2 <sup>X</sup> clocks |
| Y        | Period override (optional)            |
| Z        | ADC sample/accumulator                |
| IN       | Sample complete                       |

**Key Constants**

|                   |                     |
|-------------------|---------------------|
| P_ADC_GIO   P_ADC | ' Ground-referenced |
| P_ADC_10X   P_ADC | ' 10x gain          |

**Quick Example**

|                               |                              |
|-------------------------------|------------------------------|
| WRPIN(pin, P_ADC_GIO   P_ADC) |                              |
| WXPIN(pin, %00_0111)          | ' SINC2 sampling, 128 clocks |
| PINH(pin)                     |                              |
| sample := RDPIN(pin)          |                              |

**Reference**

Chapter 16: ADC (Analog Input)

**Mode %11001: P\_ADC\_EXT**

**ADC with external clock**

Samples A-input data on B-input clock edges. For external delta-sigma ADCs.

**Register Usage**

| Register | Function               |
|----------|------------------------|
| X[5:4]   | Filter mode            |
| X[3:0]   | Base sample period     |
| Y        | Period override        |
| Z        | ADC sample/accumulator |
| A-input  | External ADC data      |
| B-input  | External clock         |
| IN       | Sample complete        |

**Key Constants**

```
P_ADC_EXT | P_PLUS1_B
```

**Quick Example**

```
WRPIN(pin, P_ADC_EXT | P_PLUS1_B)
WXPIN(pin, %00_0111)
PINH(pin)
```

**Reference**

Chapter 16: ADC (Analog Input)

**Mode %11010: P\_ADC\_SCOPE**

**ADC triggered scope capture**

Four-channel oscilloscope-style ADC with hysteretic triggering.

**Register Usage**

| Register | Function                                |
|----------|-----------------------------------------|
| X[15:10] | B trigger value (6-bit)                 |
| X[7:2]   | A trigger value (6-bit)                 |
| X[1:0]   | Filter select (%00/%01 Tukey, %1x Hann) |
| Z        | 8-bit sample (RDPIN; C = armed)         |
| IN       | Trigger fired                           |

**Key Constants**

P\_ADC\_GIO | P\_ADC\_SCOPE

**Quick Example**

```
WRPIN(52, P_ADC_GIO | P_ADC_SCOPE)      ' pin must be a multiple of 4
WXPIN(52, (48 << 10) | (16 << 2))      ' B=48, A=16, 68-tap filter
PINH(52)
```

**Reference**

Chapter 16: ADC (Analog Input)

**Mode %11011: P\_USB\_PAIR**

**USB differential pair**

USB 1.1 physical layer for even/odd pin pair. Handles differential signaling.

**Register Usage**

| Register | Function         |
|----------|------------------|
| X        | Configuration    |
| Y        | Protocol control |
| Z        | Data/status      |
| Even pin | D- (DM)          |
| Odd pin  | D+ (DP)          |
| IN       | USB event        |

**Key Constants**

```
P_USB_PAIR | P_OE
```

**Quick Example**

```
' Pins must be consecutive even/odd pair
WRPIN(56, P_USB_PAIR | P_OE)      ' 56=D-, 57=D+
PINH(56)
PINH(57)
```

**Reference**

Chapter 19: USB Host/Device

**Mode %11100: P\_SYNC\_TX**

**Synchronous serial transmit**

Clocked serial transmission for SPI master and similar protocols.

**Register Usage**

| Register | Function                         |
|----------|----------------------------------|
| X[5]     | Mode: 0=continuous, 1=start-stop |
| X[4:0]   | Bits minus 1                     |
| Y        | Transmit data (LSB first)        |
| B-input  | Clock source                     |
| IN       | Buffer empty                     |

**Key Constants**

|                               |                       |
|-------------------------------|-----------------------|
| P_SYNC_TX   P_OE   P_PLUS1_B  | ' Clock from next pin |
| P_SYNC_TX   P_OE   P_MINUS1_B | ' Clock from prev pin |

**Quick Example**

```
' Data on pin 41, clock on pin 40
WRPIN(41, P_SYNC_TX | P_OE | P_MINUS1_B)
WXPIN(41, %1_00111)           ' Start-stop, 8 bits
PINH(41)
WYPIN(41, data)
```

**Reference**

Chapter 11: Serial Transmission



**Mode %11101: P\_SYNC\_RX**

**Synchronous serial receive**

Clocked serial reception for SPI slave and similar protocols.

**Register Usage**

| Register | Function                         |
|----------|----------------------------------|
| X[5]     | Mode: 0=continuous, 1=start-stop |
| X[4:0]   | Bits minus 1                     |
| Y        | Not used                         |
| Z        | Received data (left-justified)   |
| B-input  | Clock source                     |
| IN       | Data ready                       |

**Key Constants**

|                       |                       |
|-----------------------|-----------------------|
| P_SYNC_RX   P_PLUS1_B | ' Clock from next pin |
|-----------------------|-----------------------|

**Quick Example**

|                                   |                              |
|-----------------------------------|------------------------------|
| WRPIN(pin, P_SYNC_RX   P_PLUS1_B) |                              |
| WXPIN(pin, %1_00111)              | ' Start-stop, 8 bits         |
| PINH(pin)                         |                              |
| REPEAT UNTIL PINREAD(pin)         |                              |
| data := RDPIN(pin) >> 24          | ' Left-justified, shift down |

**Reference**

Chapter 17: Serial Receive

**Mode %11110: P\_ASYNC\_TX**

**Asynchronous serial transmit**

UART-style transmission with automatic start/stop bit generation.

**Register Usage**

| Register | Function                  |
|----------|---------------------------|
| X[31:16] | Bit period (clocks)       |
| X[15:10] | Fractional (1/64 clock)   |
| X[4:0]   | Data bits minus 1         |
| Y        | Transmit data (LSB first) |
| IN       | Ready for next byte       |

**Key Constants**

|            |  |      |                 |
|------------|--|------|-----------------|
| P_ASYNC_TX |  | P_OE |                 |
| P_ASYNC_TX |  | P_OE | P_INVERT_OUTPUT |
|            |  |      | ' RS-232        |

**Quick Example**

```
bit_period := (_clkfreq / 115200) << 16
WRPIN(pin, P_ASYNC_TX | P_OE)
WXPIN(pin, bit_period | 7)           ' 8 data bits
PINLOW(pin)
REPEAT UNTIL PINREAD(pin)
WYPIN(pin, byte_value)
```

**Reference**

Chapter 11: Serial Transmission

**Mode %11111: P\_ASYNC\_RX**

**Asynchronous serial receive**

UART-style reception with automatic start bit detection and framing.

**Register Usage**

| Register | Function                                    |
|----------|---------------------------------------------|
| X[31:16] | Bit period (clocks)                         |
| X[15:10] | Fractional (1/64 clock)                     |
| X[4:0]   | Data bits minus 1                           |
| Z        | Received data (MSB-justified, MSB at Z[31]) |
| IN       | Byte received                               |

**Key Constants**

|                          |          |
|--------------------------|----------|
| P_ASYNC_RX               |          |
| P_ASYNC_RX   P_INVERT_IN | ' RS-232 |

**Quick Example**

```
bit_period := (_clkfreq / 115200) << 16
WRPIN(pin, P_ASYNC_RX)
WXPIN(pin, bit_period | 7)
PINH(pin)
REPEAT UNTIL PINREAD(pin)
data := RDPIN(pin) >> 24
```

' MSB-justified: shift right 32-8

**Reference**

Chapter 17: Serial Receive

*For full mode details, see the referenced chapters. For P\_ constant values, see Appendix B.*

# Appendix G: FPGA Board Differences

The target for this guide is the **Propeller 2 ASIC** — the production silicon. Before the ASIC existed, the P2 design was emulated on FPGA development boards, and those boards remain in limited use. An FPGA emulation reproduces the P2's digital logic faithfully, but it cannot reproduce the analog and mixed-signal hardware built into the ASIC's pins, and it runs from a fixed development clock rather than the ASIC's configurable clock generator.

Everywhere else in this guide, behavior is described for the ASIC. Where a smart pin or I/O behavior depends on hardware the FPGA does not have, this appendix records the difference. When running on an FPGA board, read this appendix first.

## USB — No Built-In Resistors

The ASIC's USB smart pins (mode %11011, Chapter 19) include the 1.5 k $\Omega$  and 15 k $\Omega$  resistors that USB signaling requires, built directly into the pin hardware. An FPGA emulation has no such resistors — they must be fitted externally on the DP and DM lines.

## Clock — Fixed 20 MHz or 80 MHz

The ASIC derives its system clock from an on-chip oscillator and PLL, configurable across a wide frequency range. On an FPGA, the clock generator is not emulated: the only supported system-clock settings are **20 MHz** and **80 MHz**.

The timing relationships throughout this guide still hold — baud rates, NCO frequencies, and measurement windows are all expressed relative to the system clock. Substitute the FPGA's actual clock frequency (20 MHz or 80 MHz) wherever a calculation uses the system clock.

## Other Board-Level Differences

FPGA boards also differ from the ASIC in ways that are not specific to the pins — most notably the amount of hub RAM (an FPGA image provides only a portion of the ASIC's 512 KB, as little as 32 KB on small boards) and, on the smallest boards, the number of emulated cogs. These vary from board to board and from image to image; consult the board's documentation for its exact configuration before sizing buffers or assuming eight cogs are available.

# Index

Alphabetical index of terms, constants, and concepts in this guide.

## A

- **A-input** - Smart pin primary input, Ch. 3, 5
- **A-input routing** - P\_PLUS1\_A, P\_MINUS1\_A, etc., Ch. 3, App. B
- **Accumulator, phase** - NCO Z register, Ch. 8
- **ADC** - see Analog-to-Digital Conversion
- **ADC external clock** - P\_ADC\_EXT (%11001), Ch. 16
- **ADC gain** - P\_ADC\_1X through P\_ADC\_100X, Ch. 16
- **ADC internal clock** - P\_ADC (%11000), Ch. 16
- **ADC scope** - P\_ADC\_SCOPE (%11010), Ch. 16
- **AKPIN** - Acknowledge pin (PASM2), Ch. 4
- **Analog input** - see ADC, Ch. 16
- **Analog output** - see DAC, Ch. 10
- **Asynchronous serial** - UART modes, Ch. 11, 17
- **Audio** - NCO tones Ch. 8, DAC waveforms Ch. 10, 18

## B

- **B-input** - Smart pin secondary input, Ch. 3, 5
- **B-input routing** - P\_PLUS1\_B, P\_MINUS1\_B, etc., Ch. 3, App. B
- **Base period** - X[15:0] in PWM and other modes, Ch. 9
- **Baud rate** - Serial timing calculation, Ch. 11, App. C
- **Baud rate error** - Calculation and tolerance, Ch. 11, App. E
- **Bit period** - Serial bit timing, Ch. 11, 17
- **BITDAC** - Single-bit DAC mode, Ch. 10
- **Bitstream** - Raw ADC output (X[5:4]=%11), Ch. 16
- **Buffer, double** - Serial TX modes, Ch. 11

## C

- **C flag** - State indicator in timing modes, Ch. 13
- **Clock generation** - P\_TRANSITION, P\_NCO\_FREQ, Ch. 7, 8
- **Clock routing** - B-input for sync serial, Ch. 11, 17
- **Cog** - Processor core, inter-cog sharing Ch. 18
- **Comparator input** - P\_COMPARE\_AB, Ch. 12
- **Continuous mode** - X=0 for counting, Ch. 14
- **Counter modes** - %01100-%01111, Ch. 14

- **Counting** - Event and period counting, Ch. 14
- **CPOL/CPHA** - SPI clock polarity/phase, Ch. 11

## D

- **DAC** - see Digital-to-Analog Conversion
- **DAC dithering** - 16-bit resolution modes, Ch. 10, 18
- **DAC noise** - P\_DAC\_NOISE (%00001), Ch. 18
- **DAC resistor modes** - P\_DAC\_990R\_3V, etc., Ch. 10
- **Data bits** - Serial frame size, Ch. 11, 17
- **DIR** - Pin direction control, Ch. 3, 6
- **DIRH** - Set pin as output (PASM2), Ch. 4, 6
- **DIRL** - Set pin as input (PASM2), Ch. 4, 6
- **Dithering** - DAC resolution enhancement, Ch. 18
- **Drive strength** - P\_HIGH\_FAST, P\_LOW\_FAST, etc., Ch. 6, App. B
- **DRVH** - Drive pin high (PASM2), Ch. 4, 6
- **DRVL** - Drive pin low (PASM2), Ch. 4, 6
- **Duty cycle** - PWM ratio, Ch. 9; measurement Ch. 13, 15

## E

- **Edge counting** - P\_COUNT\_RISES, P\_COUNT\_HIGHS, Ch. 14
- **Encoder** - see Quadrature encoder
- **ENOB** - Effective number of bits (ADC), Ch. 16
- **Event timing** - P\_EVENTS\_TICKS, Ch. 13

## F

- **Filtering, input** - P\_FILT0\_AB through P\_FILT3\_AB, Ch. 12
- **Float** - see PINFLOAT
- **FPGA board differences** - USB resistors, clock, hub RAM, App. G
- **Fractional baud** - X[15:10] precision, Ch. 11
- **Frame period** - X[31:16] in PWM, Ch. 9
- **Frequency counter** - P\_COUNTER\_PERIODS, Ch. 15
- **Frequency generation** - NCO modes, Ch. 8
- **Frequency measurement** - Period modes, Ch. 15
- **Full Speed USB** - 12 Mbps, Ch. 19

## G

- **Gain, ADC** - P\_ADC\_1X through P\_ADC\_100X, Ch. 16
- **Gate time** - Frequency counter window, Ch. 15

- **Gated counting** - P\_REG\_UP, Ch. 14

## H

- **High-state counting** - P\_COUNT\_HIGHS, Ch. 14
- **High-state timing** - P\_HIGH\_TICKS, Ch. 13
- **Hub RAM** - vs. Repository, Ch. 18

## I

- **I2C** - Implementation notes, App. A
- **IN flag** - Smart pin status, Ch. 3, 4
- **INA/INB** - Input registers, Ch. 12
- **Input conditioning** - Schmitt, filter, compare, Ch. 12
- **Input routing** - A/B input selection, Ch. 3
- **Inter-cog** - Data sharing via Repository, Ch. 18
- **Inversion** - P\_INVERT\_A, P\_INVERT\_B, P\_INVERT\_OUTPUT, App. B

## L

- **Latency** - Pin I/O timing, Ch. 5
- **LED dimming** - PWM application, Ch. 9
- **Level comparison** - P\_LEVEL\_A modes, Ch. 12
- **LOCK bits** - vs. Repository, Ch. 18
- **Logic input** - P\_LOGIC\_A, Ch. 12
- **Low Speed USB** - 1.5 Mbps, Ch. 19
- **LSB first** - Serial bit order, Ch. 11, 17

## M

- **Mode number** - Smart pin mode (%XXXXX), Ch. 3, App. F
- **Motor control** - PWM application, Ch. 9
- **MSB first** - Bit reversal for SPI, Ch. 11

## N

- **NCO** - Numerically Controlled Oscillator, Ch. 8
- **NCO duty** - P\_NCO\_DUTY (%00111), Ch. 8
- **NCO frequency** - P\_NCO\_FREQ (%00110), Ch. 8
- **Noise, DAC** - P\_DAC\_NOISE, Ch. 18

## O

- **Open-drain** - P\_HIGH\_FLOAT configuration, Ch. 6
- **OUT** - Pin output state, Ch. 3, 6
- **OUTA/OUTB** - Output registers, Ch. 6
- **Output enable** - P\_OE (TT bits), Ch. 3, App. B

## P

- **P\_ADC** - ADC internal clock (%11000), Ch. 16, App. F
- **P\_ADC\_100X** - 100x gain ADC, Ch. 16
- **P\_ADC\_10X** - 10x gain ADC, Ch. 16
- **P\_ADC\_1X** - Unity gain ADC, Ch. 16
- **P\_ADC\_30X** - 31.6x gain ADC, Ch. 16
- **P\_ADC\_3X** - 3.16x gain ADC, Ch. 16
- **P\_ADC\_EXT** - ADC external clock (%11001), Ch. 16, App. F
- **P\_ADC\_FLOAT** - Floating ADC input, Ch. 16
- **P\_ADC\_GIO** - Ground-referenced ADC, Ch. 16
- **P\_ADC\_SCOPE** - Triggered scope (%11010), Ch. 16, App. F
- **P\_ADC\_VIO** - VIO-referenced ADC, Ch. 16
- **P\_ASYNC\_RX** - Async serial receive (%11111), Ch. 17, App. F
- **P\_ASYNC\_TX** - Async serial transmit (%11110), Ch. 11, App. F
- **P\_BITDAC** - Bit DAC enable, Ch. 10
- **P\_CHANNEL** - DAC channel enable, Ch. 10
- **P\_COMPARE\_AB** - A>B comparator, Ch. 12
- **P\_COUNT\_HIGHS** - Count high states (%01111), Ch. 14, App. F
- **P\_COUNT\_RISES** - Count rising edges (%01110), Ch. 14, App. F
- **P\_COUNTER\_HIGHS** - High time in window (%10110), Ch. 15, App. F
- **P\_COUNTER\_PERIODS** - Period count in window (%10111), Ch. 15, App. F
- **P\_COUNTER\_TICKS** - Period time in window (%10101), Ch. 15, App. F
- **P\_DAC\_124R\_3V** - 124  $\Omega$ , 3.3V DAC, Ch. 10
- **P\_DAC\_600R\_2V** - 600  $\Omega$ , 2.0V DAC, Ch. 10
- **P\_DAC\_75R\_2V** - 75  $\Omega$ , 2.0V DAC, Ch. 10
- **P\_DAC\_990R\_3V** - 990  $\Omega$ , 3.3V DAC, Ch. 10
- **P\_DAC\_DITHER\_PWM** - PWM dithered DAC (%00011), Ch. 18, App. F
- **P\_DAC\_DITHER\_RND** - PRNG dithered DAC (%00010), Ch. 18, App. F
- **P\_DAC\_NOISE** - DAC noise output (%00001), Ch. 18, App. F
- **P\_EVENTS\_TICKS** - Event timing (%10010), Ch. 13, App. F
- **P\_FILT0\_AB** through **P\_FILT3\_AB** - Input filtering, Ch. 12
- **P\_HIGH\_FAST** - Fast high drive, Ch. 6
- **P\_HIGH\_FLOAT** - Float high (open-drain), Ch. 6
- **P\_HIGH\_TICKS** - Measure high time (%10001), Ch. 13, App. F



- **P\_INVERT\_A** - Invert A-input, App. B
- **P\_INVERT\_B** - Invert B-input, App. B
- **P\_INVERT\_IN** - Invert IN bit, App. B
- **P\_INVERT\_OUTPUT** - Invert output, App. B
- **P\_LEVEL\_A** - Level comparison modes, Ch. 12
- **P\_LOCAL\_A** - Select local pin for A, App. B
- **P\_LOCAL\_B** - Select local pin for B, App. B
- **P\_LOGIC\_A** - Logic level input, Ch. 12
- **P\_LOW\_FAST** - Fast low drive, Ch. 6
- **P\_LOW\_FLOAT** - Float low (high-Z), Ch. 6
- **P\_MINUS1\_A** through **P\_MINUS3\_A** - A-input from pin-N, App. B
- **P\_MINUS1\_B** through **P\_MINUS3\_B** - B-input from pin-N, App. B
- **P\_NCO\_DUTY** - NCO variable duty (%00111), Ch. 8, App. F
- **P\_NCO\_FREQ** - NCO 50% duty (%00110), Ch. 8, App. F
- **P\_NORMAL** - Normal I/O mode (%00000), Ch. 6, App. F
- **P\_OE** - Output enable (TT=%01), Ch. 3, App. B
- **P\_OUTBIT\_A** - OUT bit to A-input, App. B
- **P\_OUTBIT\_B** - OUT bit to B-input, App. B
- **P\_PERIODS\_HIGHS** - High time for periods (%10100), Ch. 15, App. F
- **P\_PERIODS\_TICKS** - Time for periods (%10011), Ch. 15, App. F
- **P\_PLUS1\_A** through **P\_PLUS3\_A** - A-input from pin+N, App. B
- **P\_PLUS1\_B** through **P\_PLUS3\_B** - B-input from pin+N, App. B
- **P\_PULSE** - Pulse output (%00100), Ch. 7, App. F
- **P\_PWM\_SAWTOOTH** - Sawtooth PWM (%01001), Ch. 9, App. F
- **P\_PWM\_SMPS** - SMPS PWM (%01010), Ch. 9, App. F
- **P\_PWM\_TRIANGLE** - Triangle PWM (%01000), Ch. 9, App. F
- **P\_QUADRATURE** - Quadrature encoder (%01011), Ch. 14, App. F
- **P\_REG\_UP** - Gated increment (%01100), Ch. 14, App. F
- **P\_REG\_UP\_DOWN** - Up/down counter (%01101), Ch. 14, App. F
- **P\_REPOSITORY** - Inter-cog data (%00001), Ch. 18, App. F
- **P\_SCHMITT\_A** - Schmitt trigger A, Ch. 12
- **P\_STATE\_TICKS** - Time both states (%10000), Ch. 13, App. F
- **P\_SYNC\_IO** - Synchronous I/O, App. B
- **P\_SYNC\_RX** - Sync serial receive (%11101), Ch. 17, App. F
- **P\_SYNC\_TX** - Sync serial transmit (%11100), Ch. 11, App. F
- **P\_TRANSITION** - Transition output (%00101), Ch. 7, App. F
- **P\_TRUE\_A** - Non-inverted A, App. B
- **P\_TRUE\_B** - Non-inverted B, App. B
- **P\_TT\_00** through **P\_TT\_11** - TT bit values, App. B
- **P\_USB\_PAIR** - USB differential (%11011), Ch. 19, App. F
- **Parity** - Software implementation, Ch. 11
- **Period measurement** - Modes %10011-%10111, Ch. 15

- **Periodic mode** -  $X > 0$  for counting, Ch. 14
- **Phase accumulator** - NCO Z register, Ch. 8
- **Phase synchronization** - NCO X[31:16], Ch. 8
- **PINFLOAT** - Float pin (Spin2), Ch. 4, 6
- **PINHIGH** - Drive pin high (Spin2), Ch. 4, 6
- **PINLOW** - Drive pin low (Spin2), Ch. 4, 6
- **PINTOGGLE** - Toggle pin (Spin2), Ch. 1
- **PINREAD** - Read IN flag (Spin2), Ch. 4
- **PINSTART** - Configure and start (Spin2), Ch. 4
- **PINWRITE** - Write pin value (Spin2), Ch. 4, 6
- **PRNG dithering** - Random DAC dither, Ch. 18
- **Pull-up/pull-down** - P\_HIGH\_15K, P\_LOW\_15K, etc., Ch. 6
- **Pulse measurement** - P\_HIGH\_TICKS, Ch. 13
- **Pulse output** - P\_PULSE mode, Ch. 7
- **PWM** - Pulse Width Modulation, Ch. 9
- **PWM dithering** - Deterministic DAC dither, Ch. 18

## Q

- **Quadrature encoder** - P\_QUADRATURE (%01011), Ch. 14
- **Quantization error** - Measurement accuracy, Ch. 13

## R

- **RDPIN** - Read Z, clear IN (PASM2), Ch. 4
- **Repository** - P\_REPOSITORY (%00001), Ch. 18
- **Resolution** - ADC bits Ch. 16, PWM bits Ch. 9
- **RPM measurement** - Period counting, Ch. 15
- **RQPIN** - Read Z, keep IN (PASM2), Ch. 4
- **RS-232** - Inverted serial, Ch. 11, 17

## S

- **Sample period** - ADC X register, Ch. 16
- **Sample rate** - ADC calculation, Ch. 16, App. C
- **Sawtooth PWM** - P\_PWM\_SAWTOOTH, Ch. 9
- **Schmitt trigger** - P\_SCHMITT\_A, Ch. 12
- **Servo control** - Pulse output, Ch. 7, 9
- **SINC2 filter** - ADC filter mode, Ch. 16
- **SINC3 filter** - ADC filter mode, Ch. 16
- **Smart pin** - Hardware-autonomous I/O, Ch. 1, 3
- **SMPS** - Switch-mode power supply, Ch. 9

- **SPI** - Synchronous serial, Ch. 11, 17
- **Square wave** - NCO 50% duty, Ch. 8
- **Start bit** - Async serial framing, Ch. 11, 17
- **State timing** - P\_STATE\_TICKS, Ch. 13
- **Stop bit** - Async serial framing, Ch. 11, 17
- **Synchronous serial** - SPI modes, Ch. 11, 17
- **sysclk** - System clock frequency, Ch. 2

## T

- **TESTP** - TEST IN flag (PASM2), Ch. 4
- **Three-phase** - NCO phase synchronization, Ch. 8
- **Timeout detection** - P\_EVENTS\_TICKS, Ch. 13
- **Timing measurement** - Modes %10000-%10010, Ch. 13
- **Transition output** - P\_TRANSITION, Ch. 7
- **Triangle PWM** - P\_PWM\_TRIANGLE, Ch. 9
- **Trigger, hysteretic** - ADC scope, Ch. 16
- **TT bits** - DIR/OUT control, Ch. 3, App. B

## U

- **UART** - Async serial, Ch. 11, 17
- **Up/down counter** - P\_REG\_UP\_DOWN, Ch. 14
- **USB** - P\_USB\_PAIR (%11011), Ch. 19

## V

- **Velocity measurement** - Quadrature periodic mode, Ch. 14
- **Voltage, DAC** - Output calculation, Ch. 10, App. C

## W

- **Watchdog** - Timeout detection, Ch. 13
- **WRPIN** - Write mode (PASM2), Ch. 4
- **WXPIN** - Write X (PASM2), Ch. 4
- **WYPIN** - Write Y (PASM2), Ch. 4

## X

- **X register** - Smart pin configuration, Ch. 3, 4

**Y**

- **Y register** - Smart pin parameter, Ch. 3, 4

**Z**

- **Z register** - Smart pin accumulator/result, Ch. 3, 4

*Page numbers refer to chapter numbers in this digital document.*